Snapper Documentation

Comprehensive project guide

Generated on 2026-10-22

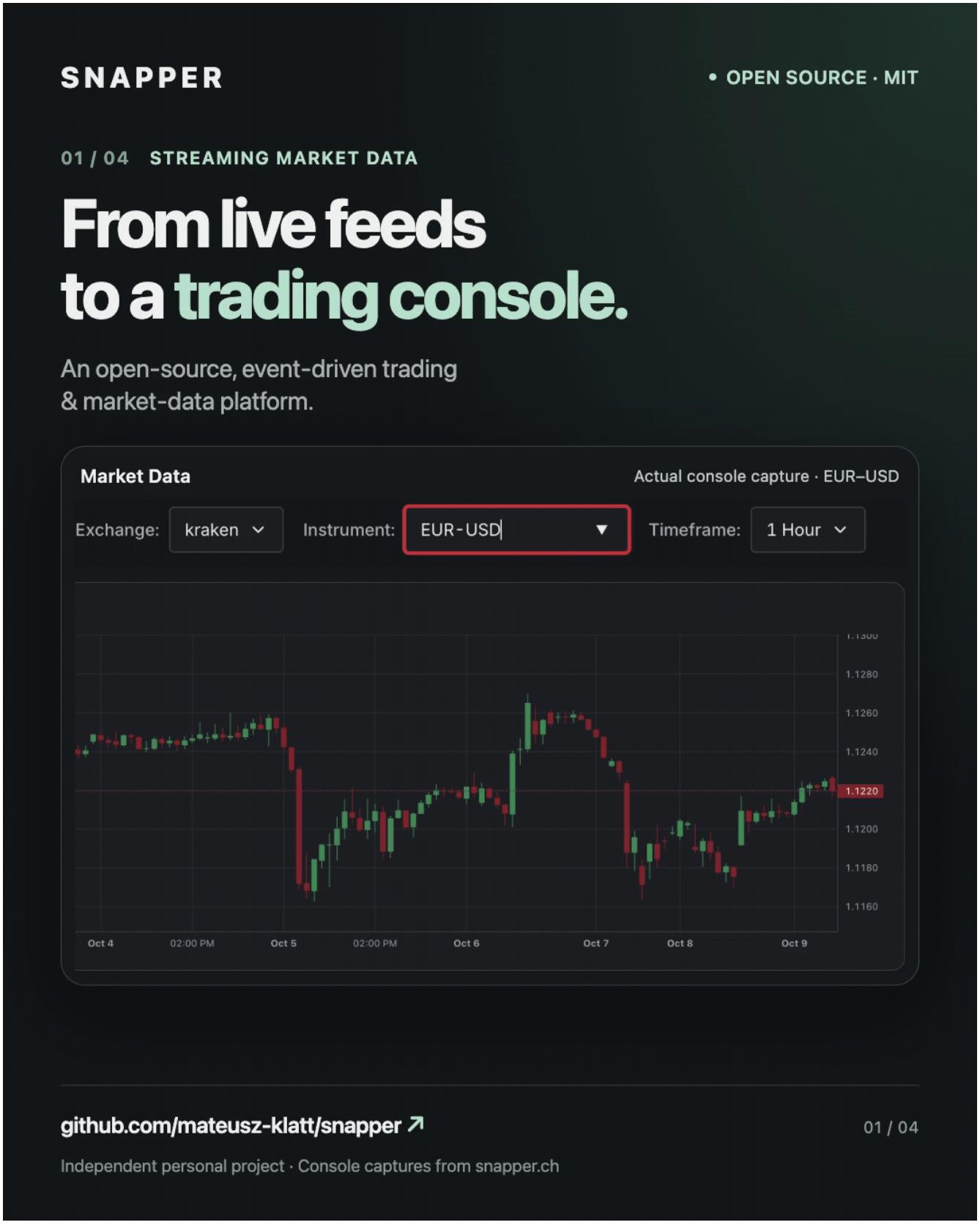
Table of Contents

- Snapper
- AI Integration
- Backend-owned Android push
- API
- Architecture
- Backtesting
- CLI
- Configuration
- Development
- Messaging (ZeroMQ)
- Observability
- Operations runbook — multi-instance trade coordinator
- Operations runbook — paired-execution guard
- snapper-egress sidecar — operator runbook
- Trading Strategies

Snapper



Trading platform with market data collection, trade runtime, and backtester. Supports Kraken (WebSocket; spot + futures + equities), Walutomat (REST polling), and Polygon.io (REST) market data.



A short tour of the Snapper console: live market data, linked instruments, historical navigation, and configuration.

Quick Steps

From a fresh checkout, initialize the public submodules first. The private proprietary strategies/seed profiles and data/polygon historical cache are optional and require separate repository access.

```
# One-time bootstrap (HTTPS also works without GitHub SSH credentials).
git -c url."https://github.com/".insteadOf=git@github.com: \
  submodule update --init frontend integrations/snapper-mcp ios android
make system-deps
make setup
cp .env.example .env
make ui-setup

# Build the static UI so the FastAPI server can serve the dashboard at `/`.
make ui-build

# Initialize the database + seed dev data, then refresh the verified
# symbol mappings, underlying mappings, and non-Polygon market snapshots
# so the dashboard has data to show.
make migrate-dev run-static

# Start the server
make run-server
```

Open http://localhost:8000/ and log in with the dev seed credentials — the default values are defined in the bundled seed file src/snapper/data/seed/dev.toml (admin / change-me-after-first-login); maintainers with the proprietary submodule get proprietary/data/seed/dev.toml via the three-tier seed lookup. Override per-environment by editing the seed file before running make migrate-dev, or rotate after first login via POST /api/auth/users/{user_id}/change-password (self-service) or POST /api/auth/users/{user_id}/admin-reset-password (admin reset).

Features

- **Market data collection** — Kraken (WebSocket; spot + futures + equities), Walutomat (REST polling), Polygon.io (REST)
- **Candle synthesis** — Multi-timeframe candle synthesis from the 1m base stream (in-process per-publisher rollups) with provenance tagging (native / calculated / synthesized), controlled by the timeframes, candle_forward_fill, and persist_intermediate_candles settings. candle_minute_completion additionally completes the base 1m plane on continuously-open trade-driven venues (Kraken spot in trade_built mode, Kraken futures), emitting a flat zero-volume bar for every minute the publisher observed the venue live and the instrument did not trade
- **Egress routing** — Per-venue VPN egress routing via the WireGuard/SOCKS5 snapper-egress sidecar. feed_egress_enabled gates feed-publisher pools; the API process initializes its own pool from egress_pool, so API-side Kraken public REST reads may

route through it independently. Private/executor traffic stays direct-first, with private_fallback_route_id used only when direct is unavailable; private mutations remain direct-only

- **Symbol correlation** — Underlying asset model linking instruments across exchanges (e.g., SPY, ESM6-CME, SPYX-USD-PERP all map to S&P 500). YAML-driven pattern matching, front-month rollover, contract ladder API
- **Trade runtime** — Facts-canonical live and paper trading with canonical Order and Execution facts, rebuildable Position and cash projections, dispatched via a durable outbox
- **Strategies** — Framework for creating strategies based on RSI, MACD, cointegration, and TA-Lib indicators
- **ZeroMQ messaging** — Pub/sub architecture for market data and signals
- **Provenance and audit** — Every payload item carries session_id and sequence_id for gap detection. Control table (always-on) and telemetry table (toggleable) record all commands and operational events
- **Web dashboard** — FastAPI + React with real-time WebSocket
- **Native mobile clients** — iOS with SwiftUI and Android with Kotlin and Jetpack Compose, both using the same backend. The Android client provides wallet-scoped positions, balances and P&L, signals with CSV export, market discovery, live candle charts and related instruments, orders and fills, alerts, desks, account settings and system inventories. Manual orders, position actions, process controls and delegated AI-review decisions require permission checks and confirmation. Interface language synchronizes with the backend account; country formatting and financial colors remain independent. Its README tracks the remaining iOS features and partial translations in 45 languages.
- **CLI** — Full system management via Typer CLI
- **Backtesting** — Strategy testing on historical data

Authorization model

Capabilities are decided from the authenticated token's effective permissions. A role is only a stable, named permission set used for account management and display; application code does not use role rank to authorize an action.

The viewer set is a read-only operator. It has the same read-side capability coverage as an operator across market data and views, account state, operator-scoped wallets and portfolio, orders, positions, signals, AI-review decisions, P&L timeline and attribution, backtests, strategies, process state, health, AI integration, and notifications. It cannot create or cancel orders, manage positions, control processes or strategies, run backtests, manage AI integration, or perform administrative mutations. It also cannot create backtest comparisons or control runtime diagnostics. Wallet visibility follows the union of wallets covered by the viewer's operator scope grants and active personal wallet read

grants, including when the viewer has no operator memberships. Personal read grants authorize visibility only; trading remains operator-scoped. This differs from the system-wide scope provided by effective impersonate:operator. That structural permission is non-downscopable for the admin named set so historically global admin scope cannot disappear when an admin token is narrowed. AI-delegate list and detail are the deliberate scope split: a read-only token uses operator memberships, while a token with the management permission uses creator ownership.

See Roles and Permissions for every named set and the token-downscoping rules.

Quick Start

Requirements

- Python 3.14+
- Poetry
- Node.js 26+ and pnpm 11+ (for frontend)
- Build tools for Python packages. The TA-Lib adapter uses TA-Lib when importable and falls back to pure Python indicators otherwise; install the native TA-Lib C library only if your local wheel build requires it.

Installation

```
# Clone repository
git clone https://github.com/mateusz-klatt/snapper.git
cd snapper
git -c url."https://github.com/".insteadOf=git@github.com: \
  submodule update --init frontend integrations/snapper-mcp ios android

# Install system dependencies
make system-deps

# Install Python dependencies
make setup

# Install frontend dependencies
make ui-setup

# Copy and configure environment variables
cp .env.example .env
```

Configuration

Edit .env file:

```
# Database (SQLite dev, PostgreSQL prod)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# Deployment environment — production/prod/staging refuse the placeholder
# MASTER_PASSWORD (the only secret; all internal keys derive from it)
SNAPPER_ENV=development

# Settings encryption in database
MASTER_PASSWORD=your_master_password

# HTTP Server
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
SERVER_API_ONLY=false
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1

# Telemetry recording (pings, heartbeats, GET reads)
TELEMETRY_RECORDING_ENABLED=false

# ZeroMQ broker
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501

# Max command age before the outbox expires stale create/submit
# commands instead of publishing them (<= 0 disables)
TRADE_COMMAND_DISPATCH_TTL_S=30.0
```

Running

```
# Initialize database and seed data
make migrate-dev

# Start server
make run-server
```

Two ways to bring up the dashboard:

- **Backend-served (production / single port).** Run make ui-setup + make ui-build to produce frontend/dist/, then start the server. The FastAPI app mounts the static UI at / only when frontend/dist/ exists; the dashboard is then at http://localhost:8000/.
- **Vite dev server (hot reload).** Run make ui-dev in a separate terminal — Vite serves the dashboard at http://localhost:3000/ with a backend proxy for /api and /api/ws. This does NOT produce frontend/dist/, so http://localhost:8000/ will still return 404 for the UI until the build step is run.

CLI snippets below use bare snapper for readability. In this repository, Makefile targets run the CLI as .venv/bin/python -m snapper; use that form unless your virtual environment is activated or the Poetry console script is already on PATH.

Trade Runtime Dispatch

The trade runtime always uses durable, outbox-driven dispatch: commands are persisted as TradeCommand rows and published by the outbox dispatcher. On the accepted/fill paths, executor VenueEvent persistence is fail-closed — a failed write raises before the corresponding orders.events.* publish. An ambiguous submit failure never fabricates a rejection: the executor verifies venue truth via find_order_by_client_id and, when the venue cannot answer, parks the order in the non-terminal UNKNOWN state — the engine holds its in-flight guard, the recon loop re-verifies the order each cycle until resolved, and the order_unknown notification rule alerts the operator. "Ambiguous" covers any failure whose placement outcome is unproven, not only a lost response: on Kraken Spot's ccxt path a venue error ccxt could not classify (the bare ccxt.ExchangeError type) takes the same verification route, while every classified subclass still rejects immediately.

Replayed submits of an already-evidenced client_order_id (outbox re-publishes after a crash between publish and the dispatched commit) are dropped by a duplicate-submit guard; its durable venue-evidence probe is fail-closed — a failed check drops the command rather than risk double-placing a market order. Dispatch is also bounded by a max-age TTL (TRADE_COMMAND_DISPATCH_TTL_S, default 30 s): the outbox CAS-expires stale CREATED create/submit commands to EXPIRED instead of publishing them, so an outage backlog cannot fire orders priced off old signals. Cancels are exempt — expiring a stale cancel would strand a live order. A submit refused by an open venue circuit breaker is neither left ambiguous nor blind-retried: the executor records durable breaker-open evidence (counted as duplicate-submit evidence), marks the command FAILED, and publishes a rejection with reason circuit_breaker_open so the engine releases its in-flight intent; if any step fails, the order is parked and the reconciliation loop reruns the disposition.

A durable live_trading_mode kill-switch setting gates every non-paper submit, read fresh per order by the executor interlock (src/snapper/messaging/executors/base.py): only enabled proceeds, halted blocks, reduce_only currently blocks like halted, and an unreadable, missing, or invalid value fails closed as live_trading_mode_unavailable. Paper venues — discriminated by the executor's own venue, never the caller-supplied order mode — always pass. A blocked submit runs a distinct interlock disposition: a durable order_interlock_blocked terminal, then the command marked FAILED, then a REJECTED publish so the engine releases intent — so a frame replayed after the mode later flips to enabled can never execute; if the disposition cannot complete, the entry is parked for the reconciliation loop to rerun.

Private fill streams are supervised: a dead venue execution stream is respawned with capped backoff, each respawn reconciles fills missed during the dark window before resubscribing, and executor startup recovery emits corrective fills for anything filled while the executor was down instead of re-baselining to the venue's current cumulative. Corrective fills carry venue-reported fees rather than fabricating fee-less executions; when the venue's fee source is transiently unavailable, the corrective is deferred for a

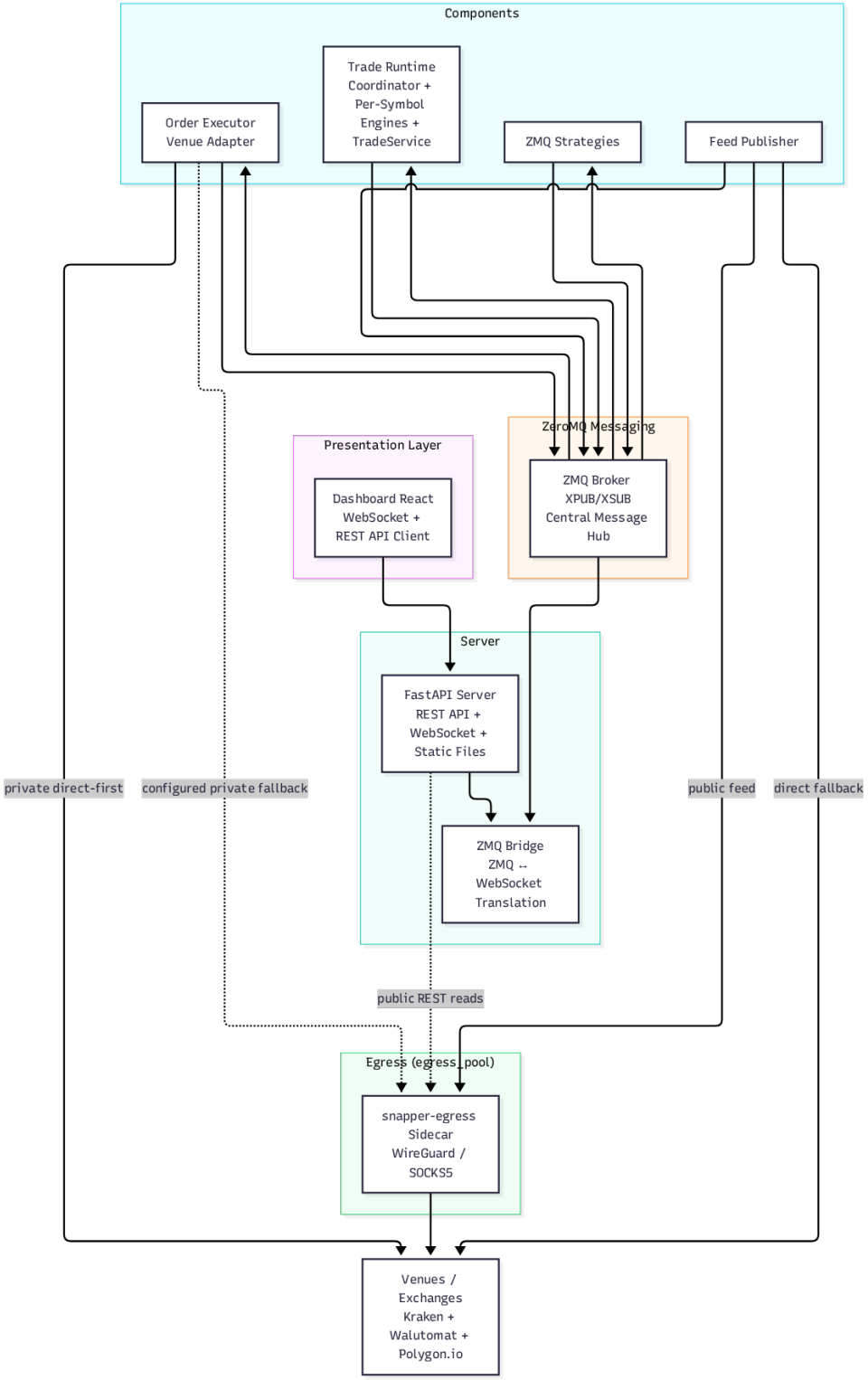
bounded number of reconciliation cycles before degrading to a fee-less emission with a critical alert. The same supervision wraps every executor core loop (order handling, reconciliation, heartbeat): a died loop respawns with capped backoff, and a persistent death streak escalates by crashing the executor so the process launcher rebuilds a fresh instance. Executor heartbeats report status derived from actual loop health — death streaks, reconciliation progress, stuck in-flight commands — instead of a hardcoded healthy, and an executor parked after exhausting its restart budget is alerted through launcher-published synthetic error heartbeats rather than vanishing silently.

Execution-plan decision events (plans.decisions.*) get the same durability: each decision row the plan executor logs commits together with an outbox row in one transaction, the plan executor publishes immediately, and a background drain loop replays unsent rows with capped exponential backoff after broker outages.

Process-Managed Executors

Process-managed deployments run one executor per (exchange, wallet) credential row. Bare executor_<exchange> process configs are templates only; the launcher expands active wallet_credentials rows into executor_<exchange>_w<wallet_short> instances, where wallet_short is the last 12 lowercase hex characters of the wallet UUID7. Each instance loads only its wallet credentials and filters incoming command frames by wallet_public_id.

System Overview



CLI Commands

This is a quick command map, not the full inventory. Run .venv/bin/python -m snapper --help or see docs/cli.md for complete options and examples.

Server and Infrastructure

```
snapper server              # Start FastAPI server
snapper broker              # Start ZMQ broker
snapper trade-zmq           # Trade runtime / coordinator (pass --instance-id +
--instance-count for N>=2, see docs/operations.md)
snapper executor            # Standalone order executor helper
snapper feed                # Direct Kraken market data publisher helper
snapper feed-engine         # Dedicated feed container entrypoint
snapper notify              # Notification sidecar + durable drift-page recovery
snapper egress              # WireGuard + SOCKS5 egress sidecar entrypoint
```

Database

```
snapper db-init             # Initialize schema
snapper db-upgrade          # Alembic migrations
snapper db-downgrade        # Rollback migrations
snapper db-seed             # Seed profile data
snapper retire-trade-projection-checkpoints --all  # Dry-run checkpoint
retirement
```

Configuration and Maintenance

```
snapper settings-rotate-encryption  # Rotate encrypted settings rows
snapper reconcile-symbol-aliases    # Close stale SCD2 alias rows
```

Users

```
snapper init-admin          # Create admin user
snapper list-users          # List users
snapper reset-password      # Reset password
```

Credentials

```
snapper token preflight --token-file PATH   # Read-only: would this credential
still authenticate?
```

preflight answers, before a deploy that tightens token acceptance, whether a live credential's hash still resolves to an active unrevoked access row naming the same jti it was signed with. It accepts the MCP bridge --config=PATH JSON envelope or a file holding the raw JWT, exits 1 when the running server would reject the credential, and prints neither the token nor its hash.

Run it on the deployment's own host, against the deployment's own .env: it reads the JWT signing algorithm and the inventory row from that database, and the signing key is derived from MASTER_PASSWORD, so a foreign environment reports a false rejection on a credential that works.

Local AI delegate PAT (for @mateusz-klatt/snapper-mcp bridge)

After make dev-backend is running and the seed has provisioned the admin user (per dev.toml), mint a long-lived AI delegate PAT into a JSON file the bridge consumes via --config=PATH:

```
snapper dev-mint-pat        # writes data/dev-pat.json (mode 0600)
```

Wire your ~/.claude.json mcpServers entry once with the file path:

```
"snapper-local": {
  "command": "node",
  "args": [
    "/path/to/snapper-mcp/dist/index.js",
    "--config=/path/to/snapper/data/dev-pat.json"
  ]
}
```

After running, in this order, (a) rm data/snapper.db, (b) make migrate-dev, (c) make dev-backend (in a separate terminal — it blocks the foreground with the live backend), and (d) snapper dev-mint-pat, the bridge picks up the refreshed token automatically — no manual edits to ~/.claude.json per DB wipe. CLI flags + environment overrides:

```
--base-url URL        SNAPPER_DEV_BASE_URL          (default http://
localhost:8000)
--admin-username USER SNAPPER_DEV_ADMIN_USERNAME    (default None → resolved
from seed profiles, mcp then dev)
--admin-password PASS SNAPPER_DEV_ADMIN_PASSWORD    (default None → resolved
from seed profiles, mcp then dev)
--output PATH         SNAPPER_DEV_PAT_OUTPUT        (default data/dev-pat.json)
--label LABEL                                         (default "Local Dev MCP")
```

The script drives production REST endpoints (POST /api/auth/login + POST /api/ai-delegates) so the seeded delegate exercises the same code path the AI Integration UI uses. Tokens never appear in stderr — HTTP error bodies are passed through a JWT-shape redaction helper before printing.

Market Data

```
snapper update-kraken-symbols        # Sync Kraken symbols
snapper update-kraken-futures-symbols
snapper update-kraken-equities-symbols
snapper update-walutomat-symbols
snapper update-polygon-symbols       # Sync Polygon symbols
snapper update-underlyings           # Sync underlying asset mappings from YAML
snapper update-kraken-market-snapshot
snapper update-kraken-futures-market-snapshot
snapper update-kraken-equities-market-snapshot
snapper update-walutomat-market-snapshot
snapper polygon-backfill-aggregates
# Step 1: download history to CSV cache (no DB write)
snapper polygon-load-csv --all       # Step 2: load the CSV cache into the
database
snapper polygon-backfill-grouped
snapper polygon-load-grouped-candles
snapper verify-candle-coverage
snapper kraken-futures-backfill-candles
snapper kraken-equities-backfill-candles
snapper update-kraken-futures-funding-rates
snapper build-continuous
snapper archive --day 2024-01-15     # Export candle cache to CSV
snapper archive --from 2024-01-01 --to 2024-01-31 --exchange polygon
snapper archive --table ticks --day 2024-01-15 --exchange polygon
snapper archive --table candles-audit --day 2024-01-15 --closed-only --purge
snapper restore --table candles --dir data/archive/candles/polygon/BTC-USD/
```

Backtests

```
snapper backtest-run
snapper backtest-list
snapper backtest-cancel
snapper backtest-rerun
```

See docs/cli.md for full options and examples.

Strategies

Creating your own strategy:

```
from snapper.strategies.base import BaseStrategy, StrategySignal, StrategyConfig
from snapper.strategies.decorators import register_strategy, create_strategy_pro
cess
from snapper.messaging.schemas.data import CandleData

@register_strategy("MyStrategy")
@create_strategy_process(
    process_name="my_strategy",
    default_config={
        "name": "my_strategy",
        "inputs": ["market.kraken.BTC-USD.candles.1h"],
```

```
        "outputs": ["BTC-USD"],
        "exchange": "paper",
        "params": {"threshold": 0.5},
    },
)
class MyStrategy(BaseStrategy):
    def __init__(self, config: StrategyConfig) -> None:
        super().__init__(config)
        self.threshold = self.params.get("threshold", 0.5)

    async def reset(self) -> None:
        """Clear any per-instrument indicator/state buffers."""

    async def on_candle(self, instrument: str, candle: CandleData) -> StrategySi
gnal | None:
        if some_condition:
            return StrategySignal(
                instrument=instrument,
                side="buy",
                strength=1.0,
                price=candle.close,
                reason="Condition met",
            )
        return None
```

reset() is abstract on BaseStrategy — every subclass must implement it so the runtime can recycle per-instrument state on restart / replay.

Development

Quality Gates

```
make check-all   # Full gate (backend + frontend + generated-type drift + 100%
coverage)
make fix-all     # Automatic formatting and linting fixes
```

Individual Steps

```
make fmt         # Check formatting
make lint        # Linting (ruff)
make typecheck   # Type checking (mypy)
make test        # Unit tests
make cov         # Tests with coverage (100% required)
```

Generated contracts are split by target: make ui-gen-types refreshes frontend OpenAPI/WebSocket/Zod/entity/permission types, make ios-gen-types refreshes Swift models, and make ts-bridge (or make bridge-regen) refreshes the opt-in snapper-mcp bridge wire contract.

Native Android client

The public Apache-2.0 Android client is a separate native application in the android/ submodule. Open that directory in Android Studio or use make android-check and make android-build with JDK 17 and Android SDK 37.0 / Build Tools 36.0.0 installed. Android build, unit tests, lint and debug APK artifacts also run in the client's own GitHub Actions workflow. Its SonarCloud analysis imports combined JVM/device coverage and checks the quality gate in that workflow.

The Android release guide describes the manual workflow for unsigned bundles or signed bundles verified against an upload-certificate pin. Google Play publication requires separate account setup and credentials, plus successful native CI for the exact commit.

Use http://10.0.2.2:8000 in the debug emulator, or an HTTPS origin for a self-hosted backend. The Android README includes USB/ADB instructions for a physical phone. Release builds require HTTPS. Optional backend Android push uses a separate FCM FID registration contract; the APNs endpoint remains unchanged. See Android push configuration for explicit, disabled-by-default server configuration and the remaining real-provider validation boundary.

Remembered Android sessions require Android 11 or newer, a secure screen lock, and device authentication within the last five minutes. Otherwise, credentials stay in memory and restarting the app requires signing in again.

Deterministic P&L browser UAT

scripts/pnl_uat_fixture.py seeds a one-shot paper-only P&L scenario and writes its credential-free browser manifest. It accepts the database URL only through DB_URL and refuses non-PostgreSQL, non-loopback, non-fresh/non-canonical OSS seed baselines, non-head, or non-snapper_pnl_uat_* targets, as well as any active non-paper credential.

Prepare a new disposable local PostgreSQL database at Alembic head with the package-bundled OSS dev seed, then run:

```
export DB_URL='postgresql+asyncpg://<local-user>:<password>@127.0.0.1:5432/
snapper_pnl_uat_<run-id>'
.venv/bin/python scripts/pnl_uat_fixture.py \
  --manifest /absolute/path/to/new-pnl-uat-manifest.json
```

The manifest path must not already exist. Fixture facts commit first; six activation anchors are then created through the production service in independent transactions. If anchor creation fails, discard the database rather than rerunning the fixture. Use --anchor only for an explicit non-future UTC minute; otherwise the script captures the current UTC minute. This command is never suitable for a shared, staging, or production database.

Frontend

```
make ui-dev      # Vite development server
make ui-build    # Production build
make ui-lint     # ESLint
make ui-format   # Prettier
```

Docker

```
make docker-build-dev    # Build dev image (backend + caddy binary baked in)
make docker-build-prod   # Build production image
make docker-migrate-dev  # Initialize and seed the Docker SQLite database
make docker-run          # Run container
make docker-stop         # Stop containers
```

Or with docker compose:

```
make docker-migrate-dev
docker compose up -d
```

Compose topology

The compose stack runs seven application services on the internal snapper-internal bridge network, all from a **single image** (klattm/snapper:latest) that bundles the python runtime + the Caddy binary. Each service overrides entrypoint / command to launch the right process. The optional postgres service is gated behind the dev compose profile. A separate runner-only delegate canary is gated behind the delegate profile and is not part of the default stack.

- snapper-broker — dedicated ZeroMQ XPUB/XSUB broker container (command: ["broker", "--xsub", "tcp://0.0.0.0:7500", "--xpub", "tcp://0.0.0.0:7501"], expose: 7500/7501 internally). Every other service connects to it via tcp://snapper-broker:7500/7501, so restarting the backend no longer bounces the bus.
- snapper — FastAPI backend, strategy/process control, and non-publisher runtime processes. PROCESS_AUTOSTART_PROFILE=api, ZMQ_BROKER_EMBEDDED=false (the dedicated broker container owns the bus), command: ["server"], expose: 8000 internally.
- snapper-feed — dedicated market-data publisher container. PROCESS_AUTOSTART_PROFILE=feed, command: ["feed-engine"]; it connects to the broker via tcp://snapper-broker:7500/7501 and uses reduced DB pool settings so publisher subprocesses do not overrun PostgreSQL connection limits.
- snapper-strategies — dedicated strategies container (PROCESS_AUTOSTART_PROFILE=strategy, command: ["strategies-engine"]). Runs role-STRATEGY processes THREAD-mode with the same boot scope enforcement and

watchdog they had in the backend; mounts ./proprietary read-only and imports STRATEGY_EXTRA_PACKAGES so out-of-tree strategies register. The backend sets STRATEGIES_EMBEDDED=false so the same strategy can never run twice.

- snapper-egress — WireGuard + SOCKS5 sidecar for outbound publisher traffic. command: ["egress"], cap_add: NET_ADMIN, kernel module bind.
- snapper-notify — alert-rule evaluator and notification sidecar (command: ["notify"]). Subscribes to the rule registry's ZMQ prefixes (heartbeats, order events, committed portfolio-drift lifecycle), evaluates the alert rules, records alert_events, republishes alerts.{user}.{type} for the WebSocket bridge, and fans deliveries out to APNs using the apns_* settings, plus optional backend-owned FCM FID delivery configured through SNAPPER_ANDROID_PUSH_CONFIG_FILE. FCM-only startup can disable APNs without requiring APNs credentials. Without this container the ENTIRE alert chain is dark — no alert rows, no UI alerts, no push. The process supervises its receive and retry workers, awaits their cleanup on shutdown, and propagates unexpected worker failure to the configured container restart policy.
- snapper-delegate — optional runner-only Python PID1 under the delegate profile. This no-egress increment is deliberately inert: network_mode: none, no coordinator, DB, broker, Docker socket, source-tree bind mount, full data/ mount, secret mount, or host port. It runs as 888:888 on a read-only root filesystem with all capabilities dropped, a no-exec temporary filesystem, fixed resource limits, and only data/log/delegate/no-egress mounted at its model log path. The host log directory must already exist and be writable by UID/GID 888; Compose will not create it. Missing configuration or secret files leaves the PID1 safely idle. This profile does not enable a vendor request or any network path.
- snapper-web — Caddy serving the React SPA from /srv/dist and reverse-proxying /api/*, /api/ws, /api/mcp, /docs, /redoc, /openapi.json to snapper:8000. entrypoint: ["/usr/bin/caddy"], command: ["run", "--config", "/etc/caddy/Caddyfile"]. Owns the host 127.0.0.1:8000:8000 bind.

Every listed application service is restart: unless-stopped so it comes back automatically after a host reboot (assuming systemctl is-enabled docker returns enabled). Services explicitly stopped via docker compose stop stay stopped.

Targeted restarts (no tick-stream drop)

Restart targets are now strictly recreate-only — they use whatever image tag exists locally. Rebuild explicitly first if you need new code baked in:

```
# Build (explicit, when you want new code in the image):
make docker-build-dev    # caller UID (host user)
make docker-build-prod   # UID 888 (matches log file ownership)

# Recreate containers (uses current image, no rebuild):
make restart-frontend    # Recreate Caddy sidecar [does NOT touch backend]
make restart-backend     # Recreate backend [API/trade-runtime interruption;
```

```
feed and broker stay up]
make restart-all         # Recreate full stack [drops ticks]

# Common idiom — build + restart in one shot:
make docker-build-prod restart-backend
```

make restart-frontend is the fast path for shipping UI changes without restarting publishers. Only the Caddy container is recreated; WS clients must reconnect, but the backend snapper container's PID is unchanged and the Kraken / Walutomat tick streams continue uninterrupted.

Footgun: a plain docker compose up -d (without --no-deps) follows depends_on edges and may recreate the backend too. Use the targeted Makefile commands for selective restarts, and reserve docker compose up -d for full-stack restarts when both should restart anyway.

Documentation

Detailed documentation in docs/ directory:

- Architecture — System structure and layers
- Configuration — Environment variables and settings
- CLI — Full command documentation
- Strategies — Creating trading strategies
- Backtesting — Backtest engines and artifacts
- API — REST API and WebSocket
- Messaging — ZeroMQ architecture
- Operations — Multi-instance coordinator runbook
- Observability — Process, notification, retention, and DB table metrics surface
- AI integration — MCP endpoint and AI service-principal tokens
- Development — Developer guidelines (incl. Internationalization)
- Paired execution — Multi-leg guard operator runbook
- Egress — WireGuard + SOCKS5 egress sidecar runbook

Regenerate frontend/public/snapper.pdf with make docs-pdf before building the frontend bundle. The PDF belongs to the frontend submodule; publish its commit there before updating the parent repository's frontend gitlink.

License

Original code and documentation are licensed under the Apache License, Version 2.0. See NOTICE for attribution. Third-party components, including the vendored CCXT wheel, retain their own licenses. Each submodule has its own licensing terms; cached market data is not covered by this license.

Revisions through the last-mit tag were released under MIT. Subsequent revisions use Apache-2.0 for original work; earlier MIT grants remain valid.

AI Integration

Snapper exposes a **Model Context Protocol (MCP)** endpoint at /api/mcp so Claude Desktop, Cursor, Windsurf, or a plain curl client can authenticate with a long-lived AI delegate or researcher access JWT and call a permission-filtered set of trading and market-data tools. The surface is vendor-neutral — no Anthropic or OpenAI SDK is bundled in Snapper core.

This doc covers:

1. Feature flag
2. Creating an AI delegate
3. Creating an AI researcher
4. Token model
5. Client configuration examples
6. Available tools
7. Safety caps
8. Kill switch + deactivation
9. Rate limits
10. Error catalog
11. Security model

Feature flag

The /api/mcp sub-app is always mounted and gated by the ai_integration_enabled database setting.

1. The flag **defaults to true** — a fresh install exposes the MCP endpoint and the AI Integration navigation entry with no manual setup. A caller with configure:system can disable the feature through the Settings UI or directly:

   ```
   curl -X POST http://localhost:8000/api/settings/ai_integration_enabled/set \
     -H "Authorization: Bearer <settings-manager-jwt>" \
     -H "Content-Type: application/json" \
     -d '{"session_id":"cli","sequence_id":1,"public_id":"'$
   (uuidgen)'","timestamp":"2026-04-20T00:00:00Z","payload":
   {"value":"false","category":"system"}}'
   ```

   Note: the REST settings write is cached verbatim as a string in the serving process; the flag change takes effect only after the API server restarts (setting values are parsed to booleans when settings are loaded at startup).

2. The feature endpoint itself is public, but the frontend route and navigation entry require read:ai_integration. The page derives all management controls separately from manage:ai_integration, so the current viewer set can inspect its operator-scoped integration state without gaining a mutation. After authentication, the page reads GET /api/settings/features and renders the enabled or disabled state. The MCP endpoint, /api/ai-delegates/*, and /api/ai-researchers return 503 feature_disabled only when the flag is explicitly set to false.
3. When the flag is on, the MCP endpoint requires every request to carry a valid Authorization: Bearer <jwt> header; anonymous requests receive 401 missing_bearer_token. When the flag is off, all requests — anonymous or authenticated — short-circuit to 503 feature_disabled before any token verification runs.

Authorization uses each token's effective grant from Snapper's 34-permission catalogue, not a role hierarchy. Roles are named permission sets. The current viewer set includes read:ai_integration and read:ai_reviews, uses its operator memberships for read-only integration visibility, and contains no AI-integration management permission. A token with manage:ai_integration instead receives the creator-owned delegate view. Only an effective impersonate:operator grant provides global operator scope on surfaces that support global scoping.

Creating an AI delegate

An **AI delegate** is a dedicated user assigned the ai_delegate named permission set and minted per MCP client by a caller with manage:ai_integration. Delegates:

- Cannot log in via the web UI password form (the placeholder password hash is opaque to humans).
- Cannot see other delegates, operators, or wallets — their wallet scope comes from the operator they are bound to.
- Carry per-delegate safety caps independent of the operating operator's caps.

Each integration owner may own at most 5 active delegates (deactivated delegates do not count toward the cap, so rotation is unbounded).

Create one via POST /api/ai-delegates:

```
curl -X POST http://localhost:8000/api/ai-delegates \
  -H "Authorization: Bearer <integration-manager-jwt>" \
  -H "Content-Type: application/json" \
  -H "X-CSRF-Token: <csrf>" \
  -d '{
    "session_id": "cli",
    "sequence_id": 1,
    "public_id": "'$(uuidgen)'",
    "timestamp": "2026-04-20T00:00:00Z",
```

```
    "payload": {
      "label": "Claude Desktop",
      "operator_public_id": "<operator-public-id>",
      "permissions": [
        "read:market_data",
        "read:orders",
        "read:positions",
        "read:strategies",
        "read:signals",
        "read:system_status",
        "read:backtests"
      ],
      "caps": {
        "max_open_orders": 3,
        "max_daily_notional_usd": 1000.0,
        "max_cancels_per_minute": 10
      }
    }
  }'
```

payload.operator_public_id is optional. When omitted, Snapper binds the delegate to the caller's primary_operator_public_id; if the caller has no primary operator, create returns 422 until an explicit operator is supplied. When the caller's effective grant lacks impersonate:operator, a supplied operator must be in the caller's authenticated operator claims. An effective grant containing impersonate:operator may bind explicitly to any operator. The minted delegate token inherits scope from that bound operator, so later scope grant changes for that operator control which wallets and instruments the delegate can act on.

payload.permissions independently downscopes what that credential may do. The read-only monitoring example above omits order creation, cancellation, and position management even though the ai_delegate named set contains them. The server enforces the intersection of the named set and token grants and returns 422 if the request includes a permission outside that set. Omit permissions to retain the complete named-set grant.

The response is **one-shot**. Copy the token out of the HTTP session immediately — Snapper never re-serves it:

```
{
  "type": "delegate_created_response",
  "sequence_id": 1,
  "public_id": "<envelope-uuid7>",
  "timestamp": "2026-04-20T00:00:00Z",
  "session_id": "<server-session>",
  "topic": null,
  "payload": {
    "delegate": {
      "public_id": "019da9e...",
      "username": "ai-claudedesktop-a1b2c3",
      "label": "claudedesktop",
      "created_by_user_public_id": "<creator-user-id>",
      "created_at": "2026-04-20T00:00:00Z",
      "is_active": true,
```

```
      "caps": { "max_open_orders": 3, "max_daily_notional_usd": 1000.0, "max_can
cels_per_minute": 10 }
    },
    "access_token": "<jwt-with-90d-exp>",
    "expires_in": 7776000
  }
}
```

Other endpoints on /api/ai-delegates:

- GET /api/ai-delegates — requires read:ai_integration; a management token sees creator-owned delegates, while a read-only token sees delegates bound to its operator memberships. Tokens are never re-served.
- GET /api/ai-delegates/{id} — requires read:ai_integration and applies the same creator-owned or membership-scoped view.
- PATCH /api/ai-delegates/{id} — update caps (SCD2 close+insert). Requires manage:ai_integration; label and username are immutable post-mint.
- POST /api/ai-delegates/{id}/deactivate — kill switch. Publishes admin.user_deactivated on the bus for immediate fanout. Requires manage:ai_integration; every Snapper instance also polls the DB-backed deactivation registry so matching WebSocket sessions close and token LRU entries are evicted even if the broker is unavailable.

Creating an AI researcher

An **AI researcher** is a dedicated principal assigned the ai_researcher named permission set for contexts that ingest hostile third-party material. That set is exactly:

- read:market_data
- read:market_views
- submit:market_view

It does not receive signal, order, position, or system-status permissions. Consequently it cannot subscribe to ai_reviews. or call the order-, position-, signal-, and review-shaped MCP tools. It can subscribe to the ai_research. wake root when its token retains submit:market_view. Periodic wakes arrive on ai_research.{round_public_id}.request with an ai_research.request frame carrying round_public_id and trigger; the round is durable even when the auxiliary wake is lost. The default snapper-mcp watch topic set includes this research root.

Create one via POST /api/ai-researchers with a ResearcherCreateRequest envelope:

```
{
  "type": "researcher_create_request",
  "sequence_id": 1,
  "public_id": "<client-uuid7>",
  "timestamp": "2026-07-21T00:00:00Z",
```

```
  "session_id": "cli",
  "payload": {
    "label": "Macro Research",
    "permissions": ["read:market_data", "submit:market_view"]
  }
}
```

The response returns the researcher projection and its long-lived access token once. Each owner may have at most two active researchers, independently of the five-delegate cap. Provisioning persists only users and user_active_tokens rows: no user_trading_caps, operator membership, or ai_delegates row is created, so researchers never enter consult admission accounting.

Token model

Each AI delegate or researcher mints a single long-lived (~3-month) access JWT. The same token authenticates both the proxy MCP server and the optional push-wakeup watch monitor. Delegate revocation is server-side: POST /api/ai-delegates/{id}/deactivate flips users.is_active=False, revokes the delegate's user_active_tokens row, publishes admin.user_deactivated on the bus, and each Snapper instance evicts matching verify-cache entries on receipt or through the DB-backed fallback scanner. The local JTI blacklist uses a 10-second grace window for requests that raced the kill switch. The 90-day exp is a ceiling, not a commitment; integration owners are expected to rotate delegate tokens on the cadence that fits their key-management hygiene.

Researcher tokens use the same active-token inventory and shared user deactivation enforcement. The dedicated researcher surface currently provisions principals only; a caller with manage:users uses the standard user-deactivation route to revoke one.

Client configuration examples

Claude Code plugin (recommended)

In any Claude Code session:

```
/plugin marketplace add mateusz-klatt/snapper-mcp
/plugin install snapper-mcp@mateusz-klatt-snapper-mcp
/reload-plugins
```

Claude Code prompts for two required values at install time (per the plugin's userConfig schema in integrations/snapper-mcp/.claude-plugin/plugin.json):

- **Snapper API URL** -- your backend's /api/mcp endpoint. The @mateusz-klatt/snapper-mcp bridge accepts the value with or without a trailing slash and normalizes it before connecting.
- **Access token** -- the access_token from the delegate_created response (or paste from the AI Integration page's config-snippet generator). Delegates are PAT-style: the JWT lifetime is ~3 months (LONG_LIVED_TOKEN_EXPIRE_DAYS = 90); operators are expected to rotate by minting a fresh delegate via the same flow before the existing one expires. Revocation is immediate: deactivating the delegate in Snapper invalidates the associated user_active_tokens row server-side within one bus round-trip.

Run /mcp list to confirm the snapper server is connected. The plugin pins the runtime to a specific @mateusz-klatt/snapper-mcp version — the manifest hardcodes the exact version string in mcpServers.snapper.args (currently @0.15.0), kept in lockstep with the plugin's own version field by the integrations/snapper-mcp/test/plugin_manifest.test.ts parity test so a bump to either side fails CI until both match. Future runtime publishes do not silently upgrade existing installs; /plugin update snapper-mcp opts in. The manifest marks the access token as sensitive: true; the host controls storage of its user configuration. In plugin context, the bridge attempts to seed a separate env.json under CLAUDE_PLUGIN_DATA, containing the API URL and token for the watch monitor. It writes and chmods the temporary file to mode 0600 before atomically renaming it; a seeding failure is logged. Do not print this credential file.

Claude Desktop

Add to ~/Library/Application Support/Claude/claude_desktop_config.json:

```
{
  "mcpServers": {
    "snapper": {
      "command": "npx",
      "args": ["-y", "@mateusz-klatt/snapper-mcp"],
      "env": {
        "SNAPPER_BASE_URL": "https://snapper.example.com/api/mcp",
        "SNAPPER_ACCESS_TOKEN": "<copied-from-create-response>"
      }
    }
  }
}
```

The @mateusz-klatt/snapper-mcp npm wrapper presents the access token as a Bearer header on every request. On 401 the bridge surfaces the auth failure to the MCP host once per session; recovery is via recreating the AI delegate in Snapper.

Cursor

In ~/.cursor/config.json:

```
{
  "mcpServers": {
    "snapper": {
      "url": "https://snapper.example.com/api/mcp/",
      "headers": {
        "Authorization": "Bearer <access-token>"
      }
    }
  }
}
```

The delegate access token Cursor sends as a Bearer header is a long-lived PAT JWT (~3 months), so day-to-day operation does not require token rotation more often than the 90-day expiry. Revocation is by deactivating the delegate in Snapper (which invalidates the underlying user_active_tokens row); recovery is to recreate the delegate and update Cursor's Authorization header with the new token.

Windsurf

~/.codeium/windsurf/mcp_config.json:

```
{
  "mcpServers": {
    "snapper": {
      "serverUrl": "https://snapper.example.com/api/mcp/",
      "auth": { "type": "bearer", "token": "<access-token>" }
    }
  }
}
```

curl (diagnostics)

Quick reachability check:

```
# Feature flag
curl http://localhost:8000/api/settings/features

# Hello-world MCP initialise
curl -X POST http://localhost:8000/api/mcp/ \
  -H "Authorization: Bearer <access-token>" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize","params":
{"protocolVersion":"2024-11-05","capabilities":{},"clientInfo":
{"name":"curl","version":"8"}}}'
```

Available tools

Read-only tools surface the delegate's order book, position state, signals, and venue market data. Write and decision tools (submit_manual_order, cancel_order, submit_ai_review_decision, submit_market_view) are gated by per-tool permissions. Only submit_manual_order is unconditionally guarded by TradingCapsEnforcer.guard; cancel_order requires the caps enforcer to be initialized, then the cancel service enters the guard only when emitting a venue cancel command for a plan with a child venue order. Plans without a child venue order use the bare cancellation transition and do not enter the guard. submit_ai_review_decision is not wired to caps_enforcer_getter at registration.

- **list_instruments(exchange: str)** — returns sorted native symbols for the exchange inventory. It is not wallet/operator scoped; read-only access is gated by READ_MARKET_DATA; the current ai_delegate named set contains this permission.
- **list_orders(wallet_public_id?, status?, exchange?, instrument?, limit=50, offset=0)** — paged read of the delegate's order history within accessible wallets. Requires READ_ORDERS. Surfaces order_not_found for inaccessible wallets (anti-enumeration).
- **get_order_status(command_public_id: str)** — single-order lookup keyed by the trade-command public id returned from submit_manual_order. Requires READ_ORDERS; unknown or out-of-scope command ids return order_not_found (anti-enumeration).
- **list_positions(wallet_public_id?, exchange?, instrument?)** — active positions across the caller's accessible wallets. Requires READ_POSITIONS; wallet scope violations return position_not_found (anti-enumeration).
- **get_position_cycle(cycle_public_id: str)** — full open→close audit trail for a position cycle. Requires READ_POSITIONS; unknown or out-of-scope cycles return position_cycle_not_found (anti-enumeration).
- **list_venue_account_states(wallet_public_id?, exchange?)** — truthful venue account states (balances + open positions per wallet/exchange/mode) across the caller's accessible wallets, each mapped through the same fail-closed read surface as REST and carrying an always-present strict reconciliation object. Requires READ_ACCOUNT_STATE; the current ai_delegate named set does **not** contain this permission, so a delegate call returns permission_denied. Wallet scope violations return account_state_not_found (anti-enumeration). Consumers must trust the account's derived effective_status and its reconciliation object's independently derived effective_status / is_authoritative fields, not raw stored statuses. Reconciliation is authoritative only for a fresh, fully revalidated current matched or mismatched verdict; stale evidence and open drift episodes remain visible, while corrupt evidence is cleared.

- **get_ohlcv(exchange, instrument, timeframe, since?, until?, limit=200)** — OHLCV candles for a venue + instrument. Range mode (both since + until ISO 8601 UTC) returns chronological candles in the closed window; latest-as-of mode (both omitted) returns the most recent limit candles in DESC order. Allowed timeframes: 1m, 5m, 15m, 1h, 4h, 1d. Requires READ_MARKET_DATA; market data is public so no wallet scope applies.
- **list_recent_signals(since, instrument?, strategy?, exchange?, wallet_public_id?, limit=50)** — strategy signals fired after a watermark, ordered by fired_at DESC. since is REQUIRED ISO 8601 UTC. Requires READ_SIGNALS; surfaces signal_not_found on wallet scope violation (anti-enumeration).
- **get_latest_research()** — returns the complete current market-view artifact, including its ordered source citations, at the server clock. Requires READ_MARKET_VIEWS. When no causally eligible, unexpired view exists, the successful envelope carries details.market_view = null.
- **get_ai_review_aftermath(review_public_id: str)** — read-only projection for a terminal CONSULT round. Returns the complete persisted review plus orders, executions/fills, position-cycle transitions, and current position snapshots for the review's wallet and instrument over the inclusive [created_at, as_of] window. Requires READ_SIGNALS, a registered AI delegate, and a currently active grant for that exact wallet and instrument. Unknown and out-of-scope reviews both return review_not_found; pending and fanout-dispatched rows return review_not_terminal. Executions include stable order, instrument, mode, and scope-sequence lineage even when the order itself predates the window. The tool never changes review state or emits an audit event.
- **submit_market_view(research_round_public_id, payload)** — validates a strict SubmittedMarketView JSON object and completes the target pending research round atomically. Requires SUBMIT_MARKET_VIEW. Sources are submitted inline as {url, title, retrieved_at} objects; identity, trigger, status, and submitted_at are server-owned. Success returns market_view_public_id. A missing or no-longer-pending round returns research_round_not_pending; invalid source/rationale defenses return invalid_market_view.
- **submit_manual_order(exchange, instrument, instrument_public_id, side, order_type, quantity, idempotency_key, wallet_public_id?, price?, stop_price?, operator_public_id?, ai_review_public_id?, leverage?, post_only?, reduce_only?)** — enqueues a trade command under the delegate's user_public_id with source_surface='mcp' + the caps check from TradingCapsEnforcer.guard. Requires CREATE_ORDERS, rejects out-of-membership operator_public_id values when the caller's effective grant lacks impersonate:operator, rechecks wallet scope against active grants on every call, and fails closed on any cap violation. Validates order params with the same evaluator rule as REST: price is required for limit/stop_limit and stop_price is required for stop/

stop_limit order types. wallet_public_id is optional — when omitted, Snapper resolves the caller's single accessible live wallet; multiple candidates return a structured wallet_ambiguous envelope and zero candidates wallet_unresolved (a blank string returns invalid_argument). Wraps the REST create_order route. leverage, post_only, and reduce_only are optional execution modifiers threaded to the same destinations the REST route uses, and each is omitted from persistence when left at its default:

- leverage (int, default absent) goes to the plan params only when supplied, and always to the trade_commands.leverage column. Trading caps measure notional as quantity x price, so leverage does not raise the exposure caps evaluate.
- post_only (bool, default false) reaches the venue since migration 0051 carried it onto the trade_commands column the executor builds its request from. Before that it was plan-params metadata only and nothing downstream could act on it.
- reduce_only (bool, default false) goes to the trade_commands.reduce_only column, and is forwarded on both Kraken spot paths as well as Kraken futures.

Modifiers are fail-closed: a flag either reaches the exchange or the submit is refused. Persisting an order the caller believes is constrained and then submitting it unconstrained is the failure this rules out — a caller told nothing went wrong has no way to learn the protection was dropped.

Which flags survive is a property of **our client**, never of the exchange, and the refusal messages say so. Kraken spot's API accepts all three; what varies is what Snapper sends:

flag	kraken	kraken_futures	kraken_equities	walutomat	paper
leverage	yes	no	no	no	no
post_only	limit, stop_limit	limit	no	no	no
reduce_only	yes, margin only	yes	no	no	no

A venue absent from that table honours nothing, so a venue added later cannot silently inherit permission to accept a flag its client drops.

The refusals, and why each is worded the way it is:

- order_flags_unsupported — the venue's client does not send the flag. The message says **Snapper** does not send it, never that the exchange lacks the feature, because a caller acting on the latter would go hunting for a different venue when the real remedy is a different client.
- leverage_not_an_order_parameter — kraken_futures only. Futures trade on leverage; it is simply not an order field there but a per-symbol account preference, set separately and outliving the order. Snapper does not set it on the

caller's behalf: it is account-wide for the symbol, so one manual order would silently re-lever every other position on it. A refusal is recoverable; a silent re-lever is not.

- post_only_order_type_unsupported — maker-only constrains an order that rests on the book, so it cannot apply to one that must take liquidity. The two Kraken venues differ on exactly this: a spot stop_limit reaches the wire with oflags=post, while a futures stop_limit maps to stp and the flag is dropped with no error, which is why the gate keys on the (venue, order type) pair.
- reduce_only_requires_margin — kraken only. Spot binds the clamp to a margin position and a cash order has none. The rule is venue-scoped on purpose: kraken_futures carries reduceOnly with no leverage precondition, and applying the spot rule there would make the flag unreachable in both directions, since futures also refuse per-order leverage.

  It is enforced at this boundary and deliberately **not** in the exchange client, which serves every producer of the flag. The strategy engine's closing order, the paired-execution guard scanner's protective flatten, and the bracket and trailing-stop exits all emit reduce_only with no leverage; a client-side version of the rule would refuse every one of them, leaving spot positions that can be opened and never closed.

paper is the exception and is accepted, not refused. It reads none of the flags, but there is no real position to overshoot and no real money, so the only cost is rehearsal fidelity. Refusing would block the default practice venue for a workflow that becomes legitimate the moment it moves to Kraken, so the order goes through and the inertness is disclosed instead.

REST applies none of these gates and is unchanged. Parity with an ungated REST is not the right target for a surface an autonomous agent drives. - **inert_flags** on every accepted envelope lists the requested flags the resolved venue will not act on. Now that every other venue refuses instead, paper is the only thing that can make this non-empty — which is exactly why the field survives the move to fail-closed rather than becoming dead. The key is always present and is an empty list when every requested flag is honoured, so a caller can read it unconditionally and can tell "nothing inert" apart from "an older server that never disclosed". A flag that is set and read by nothing is otherwise a silent lie by omission: the submit succeeds, so the caller concludes the constraint was applied.

- **cancel_order(plan_public_id, idempotency_key)** — cancels an active execution plan via the same PlansCancelService REST goes through. Idempotent: same idempotency_key on retry returns the current plan state without re-executing. Requires CANCEL_ORDERS; unknown and out-of-scope plans collapse to order_not_found (anti-enumeration).

- **submit_ai_review_decision(review_id, decision, rationale?)** — REST counterpart to the POST /api/ai-reviews/{review_public_id}/decision route for the in-process MCP surface; lets the delegate approve or reject a pending CONSULT review. review_id is the UUID7 of the ai_reviews row. Requires submit:ai_review_decision. Tool-catalog visibility, the tool's call gate, and the REST route all answer to one shared capability projection, so a token that can see the tool can also call it and behaves identically on either transport. A legacy branch also admits decision-capable tokens inside the pre-versioning window — a permission-scope version that is absent (the production delegate token predates scope versioning entirely) or a literal 1 — provided the role ceiling grants the decision permission, carries no manage:users, and the token retained create:orders. Order-creation authority alone reaches the decision write on neither surface: a create-only operator token is denied at any version, and a create-only admin token is denied even with an absent version. The review service also verifies the caller has an active operational delegate lifecycle identity and scope grant for the review wallet and instrument. Any such delegate may decide after fanout opens (or once the review has entered fanout_dispatched); while a review is pending and before fanout_after, only the selected delegate may decide. Another delegate receives not_selected_before_fanout. A delegate racing an already-resolved review receives review_already_resolved_by_peer. Not gated by the caps enforcer (the underlying review-decision path does its own SCD2 close-and-insert + bus fanout).

The tool catalog is discoverable via the MCP tools/list JSON-RPC method:

```
curl -X POST http://localhost:8000/api/mcp/ \
  -H "Authorization: Bearer <access-token>" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}'
```

CONSULT wake path (runbook)

A strategy issues a CONSULT round via the create_ai_review_and_await primitive; the service admits it only when an eligible delegate's ai_delegates.last_seen_at is inside the heartbeat window (default 15s). Liveness is maintained by the delegate's WebSocket session: the connect handshake bumps last_seen_at, and every client ping re-bumps it (throttled server-side to at most one write per 5s per delegate). Admission ranks the review owner's own live delegate first, then uses immutable delegate creation time and public ID for deterministic fallback order. A timeout releases the selected delegate's in-flight counter; it does not penalize that delegate or let heartbeat arrival timing reorder the next consult. The bundled HeartbeatConsult strategy exercises the full loop with one consult per 1h candle and, on approval, a paper signal at the configurable heartbeat_signal_strength param (default 0.0 = target-flat, opening no position; a value

in [0.0, 1.0] opens an actionable paper long so the signal→order→fill→position plane is exercised — still paper-only by construction). Its process config must supply UUID7 ai_review_user_public_id and ai_review_strategy_public_id params plus a scoped wallet/operator pair whose grant covers the output instrument.

Every HeartbeatConsult envelope also attempts to include a macro cross-asset snapshot built at consult time from complete persisted 1m kraken_equities candles. macro_contract_symbol selects the quarterly CME equity-index contract and defaults to MNQU6-CME (the September 2026 Micro Nasdaq-100 future); rotate this parameter as part of the operator's quarterly contract-roll runbook. macro_stale_after_minutes defaults to 10.0. The snapshot carries symbol, as_of, server-computed age_minutes, session, change_1h_pct, change_since_session_open_pct, and realized_vol_24h_pct. The change fields use the latest complete print against the print at least one hour earlier and the current CME session's opening price, while realized volatility is the population standard deviation of one-minute simple returns from the 24 wall-clock hours ending at the last print.

Session state reuses the shared CME calendar. A fresh open-session feed reports session: "open". If CME is open but the newest candle is older than the configured threshold, it reports session: "halted", adds the presence-only stale: true key, and sets all three derived values to null. Scheduled daily, weekend, and holiday closures instead report session: "closed" without a stale key: the last print remains the calculation anchor and its honest age and available values remain in the envelope. No proxy series is substituted during a closure. Macro and traded-market snapshot failures are independent and fail-soft, so either section may be omitted without preventing the consult.

Integration-owner steps to arm the wake surface:

1. Mint a delegate (POST /api/ai-delegates) and grant it scope on the strategy's (wallet, instrument) pair.
2. Run snapper-mcp watch with the delegate token. The default subscription already includes the ai_reviews. family; each ai_review.request frame appears as one JSONL line on stdout.
3. Wire the watch stdout into the monitoring host (for example a Claude Code monitor primitive) so a request frame wakes the delegate's session. This host-side wiring lives outside this repository by design — any host able to read a subprocess's stdout can implement it.
4. The woken delegate reads the frame's review_public_id and passes it as the review_id argument of the submit_ai_review_decision MCP tool before the review deadline (default 25s for HeartbeatConsult); the strategy resumes with the outcome, and a decision_ack frame follows on the same ai_reviews. family.

The request is published once to every delegate whose grant passes the bridge scope filter. Before the persisted fanout_after instant, only the selected delegate may resolve it; another granted delegate receives not_selected_before_fanout with fanout_opens_at for

retry. At the exact opening instant and afterward, any granted delegate may win the resolve CAS. The fanout scanner changes database state only and publishes no second request frame.

After a watch reconnect, GET /api/ai-reviews/pending is the catch-up read for reviews whose fanout window (fanout_after, default creation + 30s) has already opened while the delegate was offline. Short-deadline rounds such as HeartbeatConsult (25s) time out before that window opens, so a missed heartbeat frame is simply lost and the next 1h round retries — the catch-up read matters for strategies configured with deadlines longer than the fanout window. The route requires READ_SIGNALS plus the caller's active operational delegate_public_id; the second condition is lifecycle state rather than a role gate.

For a round that already became terminal while the delegate was away, call get_ai_review_aftermath(review_public_id) or its REST mirror, GET /api/ai-reviews/{review_public_id}/aftermath. The returned as_of anchors every temporal row in the projection, while window_started_at is the review's persisted created_at. This is retrospective evidence only: it does not reopen the review and grants no order authority.

Callers with read:ai_reviews, including the current viewer, operator, and admin sets, audit what the AI decided through GET /api/ai-reviews. A caller with effective impersonate:operator sees every operator's reviews; every other reader is narrowed server-side to its explicit operator memberships. The route returns AdminAiReviewListResponse (items + count) with the full per-row outcome: status, decision, rationale, resolution_mode, and the responding delegate. It accepts optional exact-match status, wallet_public_id, and strategy_public_id filters plus a limit (1-500, default 100), newest first. Unlike /pending, it is not keyed by the delegate identity and returns terminal decided rows rather than only pending ones.

Safety caps

Every delegate has its own user_trading_caps row. All fields are optional; null means "unbounded on this axis".

- max_order_quantity_per_instrument — JSON dict {instrument_public_id: max_qty} keyed by the Snapper instrument UUID7 (NOT the native venue symbol). The delegate API accepts only a JSON dict or null; a scalar value stored directly on the caps row (legacy/manual writes) is applied to every instrument by the enforcer, but cannot be set via POST/PATCH /api/ai-delegates.
- max_open_orders — all-time count of the delegate's in-flight commands (every non-terminal status: created/dispatched/ direct_dispatched/accepted/partially_filled).
- max_daily_notional_usd — rolling 24h sum of submit_quantity * submit_price per prior non-rejected LIVE command (mode='paper' history is excluded: paper commands carry a simulator reference price and simulated notional must not consume the live

allowance; raw submit-time price, no USD re-conversion; prior live market orders with no submit price are skipped with a WARN log), plus the new submission's USD-converted notional via the USD price oracle (price_unavailable caps violation when the oracle is stale or missing) — the current submission is evaluated regardless of mode. Submit-time commitment basis; partial fills don't change accounting.

- max_cancels_per_minute — sliding-window cancel rate.

Caps are enforced at trade-command insert sites via the TradingCapsEnforcer.guard() surface — submit_manual_order routes through it unconditionally; cancel_order requires the caps enforcer to be initialized and only enters the guard when the cancel service emits a venue cancel command for a plan with a child venue order (the bare cancellation transition skips the guard). The other MCP tools (submit_ai_review_decision, etc.) are not currently wired to caps_enforcer_getter at registration.

Update caps via PATCH /api/ai-delegates/{id} with manage:ai_integration. The write is SCD2 close+insert, so cap history is auditable.

Kill switch + deactivation

POST /api/ai-delegates/{id}/deactivate runs the same code path as the shared user-deactivation flow:

1. SCD2 close+insert on the users row flipping is_active=False.
2. TokenManager.revoke_user_sessions loads every user_active_tokens.jti for the delegate, flips revoked_at=NOW() in one SQL UPDATE, and seeds the in-memory JTI blacklist with a 10-second grace period.
3. UserService.deactivate_user publishes admin.user_deactivated on the bus as the immediate fanout path (sole publisher).
4. Every Snapper instance's WebSocketAuthManager subscribes to the topic and closes matching WebSocket connections with code 4003.
5. Every Snapper instance's TokenManager subscribes to the topic and evicts matching verify-cache entries via invalidate_user_cache.
6. Every lifespan-wired WebSocketAuthManager and TokenManager also runs a DB-backed fallback scan against the SCD2-active users.is_active row, so broker outages delay fanout by the 5-second scan interval (DEACTIVATION_FALLBACK_SCAN_INTERVAL_S) rather than by token expiry or reconnect.

Latency: the token inventory is revoked before the deactivated-user bus event is published. Existing positive verify-cache entries are evicted when each instance receives admin.user_deactivated; if the bus listener is unavailable, the fallback scanner evicts

cache entries and closes matching WebSockets by reading the committed users.is_active=False row. The local JTI blacklist is consulted before the LRU but has a 10-second grace window, so requests that race deactivation can still complete.

In-flight MCP tool handlers are **not** force-cancelled. The guarantee is narrowly "later MCP requests are rejected once the inventory/cache revocation path has observed the kill switch."

Rate limits

Outgoing exchange REST traffic is measured by the process-local RestCallTracker, exposed through GET /api/metrics/rest-rate. Call paths using its acquire() method also wait for local per-exchange capacity; this is not a deployment-wide quota shared by every container. MCP-specific rate limits are enforced by a separate per-principal middleware on /api/mcp (default 60/minute); the per-delegate max_cancels_per_minute cap then applies on top inside cancel_order. Bursts are served best-effort; sustained abuse over published exchange limits (Walutomat 20 req/s, Kraken 15 req/s, Polygon 5/min) surfaces as REST utilization …% of limit log entries — WARNING at 80% and ERROR at 95% utilisation, rate-limited to one log per exchange per 60 s.

Error catalog

Transport and middleware failures on /api/mcp use standard HTTP status codes plus a Snapper-specific error_code in the JSON body. Clients should branch on error_code, not on status text.

Envelope-based MCP tools return a normal MCP CallToolResult instead: the single TextContent.text entry contains JSON with success, error_code, message, and details, and CallToolResult.isError mirrors not success. A few older tools still return raw dictionaries on success or surface FastMCP ToolError/permission exceptions on failure; client code should handle both shapes until those tools are migrated.

Status	error_code	When it fires	Client action
503	feature_disabled	Flag explicitly set to false	Show "AI Integration disabled" banner
429	rate_limit_exceeded	Per-principal MCP middleware quota exhausted	Back off and retry after Retry-After seconds
503	mcp_unavailable	Repository dep unavailable (lifespan not ready)	Retry with backoff

Status	error_code	When it fires	Client action
401	missing_bearer_token	No Authorization: Bearer … header	Prompt user to authenticate
401	invalid_bearer_token	JWT signature/expiry/blacklist/inventory failure	Long-lived AI-principal tokens have no refresh token; deactivate and recreate the principal, then update the client's bearer token. Interactive sessions can use POST /api/auth/refresh.
401	user_deactivated	Owner account deactivated	Prompt re-login; don't auto-refresh
401	Refresh token redeemed	Replay of a spent refresh JWT	Re-login
401	Authentication required *(detail string, no error_code)*	Session cookie flow — deactivated account, revoked session, or expired token fails DB-backed verify; require_authentication collapses all of these into one generic 401	Re-login
403	MCP write helpers: wallet_out_of_scope: *(prefixed message)* / REST: "Wallet not in accessible set" *(detail string)*	Mutating tool targets a wallet outside the caller's scope. The helper path raises PermissionError(f"wallet_out_of_scope: ...") lifted by FastMCP into a ToolError; REST raises HTTPException(403, detail="Wallet not in accessible set") from server/scoping.py. Tool-level read/cancel paths may instead return structured anti-enumeration envelopes such as order_not_found, position_not_found, or signal_not_found.	Pick a wallet the caller still has a live grant on
403	MCP write helpers: operator_out_of_scope: *(prefixed message)* / REST:	Same write-helper shape as the wallet variant. Callers lacking effective impersonate:operator must pick an operator from their authenticated set; an effective grant containing that permission has global	Pick an operator from the caller's authenticated set unless its effective grant contains impersonate:operator

Status	error_code	When it fires	Client action
	"Operator not in accessible set" *(detail string)*	operator scope. Read/cancel tools can intentionally collapse out-of-scope and not-found cases into structured not-found envelopes to avoid leaking resource existence.	

Delegate CRUD:

Status	Detail	When it fires
401	Requires populated user_public_id	Principal has blank user_public_id (older token rollout)
403	Permission 'read:ai_integration' required	Token lacks delegate list/detail visibility
403	Permission 'manage:ai_integration' required	Token attempts delegate or researcher creation, delegate update, or delegate deactivation without the management permission
404	Delegate not found	Unknown ID OR cross-tenant (no existence leak)
409	Could not derive a unique username …	Label slug collides 8+ times (pathological)
409	Operator … already owns N active AI delegates (limit 5)	Integration owner hit the five-active-delegates cap; deactivate an existing delegate before creating another
422	Operator '<id>' is not in …	Caller without effective impersonate:operator picked operator_public_id outside its claim set
422	Caller has no primary operator …	No explicit operator and no primary → binding is ambiguous

---

Security model

Token lifetime

- Interactive session access tokens live **15 minutes** (configurable via auth_access_token_expire_minutes). AI delegate and researcher access tokens are PAT-style and live for LONG_LIVED_TOKEN_EXPIRE_DAYS (about 3 months / 90 days);

integration owners rotate them on the cadence that fits their key-management hygiene, and revocation is immediate through principal deactivation rather than waiting for expiry.

- Refresh tokens on the login/refresh route path currently live auth_refresh_token_expire_days (default **7 days**). The remember_me request field is accepted by the schema but is not currently threaded into token creation.
- Refresh rotation is **atomic**: one DB transaction revokes the old refresh JTI AND persists the new access+refresh pair. Replay of a spent refresh JWT fails with 401 Refresh token already redeemed — the transaction rolls back cleanly.

Inventory-based revocation

user_active_tokens is the ground truth for "is this JWT still valid?". Every mint persists a row; every verify consults it via a 30-second LRU. Deactivation flips revoked_at=NOW() in one UPDATE and fans out a bus event so every cross-instance LRU evicts the matching entry on receipt.

Inventory-based token purpose

The same row is also the ground truth for "what is this JWT FOR?". Every DB-backed verify entry point takes a required expected_token_type — access at REST, MCP and the WebSocket upgrade; refresh only at refresh rotation — and a credential is accepted only when its row is unrevoked, unexpired, owned by an active user, typed as the caller demanded, and naming the same jti the JWT was signed with (with the refresh_ jti prefix agreeing with the row's type). A mismatch fails closed and emits the structured token_purpose_mismatch security event naming the expected and actual purpose, never the credential.

No purpose CLAIM is required inside the JWT, and none may be added: the long-lived delegate tokens already in the field predate the idea and carry a bare UUID jti with no permission_scope_version. The issuer has written token_type on the hash-bound row since before those tokens were minted, so the row works identically for old and new credentials. snapper token preflight --token-file PATH confirms, read-only and before a deploy, that a given live credential still resolves to an active unrevoked access row with a matching jti; it prints neither the token nor its hash.

No secrets leaked at rest

Delegate password hashes are bcrypt-derived from 32 bytes of cryptographically random data the integration owner never sees. An attacker with DB read rights cannot brute-force the hash.

Single-publisher invariant

UserService.deactivate_user is the SOLE publisher of admin.user_deactivated across the entire process. TokenManager and WebSocketAuthManager are pure subscribers/fallback readers: they evict + close on receipt or on DB scan, but never emit the event themselves. This holds for interactive-user deactivation and delegate deactivation (the route reuses UserService.deactivate_user).

MCP transport = Streamable HTTP

The MCP endpoint uses Model Context Protocol's Streamable HTTP transport. No cookies, no CSRF on MCP calls (bearer header bypass). Every tool call opens a sub-request inside the same bearer-authenticated HTTP connection.

Related reading

- docs/architecture.md — repository + SCD2 + bus layering.
- docs/operations.md — multi-instance trade coordinator and static-hash shard partitioning.
- docs/api.md — full REST surface.

Backend-owned Android push

Android push is optional and disabled by default. Existing APNs registration, payloads and preferences remain unchanged. Each backend supplies its own Firebase project and server credentials; no publisher relay or shared publisher credential is needed. Configured metadata means a validated local sender, not demonstrated Google reachability or device delivery.

Runtime configuration

Apply database migrations before starting the API or notification sidecar. Set SNAPPER_ANDROID_PUSH_CONFIG_FILE to an explicitly provisioned, private JSON file in each process that should serve Android metadata or send Android notifications. An empty variable leaves that process Android-disabled and retains APNs defaults. It does not retire shared configuration or stop another configured sender.

For container deployments, provision read-only mounts for the runtime JSON and its referenced private credential file into the API and notification processes that need them. Mount paths are deployment-specific; setting the environment variable does not create mounts or copy credentials.

Read the current operator CAS state without changing it:

```
snapper android-push-state
```

The result contains revision and active_configuration_id, both initially null. Copy the observed revision into expected_configuration_revision when preparing an activation or disable decision. A first activation can use:

```
{
    "enabled": true,
    "apns_enabled": false,
    "expected_configuration_revision": null,
    "credentials_file": "private/service-account.json",
    "configuration": {
        "configuration_id": "11111111-1111-4111-8111-111111111111",
        "protocol": "fcm_fid",
        "package_name": "ch.snapper.android",
        "project_id": "your-firebase-project",
        "application_id": "your-registered-firebase-android-app-id",
        "api_key": "your-public-client-api-key",
        "messaging_sender_id": "your-project-sender-id"
    }
}
```

Use a newly generated UUID for the real configuration; these are illustrative public values. Relative credential paths resolve from the runtime JSON directory, not the current working directory. The service-account file must authorize Firebase Messaging for the same project. OAuth uses the fixed Google token endpoint and Firebase Messaging scope. No ambient credential discovery occurs. Never commit the private file or include it in an image. Credential validation errors omit raw private input. Metadata exposes only public Firebase options.

Register the actual Android package in the backend owner's Firebase project. Optional Android API-key restrictions must match the installed APK's app-signing certificate, including the Play app-signing certificate for Play distribution. The upload certificate is a separate key. Public client options do not grant permission to send messages.

API and snapper notify startup validate the sender before atomically activating the configuration. Same active UUID and identical public options are restart idempotent. Change the UUID when public options change; credential-only rotation may keep it. Retired UUIDs never reactivate. A competing or stale predecessor fails startup rather than overwriting a newer decision.

For a global disable, read the current revision and provision explicit JSON with enabled:false and that expected_configuration_revision. This atomically retires the active configuration and its Android bindings and records a disabled head. Initial null is never valid again after any decision, including initial disable. Read the new revision before another decision or reapplication of a disable. Removing a process's filename is only a local disable.

Registration and delivery

Authenticated Android clients read /api/devices/android/config, then prepare /api/devices/android/registration-state?configuration_id=...&installation_id=.... Registration uses POST /api/devices/android with CSRF, explicit fcm_fid, the captured installation revision, expected user and a fresh binding ID. No trading permission or wallet is required. Registration FIDs are never returned by inventory. The legacy APNs list excludes Android rows; Android inventory uses GET /api/devices/android. Existing owned delete and preference endpoints support both providers and retain all seven notification types.

Same-installation replacement requires the current revision. A delayed first submission cannot retire a newer binding. Retired bindings never reactivate; another installation claiming an active FID receives a conflict without takeover or owner disclosure. Mutations are not automatically replayed by the client.

FCM HTTP v1 sends message.fid and data-only opaque registration, binding, configuration and alert IDs with a fixed 24-hour expiry. No account or financial content is sent to the provider. Every attempt revalidates current user, registration, configuration, captured

scope, preferences and authority. Retry and terminal transitions compare the charged attempt count. Provider acceptance is recorded separately from APNs IDs and is not proof of device delivery.

Only explicit FCM UNREGISTERED invalidates a binding. Other permanent 4xx errors do not; 429/5xx retry within the original expiry and three-attempt budget. Retry-After is respected. A message 401 clears its OAuth token for a later hint but does not replay that message. Retired configurations fail subsequent admission checks, and workers cannot retarget an old outbox row to a replacement registration. Admission checks cannot recall a provider request already in flight and do not guarantee exactly-once delivery.

Native handling requires explicit consent, notification permission and the current accepted durable binding. It may show only a generic invitation to open Snapper. Private content and navigation wait for current foreground authentication and permission checks. A background service must not read protected session cookies or construct an authenticated repository. One Firebase project per process remains the conservative lifecycle boundary.

Validation boundary

Automated tests use disposable SQLite databases, generated synthetic signing keys and in-process fake HTTP/providers. They do not send to Google or configure a phone. Real FID registration, killed-process delivery, offline cleanup and cross-project process lifecycle must be validated with disposable Firebase projects before claiming working device delivery. Backend availability alone does not prove those behaviors.

Primary references: HTTP v1 authorization, FID message target, FCM error handling, runtime Firebase options.

API

Snapper provides a REST API and WebSocket interface for platform interaction. The API accepts JWT authentication either through HTTP-only cookies or an Authorization: Bearer <jwt> header. When both are present, Bearer auth wins. Cookie-authenticated mutating requests that use the CSRF guard require a valid X-CSRF-Token header; Bearer-authenticated requests skip CSRF because they are not ambient browser credentials. The auth token lifecycle routes (/api/auth/login, /api/auth/refresh, /api/auth/logout, /api/auth/ws_token) intentionally omit the CSRF guard.

Authentication

Browser sessions usually use HTTP-only cookies. After login, the server sets access_token, refresh_token, and csrf_token cookies automatically. API clients and MCP/AI delegates may send the access JWT as a Bearer token instead.

Roles and Permissions

Every capability decision is based on effective permissions. A role is only a stable name for a base permission set, plus a user-management value and display label; it is not an authorization hierarchy.

Named set	Count	Access summary
ai_researcher	3	Research-only access: read market data and market views, then submit market views. It has no signal, order, position, process, or health access.
ai_reviewer	11	Read market data, market views, orders, positions, strategies, signals, system status, and backtests; create backtest comparisons; control runtime diagnostics; and see the MCP AI-review decision tool in its catalog. It has no order-execution or position-management permission, so current invocation of that tool remains unavailable.
ai_delegate	14	Scoped MCP and AI-review automation: the reviewer set plus scoped order creation, cancellation, and position management. It cannot manage users, settings, processes, AI integration, credentials, scope grants, or paired execution, and cannot subscribe to alerts. WebSocket topics.
viewer	14	Full read-only operator visibility: account state; operator-scoped wallets and portfolio; market data and views; orders and positions; signals and AI-review decisions; P&L timeline and attribution;

Named set	Count	Access summary
		backtests; strategies; process state; health; AI integration; and notifications. It cannot trade, manage positions, create backtest comparisons, control runtime diagnostics, configure or control strategies and processes, manage AI integration, or perform administrative mutations. Self-service notification-device management remains available.
operator	26	The complete viewer read set plus trade execution, position management, strategy configuration and lifecycle control, non-strategy process management, AI-integration management, runtime diagnostics, backtest comparisons and run management, and paired-execution terminalization.
admin	34	Every permission in the current Permission catalog, including user and system configuration, wallet credentials, scope grants, and operator impersonation. The mapping is defined as the complete catalog, so newly introduced permissions are included automatically.

Wallet and portfolio reads use the union of active scope grants from the caller's operator memberships and its active personal wallet_user_read_grants. A caller with no operator membership can therefore read a personally granted wallet. Personal read grants never authorize orders, cancellation, or plan actions, which use the operator scope-grant plane. Runtime scope decisions consume effective impersonate:operator; the current admin set is the only set containing it. This structural permission is non-downscopable, so an explicitly narrowed admin token retains the historical system-wide operator and wallet scope.

JWT access is the intersection of the named role set and the token's optional permissions claim, plus any non-downscopable structural grant in that named set. The named set remains an immutable ceiling: a token may remove ordinary permissions but cannot add one outside that set or remove impersonate:operator. Tokens minted without an explicit scope retain the complete named set. For compatibility, an older access token with no permissions claim also receives the complete named set until it expires. Authenticated profile responses expose the result as effective_permissions; clients use that field for capability decisions.

Version-two explicit scopes are migrated by capability equivalence. An exact full named-set scope adopts the current named set. For a narrowed operator or admin scope, retained read:signals adds read:ai_reviews, while retained manage:processes adds the process and AI-integration read and management permissions plus the strategy configure/start/stop permissions that replace its historical controls. For every named set, retained read:backtests adds create:backtest_comparisons, and retained read:system_status adds manage:runtime_diagnostics, only when that named set

contains the replacement permission. The viewer set contains none of those mutation permissions, and an intentionally narrowed version-two viewer scope does not inherit the new read permissions.

POST /api/auth/login

Authenticate and create a session. Sets access_token, refresh_token, and csrf_token cookies on the response.

Request: the body is a LoginRequest envelope (PayloadRequest) wrapping a LoginBody payload — the client stamps its own provenance on the outer envelope.

```
POST /api/auth/login
Content-Type: application/json

{
    "type": "login_request",
    "sequence_id": 1,
    "public_id": "<client-uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<client-session>",
    "topic": null,
    "payload": {
        "username": "admin",
        "password": "password123",
        "remember_me": false,
        "permissions": [
            "read:market_data",
            "read:orders",
            "read:positions"
        ]
    }
}
```

Response (200):

```
{
    "type": "login_response",
    "sequence_id": 1,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<server-session>",
    "topic": null,
    "payload": {
        "type": "login",
        "sequence_id": 1,
        "public_id": "<uuid7>",
        "timestamp": "2026-01-18T12:00:00Z",
        "session_id": "<server-session>",
        "topic": null,
        "message": "Login successful",
        "expires_in": 900,
        "user": {
            "type": "user_profile",
            "sequence_id": 1,
```

```
            "public_id": "<uuid7>",
            "timestamp": "2026-01-18T12:00:00Z",
            "session_id": "<server-session>",
            "topic": null,
            "username": "admin",
            "email": "admin@example.com",
            "role": "admin",
            "is_active": true,
            "created_at": "2026-01-10T08:00:00Z",
            "operator_public_ids": [],
            "primary_operator_public_id": null,
            "active_wallet_public_id": null,
            "default_language": null,
            "effective_permissions": [
                "impersonate:operator",
                "read:market_data",
                "read:orders",
                "read:positions"
            ],
            "delegate_public_id": null
        },
        "access_token": null,
        "refresh_token": null
    }
}
```


The login handler returns the raw UserProfile from authenticate_user() and overlays principal.active_wallet_public_id onto the returned user via model_copy. On the initial login path the freshly-built principal has no wallet selected yet, so active_wallet_public_id lands as null; the operator-membership fields are not enriched on this surface either (operator_public_ids = [], primary_operator_public_id = null). Call GET /api/auth/me after login to get a fully-enriched profile with operator memberships resolved.

The outer envelope (LoginResponse) and the inner payload (LoginData) both carry the standard provenance fields (type, sequence_id, public_id, timestamp, session_id, topic). access_token and refresh_token are null in the cookie flow; they are populated only when the caller passes ?return_tokens=true for headless integrations.

payload.permissions is optional. When supplied, every value must already belong to the authenticated user's named role set; otherwise login returns 422 rather than silently dropping the invalid permission. Omit the field for the complete named-set behavior. A refresh preserves the selected scope, including an intentionally empty list, and cannot re-broaden it.

remember_me is currently accepted for compatibility but is not wired into /api/auth/login: refresh JWTs use auth_refresh_token_expire_days, and browser refresh cookies use a fixed 7-day Max-Age. auth_refresh_token_expire_days_extended is reserved until login passes the flag through to token creation.

Cookies set:

Cookie	HttpOnly	Secure	SameSite	Path	Max-Age	Description
access_token	Yes	Yes (prod)	Lax/Strict	/	session	JWT access token (15 min)
refresh_token	Yes	Yes (prod)	Lax/Strict	/api/auth	7 days	JWT refresh token
csrf_token	No	Yes (prod)	Lax/Strict	/	session	CSRF protection token

POST /api/auth/refresh

Refresh session tokens. The refresh_token cookie is sent automatically by the browser. Returns new tokens in cookies plus a WebSocket authentication token in the response body. Long-running clients that already hold a valid access token and only need a fresh ws_token should call POST /api/auth/ws_token instead — that route does not rotate the refresh-token pair.

Request:

```
POST /api/auth/refresh
```

Response (200):


```
{
    "type": "refresh_response",
    "sequence_id": 2,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<server-session>",
    "topic": null,
    "payload": {
        "type": "refresh",
        "sequence_id": 2,
        "public_id": "<uuid7>",
        "timestamp": "2026-01-18T12:00:00Z",
        "session_id": "<server-session>",
        "topic": null,
        "message": "session refreshed",
        "ws_token": "eyJhbGciOi...",
        "ws_token_exp": "2026-01-18T12:30:00Z",
        "csrf_token": "abc123def456...",
        "user": {
            "type": "user_profile",
            "sequence_id": 2,
            "public_id": "<uuid7>",
            "timestamp": "2026-01-18T12:00:00Z",
            "session_id": "<server-session>",
            "topic": null,
            "username": "admin",
            "email": "admin@example.com",
            "role": "admin",
```

```
            "is_active": true,
            "created_at": "2026-01-10T08:00:00Z",
            "operator_public_ids": [],
            "primary_operator_public_id": null,
            "active_wallet_public_id": null,
            "default_language": null,
            "effective_permissions": [
                "impersonate:operator",
                "read:market_data",
                "read:orders",
                "read:positions"
            ],
            "delegate_public_id": null
        },
        "access_token": null,
        "refresh_token": null
    }
}
```


The refresh handler returns the UserProfile from get_user_by_id() and overlays the principal's active_wallet_public_id onto it — that value is seeded from token_data.active_wallet_public_id (and may be swapped via an optional RefreshTokenPayload wallet hint), so the refresh response generally carries the active wallet (it is only null when the refresh JWT itself carries no wallet claim and no hint is supplied). Operator memberships are still NOT resolved here (operator_public_ids = [], primary_operator_public_id = null); use GET /api/auth/me for the enriched view.

access_token and refresh_token are populated only when ?return_tokens=true is set (headless integrations); the cookie flow leaves them null and writes the rotated JWTs as Set-Cookie headers. The RefreshTokenRequest body (optional) carries active_wallet_public_id / clear_active_wallet to atomically swap the caller's active wallet during the rotation; an Authorization: Bearer <refresh-jwt> header is read first, with the refresh_token cookie used as fallback for browser callers.

POST /api/auth/ws_token

Mint a one-shot WebSocket authentication token without rotating the caller's refresh-token pair. Authenticates via the access bearer (header or cookie) and returns a fresh ws_token bound to the access JWT's session. Per-source-IP rate-limited so a reconnect storm cannot exhaust the token store.

Use this route when a long-running client (e.g. a push-wakeup monitor) needs to mint successive ws_tokens on its own cadence without rotating the refresh-token pair shared with sibling processes.

Request:

```
POST /api/auth/ws_token
Authorization: Bearer <access_token>
```

Response (200):

```
{
    "type": "ws_token_response",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:15:00Z",
    "topic": null,
    "payload": {
        "type": "ws_token",
        "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5c",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:15:00Z",
        "topic": null,
        "message": "ws_token issued",
        "ws_token": "eyJhbGciOi...",
        "ws_token_exp": "2026-01-18T12:30:00Z",
        "expires_in": 900
    }
}
```

expires_in mirrors ws_token_exp as relative seconds for clients that prefer relative-deadline math.

Errors:

- 401 Unauthorized — access bearer is absent, expired, or invalid.
- 429 Too Many Requests — per-source-IP minute budget exhausted.

POST /api/auth/logout

Logout and invalidate the current session. Clears all authentication cookies and blacklists tokens.

Request:

```
POST /api/auth/logout
```

Response (200):

```
{
    "type": "message",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": "Logged out successfully"
}
```

GET /api/auth/me

Get the currently authenticated user's profile.

Request:

```
GET /api/auth/me
```

Response (200):

Handler returns a UserResponse envelope (PayloadResponse[user_response, UserProfile]) wrapping the full UserProfile payload, which carries the multi-tenant trio:


```
{
    "type": "user_response",
    "sequence_id": 3,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<server-session>",
    "topic": null,
    "payload": {
        "type": "user_profile",
        "sequence_id": 3,
        "public_id": "<uuid7>",
        "timestamp": "2026-01-18T12:00:00Z",
        "session_id": "<server-session>",
        "topic": null,
        "username": "admin",
        "email": "admin@example.com",
        "role": "admin",
        "is_active": true,
        "created_at": "2026-01-10T08:00:00Z",
        "operator_public_ids": ["019d6ca4-..."],
        "primary_operator_public_id": "019d6ca4-...",
        "active_wallet_public_id": "019d7e9a-...",
        "default_language": "pl",
        "effective_permissions": [
            "impersonate:operator",
            "read:market_data",
            "read:orders",
            "read:positions"
        ],
        "delegate_public_id": null
    }
}
```


Only /api/auth/me resolves the operator_public_ids and primary_operator_public_id membership fields. A caller with effective impersonate:operator receives every active operator; all other callers, including the current operator and viewer sets, receive only their explicit memberships. active_wallet_public_id reflects the active wallet selection. These fields power the frontend OperatorPicker without a second round trip. effective_permissions is the role-bounded token grant that clients use for capability checks, and delegate_public_id identifies an operational AI-delegate lifecycle identity when one exists.

The login and refresh responses return the UserProfile from authenticate_user() / get_user_by_id() with the active wallet overlaid from the request principal. Operator memberships are not resolved on those two surfaces (operator_public_ids = [], primary_operator_public_id = null). Active-wallet semantics differ between the two: login lands active_wallet_public_id = null because no wallet is selected at first authentication; refresh propagates whatever wallet the refresh JWT carries or the request hint. Clients should call /api/auth/me to get a fully resolved profile.

POST /api/auth/me/update

Update the authenticated caller's self-service preferences. Currently exposes default_language only; additional preference fields may be added later without changing the endpoint contract. Mirrors the admin POST /api/auth/users/{user_id}/update shape (codebase convention: POST + verb, no REST PATCH).

Request:

```
POST /api/auth/me/update
Content-Type: application/json
X-CSRF-Token: <token>
```

```
{
    "type": "update_auth_me_request",
    "payload": {
        "default_language": "pl"
    }
}
```

default_language is validated against the union of supported client codes (iOS-canonical + frontend-canonical forms accepted — e.g. "zh-Hans", "zh", "pt-BR", "pt", "nb", "no"). null clears the preference.

Response (200):

Returns the same UserResponse envelope shape as GET /api/auth/me, with the updated default_language echoed in payload.

Errors:

- 404 — caller's user row was not found (rare; only surfaces if an admin concurrently deactivates the user between auth-dep resolution and the update commit).
- 422 — default_language not in the supported-language allowlist.

GET /api/auth/users

List users. Requires manage:users.

Query parameters:

Parameter	Type	Description
include_inactive	bool	Include deactivated users (default false)
as_of	datetime	Optional point-in-time query timestamp

Returns UserListResponse with payload and count.

Desk membership management

Desk membership management is available to callers with the effective manage:desk_memberships permission. A non-global caller must also have a target-desk claim in the authenticated token and a currently active membership in that desk; the live membership is re-read from the database for each operation. The intersection keeps the token as a ceiling, so a desk attached after login cannot be managed until explicit login. A caller with effective impersonate:operator in both the token and the current database role is the explicit global exception. The current named sets grant desk management to operator and admin, while viewer does not hold it.

The first CRUD slice may attach or detach active human viewer users only. AI-delegate membership remains owned by delegate creation and deactivation, and human operator / admin memberships are visible in the directory but cannot be mutated through these endpoints.

GET /api/auth/desks/{operator_public_id}/members

List active human members of one desk. The response is a UserListResponse, ordered by username. Its UserProfile entries are deliberately desk-scoped:

- operator_public_ids contains only the requested desk;
- primary_operator_public_id contains that desk only when this membership is the user's primary membership, otherwise it is null;
- email is null, and memberships in other desks are never disclosed.

The optional as_of query returns the membership directory at that UTC timestamp for time-travel audit. The caller's permission and target-desk membership are still revalidated against the live database state.

This endpoint does not require manage:users. It therefore gives an operator the directory needed to manage its own desk without exposing the global user directory.

POST /api/auth/desks/{operator_public_id}/members/{username}

Attach the active human viewer identified by the exact username to the desk. The final path parameter uses the path converter, so clients must percent-encode the username and names containing / remain addressable. The operation is idempotent: repeating an

active pair returns 200 without creating another membership. The first membership becomes primary; later memberships are non-primary. Cookie-authenticated requests require CSRF.

Attachment deliberately takes effect on the target's next login. Existing access tokens and WebSocket principals are not widened in place, and there is no admin.membership_granted event. Established deployments must use this explicit endpoint and then have the target log in again; local seed or demo commands do not attach production users. Refresh rotation preserves this boundary: it intersects desk IDs and membership generations with the signed session, and a database role change requires explicit authentication instead of upgrading the existing refresh session.

The response is a MessageResponse confirming that the user is attached.

DELETE /api/auth/desks/{operator_public_id}/members/{username}

Detach an active human viewer from the desk. The operation is idempotent: an already-absent pair still returns 200 and does not emit another revocation event. If the removed membership was primary, the oldest surviving membership by (timestamp, public_id) is promoted deterministically; removing the last membership leaves the user with no primary desk. Cookie-authenticated requests require CSRF.

For a real detachment, every active access and refresh session belonging to the target is revoked immediately and positive token-verification caches are invalidated before the authority reduction is committed. After commit the service emits the backend-internal admin.membership_revoked event. Auth listeners invalidate cross-instance token caches, retire the target's active WebSocket subscriptions, and close all of its sockets with code 4003. Database-backed membership checks provide the broker-outage fallback. The target must log in again to receive credentials containing only its surviving desk memberships.

The endpoints use the following error classes:

- 403 — missing manage:desk_memberships, or a non-global caller is not a current member of the target desk;
- 404 — the active target desk does not exist; mutations also return it when the active target user does not exist;
- 422 — an attach or detach target is not a human viewer, including an AI delegate.

POST /api/auth/users

Create a user. Requires manage:users and CSRF for cookie auth. Body is CreateUserRequest wrapping username, password, optional email, role, and is_active.

POST /api/auth/users/{user_id}/update

Update email, role, and active state for an existing user. Requires manage:users and CSRF for cookie auth. Returns UserResponse.

POST /api/auth/users/{user_id}/deactivate

Deactivate a user through the canonical kill-switch flow. Requires manage:users and CSRF for cookie auth. The path segment is resolved as the target username; self-deactivation returns 400, unknown active users return 404. The service also revokes active sessions and emits admin.user_deactivated for immediate verify-cache eviction and WebSocket close fanout; auth listeners also poll the committed users.is_active=False row as broker-outage fallback.

POST /api/auth/users/{user_id}/change-password

Change a password. Users may change their own password; a caller with manage:users may change another user's password. Requires CSRF for cookie auth and is account-rate limited. Body is ChangePasswordRequest with current and new password.

POST /api/auth/users/{user_id}/admin-reset-password

Admin password reset without the current password. Requires manage:users, CSRF for cookie auth, and is account-rate limited. Body is AdminResetPasswordRequest.

CSRF Protection

The csrf_token cookie is readable by JavaScript (not HttpOnly). For cookie-authenticated mutating requests outside /api/auth/login, /api/auth/refresh, /api/auth/logout, and /api/auth/ws_token, include its value as a header:

```
X-CSRF-Token: <value from csrf_token cookie>
```

New CSRF tokens are issued on login and refresh. There is no separate endpoint for obtaining CSRF tokens.

Bearer-authenticated requests do not require X-CSRF-Token. This is intentional: the request is authorized by the explicit Authorization: Bearer ... header rather than an ambient browser cookie.

REST Endpoints

REST endpoints return the same Data schemas used by WebSocket messages (OrderData, SignalData, ExecutionData, PositionData, CandleData from messaging.schemas.data). This means the wire format is identical whether data arrives via REST or the WebSocket feed.

REST envelopes — PayloadResponse / PayloadListResponse

REST responses are JSON objects with a typed envelope plus a payload. Two shapes ship from src/snapper/api/schemas/base.py:

- PayloadResponse[T, P] — singleton response, fields: type (Literal discriminator), envelope provenance (public_id, session_id, sequence_id, timestamp, topic) and payload: P.
- PayloadListResponse[T, P] — list response, same envelope plus payload: list[P] and count: int (always equal to len(payload)).

Per-item provenance is preserved on items that originate from a bitemporal DB projection (the item carries its own public_id/session_id/sequence_id); minted results share the envelope's provenance. Each item's data-type shape is identical to what the WebSocket feed delivers for the same domain object — only the wrapping differs.

OpenAPI request-body schemas for routes that use the json_body dependency are patched into FastAPI's generated OpenAPI from openapi_extra metadata. Optional request bodies, such as refresh and delegate deactivate, stay optional in the generated schema. The mounted /api/mcp Streamable HTTP sub-app is intentionally outside FastAPI's OpenAPI route list; its tool contract is documented in ai-integration.md.

Provenance on Reads vs. Mutations

- **GET reads** — No client provenance is expected. When telemetry recording is enabled (disabled by default), the server records the request in the telemetry table for observability; otherwise no provenance is recorded for reads. Either way, it does not stamp provenance onto the response items beyond what was stored at write time.
- **Mutations (POST/PATCH/DELETE)** — Most mutation bodies use command-style PayloadRequest envelopes carrying provenance (public_id, session_id, sequence_id, timestamp) and domain intent in payload; most write routes follow the POST + verb convention, with a small number of PATCH surfaces (AI delegate caps, device prefs, alert defaults) using the same envelope. Body-less or query-only mutations — for example DELETE /api/devices/{device_public_id}, POST /api/processes/{name}/stop, POST /api/paired-execution/groups/{group_public_id}/terminalize, and the tracemalloc start/stop routes — carry no client envelope; their control rows record server-side provenance with no extracted payload. The server-side

ClientProvenanceMiddleware extracts envelope fields when a request body is present, emits a structured info log, and runs a per-session GapDetector to warn on sequence gaps. Gaps are logged as warnings but never reject requests.

ClientProvenanceMiddleware

ClientProvenanceMiddleware is an ASGI middleware that processes every mutation request:

1. Intercepts the request body transparently (replays chunks to downstream handlers).
2. Extracts session_id, sequence_id, and public_id from the JSON body when present.
3. Emits a structured log with client_session_id, client_sequence_id, client_public_id, and the request path.
4. Runs a per-session GapDetector to detect sequence gaps from the client.
5. Records the mutation in the control table via a finally block, so the row is persisted whether the request succeeded or failed.

The control recording is non-blocking: any DB failure is logged and swallowed so the response already sent to the client is never invalidated. The control row includes the redacted request payload, HTTP method, path, outcome (ok, error, or exception), and server-side provenance (session_id and sequence_id from the middleware's own SequenceTracker).

All data endpoints support bitemporal querying via the optional as_of parameter (UTC datetime). When provided, the query returns data as it was known at that point in time. When omitted, the current time is used.

GET /api/health

Public health check endpoint. No authentication required.

status reflects the health of enabled long-running CORE processes: "healthy" when all are running, "error" if any are missing. In SERVER_API_ONLY mode the status is "healthy" (no processes are started by design) unless a manually started process has parked after exhausting its restart budget, in which case "error" is reported.

Response (200):


```
{
    "type": "health_check_response",
    "sequence_id": 1,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<server-session>",
    "topic": null,
    "payload": {
        "type": "health_check",
        "sequence_id": 1,
```

```
        "public_id": "<uuid7>",
        "timestamp": "2026-01-18T12:00:00Z",
        "session_id": "<server-session>",
        "topic": null,
        "status": "healthy",
        "version": "0.1.0",
        "connections": {
            "active_connections": 5,
            "zmq_subscribers": 12,
            "subscriber_tasks": 12,
            "active_topics": 8,
            "active_clients": 3
        },
        "topics": {
            "active": 8
        },
        "gap_detection": {
            "bridge": {
                "gaps_detected": 0,
                "session_resets": 0,
                "duplicates": 0,
                "mid_stream_joins": 0,
                "rejected_unstamped": 0
            },
            "rest_clients": {}
        }
    }
}
```


The outer health_check_response envelope carries server-side provenance; the inner health_check payload holds the runtime snapshot. gap_detection provides observability into sequence gap detection across the ZMQ bridge and per-session REST client detectors.

GET /api/health/egress

Egress route snapshot. Requires read:system_status permission and uses the same CSRF guard as the detailed monitoring health routes.

The endpoint returns the API process's local egress snapshot merged with the latest system.egress.snapshot frames from other pool-bearing processes such as snapper-feed. Missing remote snapshots are not an error: the payload still includes the API process. Stale remote snapshots remain visible and are flagged in containers.

Response (200):


```
{
    "type": "egress_health_response",
    "sequence_id": 1,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<server-session>",
    "topic": null,
    "payload": {
```

```
"type": "egress_health",
"sequence_id": 1,
"public_id": "<uuid7>",
"timestamp": "2026-01-18T12:00:00Z",
"session_id": "<server-session>",
"topic": null,
"enabled": true,
"on_all_quarantined": "wait",
"private_fallback_route_id": "pl",
"private_on_fallback": false,
"containers": [
    {
        "container": "api:coord-0@snapper",
        "last_seen_age_seconds": 0.0,
        "stale": false,
        "route_count": 1
    },
    {
        "container": "publisher:kraken@snapper-feed",
        "last_seen_age_seconds": 1.4,
        "stale": false,
        "route_count": 1
    }
],
"routes": [
    {
        "id": "default",
        "kind": "direct",
        "proxy_url": null,
        "region": "host",
        "exit_ip": "198.51.100.11",
        "provider": "isp",
        "priority": 100,
        "allowed_exchanges": [],
        "enabled": true,
        "quarantined": false,
        "quarantine_seconds_remaining": null,
        "in_use_count": 2,
        "active_reservations": [
            {
                "exchange": "kraken",
                "traffic_class": "private",
                "container": "api:coord-0@snapper"
            },
            {
                "exchange": "kraken",
                "traffic_class": "public",
                "container": "publisher:kraken@snapper-feed"
            }
        ],
        "connections": [
            {
                "host": "ws-auth.kraken.com",
                "kind": "ws",
                "exchange": "kraken",
                "traffic_class": "private",
                "container": "api:coord-0@snapper",
                "count": 1,
                "last_seen_at": null
            },
```

```
                    {
                        "host": "api.kraken.com",
                        "kind": "rest",
                        "exchange": "kraken",
                        "traffic_class": "public",
                        "container": "publisher:kraken@snapper-feed",
                        "count": 0,
                        "last_seen_at": "2026-01-18T12:00:01Z"
                    }
                ],
                "transfer": null
            }
        ]
    }
}
```


region, exit_ip, and provider are optional operator metadata copied from the egress_pool.routes[] entry. proxy_url is present so the API can join sidecar transfer samples to SOCKS5 routes by listener port. active_reservations lists the unique (container, exchange, traffic_class) tuples currently reserved on the route. connections lists target hostnames by (container, host, kind, exchange, traffic_class): WebSocket rows carry currently open counts, while REST rows retain the capped last-seen host with count zero after the short reservation releases. Only hostnames are reported; URL paths, queries, headers, bodies, and credentials are never included.

SOCKS5 routes may include transfer when the API has a matching system.egress.transfer sample from snapper-egress. The nested object contains interface, socks5_listen_port, cumulative rx_bytes and tx_bytes, current rx_rate_bytes_per_second and tx_rate_bytes_per_second when a reliable delta exists, latest_handshake_at, counter_reset, sampled_at, sample_age_seconds, and stale. Direct routes, missing samples, and ambiguous port joins return transfer: null.

containers lists each reporting process, the age of its latest snapshot as observed by the API process, whether that snapshot is stale, and how many routes it reported. private_on_fallback is true when any private reservation is active on a non-direct route.

GET /api/candles

Fetch OHLCV candlestick data. Requires read:market_data permission.

Request:

```
GET /api/candles?instrument=BTC-USD&exchange=kraken&timeframe=1h&limit=100
```

Parameters:

Parameter	Type	Required	Description
instrument	string	yes	Instrument symbol (e.g., BTC-USD)

Parameter	Type	Required	Description
exchange	string	yes	Exchange name (kraken, kraken_futures, kraken_equities, walutomat, polygon)
timeframe	string	yes	Candle timeframe (e.g., 1m, 5m, 15m, 30m, 1h, 4h, 1d)
limit	int	no	Number of candles, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)
start	datetime	no	Market-time window start (open_at, UTC); pair with end
end	datetime	no	Market-time window end (open_at, UTC); pair with start

Returns 200 OK with an empty payload array when no candles match the query (unknown instrument, no warm-cache rows, etc.) — the CandleListResponse envelope is the canonical shape for empty results too.

When both start and end are supplied the route reads a market-time range by open_at (ascending, capped at limit), bypassing the cache and the as_of write-time routing. This navigates the full persisted history — including bulk-backfilled corpora whose database write time is unrelated to their market time — and powers the market view's time-travel scrubber. Supplying only one of the pair, or start >= end, returns 400.

Response (200):


```
{
    "type": "candle_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "candle",
            "timestamp": "2026-01-18T12:00:00Z",
            "instrument": "BTC-USD",
            "exchange": "kraken",
            "timeframe": "1h",
            "open_at": "2026-01-18T11:00:00Z",
            "open": 42000.0,
            "high": 42500.0,
            "low": 41800.0,
            "close": 42300.0,
            "volume": 1234.56,
            "vwap": 42150.0,
            "trades": 5678,
            "complete": true
        }
    ],
    "count": 1
}
```

The complete flag marks a provisional intra-minute update of a still-forming bar when false, and the final bar for its window when true (default true).

GET /api/candles/db

Explicit DB-only candle read for tokens with read:market_data. Parameters and response shape match GET /api/candles except the start/end market-time window pair is not accepted; the route bypasses the in-process market cache unconditionally so incident response can verify persisted candles rows directly.

GET /api/candles/cache

Explicit cache-only candle diagnostic read. Parameters match GET /api/candles except as_of and the start/end window pair are not accepted, and timeframe is optional (default 1m). For cache-eligible timeframes (1m, 5m, 15m, 30m) the route returns cache-shaped payload data with diagnostic fields such as is_warm, source, and sample_count; cold cache returns an empty payload with is_warm=false instead of silently falling back to DB. Long frames (1h, 4h, 1d) fall through to persisted rows because the cache does not serve them.

GET /api/orders

Fetch orders with optional filtering. Requires read:orders permission.

Request:

```
GET /api/orders?symbol=BTC-USD&exchange=kraken&limit=100&offset=0
```

Parameters:

Parameter	Type	Required	Description
symbol	string	no	Filter by instrument symbol
exchange	string	no	Filter by exchange (paper, kraken, kraken_futures, walutomat)
limit	int	no	Number of orders, 1-1000 (default 100)
offset	int	no	Number of orders to skip (default 0)
as_of	datetime	no	Point-in-time query, UTC (default: current time)
operator_public_id	string	no	Scope to a single operator (403 if foreign)
wallet_public_id	string	no	Scope to a single wallet (403 if inaccessible)

Response (200):

```
{
    "type": "order_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "order",
            "timestamp": "2026-01-18T12:00:00Z",
            "instrument": "BTC-USD",
            "exchange": "kraken",
            "client_order_id": "signal-a1b2c3d4",
            "exchange_order_id": "KRAKEN-456",
            "created_at": "2026-01-18T12:00:00Z",
            "updated_at": "2026-01-18T12:01:00Z",
            "side": "buy",
            "order_type": "limit",
            "price": 42000.0,
            "size": 0.1,
            "filled_size": 0.1,
            "average_price": 42000.0,
            "status": "filled",
            "time_in_force": "GTC",
            "mode": "live",
            "error": null
        }
    ],
    "count": 1
}
```

POST /api/orders

Create a manual order via a manual_once execution plan. Requires create:orders permission.

Request:

```
POST /api/orders
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "create_order_command",
    "session_id": "ui",
    "sequence_id": 0,
    "public_id": "<uuid7>",
    "timestamp": "2026-04-10T12:00:00Z",
    "payload": {
        "instrument": "BTC-USD",
        "instrument_public_id": "<uuid>",
        "exchange": "kraken",
```

```
        "mode": "live",
        "side": "buy",
        "order_type": "limit",
        "quantity": 0.5,
        "price": 50000.0,
        "time_in_force": "GTC",
        "post_only": false,
        "leverage": null,
        "reduce_only": false,
        "wallet_public_id": "<uuid>",
        "idempotency_key": "<uuid7>",
        "ai_review_public_id": null
    }
}
```

Payload fields:

Field	Type	Required	Description
instrument	string	yes	Native symbol (e.g., BTC-USD)
instrument_public_id	string	yes	Required by the request schema for client-side consistency; the server resolves the persisted instrument UUID from instrument + exchange, so this field is not authoritative
exchange	string	yes	Exchange name
mode	string	no	live (default) or paper
side	string	yes	buy or sell
order_type	string	yes	market, limit, stop, stop_limit
quantity	float	yes	Finite order size greater than zero
price	float	cond	Finite value greater than zero when supplied; required for limit and stop_limit
stop_price	float	cond	Finite value greater than zero when supplied; required for stop and stop_limit
time_in_force	string	no	Time-in-force policy (GTC default)
post_only	boolean	no	Maker-only order flag (false default)
leverage	int	no	Optional integer multiplier from 1 through 2147483647; omitted or null means no leverage override
reduce_only	boolean	no	Reduce-only flag for closing exposure (false default)
wallet_public_id	string	no	Target wallet UUID; when omitted, the server resolves the caller's single accessible wallet for the requested mode

Field	Type	Required	Description
operator_public_id	string	no	Operator identity
idempotency_key	string	no	Dedup key (409 on duplicate)
ai_review_public_id	string	no	Approved ai_reviews row cited for AI-mediated manual orders

Response (200): ExecutionPlanResponse envelope with plan details.

Errors: 403 (disabled/forbidden), 409 (idempotency conflict), 422 (validation).

Numeric strings and booleans are rejected. Quantity and supplied prices must be finite and positive; leverage must be an integer, so values such as 2.5 and 2.0 are rejected for that field. The leverage range reflects its storage type, not a guarantee that a venue or instrument supports the requested multiplier. Optional prices may be omitted or null when the order type does not require them; market orders must omit price or set it to null.

These numeric checks also apply to the MCP submit_manual_order tool and run before order-related database lookups or writes. Authentication and authorization retain their normal checks. REST validation failures return JSON with structured error locations and messages; rejected input values and raw request bodies are omitted from that error detail.

POST /api/orders/{plan_public_id}/cancel

Cancel an active execution plan by its plan public id. Requires cancel:orders permission and caller access to the plan's wallet.

The route transitions the plan to cancel_requested, hydrates the venue-assigned exchange_order_id from the active orders row if available, and inserts a cancel TradeCommand so the outbox dispatcher can publish OrderCancelData on the orders.commands.{ex}.{instr}.cancel topic. If the cancel TradeCommand insert fails the plan is transitioned to failed with last_error, and HTTP 500 is returned — PlanExecutorService will re-emit the cancel on the next startup (idempotent via has_pending_cancel_command).

Request:

```
POST /api/orders/<plan_public_id>/cancel
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "cancel_order_command",
```

```
    "session_id": "ui",
    "sequence_id": 0,
    "public_id": "<uuid7>",
    "timestamp": "2026-04-10T12:01:00Z",
    "payload": {
        "reason": "changed mind"
    }
}
```

Response (200): ExecutionPlanResponse with status cancel_requested.

Errors: 403 (wallet not accessible), 404 (not found), 409 (already terminal or concurrent change), 422 (caps violation), 500 (cancel command insert failed).

POST /api/orders/by-client-order-id/{client_order_id}/cancel

UI convenience route that cancels by the child order's client_order_id (the value displayed on the Orders table). Resolves the owning plan via the trade_commands table and delegates to the shared cancel flow described above.

Requires cancel:orders permission. Same response shape and error codes as the plan-id route, plus 404 when no plan-linked trade_commands row is found for the given client_order_id.

Request:

```
POST /api/orders/by-client-order-id/<client_order_id>/cancel
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "cancel_order_command",
    "session_id": "ui",
    "sequence_id": 0,
    "public_id": "<uuid7>",
    "timestamp": "2026-04-11T12:00:00Z",
    "payload": {
        "reason": "cancelled from Orders table"
    }
}
```

Response (200): ExecutionPlanResponse with status cancel_requested.

Errors: 403 (wallet not accessible), 404 (no linked plan), 409 (already terminal), 422 (caps violation), 500 (cancel command insert failed).

GET /api/instrument-capabilities

Fetch instrument order capability matrix. Requires read:market_data.

Parameters:

Parameter	Type	Required	Description
exchange	string	no	Filter by exchange
instrument_public_id	string	no	Filter by instrument
as_of	datetime	no	Point-in-time query

Response (200): InstrumentCapabilityListResponse with capability flags.

GET /api/venue-fee-schedules

Fetch venue fee schedules. Requires read:market_data.

Parameters:

Parameter	Type	Required	Description
exchange	string	no	Filter by exchange
as_of	datetime	no	Point-in-time query

Response (200): VenueFeeScheduleListResponse with fee tiers.

GET /api/signals

Fetch trading signals with optional filtering. Requires read:signals permission.

Request:

```
GET /api/signals?instrument=BTC-
USD&strategy=rsi_btc_1h&exchange=paper&hours=24&limit=100
```

Parameters:

Parameter	Type	Required	Description
instrument	string	no	Filter by instrument symbol
strategy	string	no	Filter by strategy name
exchange	string	no	Filter by exchange (paper, kraken, kraken_futures, walutomat)
hours	int	no	Hours of history, max 168 (default 24)
limit	int	no	Number of signals, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)
	string	no	Scope to a single operator (403 if foreign)

Parameter	Type	Required	Description
operator_public_id			
wallet_public_id	string	no	Scope to a single wallet (403 if inaccessible)

Response (200):


```
{
    "type": "signal_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "signal",
            "wallet_public_id": "019e1a2b-0000-7000-8000-000000000301",
            "timestamp": "2026-01-18T12:00:00Z",
            "instrument": "BTC-USD",
            "exchange": "paper",
            "side": "buy",
            "strength": 0.85,
            "reason": "RSI 28.5 <= 30",
            "strategy_name": "rsi_btc_1h",
            "price": 42000.0,
            "fired_at": "2026-01-18T12:00:00Z"
        }
    ],
    "count": 1
}
```


GET /api/executions

Fetch order fills/executions. Requires read:orders permission.

Request:

```
GET /api/executions?limit=100
```

Parameters:

Parameter	Type	Required	Description
limit	int	no	Number of executions, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)
operator_public_id	string	no	Scope to a single operator (403 if foreign)

Parameter	Type	Required	Description
wallet_public_id	string	no	Scope to a single wallet (403 if inaccessible)

Response (200):


```
{
    "type": "execution_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:01:00Z",
    "topic": null,
    "payload": [
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "execution",
            "timestamp": "2026-01-18T12:01:00Z",
            "trade_id": "TTRAD-456",
            "exchange_order_id": "KRAKEN-456",
            "client_order_id": "signal-a1b2c3d4",
            "instrument": "BTC-USD",
            "exchange": "kraken",
            "side": "buy",
            "size": 0.1,
            "price": 42000.0,
            "last_size": 0.1,
            "last_price": 42000.0,
            "fee": 0.001,
            "fee_asset": "USD",
            "status": "filled",
            "executed_at": "2026-01-18T12:00:59Z"
        }
    ],
    "count": 1
}
```


GET /api/positions

Fetch current portfolio positions. Requires read:positions permission.

Request:

```
GET /api/positions
```

Parameters:

Parameter	Type	Required	Description
as_of	datetime	no	Point-in-time query, UTC (default: current time)
operator_public_id	string	no	Scope to a single operator (403 if foreign)

Parameter	Type	Required	Description
wallet_public_id	string	no	Scope to a single wallet (403 if inaccessible)

Response (200):


```
{
    "type": "position_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "position",
            "timestamp": "2026-01-18T12:00:00Z",
            "instrument": "BTC-USD",
            "exchange": "kraken",
            "quantity": 0.5,
            "average_price": 41500.0,
            "unrealized_pnl": 250.0,
            "realized_pnl": 100.0,
            "mark_price": 41650.0,
            "marked_at": "2026-01-18T11:59:30Z",
            "source_venue_event_id": 4711,
            "mode": "live",
            "position_cycle_public_id": "019e1a2b-4d5e-7f6a-8b9c-0d1e2f3a4b5c"
        }
    ],
    "count": 1
}
```


The position_cycle_public_id field is null when no open position cycle exists for the position (e.g. flat positions or positions without cycle tracking).

Truthful-valuation semantics: economic rows are written by the trader's position projection and NULLs are honest, never zero-coerced. Current reads overlay the stored event-time mark with the newest durable tick for the exact valuation instrument when it is no more than ten minutes old. Missing, stale, unmapped-paper, non-positive, and non-finite tick evidence returns mark_price, marked_at, and unrealized_pnl as null. Historical as_of reads retain the projection's event-time mark. average_price is null when an aggregate of opposing paper strategy shards has no single truthful entry (or a component entry is unknown). mark_price / marked_at on historical rows echo the active market snapshot captured by the triggering projection event; both are null together with unrealized_pnl when no usable mark exists. source_venue_event_id is the maximum durable venue-event watermark consumed into the row's state (a recovery watermark, not the exact causal fill), null when unknown.

GET /api/portfolio/accounts

Fetch truthful venue account state and portfolio-reconciliation truth per (wallet, exchange, mode). The venue observation remains distinct from the fill-derived position projection, while the nested reconciliation view compares the two truth planes. Requires read:account_state; the current ai_delegate named set does not contain that permission. There is no as_of parameter: an account state has no truthful historical projection, only its current observation.

Live clients subscribe to portfolio.accounts.. Executors publish a thin account_state_changed_event after snapshot and reconciliation commits; the event is an invalidation signal, not an account payload. Clients refetch this REST endpoint to rebuild the read-time fail-closed view and retain a 60-second safety-net poll for missed frames. The bridge enforces the same accessible-wallet scope per frame, with system-wide scope for callers holding effective impersonate:operator and fail-closed malformed-frame handling.

Request:

```
GET /api/portfolio/accounts
```

Parameters:

Parameter	Type	Required	Description
operator_public_id	string	no	Scope to a single operator (403 if foreign)
wallet_public_id	string	no	Scope to a single wallet (403 if inaccessible)

Response (200):


```
{
    "type": "portfolio_account_state_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "portfolio_account_state",
            "timestamp": "2026-01-18T12:00:00Z",
            "wallet_public_id": "019e1a2b-1111-7000-8000-000000000001",
            "exchange": "kraken",
            "mode": "live",
            "sync_status": "observed",
            "effective_status": "observed",
            "is_authoritative": true,
            "balance_status": "observed",
            "position_status": "observed",
            "valuation_status": "native_only",
```

```
            "balances": [
                {
                    "currency": "USD",
                    "total": 10000.0,
                    "free": 9000.0,
                    "used": 1000.0
                }
            ],
            "open_positions": [
                {
                    "symbol": "PF_XBTUSD",
                    "side": "buy",
                    "size": 0.5,
                    "entry_price": 41500.0,
                    "mark_price": 41650.0,
                    "unrealized_pnl": 75.0,
                    "unrealized_funding": -1.25,
                    "timestamp": "2026-01-18T11:59:30Z"
                }
            ],
            "balance_observed_at": "2026-01-18T11:59:30Z",
            "position_observed_at": "2026-01-18T11:59:30Z",
            "authoritative_until": "2026-01-18T12:05:00Z",
            "current_attempt_observation_id": 4711,
            "balance_payload_source_observation_id": 4711,
            "position_payload_source_observation_id": 4711,
            "error": null,
            "reconciliation": {
                "method": null,
                "evaluation_status": null,
                "effective_status": "incomplete",
                "is_authoritative": false,
                "evaluated_at": null,
                "current_observation_id": null,
                "last_full_observation_id": null,
                "detail_source_observation_id": null,
                "last_full_outcome": null,
                "consecutive_full_mismatches": 0,
                "anchor_public_id": null,
                "venue_account_state_public_id": null,
                "venue_account_observation_id": null,
                "source_watermark_kind": null,
                "source_watermark": null,
                "expected": null,
                "actual": null,
                "difference": null,
                "tolerance": null,
                "reconciled_at": null,
                "authoritative_until": null,
                "error": null,
                "open_drift_episode": null
            }
        }
    ],
    "count": 1
}
```

Fail-closed read semantics: sync_status is the raw stored outcome of the last observation attempt (observed / simulated / unsupported / error), while effective_status is DERIVED at read time and is the one consumers must trust. It demotes an observed row to stale once its authoritative_until window has elapsed (or is missing), to clock_error on a future-dated observation clock, and to corrupt when a stored payload fails to revalidate. is_authoritative is true ONLY when effective_status is exactly observed; simulated (paper), unsupported (market-data-only), error, stale, clock_error, and corrupt are never authoritative. balances and open_positions are honest NULLs — never fabricated and cleared only when the state is corrupt. A stale row can retain labeled last-known values. Balance and positions are independent reads, each carrying its own *_observed_at timestamp and *_payload_source_observation_id provenance. Values are venue-native only (valuation_status = native_only); no currency conversion is synthesized.

The strict reconciliation object is always present. With no persisted reconciliation state it has effective_status="incomplete", is not authoritative, and carries no evidence. For a persisted state, the read surface revalidates its method config, referenced observations, latest-observation lineage, and open drift episode before returning it. Any inconsistency yields effective_status="corrupt", is_authoritative=false, and clears all evidence. A valid verdict becomes clock_error when its evaluation stamp is more than five seconds in the future and stale after 900 seconds. Only a fresh, fully revalidated current matched or mismatched verdict is authoritative. Stale views retain their validated expected, actual, difference, and tolerance JSON objects and continue to surface an open drift episode; raw persisted JSON strings are never exposed.

Portfolio P&L

The P&L surfaces require read:positions; the current viewer set contains this permission. They never mutate trading, review, or portfolio-projection resources. A current-horizon request that omits as_of may atomically persist the wallet/mode/currency scope's missing immutable activation anchor only when the caller also has access to that wallet through the trade-scope resolver. A personal read grant alone can read existing P&L but cannot create the anchor; without one, it receives an empty series. Supplying as_of disables anchor creation and performs a strict historical read; when no anchor was visible at that horizon, the series is empty. Both routes use the same readable-wallet scope as positions. An operator_public_id filter narrows the operator-grant half of that scope; the caller's personal read grants remain included.

GET /api/portfolio/pnl/series

Reconstruct one wallet and mode's net-P&L-since-activation series. The response decomposes realized, fee, accrual, unrealized, and net P&L at the requested granularity, with per-exchange and per-instrument attribution. Normal reconstruction uses the

response's bitemporal as_of horizon for source reads; first activation additionally certifies independent request-horizon and activation-minute execution-prefix cuts before persisting the anchor.

GET /api/portfolio/pnl/timeline

Return the same P&L series plus bounded attribution markers for fills, signals, and AI-review decisions. Its marker overlay is read-only and explains when trading and review events affected the portfolio; it does not grant authority to mutate any of those resources. At a historical horizon with no visible activation anchor, the empty series may still carry independently read markers.

Query parameters shared by both routes:

Parameter	Type	Required	Description
wallet_public_id	string	yes	Single wallet scope; 403 when inaccessible
operator_public_id	string	no	Optional operator scope; 403 when inaccessible
mode	string	no	Trading mode, default live
granularity	string	no	1m, 5m, 1h, or 1d; default 1m
from	datetime	yes	Inclusive UTC window start
to	datetime	yes	Inclusive UTC window end
as_of	datetime	no	Explicit bitemporal knowledge horizon; disables missing-anchor creation
valuation_ccy	string	no	Three-letter valuation currency, default USD

Invalid or excessive windows return 400. Both routes return 403 for a wallet outside the caller's permission-derived accessible set.

POST /api/execution-plans

Create a bracket (SL/TP) execution plan on an open position cycle. Requires create:orders permission.

Request:

```
POST /api/execution-plans
Content-Type: application/json
X-CSRF-Token: <csrf_token>
```

Body (BracketCreateCommand envelope):

```
{
    "type": "create_bracket_command",
    "public_id": "019e1a2b-0000-7000-8000-000000000001",
    "session_id": "ui-session-1",
    "sequence_id": 1,
    "timestamp": "2026-04-12T18:00:00Z",
    "payload": {
        "position_cycle_public_id": "019e1a2b-4d5e-7f6a-8b9c-0d1e2f3a4b5c",
        "sl_price": 48000.0,
        "tp_price": 55000.0
    }
}
```

At least one of sl_price or tp_price is required.

Response (200): ExecutionPlanResponse wrapping the new armed bracket plan.

Errors: 422 (invalid params, missing capability, no truthful live position for the cycle, position direction/exchange disagreeing with the cycle, ambiguous cycle link, or a position with an honest-NULL entry price — SL/TP sanity cannot be validated without one), 409 (cycle not open, duplicate), 403 (wallet inaccessible), 503 (executor unavailable).

POST /api/execution-plans/{plan_public_id}/cancel

Cancel a bracket execution plan. Armed brackets transition to cancelled directly. Active brackets transition to cancel_requested with cancel TradeCommands emitted. Requires cancel:orders permission.

Request:

```
POST /api/execution-plans/{plan_public_id}/cancel
Content-Type: application/json
X-CSRF-Token: <csrf_token>
```

Response (200): ExecutionPlanResponse wrapping the updated plan.

Errors: 404 (not found), 409 (already terminal or concurrent cancel).

GET /api/execution-plans/{plan_public_id}

Retrieve a single execution plan by public_id. Requires read:orders permission.

Response (200): ExecutionPlanResponse wrapping the plan.

GET /api/execution-plans/{plan_public_id}/decisions

List decision audit rows for an execution plan. Requires read:orders permission.

Response (200): List of decision rows with action, reason, and timestamp.

POST /api/trailing-stops

Create a trailing stop execution plan on an open position cycle. The trailing stop ratchets the stop price as the market moves favorably and triggers a reduce_only market close on breach.

Request body:

- position_cycle_public_id (string, required): Target position cycle.
- trailing_pct (float, required): Trailing distance as percentage (0 < x < 100).
- min_lock_pct (float, optional, default 0): Minimum profit % before trailing activates.
- idempotency_key (string, optional): Dedup key for retries.

Permission: create:orders. CSRF token required.

Response (200): ExecutionPlanResponse with plan_type: "trailing_stop", status: "armed".

Response (409): Cycle not open or duplicate trailing stop on same cycle.

Response (422): Invalid params, missing capability, no truthful live position for the cycle (the historical peak-quantity fallback is not used), or a position without a usable entry price.

POST /api/trailing-stops/{plan_public_id}/cancel

Cancel a trailing stop. Armed stops transition directly to cancelled. Active stops (with in-flight child orders) transition to cancel_requested.

Permission: cancel:orders. CSRF token required.

GET /api/trailing-stops/{plan_public_id}

Retrieve a trailing stop plan by public_id. Only returns plans with plan_type: "trailing_stop".

Permission: read:orders.

GET /api/trailing-stops/{plan_public_id}/decisions

List decision audit rows for a trailing stop plan.

Permission: read:orders.

GET /api/trailing-stops/by-cycle/{cycle_public_id}

Get live trailing stop state for a position cycle. Returns peak_price and current_stop from the evaluator's in-memory state.

Permission: read:orders.

Response (200): TrailingStopStateResponse with live state, or { "type": "message", "payload": "none" } if no active trailing stop.

Response (404): Position cycle not found.

Paired Execution

The paired-execution operator surface exposes halted or exposed multi-leg groups and the manual attestation used after an operator resolves a manual_intervention group at the venue. Callers lacking effective impersonate:operator are filtered to their accessible wallets; callers with that effective permission are unscoped.

GET /api/paired-execution/incidents

List halted or currently exposed paired-execution scopes visible to the caller. Each incident is keyed by (wallet_public_id, strategy_id, group_key) and may contain an active durable halt, exposed groups, or both. halt_missing: true marks the anomalous window where an exposed group exists before the scanner has restored the durable halt row.

Permission: read:positions.

Response (200): PairedExecutionIncidentListResponse with paired_execution_incident payload items. Each item includes the scope, optional halt, halt_missing, and groups; each group contains per-leg signed exposure with open_qty = filled_signed_qty - compensated_signed_qty.

Response (503): Paired-execution tables are unavailable for the active repository backend.

POST /api/paired-execution/groups/{group_public_id}/terminalize

Attest that a paired-execution group in manual_intervention (or a compensating group reopened onto a manual leg) has been resolved at the venue. The repository completes the group; the guard scanner clears the scope's durable halt and in-memory mirrors within one scan cycle.

Permission: manage:paired_execution. CSRF token required for cookie-authenticated requests.

Response (200): PairedGroupTerminalizeResponse carrying the completed group projection and its legs' true accounting.

Response (404): No current active group exists with that id, or the group belongs to a wallet outside the caller's accessible set.

Response (409): The group is not currently attestable because its status is not manual, it has no legs or a held original command, or a sibling leg still has automation in flight.

Response (503): Paired-execution tables are unavailable for the active repository backend.

GET /api/position-cycles/open

List all open position cycles with age information. Requires manage:users for diagnosing orphaned cycles (open cycles without a matching trading engine).

Query parameters:

- min_age_hours (float, optional, default 0): Only return cycles open longer than this.

Permission: manage:users, which is present only in the current admin set.

Response (200): PositionCycleListResponse envelope whose payload items carry cycle_public_id, shard_key, instrument_public_id, exchange, mode, wallet_public_id, operator_public_id, direction, max_qty (per-cycle peak, not lifetime), opened_at, and age_hours.

POST /api/position-cycles/close-orphan

Close a specific orphaned position cycle. The operator should verify via GET /open that the cycle is genuinely orphaned before calling this.

Query parameters:

- cycle_public_id (string, required): Public ID of the cycle to close.

Permission: manage:users, which is present only in the current admin set. CSRF token required.

Response (200): OrphanSweepResponse envelope with payload { "closed_count": 1, "closed_cycle_ids": ["<id>"] }.

Response (404): No active open cycle found for the given public_id.

POST /api/position-cycles/sweep-orphans

Bulk-close all open position cycles older than min_age_hours. Default threshold is 72 hours (3 days). Review via GET /open?min_age_hours=72 before running.

Query parameters:

- min_age_hours (float, optional, default 72, minimum 1): Minimum age threshold.

Permission: manage:users, which is present only in the current admin set. CSRF token required.

Response (200): OrphanSweepResponse envelope with payload { "closed_count": N, "closed_cycle_ids": [...] }.

Response (400): min_age_hours must be at least 1.

GET /api/exchanges

List distinct exchange names from active symbol aliases. Requires read:market_data permission.

Request:

```
GET /api/exchanges
```

Response (200):


```
{
    "type": "exchange_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": ["kraken", "polygon", "walutomat"],
    "count": 3
}
```


GET /api/exchanges/{exchange}/instruments

List distinct native symbols available on a given exchange. Requires read:market_data permission.

Request:

```
GET /api/exchanges/kraken/instruments
```

Response (200):


```
{
    "type": "instrument_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": ["BTC-USD", "ETH-USD", "SOL-USD"],
    "count": 3
}
```

GET /api/exchanges/{exchange}/instruments/detail

List capability-aware instrument rows for an exchange. Requires read:market_data. Each row includes the native symbol plus trade and market-data capability flags, instrument kind, and expiry metadata so clients can show market-data-only instruments without a second request.

GET /api/instruments/{exchange}/{native_symbol}/related

Return related instruments for a concrete exchange/native-symbol pair. Requires read:market_data. Used by cross-asset and continuous-contract UI flows to navigate from an instrument to its configured underlying, front month, and sibling contracts.

GET /api/status

System-wide status including trader process, backtests, and active strategies. Requires read:system_status permission.

Request:

```
GET /api/status
```

Response payload excerpt (200):

The actual response is a SystemStatusResponse envelope with provenance fields and this object under payload.


```
{
    "trader": {
        "status": "running",
        "pid": null,
        "started_at": null,
        "command": null,
        "exit_code": null,
        "error": null
    },
    "backtests": {},
    "strategies": [
        {
            "strategy_name": "rsi_btc_1h",
            "status": "running",
            "details": {},
            "signals_generated": 42,
            "trades_executed": 5,
            "last_signal": "buy",
            "last_signal_time": "2026-01-18T11:45:00Z",
            "pnl": 150.25,
            "pid": 12345,
            "uptime": "2h 15m"
```

```
        }
    ]
}
```

GET /api/ws/stats

WebSocket and ZMQ bridge statistics. Requires read:system_status permission.

Request:

```
GET /api/ws/stats
```

Response payload excerpt (200):

The actual response is a WsStatsResponse envelope with provenance fields and this object under payload.


```
{
    "websocket": {
        "active_connections": 5,
        "topic_subscribers": {
            "market.kraken.BTC-USD.candles.1h": 3,
            "signals.paper.BTC-USD.rsi_btc_1h": 2
        },
        "client_count": 5
    },
    "zmq_bridge": {
        "active_topics": 12,
        "subscriber_tasks": 12,
        "available_topics": ["market.", "signals.", "system.egress.", "system.heartbeats.", "admin.", "orders.commands.", "orders.events.", "accruals.", "backtest.", "alerts.", "portfolio.accounts.", "plans.decisions.", "ai_reviews.", "ai_research.", "processes.events.summary.", "processes.events.configured.", "processes.events.runs.", "strategies.events.list."]
    },
    "connections": {
        "active_connections": 5,
        "zmq_subscribers": 12,
        "subscriber_tasks": 12,
        "active_topics": 8,
        "active_clients": 5
    },
    "topics": {
        "market.kraken.BTC-USD.candles.1h": {
            "active_subscribers": 3,
            "received": 1200,
            "forwarded": 1180,
            "throttled": 20,
            "dropped": 0,
            "timeout": 0,
            "errors": 0,
            "invalid_messages": 0,
            "last_message_ts": 1737208800.0,
            "throttle_ms": 100,
            "pattern": "market."
        }
```

```
    },
    "subscriptions": {
        "per_topic": {
            "market.kraken.BTC-USD.candles.1h": 3
        },
        "per_client": {
            "140234567890": ["market.kraken.BTC-USD.candles.1h"]
        }
    },
    "config": {
        "broker_xpub": "tcp://127.0.0.1:7501",
        "heartbeat_interval_ms": 1000
    }
}
```


GET /api/zmq/health

ZMQ bridge health check. Requires read:system_status permission.

Request:

```
GET /api/zmq/health
```

Response payload excerpt (200):

The actual response is a ZmqHealthResponse envelope with provenance fields and this object under payload.


```
{
    "status": "healthy",
    "timestamp": "2026-01-18T12:00:00Z",
    "components": {
        "zmq_context": "ok",
        "websocket_manager": "ok",
        "active_connections": 5
    },
    "config": {
        "available_topics": ["market.", "signals.", "system.egress.", "system.he
artbeats.", "admin.", "orders.commands.", "orders.events.", "accruals.", "backte
st.", "alerts.", "portfolio.accounts.", "plans.decisions.", "ai_reviews.", "ai_r
esearch.", "processes.events.summary.", "processes.events.configured.", "process
es.events.runs.", "strategies.events.list."]
    },
    "connections": {
        "active_connections": 5,
        "zmq_subscribers": 12,
        "subscriber_tasks": 12,
        "active_topics": 8,
        "active_clients": 3
    },
    "message_stats": {},
    "errors": []
}
```

GET /api/market/cache/health

Diagnostic snapshot of the in-process market cache, configured pair stats, and persist-policy universe. Requires read:market_data. Returns counts for cached instruments, cached pairs, and persisted instrument universe size.

GET /api/market/cache/stats/configured

Return cached Pearson and cointegration stats for every configured market-stats pair. Requires read:market_data. Configured-but-cold pairs return placeholders with is_warm=false so dashboards can render stable "computing" rows.

GET /api/market/cache/stats/{exchange_a}/{symbol_a}/{exchange_b}/{symbol_b}

Return cached stats for one configured pair. Requires read:market_data. Unknown exchanges return 400; unconfigured pairs return 404; configured-but-not-yet-computed pairs return 200 with is_warm=false.

GET /api/market/coverage

Per-exchange market-data coverage over active instruments. Requires read:system_status.

Query parameters:

Parameter	Type	Description
tick_window_seconds	int	Freshness window for ticks (default 600)
candle_window_seconds	int	Freshness window for candles (default 1800)

The payload includes one row per exchange with instrument count, fresh_ticks, fresh_candles, gated_off, and dark counts.

GET /api/market/feed-health

Current per-symbol feed-health rows. Requires read:system_status.

Query parameters:

Parameter	Type	Description
exchange	string	Optional lowercase exchange filter
fresh_within_seconds	int	Optional staleness filter; older snapshots are dropped

Process Management

Process endpoints manage background services (feeds, strategies, executors, brokers). Read endpoints require read:processes; the lightweight summary uses read:system_status because it is part of the health surface. Non-strategy mutations require manage:processes. Strategy targets use the narrow permissions configure:strategies, start:strategies, and stop:strategies according to the requested operation. The current operator and admin sets contain all three strategy permissions; viewer contains the read permissions and no process or strategy mutation permission.

GET /api/processes/available

List registered process templates that can be instantiated. Requires read:processes permission.

Response (200):


```
{
    "type": "available_processes",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "type": "available_process",
            "sequence_id": 1,
            "public_id": "<uuid7>",
            "timestamp": "2026-01-18T12:00:00Z",
            "session_id": "<server-session>",
            "topic": null,
            "name": "zmq_broker",
            "class_path": "snapper.messaging.infrastructure.broker.ZmqBrokerProc
ess",
            "method": "start",
            "description": "ZeroMQ XPUB/XSUB message broker",
            "lifecycle": "long_running",
            "role": "core",
            "tags": ["infrastructure"],
            "parameters_schema": null
        }
    ],
    "count": 1
}
```


GET /api/processes/configured

List configured process instances with runtime state. Requires read:processes permission.

Response (200):


```
{
    "type": "configured_processes",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "type": "configured_process",
            "sequence_id": 1,
            "public_id": "<uuid7>",
            "timestamp": "2026-01-18T12:00:00Z",
            "session_id": "<server-session>",
            "topic": null,
            "name": "zmq_broker",
            "enabled": true,
            "running": true,
            "mode": "thread",
            "class_path": "snapper.messaging.infrastructure.broker.ZmqBrokerProc
ess",
            "method": "start",
            "parameters": {},
            "note": null,
            "lifecycle": "long_running",
            "role": "core",
            "tags": ["infrastructure"],
            "parameters_schema": null,
            "is_one_shot": false,
            "active_public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "kind": "instance",
            "wallet_public_id": null,
            "parent_template": null,
            "template": "zmq_broker",
            "coordinator": "coord-0",
            "managed_remotely": false
        }
    ],
    "count": 1
}
```


GET /api/processes/summary

Lightweight process category counts for the overview dashboard. Requires read:system_status permission.

Response (200):


```
{
    "type": "process_summary_response",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
```

```
    "topic": null,
    "payload": {
        "type": "process_summary",
        "sequence_id": 1,
        "public_id": "<uuid7>",
        "timestamp": "2026-01-18T12:00:00Z",
        "session_id": "<server-session>",
        "topic": null,
        "coordinator": "coord-0",
        "feeds": { "running": 2, "total": 3 },
        "strategies": { "running": 1, "total": 2 },
        "executors": { "running": 1, "total": 1 },
        "brokers": { "running": 1, "total": 1 },
        "processes": []
    }
}
```

POST /api/processes

Create a new process configuration from a registered template. A non-strategy template requires manage:processes. A strategy template requires configure:strategies; creating it enabled also requires start:strategies. Returns 201 on success.

Request:

```
POST /api/processes
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "process_create_request",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "name": "kraken_feed_btc",
        "template": "kraken_feed_publisher",
        "enabled": true,
        "mode": "thread",
        "parameters": { "symbols": ["BTC-USD"] },
        "note": "Kraken BTC feed"
    }
}
```

Response (201):

```
{
    "type": "process_create_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
```

```
        "type": "process_create",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "status": "created",
        "process": {
            "name": "kraken_feed_btc",
            "template": "kraken_feed_publisher"
        }
    }
}
```


GET /api/processes/schema/{name}

Get the configuration schema and defaults for a registered process template. Requires read:processes permission.

Response (200):


```
{
    "type": "process_schema_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "process_schema",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "name": "kraken_feed_publisher",
        "description": "Kraken market data feed publisher",
        "class_path": "snapper.messaging.publishers.kraken.KrakenMarketDataPublisher",
        "method": "start",
        "default_enabled": true,
        "default_mode": "thread",
        "default_parameters": {},
        "lifecycle": "long_running"
    }
}
```


POST /api/processes/{name}/start

Start a configured process. A strategy target requires start:strategies; a non-strategy target requires manage:processes.

Request:

```
POST /api/processes/zmq_broker/start
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "process_start_request",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "mode": "process"
    }
}
```

All payload fields are optional. mode overrides the stored execution mode for this run only; parameters supplies run-only constructor overrides. Neither field persists to Settings, and omitting either value uses the stored configuration. Strategy processes reject start-time parameters so their operator/wallet/output scope check always runs against persisted launch parameters. operator_public_id and wallet_public_id can never be overridden at start time. Executor templates cannot be started directly; start the generated executor_<exchange>_w<wallet_short> instance instead.

Response (200):

```
{
    "type": "process_start_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "process_start",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "status": "success",
        "name": "zmq_broker",
        "process_public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
        "message": "Process started"
    }
}
```

POST /api/processes/{name}/stop

Stop a running process. A strategy target requires stop:strategies; a non-strategy target requires manage:processes.

Request:

```
POST /api/processes/zmq_broker/stop
X-CSRF-Token: <csrf_token>
```

Response (200):

```
{
    "type": "process_stop_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "process_stop",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "status": "success",
        "name": "zmq_broker",
        "message": "Process stopped"
    }
}
```

PATCH /api/processes/{name}/desired-state

Set the persistent desired state (enable, disable, or restart) of the process_<name> config — the source of truth for the reconcile loop. This NEVER starts or stops anything locally, so it is safe for a process owned by another container: the owning coordinator's reconcile loop converges the running state to what is written here. A non-strategy target requires manage:processes. For a strategy, enable requires start:strategies, disable requires stop:strategies, and restart requires both permissions. Cookie-authenticated requests also require CSRF.

Request:

```
PATCH /api/processes/strategy_rsi_btc_1h/desired-state
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "process_desired_state_request",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "action": "restart",
        "restart_nonce": "01960a7e-2c1a-7c00-8000-000000000000"
    }
}
```

action is enable, disable, or restart. restart_nonce is a client-minted idempotency token (pattern ^[A-Za-z0-9_-]+$, length 8-64), **required** for restart and ignored otherwise; re-sending the same nonce does not double-bounce the process. Enabling a strategy re-runs the operator/wallet/grant scope check (fail-closed) before the write.

Response (200):

```
{
    "type": "process_desired_state_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "process_desired_state",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "status": "success",
        "name": "strategy_rsi_btc_1h",
        "action": "restart",
        "coordinator": "coord-2",
        "managed_remotely": true,
        "message": "Desired state persisted; coord-2 will reconcile"
    }
}
```

Errors: 404 (per-wallet executor instance or unknown process), 422 (bare executor template, or a restart without restart_nonce), 409 (restart of a disabled process), 403/400 (strategy operator/wallet/grant scope denied).

PATCH /api/processes/{name}/config

Retarget an existing **strategy** config's scope (operator / wallet / AI-reviewer reference-identity params) without hand-editing the config JSON. Requires configure:strategies and CSRF for cookie auth. The endpoint does not restart the process: the response carries restart_required: true (surfaced by the UI as a "restart required" banner), and the change applies on the next start or restart. Authorization is fully enforced at edit time against the caller's principal; it does not defer cross-tenant checks to the trusted start resolver.

Request:

```
PATCH /api/processes/strategy_rsi_btc_1h/config
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "process_config_scope_request",
    "public_id": "<uuid7>",
```

```
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "operator_public_id": "019d6ca4-...",
        "wallet_public_id": "019d7e9a-...",
        "reference_identity_params": {
            "ai_review_user_public_id": "label:reviewer"
        }
    }
}
```

All payload fields are optional; an omitted field is left unchanged, and an empty-string operator_public_id clears it. operator_public_id / wallet_public_id accept either a concrete public_id or a label:<name> reference. The scope is authorized at edit time against the caller's principal: a label: reference is resolved scope-qualified to the caller's own operator memberships (and, for a wallet, the resolved operator's accessible wallets of the strategy's paper/live mode), so it can never resolve outside the caller's authority. A label that resolves to exactly one in-scope entity is accepted; a blank, unresolved, or ambiguous label is rejected with 400, and a concrete operator the caller has no membership on is rejected with 403. Values are persisted **verbatim** — a label: reference is stored as-is and resolved to a concrete UUID only at the next (re)start (the authorization check runs on a throwaway resolved copy). reference_identity_params values may likewise be label:<name> references (e.g. label:<username> for the AI reviewer); their keys must be reference-identity params the strategy **declares** (whitelisted against the registry; an undeclared key returns 400).

Response (200):

```
{
    "type": "process_config_scope_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "process_config_scope",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "status": "success",
        "name": "strategy_rsi_btc_1h",
        "parameters": {
            "operator_public_id": "019d6ca4-...",
            "wallet_public_id": "019d7e9a-...",
            "params": { "ai_review_user_public_id": "label:reviewer" }
        },
        "restart_required": true
    }
}
```

Errors: 404 (per-wallet executor instance or not configured), 422 (bare executor template), 400 (not a strategy, undeclared reference-identity key, blank/unresolved/ambiguous operator or wallet label, or wallet resolution failure), 403 (operator not a member / missing wallet grant / output not covered).

GET /api/processes/runs

List historical process runs. Requires read:processes permission.

Parameters:

Parameter	Type	Required	Description
limit	int	no	Number of runs to return (default 50)
name	string	no	Filter by process name

Response (200):


```
{
    "type": "process_runs",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T18:00:00Z",
    "topic": null,
    "payload": [
        {
            "type": "process_run",
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "sequence_id": 1,
            "timestamp": "2026-01-18T18:00:00Z",
            "session_id": "<server-session>",
            "topic": null,
            "process_name": "zmq_broker",
            "status": "succeeded",
            "role": "core",
            "lifecycle": "long_running",
            "parameters": {},
            "result": null,
            "error": null,
            "tags": ["infrastructure"],
            "started_at": "2026-01-18T10:00:00Z",
            "completed_at": "2026-01-18T18:00:00Z"
        }
    ],
    "count": 1
}
```


Strategies

GET /api/strategies

List configured strategy processes with lightweight status. Requires read:strategies permission.

Response (200):


```
{
    "type": "strategy_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "type": "strategy_process",
            "public_id": "<uuid7>",
            "session_id": "<server-session>",
            "sequence_id": 1,
            "timestamp": "2026-01-18T12:00:00Z",
            "topic": null,
            "name": "strategy_rsi_btc_1h",
            "running": true,
            "enabled": true,
            "mode": "process",
            "strategy_class": "RSIReversion"
        }
    ],
    "count": 1
}
```


Strategy mutations use the process routes described above: configure:strategies controls creation and scope edits, start:strategies controls starts and enables, and stop:strategies controls stops and disables. A restart requires both start and stop permissions, and creating an enabled strategy requires both configure and start permissions.

Settings

Settings-management endpoints require configure:system permission (present only in the current admin set). GET /api/settings/features is the public exception.

GET /api/settings

List all settings, optionally filtered by category.

Parameters:

Parameter	Type	Required	Description
category	string	no	Filter by setting category

Parameter	Type	Required	Description
as_of	datetime	no	Point-in-time timestamp for temporal setting reads

Response (200):


```
{
    "type": "setting_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "type": "setting_read",
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5c",
            "session_id": "<server-session>",
            "sequence_id": 1,
            "timestamp": "2026-01-18T12:00:00Z",
            "topic": null,
            "key": "polygon_api_key",
            "value": "IvLG...",
            "category": "api",
            "description": "Polygon.io market-data API key",
            "updated_at": "2026-01-15T10:00:00Z",
            "updated_by": "admin"
        }
    ],
    "count": 1
}
```


Note: per-wallet trading credentials (kraken, walutomat, kraken_futures) are NOT exposed through the settings endpoints. They live in the wallet_credentials table and are managed via the dedicated /api/wallets/{wallet_public_id}/credentials* routes (src/snapper/server/credential_routes.py — GET for the active summaries, POST for create, and POST rotate for replacing an existing credential row). Seed files (dev.toml / prod.toml — see Configuration / Wallet Credentials) remain the bootstrap source for dev/prod parity.

GET /api/settings/features

Public feature-flag projection used by the frontend before auth-gated navigation renders. No authentication required. Currently exposes ai_integration_enabled; disabled AI integration also makes /api/mcp and /api/ai-delegates/* return the shared feature_disabled envelope.

GET /api/settings/categories

List distinct setting category names.

Parameters:

Parameter	Type	Required	Description
as_of	datetime	no	Point-in-time timestamp for temporal category reads

Response (200):

```
{
    "type": "setting_categories",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": ["exchanges", "strategy", "system"],
    "count": 3
}
```

GET /api/settings/push-beta/users

Return the active push-beta gate configuration. Requires configure:system. If the underlying push_beta_config setting is absent or malformed, the endpoint returns the default disabled gate with an empty allowlist so a permission-bearing caller can see the effective routing state.

POST /api/settings/push-beta/users

Replace the push-beta gate configuration in one call. Requires configure:system and CSRF for cookie auth. The request body is UpdatePushBetaUsersCommand; user_public_ids becomes the complete allowlist after the write, so callers should read, edit locally, then submit the full desired set.

Request:

```
POST /api/settings/push-beta/users
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "update_push_beta_users_command",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "enabled": true,
        "user_public_ids": ["019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b"]
    }
}
```

Response (200):

```
{
    "type": "push_beta_config_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "push_beta_config_read",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "enabled": true,
        "user_public_ids": ["019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b"]
    }
}
```

POST /api/settings/{key}/set

Set (create or update) a setting value.

Request:

```
POST /api/settings/polygon_api_key/set
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "setting_update",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "value": "new-api-key-value",
        "category": "api",
        "description": "Polygon.io market-data API key"
    }
}
```

Payload fields:

Field	Type	Required	Description
value	string	yes	Setting value
category	string	no	Setting category (default: system)
description	string	no	Human-readable description

Response (200):

```
{
    "type": "setting_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "setting_read",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "key": "polygon_api_key",
        "value": "new-api-key-value",
        "category": "api",
        "description": "Polygon.io market-data API key",
        "updated_at": "2026-01-18T12:00:00Z",
        "updated_by": "admin"
    }
}
```


POST /api/settings/{key}/remove

Soft-delete a setting by key (sets known_to to current time).

Request:

```
POST /api/settings/polygon_api_key/remove
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "remove_setting_request",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:00:01Z",
    "payload": {}
}
```

Response (200):


```
{
    "type": "message",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:00:01Z",
    "topic": null,
    "payload": "Setting 'polygon_api_key' deleted successfully"
}
```

AI Delegates

AI delegate management is gated by ai_integration_enabled, the same feature flag used by /api/mcp. When disabled, /api/ai-delegates/* returns the shared feature_disabled envelope. Delegate tokens are bearer-only automation credentials for MCP-compatible clients and are returned exactly once, on creation.

POST /api/ai-delegates

Create an AI delegate owned by the authenticated caller and mint a long-lived access JWT.

Permission: manage:ai_integration. CSRF token required for cookie-authenticated requests.

Body (DelegateCreateRequest envelope):

```
{
    "type": "delegate_create_request",
    "public_id": "<client-uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "label": "research-agent",
        "operator_public_id": null,
        "permissions": [
            "read:market_data",
            "read:orders",
            "read:positions",
            "read:strategies",
            "read:signals",
            "read:system_status",
            "read:backtests"
        ],
        "caps": {
            "max_order_quantity_per_instrument": null,
            "max_open_orders": 2,
            "max_daily_notional_usd": 1000.0,
            "max_cancels_per_minute": 5
        }
    }
}
```

When supplied, operator_public_id must be one of the caller's operator memberships unless its effective grant contains impersonate:operator; that structural permission allows any explicit operator. A null value uses the caller's primary operator. All caps are optional; null means the delegate inherits the Snapper-wide fallback for that cap.

permissions is optional and chooses the delegate access token's actual grant. Omission keeps the complete ai_delegate named permission set. Every requested permission must belong to that set; the server rejects a superset with 422.

Response (200): DelegateCreatedResponse with the delegate projection and one-shot access_token. List and detail endpoints never re-serve the token.

Response (409): The label cannot produce a unique delegate username, or the owner has reached the per-owner delegate cap.

Response (422): The requested operator binding is outside the caller's claim set, or the requested token permission scope exceeds the ai_delegate named set.

GET /api/ai-delegates

List active delegates visible to the caller. A token with manage:ai_integration keeps the creator-owned management view; a read-only token is narrowed to delegates bound to the caller's operator memberships. Deactivated delegates are omitted.

Permission: read:ai_integration.

Response (200): DelegateListResponse with DelegateRead payload items.

GET /api/ai-delegates/{delegate_public_id}

Fetch one active delegate in the caller's permission-appropriate scope. A management token uses creator ownership; a read-only token uses operator memberships. An absent or out-of-scope identifier returns 404.

Permission: read:ai_integration.

Response (200): DelegateResponse.

Response (404): The delegate does not exist or is outside the caller's permission-appropriate scope.

PATCH /api/ai-delegates/{delegate_public_id}

Replace a delegate's trading caps while preserving username and label immutability. The write is SCD2-preserved so cap history remains auditable.

Permission: manage:ai_integration. CSRF token required for cookie-authenticated requests.

Body (DelegateCapsUpdateRequest envelope):

```
{
    "type": "delegate_caps_update_request",
    "public_id": "<client-uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:01:00Z",
    "payload": {
        "caps": {
            "max_order_quantity_per_instrument": null,
```

```
            "max_open_orders": 1,
            "max_daily_notional_usd": 500.0,
            "max_cancels_per_minute": 3
        }
    }
}
```

Response (200): DelegateResponse with the updated caps.

Response (404): The delegate does not exist or is not owned by the caller.

POST /api/ai-delegates/{delegate_public_id}/deactivate

Deactivate a delegate using the shared user kill-switch flow. The server closes the active delegate row, revokes active tokens, and publishes the same deactivation event used by admin user deactivation.

Permission: manage:ai_integration. CSRF token required for cookie-authenticated requests.

Body: Optional DelegateDeactivateRequest envelope with payload.reason for audit context.

Response (200): DelegateResponse with is_active: false.

Response (404): The delegate does not exist or is not owned by the caller.

Underlying Assets

GET /api/underlyings

List all underlying assets with instrument counts.

```
GET /api/underlyings?as_of=2026-06-20T12:00:00Z
```

Returns PayloadListResponse with UnderlyingAssetData items (ticker, name, asset_class, sector, instrument_count).

GET /api/underlyings/{ticker}/instruments

List instruments mapped to an underlying asset.

```
GET /api/underlyings/SPX/instruments?relationship_type=derivative
```

Query parameters:

- relationship_type (optional): Filter by exact, derivative, or proxy

- as_of (optional): Point-in-time query timestamp

GET /api/underlyings/{ticker}/front-month

Return the front-month (nearest non-expired) futures contract for an underlying.

```
GET /api/underlyings/SPX/front-month?exchange=kraken_equities&contract_family=ES
```

Query parameters:

- exchange (optional): Filter by exchange
- contract_family (optional): Filter by product root (e.g., ES vs MES)
- as_of (optional): Point-in-time query timestamp

Returns PayloadResponse with FrontMonthData (instrument_public_id, native_symbol, exchange, expiry_at, relationship_type, contract_family). Returns 404 if no active futures contracts exist.

GET /api/underlyings/{ticker}/contracts

List all futures contracts for an underlying asset.

```
GET /api/underlyings/SPX/contracts?include_expired=true&contract_family=ES
```

Query parameters:

- exchange (optional): Filter by exchange
- contract_family (optional): Filter by product root
- include_expired (optional, default false): Include expired contracts
- as_of (optional): Point-in-time query timestamp

Returns PayloadListResponse with ContractData items. Each item includes is_front_month (true for nearest non-expired within same contract family).

GET /api/underlyings/{ticker}/continuous

Build a continuous futures series for an underlying from active contract metadata and candle rows.

```
GET /api/underlyings/SPX/continuous?
exchange=kraken_equities&contract_family=ES&timeframe=1d&start=2026-01-01T00:00:
00Z&end=2026-01-05T00:00:00Z
```

Query parameters:

- exchange (required): Exchange for the contract ladder

- contract_family (required): Product root, e.g. ES vs MES
- timeframe (required): Candle timeframe to load
- start (required): Inclusive UTC start timestamp
- end (required): Exclusive UTC end timestamp
- method (optional, default panama): Adjustment method, unadjusted, ratio, or panama
- rollover_days_before (optional, default 0): Roll contracts this many days before expiry, range 0..365
- as_of (optional): Point-in-time query timestamp

Returns a continuous-series response with the selected contract windows and candle points. A successful response is either a continuous_candle_list (full series) or a continuous_partial envelope; partial responses include failed_roll and message fields describing the unavailable roll window. Returns 400 for invalid parameters and 404 when the underlying cannot be resolved.

Multi-Tenant (Wallets, Operators, Scope Grants, Credentials)

Effective impersonate:operator provides global operator and wallet scope on wallet-data read surfaces. Other principals see wallets covered by active operator scope grants or their own active personal wallet read grants. Both viewer and operator use this read scope, which can include a wallet even without any operator membership. Trading and authorization-topology endpoints continue to use the operator scope-grant plane; credential endpoints enforce their own permissions independently.

GET /api/wallets

List wallets accessible to the current principal. Effective impersonate:operator exposes all active wallets; other callers see the union of wallets covered by their operator set's active scope grants and their own active personal read grants.

GET /api/operators

List operators accessible to the current principal. Effective impersonate:operator exposes all operators; every other caller sees only operators in principal.operator_public_ids. Optional as_of resolves the operator catalogue at the same UTC history horizon used by the desk-membership directory; authorization remains bounded by the caller's current signed scope.

GET /api/scope-grants

List active scope grants on a given wallet. Callers lacking effective impersonate:operator must have an active operator scope grant covering the target wallet; a personal wallet read grant alone does not expose this authorization topology. Otherwise the route returns 403. Required query parameter: wallet_public_id.

POST /api/scope-grants

Create a new scope grant. Requires manage:scope_grants, which is present only in the current admin set. Returns 409 on overlap conflict. Body fields: operator_public_id, wallet_public_id, scope_kind (underlying or instrument), underlying_public_id or instrument_public_id, and optional note.

POST /api/scope-grants/handover

Atomic SCD2 close + insert transfer of a scope grant to a different operator. Requires manage:scope_grants. Body: from_grant_public_id, to_operator_public_id, optional reason. Returns 404 if source grant missing, 400 on self-handover, 409 on cross-scope overlap.

POST /api/scope-grants/{grant_public_id}/revoke

Atomic SCD2 close (no replacement row) of an active scope grant + emits admin.scope_revoked on the bus for live AI-delegate subscription revalidation. Requires manage:scope_grants, which is present only in the current admin set. Body: optional reason (flows to event payload, not to the closed row). Returns 404 if the grant does not exist or is already closed (double-revoke).

POST /api/wallets

Create a new wallet. Requires manage:wallet_credentials, which is present only in the current admin set. Returns 409 if (label, is_paper) active unique index is violated. Body: label, optional description, and is_paper (default false).

GET /api/wallets/{wallet_public_id}/credentials

List active credentials on a wallet as summaries (no encrypted payload on the wire). Requires read:wallet_credentials, which is present only in the current admin set.

POST /api/wallets/{wallet_public_id}/credentials

Create a new wallet credential. Requires manage:wallet_credentials. Plaintext credential_payload is Fernet-encrypted server-side before DB insert. Body: exchange, credential_type (api_key_secret, rsa_pem, oauth, paper), required reconciliation_method, credential_payload (dict), and optional label. A real method config is committed atomically with the credential; unclassified creates no config row. Concrete adapter policy rejects unsupported methods, and paper credentials require unclassified. Returns 409 if (wallet, exchange) already exists or durable reconciliation history conflicts.

PUT /api/wallets/{wallet_public_id}/credentials/{credential_public_id}/reconciliation-method

Idempotently classify an existing active live credential. Requires manage:wallet_credentials and CSRF validation. The body accepts one real reconciliation_method. Kraken Futures permits only futures_position; Kraken Spot permits spot_execution_replay or margin_ledger_replay; Walutomat permits only spot_execution_replay. Paper, market-data-only, unreviewed, and unknown concrete adapters reject every real method. Returns 404 when the active credential does not belong to the path wallet and 409 when reconciliation history makes a different method immutable.

POST /api/wallets/{wallet_public_id}/credentials/{credential_public_id}/rotate

SCD2 close + insert rotation. Old credential row closed, new row inserted with updated encrypted payload. Requires manage:wallet_credentials. Returns 404 if credential not found. Validates credential_payload against the existing credential type before encrypting.

WebSocket

Connection and Authentication

The WebSocket endpoint is at /api/ws. Authentication is message-based, not query-parameter-based, but the upgrade must already carry an access JWT via Authorization: Bearer <access_token> or, for browser clients, the access_token cookie. Bearer auth is checked first; the cookie is the fallback. Missing or invalid upgrade auth is rejected with close code 4401 before the auth_required challenge.

Connection flow:

1. Client connects to ws://host:port/api/ws (or wss:// over TLS) with an access bearer token or active auth session cookie
2. Server validates the origin header and bearer/cookie auth
3. Server sends auth_required message
4. Client sends authenticate message with a WebSocket token
5. Server validates the token and sends auth_ok
6. Server sends auth_complete with topics allowed by the token's effective permissions
7. Client can now subscribe to topics

Obtaining a WebSocket token:

Call POST /api/auth/ws_token with an access token (Bearer header or cookie auth). The route returns a short-lived, one-shot ws_token without rotating the refresh-token pair. POST /api/auth/refresh can also return ws_token during session rotation, but long-running clients should prefer /api/auth/ws_token.

Client-originated control frames (authenticate, reauth, subscribe, unsubscribe, get_subscriptions, ping) inherit StrictDataSchema; clients must stamp public_id, session_id, sequence_id, and timestamp before sending. topic defaults to null and can be omitted on client frames.

Authentication Messages

Server sends after connection:

```
{
    "type": "auth_required",
    "public_id": "019e1a2b-0000-7000-8000-000000000901",
    "session_id": "<server-ws-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "timeout": 10
}
```

Client sends to authenticate:

```
{
    "type": "authenticate",
    "public_id": "019e1a2b-0000-7000-8000-000000000a01",
    "session_id": "<client-ws-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "ws_token": "eyJhbGciOi..."
}
```

Server sends on success:

```
{
    "type": "auth_ok",
    "public_id": "019e1a2b-0000-7000-8000-000000000902",
    "session_id": "<server-ws-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "exp": "2026-01-18T12:30:00Z"
}
```


Server sends session info:


```
{
    "type": "auth_complete",
    "public_id": "019e1a2b-0000-7000-8000-000000000903",
    "session_id": "<server-ws-session>",
    "sequence_id": 3,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "available_topics": [
        "ai_research.",
        "ai_reviews.",
        "alerts.",
        "backtest.",
        "market.",
        "orders.commands.",
        "orders.events.",
        "plans.decisions.",
        "portfolio.accounts.",
        "processes.events.configured.",
        "processes.events.runs.",
        "processes.events.summary.",
        "signals.",
        "strategies.events.list.",
        "system.egress.",
        "system.heartbeats."
    ],
    "user_role": "operator",
    "session_expires_at": "2026-01-18T12:15:00Z",
    "ws_token_exp": "2026-01-18T12:30:00Z"
}
```


available_topics contains the registry roots derived from the token's effective permissions. Subscriptions may use a registry root such as market. or a full concrete topic such as market.kraken.BTC-USD.candles.1h. Intermediate prefixes such as market.kraken. are rejected. Backtests also allow scoped prefixes: backtest., backtest.{wallet_public_id}., and backtest.{wallet_public_id}.{run_public_id}., subject to permission and wallet scope.

Server sends on failure:


```
{
    "type": "auth_failed",
    "public_id": "019e1a2b-0000-7000-8000-000000000904",
    "session_id": "<server-ws-session>",
```

```
    "sequence_id": 4,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "reason": "Invalid token"
}
```


Reauthentication

Before the WebSocket token expires, the server sends a reauth_required message. Clients may also refresh proactively before that deadline. In both cases they obtain a new one-shot ws_token via POST /api/auth/ws_token and send it as a reauth message. Use POST /api/auth/refresh only when the HTTP access/refresh token session itself needs rotation.

Server warning:


```
{
    "type": "reauth_required",
    "public_id": "019e1a2b-0000-7000-8000-000000000905",
    "session_id": "<server-ws-session>",
    "sequence_id": 5,
    "timestamp": "2026-01-18T12:28:00Z",
    "topic": null,
    "deadline": "2026-01-18T12:29:00Z"
}
```


Client sends new token:


```
{
    "type": "reauth",
    "public_id": "019e1a2b-0000-7000-8000-000000000a02",
    "session_id": "<client-ws-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:28:30Z",
    "ws_token": "eyJhbGciOi...(new token)..."
}
```


Server confirms:


```
{
    "type": "reauth_ok",
    "public_id": "019e1a2b-0000-7000-8000-000000000906",
    "session_id": "<server-ws-session>",
    "sequence_id": 6,
    "timestamp": "2026-01-18T12:28:30Z",
    "topic": null,
    "exp": "2026-01-18T13:00:00Z"
}
```


If the client does not reauthenticate in time:


```
{
    "type": "auth_expired",
    "public_id": "019e1a2b-0000-7000-8000-000000000907",
```

```
    "session_id": "<server-ws-session>",
    "sequence_id": 7,
    "timestamp": "2026-01-18T12:30:00Z",
    "topic": null
}
```


Subscription Management

Subscribe to topics:


```
{
    "type": "subscribe",
    "public_id": "019e1a2b-0000-7000-8000-000000000a03",
    "session_id": "<client-ws-session>",
    "sequence_id": 3,
    "timestamp": "2026-01-18T12:00:01Z",
    "topics": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-
USD.rsi_btc_1h"]
}
```


Server confirms subscription:


```
{
    "type": "subscription_success",
    "public_id": "019e1a2b-0000-7000-8000-000000000908",
    "session_id": "<server-ws-session>",
    "sequence_id": 8,
    "timestamp": "2026-01-18T12:00:01Z",
    "topic": null,
    "action": "subscribe",
    "status": "subscribed",
    "topics": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-
USD.rsi_btc_1h"],
    "denied_topics": [],
    "active_subscriptions": ["market.kraken.BTC-USD.candles.1h", "signals.paper.
BTC-USD.rsi_btc_1h"],
    "message": null
}
```


Unsubscribe:


```
{
    "type": "unsubscribe",
    "public_id": "019e1a2b-0000-7000-8000-000000000a04",
    "session_id": "<client-ws-session>",
    "sequence_id": 4,
    "timestamp": "2026-01-18T12:00:02Z",
    "topics": ["market.kraken.BTC-USD.candles.1h"]
}
```


List active subscriptions:

```
{
    "type": "get_subscriptions",
    "public_id": "019e1a2b-0000-7000-8000-000000000a05",
    "session_id": "<client-ws-session>",
    "sequence_id": 5,
    "timestamp": "2026-01-18T12:00:02Z"
}
```


Response:


```
{
    "type": "subscriptions_list",
    "public_id": "019e1a2b-0000-7000-8000-000000000909",
    "session_id": "<server-ws-session>",
    "sequence_id": 9,
    "timestamp": "2026-01-18T12:00:02Z",
    "topic": null,
    "subscriptions": ["signals.paper.BTC-USD.rsi_btc_1h"],
    "available_topics": [
        "ai_research.",
        "ai_reviews.",
        "alerts.",
        "backtest.",
        "market.",
        "orders.commands.",
        "orders.events.",
        "plans.decisions.",
        "portfolio.accounts.",
        "processes.events.configured.",
        "processes.events.runs.",
        "processes.events.summary.",
        "signals.",
        "strategies.events.list.",
        "system.egress.",
        "system.heartbeats."
    ],
    "total_available": 15
}
```


Ping/Pong

Client sends:


```
{
    "type": "ping",
    "public_id": "019e1a2b-0000-7000-8000-000000000a06",
    "session_id": "<client-ws-session>",
    "sequence_id": 6,
    "timestamp": "2026-01-18T12:00:30Z"
}
```


Server responds:

```
{
    "type": "pong",
    "public_id": "019e1a2b-0000-7000-8000-00000000090a",
    "session_id": "<server-ws-session>",
    "sequence_id": 10,
    "timestamp": "2026-01-18T12:00:30Z",
    "topic": null,
    "active_connections": 5
}
```


Server Data Messages

Data messages are flat JSON objects forwarded directly from the ZMQ bus. There is no "data" wrapper. Messages with malformed JSON are dropped by the bridge before forwarding and are counted in the invalid_messages metric for the topic.

All data messages carry session_id and sequence_id provenance fields (see REST section above). All WebSocket control messages (auth, subscribe, ping/pong, errors) inherit from StrictDataSchema, so they also carry the same provenance envelope. Clients can use these fields to detect gaps without server-side replay support.

Candle


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "candle",
    "timestamp": "2026-01-18T11:00:00Z",
    "session_id": "019e0000-0000-7000-0000-000000000001",
    "sequence_id": 42,
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "timeframe": "1h",
    "open_at": "2026-01-18T11:00:00Z",
    "open": 42000.0,
    "high": 42500.0,
    "low": 41800.0,
    "close": 42300.0,
    "volume": 1234.56,
    "vwap": 42150.0,
    "trades": 5678,
    "complete": true
}
```


The complete flag marks a provisional intra-minute update of a still-forming bar when false, and the final bar for its window when true (default true).

Tick


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "tick",
```

```
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "volume": 1234.5,
    "bid": 42000.0,
    "ask": 42001.0,
    "last": 42000.5
}
```


Trade


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "trade",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "executed_at": "2026-01-18T12:00:00Z",
    "price": 42000.5,
    "volume": 0.25,
    "side": "buy"
}
```


Signal


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "signal",
    "wallet_public_id": "019e1a2b-0000-7000-8000-000000000301",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "019e0000-0000-7000-0000-000000000001",
    "sequence_id": 7,
    "instrument": "BTC-USD",
    "exchange": "paper",
    "side": "buy",
    "strength": 0.85,
    "reason": "RSI 28.5 <= 30",
    "price": 42000.0,
    "strategy_name": "rsi_btc_1h",
    "fired_at": "2026-01-18T12:00:00Z"
}
```


Execution


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "execution",
    "timestamp": "2026-01-18T12:01:00Z",
    "trade_id": "TTRAD-456",
    "exchange_order_id": "KRAKEN-456",
    "client_order_id": "signal-a1b2c3d4",
    "instrument": "BTC-USD",
    "exchange": "kraken",
```

```
    "side": "buy",
    "size": 0.1,
    "price": 42000.0,
    "fee": 0.001,
    "fee_asset": "USD",
    "status": "filled",
    "executed_at": "2026-01-18T12:00:59Z"
}
```


Order


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "order",
    "timestamp": "2026-01-18T12:00:00Z",
    "exchange_order_id": "KRAKEN-456",
    "client_order_id": "signal-a1b2c3d4",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "side": "buy",
    "status": "submitted",
    "order_type": "limit",
    "size": 0.1,
    "filled_size": 0.0,
    "price": 42000.0,
    "average_price": null,
    "reason": null,
    "time_in_force": "GTC",
    "error": null,
    "created_at": "2026-01-18T12:00:00Z",
    "updated_at": null
}
```


Heartbeat


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "heartbeat",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "019e1a2b-0000-7000-8000-000000000001",
    "sequence_id": 42,
    "component": "zmq_broker",
    "sequence": 42,
    "status": "healthy",
    "lag_ms": 5,
    "meta": {}
}
```


Error


```
{
    "type": "error",
    "public_id": "019e1a2b-0000-7000-8000-000000000b01",
```

```
    "session_id": "<server-ws-session>",
    "sequence_id": 11,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "message": "Topic 'invalid.topic' does not exist"
}
```


Available Topic Patterns

Topics use dot-separated hierarchical names. Subscribe using a full topic string or one of the registry roots returned in available_topics. Intermediate prefixes are rejected; use market., not market.kraken.. Backtest subscriptions are the exception and support the scoped prefixes documented in the WebSocket auth section above.

Market Data

- market.{exchange}.{instrument}.candles.{timeframe} -- OHLCV candles
- market.{exchange}.{instrument}.ticks -- Price ticks
- market.{exchange}.{instrument}.trades -- Individual trades
- market.paper.{source_exchange}.{instrument}.candles.{timeframe} -- Paper candles sourced from a real exchange
- market.paper.{source_exchange}.{instrument}.ticks -- Paper ticks sourced from a real exchange

Signals

- signals.{exchange}.{instrument}.live -- Live trading signals
- signals.paper.{instrument}.{strategy_name} -- Paper trading signals

Order Events

- orders.events.{exchange}.{instrument}.submitted -- Order submitted
- orders.events.{exchange}.{instrument}.accepted -- Order accepted
- orders.events.{exchange}.{instrument}.rejected -- Order rejected
- orders.events.{exchange}.{instrument}.executed -- Order fill
- orders.events.{exchange}.{instrument}.cancelled -- Order cancelled
- orders.events.{exchange}.{instrument}.expired -- Order expired
- orders.events.{exchange}.{instrument}.replaced -- Order replaced
- orders.events.{exchange}.{instrument}.unknown -- Ambiguous venue submit outcome (non-terminal; resolved to accepted or rejected via venue verification)

Order Commands

- orders.commands.{exchange}.{instrument}.submit -- Submit order

- orders.commands.{exchange}.{instrument}.cancel -- Cancel order
- orders.commands.{exchange}.{instrument}.replace -- Replace order

Portfolio Accounts

- portfolio.accounts.{wallet_public_id} -- Thin committed snapshot or reconciliation invalidation; clients refetch GET /api/portfolio/accounts

System

- system.heartbeats -- Global heartbeat
- system.heartbeats.strategy.{name} -- Strategy heartbeat
- system.heartbeats.executor.{exchange} -- Single-wallet executor heartbeat
- system.heartbeats.executor.{exchange}.{wallet_short} -- Per-wallet executor heartbeat; wallet_short is exactly 12 lowercase hex characters
- system.heartbeats.feed.{exchange} -- Live feed heartbeat
- system.heartbeats.feed.paper.{source} -- Paper replay feed heartbeat
- system.heartbeats.host.disk -- API-host disk-pressure heartbeat emitted by the system metrics snapshotter
- system.egress.snapshot -- Egress pool route snapshots
- system.egress.transfer -- WireGuard transfer samples
- admin.{resource} -- Administrative events requiring manage:users
- processes.events.summary.{coord_slug} -- Process summary snapshots
- processes.events.configured.{coord_slug} -- Configured process-name snapshots
- processes.events.runs.{process_name} -- Per-process lifecycle transitions
- strategies.events.list.{coord_slug} -- Strategy class-path snapshots

Alerts and Reviews

- alerts.{user_public_id}.{alert_type} -- Notification stream
- ai_research.{round_public_id}.request -- Committed research-round wake, available through the ai_research. subscription root gated by submit:market_view
- plans.decisions.{plan_public_id} -- Execution-plan decision events
- ai_reviews.{user_public_id}.{strategy_public_id}.{suffix} -- AI delegate review frames
- accruals.{exchange}.{instrument}.{accrual_type} -- Funding, rollover, and borrow accruals (internal ZMQ bus topic; not currently subscribable over the WebSocket because no permission-derived category includes accruals)
- backtest.{wallet_public_id}.{run_public_id}.{event} -- Backtest lifecycle and progress frames

Error Handling

HTTP Status	Description
401	Missing or invalid authentication
403	Insufficient permissions or invalid CSRF token
404	Resource not found
422	Request validation error
429	Rate limit exceeded (Retry-After header included)
500	Internal server error

Usage Example (Python)

```python
import httpx

BASE_URL = "http://localhost:8000/api"

async def main():
    async with httpx.AsyncClient() as client:
        # Login: LoginRequest envelope around LoginBody payload
        login_resp = await client.post(
            f"{BASE_URL}/auth/login",
            json={
                "type": "login_request",
                "sequence_id": 1,
                "public_id": "<client-uuid7>",
                "timestamp": "2026-01-18T12:00:00Z",
                "session_id": "<client-session>",
                "topic": None,
                "payload": {
                    "username": "admin",
                    "password": "password123",
                    "remember_me": False,
                },
            },
        )

        ws_token_resp = await client.post(f"{BASE_URL}/auth/ws_token")
        # WsTokenResponse envelope: ws_token lives under .payload
        ws_token = ws_token_resp.json()["payload"]["ws_token"]

        candles_resp = await client.get(
            f"{BASE_URL}/candles",
            params={
                "instrument": "BTC-USD",
                "exchange": "kraken",
                "timeframe": "1h",
            },
```

```
        )
        candles = candles_resp.json()
        print(candles)
```

Usage Example (JavaScript)

This example is intentionally minimal. The shipped frontend client reuses cached WS tickets, schedules proactive reauthentication before expiry, and sends heartbeat pings every 5 seconds.

```
async function connect() {
    async function mintWsToken() {
        const wsTokenResp = await fetch("/api/auth/ws_token", {
            method: "POST",
            credentials: "include",
        });
        // WsTokenResponse envelope wraps the data in `.payload`.
        const { payload } = await wsTokenResp.json();
        return payload.ws_token;
    }

    let wsToken = await mintWsToken();

    const protocol = location.protocol === "https:" ? "wss:" : "ws:";
    const ws = new WebSocket(`${protocol}//${location.host}/api/ws`);
    const clientSessionId = crypto.randomUUID();
    let sequenceId = 0;

    const controlFrame = (type, body = {}) => ({
        type,
        public_id: crypto.randomUUID(),
        session_id: clientSessionId,
        sequence_id: ++sequenceId,
        timestamp: new Date().toISOString(),
        ...body,
    });

    ws.onopen = () => {
        console.log("Connected, waiting for auth_required...");
    };

    ws.onmessage = async (event) => {
        const msg = JSON.parse(event.data);

        if (msg.type === "auth_required") {
            ws.send(JSON.stringify(controlFrame("authenticate", {
                ws_token: wsToken,
            })));
        }

        if (msg.type === "auth_complete") {
            console.log("Authenticated. Topics:", msg.available_topics);
            ws.send(JSON.stringify(controlFrame("subscribe", {
                topics: ["market.kraken.BTC-USD.candles.1h"],
            })));
        }
```

```
        if (msg.type === "candle" || msg.type === "tick") {
            console.log("Data:", msg);
        }

        if (msg.type === "reauth_required") {
            wsToken = await mintWsToken();
            ws.send(JSON.stringify(controlFrame("reauth", {
                ws_token: wsToken,
            })));
        }
    };

    setInterval(() => {
        if (ws.readyState === WebSocket.OPEN) {
            ws.send(JSON.stringify(controlFrame("ping")));
        }
    }, 5000);
}
```

Backtests

Backtest endpoints manage strategy backtesting runs. Read endpoints require read:backtests; run creation, cancellation, and reruns require manage:backtests. Comparison creation is a deliberately separate capability, create:backtest_comparisons, held by the current ai_reviewer, ai_delegate, operator, and admin sets. The viewer and ai_researcher sets cannot create comparisons. Every wallet-bound read and mutation fails with 400 when the caller has no active wallet selected. The global GET /api/backtests/strategy-classes registry read is the exception.

POST /api/backtests

Create and launch a new backtest run. Requires an active wallet selection and manage:backtests. The route accepts a BacktestCreateCommand envelope wrapping a BacktestCreateBody payload, creates a pending run row, and starts a one-shot BacktestRunnerProcess.

Request body:

```
{
    "type": "backtest_create_command",
    "sequence_id": 1,
    "public_id": "<client-uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<client-session>",
    "topic": null,
    "payload": {
        "strategy_class": "RSIReversion",
        "instrument_public_id": "<instrument-public-id>",
        "exchange": "kraken",
        "timeframe": "1h",
```

```
        "start_date": "2026-01-01T00:00:00Z",
        "end_date": "2026-06-01T00:00:00Z",
        "initial_cash": 10000.0,
        "strategy_params": {"period": 14, "upper": 70.0, "lower": 30.0},
        "execution_mode": "direct_db",
        "fill_model": "market",
        "slippage_bps": 0.0,
        "commission_bps": 0.0,
        "target_execution_exchange": null
    }
}
```

Response: BacktestRunResponse with the created run details.

GET /api/backtests

List backtest runs with optional filters.

Query parameters:

Parameter	Type	Description
strategy	string	Filter by strategy name
status	string	Filter by status (pending, running, completed, failed, cancel_requested, cancelled)
config_hash	string	Pairing-stable SHA-256 config hash used by comparison auto-pairing
limit	int	Page size (1-100, default 20)
offset	int	Page offset (default 0)
as_of	datetime	Temporal query timestamp

GET /api/backtests/strategy-classes

List registered strategy-class identifiers accepted by BacktestCreateBody.strategy_class. Used by the UI to populate the create-backtest strategy selector.

POST /api/backtests/compare

Create or return an idempotent existing comparison row for two terminal runs. Manual mode supplies run_a_public_id and run_b_public_id; auto mode supplies config_hash and optionally anchor_run_public_id. The route accepts a BacktestCompareRequest envelope wrapping BacktestCompareBody. Requires an active wallet and create:backtest_comparisons. See backtesting.md for pairing semantics.

GET /api/backtests/compare

List recent comparison rows for the caller's active wallet. Requires read:backtests.

Query parameters: limit, offset, and as_of.

GET /api/backtests/compare/{comparison_public_id}

Fetch comparison metadata plus recomputed metrics, equity, trades, and signals diffs from the current artifact rows. Requires read:backtests.

GET /api/backtests/{run_id}

Get backtest run detail by public ID.

POST /api/backtests/{run_id}/cancel

Cancel a pending or running backtest. The route accepts a BacktestCancelCommand envelope wrapping BacktestCancelBody (reason is optional), sets status to cancel_requested via SCD2 close-and-insert, and returns 409 if the run is already in a terminal state. Requires manage:backtests.

POST /api/backtests/{run_id}/rerun

Create a new backtest run with the same configuration as the original. Requires manage:backtests.

GET /api/backtests/{run_id}/trades

Get paginated trades for a backtest run.

GET /api/backtests/{run_id}/signals

Get paginated signals for a backtest run.

GET /api/backtests/{run_id}/events

Get lifecycle events for a backtest run (started, failed, etc.).

GET /api/backtests/{run_id}/equity

Get paginated equity curve points for a run.

Query parameters: limit (1-20000, default 5000), after, and as_of.

AI Reviews

Endpoints for the human-in-the-loop review queue. Strategies emit ai_reviews.* requests via the create_ai_review_and_await() primitive (see strategies.md); the selected, registered AI delegate replies via these routes.

POST /api/ai-reviews/{review_public_id}/decision

REST counterpart to the submit_ai_review_decision MCP tool. Body is AiReviewDecisionCommand, an envelope wrapping an inner AiReviewDecisionRequest:

```
{
    "type": "ai_review_decision_command",
    "sequence_id": 1,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<client-session>",
    "topic": null,
    "payload": {
        "decision": "approve",
        "rationale": "Risk profile within bounds."
    }
}
```

decision is "approve" or "reject" (validated against AiReviewDecisionEnum; unknown values yield 422 with error_code='invalid_decision'). rationale is optional free text, ≤ 4096 chars per the ai_reviews.rationale column constraint. Requires submit:ai_review_decision, resolved through the same capability projection the submit_ai_review_decision MCP tool uses for both visibility and execution, so one delegate token behaves identically on either transport.

A legacy compatibility branch also admits tokens inside the pre-versioning window. It applies only when every one of these holds:

- the token's permission-scope version is absent (minted before scope versioning existed) or is a literal 1;
- the caller's role ceiling grants submit:ai_review_decision (ai_delegate, ai_reviewer);
- that ceiling carries no manage:users grant, which excludes admin;
- the token's effective grant retained create:orders.

Tokens on scope version 2 or later are decided purely by their effective grant, with no compatibility path. So a create-only operator token is rejected at the route gate regardless of version, and a create-only admin token is rejected even with an absent version, while the pre-versioning ai_delegate production token is admitted. The capability is not sufficient by itself: the caller must still resolve to a registered ai_delegates row and

have a live scope grant for the review wallet and instrument. The dispatch fans out on bus.ai_review_decision and emits ai_reviews.{user}.{strategy}.decision_ack on the WS surface.

While a review is pending before its persisted fanout_after, only its selected_delegate_public_id may resolve it. A different granted delegate receives 409 with error_code="not_selected_before_fanout" and details.fanout_opens_at; the exact opening instant is accepted. At or after that instant, any delegate that still passes the same grant check may answer.

Responses use AiReviewDecisionResponse — the canonical envelope shared with the MCP CallToolResult payload:

```
{
    "success": true,
    "error_code": null,
    "message": "Decision recorded.",
    "details": { "...": "..." }
}
```

Error status codes: 404 (review not found), 403 (caller not authorized for this review), 409 (not selected before fanout or already resolved by peer), 410 (deadline elapsed), 422 (invalid decision).

GET /api/ai-reviews/pending

List pending CONSULT reviews where the caller is the selected AI delegate. Used by the bridge for catch-up after WS reconnect and by delegate-aware dashboards. Snapshot is bounded by limit and ordered by fanout_after ASC (oldest first). The route first requires Permission.READ_SIGNALS (callers without it receive 403) and then requires the operational delegate lifecycle identity exposed as delegate_public_id. Callers that pass the permission gate without an active delegate lifecycle identity receive 422; this is a state check, not a role capability check.

Query parameters:

Parameter	Type	Description
limit	int	Page size, 1..500 (default 100)
wallet_public_id	string	Optional wallet filter

Response: PendingReviewListResponse (items + count) where each PendingReviewSummaryItem carries the resolved instrument ticker plus the raw signal_envelope payload (thesis, side, news anchors) so the AI delegate inbox can render a meaningful row without a follow-up read.

GET /api/ai-reviews/{review_public_id}/aftermath

Return a terminal review and the exact wallet+instrument activity that became visible between the review's created_at and one captured as_of instant, inclusive. The route is read-only: it does not transition the review, write an audit event, or grant authority to place an order.

Requires Permission.READ_SIGNALS, a populated operational delegate_public_id, and an active delegate scope grant for the review's wallet and instrument. Unknown and out-of-scope identifiers both return 404 with error_code="review_not_found"; a non-terminal review returns 409 with error_code="review_not_terminal"; a permission-bearing caller without a delegate identity returns 422 with error_code="not_a_delegate".

Response: AiReviewAftermathResponse with this top-level shape:

```
{
    "review": { "public_id": "<uuid7>", "status": "timeout", "...": "..." },
    "window_started_at": "2026-07-20T10:00:00Z",
    "as_of": "2026-07-20T11:00:00Z",
    "orders": [],
    "executions": [],
    "position_cycle_transitions": [],
    "current_positions": []
}
```

review is the complete persisted ai_reviews row. Orders and executions are the versions active at as_of, ordered oldest first and bounded by the review window. Every execution carries order_public_id, instrument_public_id, exchange, mode, and scope_sequence so fills whose orders predate the window remain attributable. Its client_order_id and exchange_order_id are nullable when the parent order was not yet visible at as_of. Each position-cycle transition is an explicit opened, closed, or liquidated event derived from lifecycle timestamps retained by the cycle version active at as_of. current_positions is a list because the review row does not carry a mode and more than one mode-specific position may exist; an instrument with no position returns an empty list.

GET /api/ai-reviews

Permission-gated audit list of AI reviews, newest first: the read-only counterpart to /pending that answers "what did the AI decide, and why?". Unlike /pending, it is not keyed by the delegate identity, so it never returns 422 for a missing delegate identity, and it includes terminal decided rows rather than only pending ones. Requires read:ai_reviews; the current viewer, operator, and admin sets contain that permission. A caller with effective impersonate:operator sees every operator's reviews; every other reader is narrowed server-side to its explicit operator memberships.

Query parameters:

Parameter	Type	Description
limit	int	Page size, 1..500 (default 100)
status	string	Optional exact-match status filter
wallet_public_id	string	Optional wallet filter
strategy_public_id	string	Optional strategy filter

Response: AdminAiReviewListResponse (items + count). Each AdminAiReviewItem carries review_public_id, strategy_public_id, user_public_id, operator_public_id, wallet_public_id, instrument_public_id, selected_delegate_public_id, responding_delegate_public_id, status, decision, rationale, resolution_mode, dispatch_version, created_at, resolved_at, deadline, and the raw signal_envelope.

Devices

APNs registration for iOS and optional FCM FID registration for Android, plus shared per-device alert preferences for order, position and system events. All rows are bound to user_public_id and follow the SCD2 close-and-insert convention.

POST /api/devices

Register or refresh the caller's APNs device token. Body is RegisterDeviceCommand. Same-token re-registrations collapse onto the existing SCD2 row via upsert_notification_device and return the stable public_id reused across versions. Returns NotificationDeviceResponse.

GET /api/devices

List the caller's active APNs devices, newest registered first. Android registrations use the separate inventory below. Returns NotificationDeviceListResponse (payload + count — PayloadListResponse base); rows whose active version has token_status='user_unregistered' (tombstones) are excluded.

Android configuration and registration

These routes require authentication but no trading permission or active wallet. Mutations also require CSRF. Existing APNs request/response semantics remain unchanged.

Method	Route	Contract
GET		

Method	Route	Contract
	/api/devices/android/config	android_push_config_response; configured public Firebase options or availability: "unavailable" with null configuration
GET	/api/devices/android/registration-state	Requires configuration_id and installation_id; returns opaque revision and current_binding_id, disclosed only to its owner
POST	/api/devices/android	register_android_device_command with explicit fcm_fid, configuration/package, installation/binding IDs, FID, expected_user_public_id, expected_revision and optional app version
GET	/api/devices/android	android_device_list_response with owned active registrations and count; never returns FIDs

Successful registration returns android_device_response; its payload public ID is the server registration ID. Same equivalent active-binding retries retain that ID; retired bindings never reactivate. Installation revision CAS prevents a delayed submission from replacing a newer binding. Another installation claiming an active FID receives a conflict without takeover or counterparty disclosure. Known refusals use HTTP status plus FastAPI detail; transport loss remains uncertain and does not authorize automatic POST replay. See Android push for lifecycle, configuration and delivery rules.

The following owned delete and preference routes support both providers and preserve all seven alert types. No preferences transfer to a replacement registration.

DELETE /api/devices/{device_public_id}

Soft-delete a device via SCD2 close + tombstone successor: the active row is closed and a successor with token_status='user_unregistered' is inserted so an as_of query after the close sees an inactive row rather than a gap. Returns MessageResponse. Marking a nonexistent or not-owned device inactive returns 404.

PATCH /api/devices/{device_public_id}/prefs

Upsert a per-(device, alert_type, scope) preference for the caller. Body is UpdateDevicePrefCommand. The composite scope key is (device_public_id, alert_type, operator_public_id, wallet_public_id) — three null-permutations map to the three partial unique indexes on device_alert_prefs. Returns DeviceAlertPrefResponse.

POST /api/devices/{device_public_id}/prefs/{pref_public_id}/revoke

Close one device-scoped alert preference (SCD2 in place). Backend convention is POST + envelope (not DELETE) so every write carries client-side provenance for the gap detector — same as cancel_order / revoke_scope_grant. The pref row is closed by stamping known_to at the revoke timestamp; no successor is inserted, which frees the slot in the partial unique index. Body is RevokeDevicePrefCommand; returns RevokeDevicePrefResponse.

GET /api/devices/{device_public_id}/prefs

List active per-(alert_type, scope) prefs for a caller-owned device. Returns DeviceAlertPrefListResponse. Tombstoned device prefs are filtered out server-side via the join to notification_devices on token_status='active'.

Alerts

Real-time alert feed for authenticated web and native clients. Alert events are minted by various subsystems and surfaced via WS on alerts.* (see messaging.md for the canonical AlertType enumeration) plus this REST catch-up tail.

GET /api/alerts/history

Keyset-paginated page of the caller's active alert_events. Order: (timestamp DESC, public_id DESC).

Query parameters:

Parameter	Type	Description
limit	int	Page size, 1..200 (default 50)
before	string	Opaque cursor token from a previous page's next_cursor; None or malformed values map to page 1 (max 160 chars)

The before cursor is decoded server-side back into the internal AlertListCursor and applied as a keyset predicate against Repository.list_recent_alerts_for_user. Returns AlertHistoryResponse.

GET /api/alerts/{alert_public_id}

Fetch a single active alert_events row if owned by the caller. Returns AlertEventResponse. Ownership is enforced server-side via user_public_id equality; 404 is returned for unknown IDs **and** for IDs owned by other users, so the endpoint never leaks the existence of foreign alert events.

Alert Defaults

Per-user fallback preferences that apply when no per-device device_alert_prefs row matches.

GET /api/alert_defaults

List the caller's active user-level fallback prefs. Returns UserAlertDefaultListResponse (payload + count — PayloadListResponse base). An empty list is the legitimate "no overrides" state — clients should fall through to the in-app default UI; the route does **not** auto-create rows on first read.

PATCH /api/alert_defaults

Upsert the caller's user-level fallback pref for a given alert_type via SCD2 close-and-insert. Body is UpdateUserAlertDefaultCommand.

Metrics

Observability endpoints surfaced for the operations dashboard and the iOS Home/system-status surface. All routes require Permission.READ_SYSTEM_STATUS. See observability.md for the underlying retention / system-metrics pipeline and snapshot schemas.

GET /api/metrics/notifications

Current notify-sidecar outbox counters (per-status row counts — *not* a rolling window), aggregated across providers without a provider breakdown. sent records provider acceptance, not device delivery. Returns NotificationMetricsResponse.

GET /api/metrics/system

Most recent SystemMetricsSnapshot from the in-memory ring buffer (CPU, memory, GC, file descriptors, plus the process connection count under process.num_connections). Returns SystemMetricsResponse.

GET /api/metrics/system/history

Bounded slice of the in-memory system-metrics history (no cursor / page tokens — the caller filters with since / until + limit). Returns SystemMetricsHistoryResponse.

Query parameters:

Parameter	Type	Description
since	datetime	Lower bound (inclusive)
until	datetime	Upper bound (inclusive)
limit	int	Page size, 1..100000 (default 720 — roughly the last hour at the default 5-second cadence)

Ring-buffer depth is capped by SYSTEM_METRICS_HISTORY_CAP (default 17280, ~24 h at 5 s cadence — see configuration.md).

POST /api/metrics/system/tracemalloc/start

Arm Python tracemalloc with an auto-stop deadline. Body-less; duration_s is supplied as a query parameter. Returns TracemallocStateResponse. Requires manage:runtime_diagnostics.

POST /api/metrics/system/tracemalloc/stop

Disarm tracemalloc and cancel any pending auto-stop deadline. Returns TracemallocStateResponse. Requires manage:runtime_diagnostics.

GET /api/metrics/retention

Most recent retention-scheduler run summary. Returns RetentionRunResponse whose payload is a RetentionRunData with top-level fields run_started_at, run_completed_at, dry_run, and results: list[RetentionPolicyResult]. Each per-policy result inside results carries archived_rows, purged_rows, files_written, and the window boundaries (day_start, day_end).

GET /api/metrics/db/tables

Most recent per-table row-count snapshot. Refreshed on the cadence configured by DB_METRICS_INTERVAL_SECONDS. Returns DbStatsResponse.

GET /api/metrics/rest-rate

Per-exchange REST call rates + venue rate-limit utilization (rps_1s / rps_10s / rps_60s rolling windows, plus limit_rps + utilization when the venue's documented cap is known). Read from the in-process RestCallTracker snapshot. Returns RestRateResponse.

AI Delegates

AI delegate access is feature-gated by ai_integration_enabled; when disabled, these routes return a 503 feature_disabled envelope matching /api/mcp.

Route	Description
POST /api/ai-delegates	Create a delegate and return its 90-day access JWT once
GET /api/ai-delegates	List active delegates in the caller's creator-owned management scope or read-only operator-membership scope
GET /api/ai-delegates/{delegate_public_id}	Fetch one delegate in the same permission-appropriate scope; 404 also covers out-of-scope IDs
PATCH /api/ai-delegates/{delegate_public_id}	SCD2 close+insert new caps for an owned delegate
POST /api/ai-delegates/{delegate_public_id}/deactivate	Deactivate via the shared kill-switch flow and revoke active tokens

The routes are documented end-to-end in ai-integration.md alongside the Claude Code / Claude Desktop / Cursor / Windsurf wire-up instructions. They follow the same PayloadResponse envelope convention used by the auth routes above. List and detail reads require read:ai_integration; create, update, deactivate, and researcher provisioning require manage:ai_integration. Mutating routes also require CSRF for cookie-authenticated requests. On create, omitted operator_public_id binds to the caller's primary_operator_public_id and returns 422 when no primary exists. Callers lacking effective impersonate:operator must choose one of their authenticated operators when supplying operator_public_id; callers with that effective permission may bind explicitly to any operator.

AI Researchers

Researcher provisioning shares the ai_integration_enabled feature gate but uses a separate principal named permission set and cap from AI delegates.

Route	Description
POST /api/ai-researchers	Create a research-only principal and return its 90-day access JWT once

The route requires manage:ai_integration, plus CSRF for cookie authentication. The request is a ResearcherCreateRequest envelope whose payload contains label and an optional narrower permissions list. The ai_researcher named-set ceiling is exactly read:market_data, read:market_views, and submit:market_view; a requested permission outside that set returns 422.

At most two active researchers may be provisioned per owner. Researcher creation writes only the user identity and access-token inventory: it creates no trading caps, operator membership, or ai_delegates runtime row, and therefore cannot consume a delegate slot or enter consult admission accounting.

Paired Execution (operator surface)

Operator endpoints for the paired-execution guard (dark behind PAIRED_EXECUTION_GUARD_ENABLED; usable regardless for reading state). Both narrow to the SQL repository (503 otherwise). The operator workflow — triage, manual venue resolution, attestation — is documented in paired-execution.md.

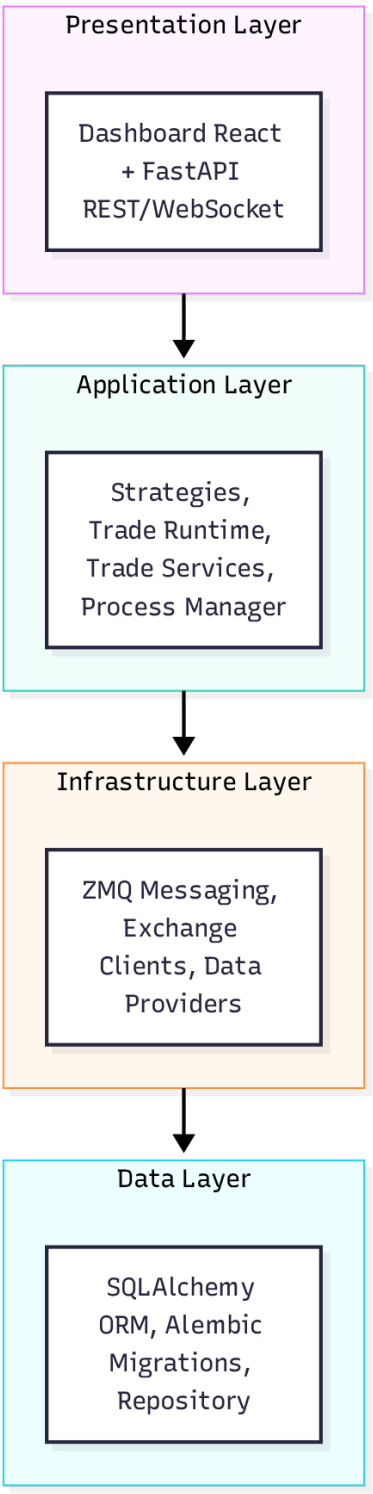
Route	Description
GET /api/paired-execution/incidents	Per-scope incidents: active halts ∪ exposed groups with per-leg signed exposure (READ_POSITIONS, wallet-scoped)
POST /api/paired-execution/groups/{group_public_id}/terminalize	Attest a manual_intervention group resolved at the venue (MANAGE_PAIRED_EXECUTION, CSRF; 404 also covers out-of-scope wallets, 409 when not attestable)

Architecture

Snapper is a trading platform built with a layered architecture using asynchronous processing and ZeroMQ messaging.

The trading path is facts-canonical: Order and Execution are the canonical business facts, while Position, cash, and equity are derived projections that can be rebuilt from facts plus checkpoints.

System Layers



Components

Core (src/snapper/core/)

Fundamental types and aliases used throughout the application:

- TradeSide — Trade direction (buy, sell)
- OrderType — Order type (market, limit, stop, stop_limit)
- OrderStatus — Order status in lifecycle
- ExecutionMode — Execution mode (live, paper)
- OrderExchange — Order-capable exchanges (paper, kraken, kraken_futures, walutomat)

Config (src/snapper/config/)

Two-layer configuration:

1. **Bootstrap Settings** (bootstrap.py)

   Settings from environment variables required before database connection:

   - Database URL
   - Deployment environment (SNAPPER_ENV)
   - Master password for encryption
   - HTTP and ZMQ server endpoints

2. **App Settings** (app.py)

   Settings from database (encrypted):

   - Exchange API keys
   - Trading parameters
   - Authentication configuration

   Production-like SNAPPER_ENV values (production, prod, staging) fail fast on a placeholder secret: bootstrap refuses the default MASTER_PASSWORD at load time. It is the only provisioned secret — the JWT and CSRF signing keys are derived from it via the single derivation site (infrastructure/security/kdf.py), so a placeholder master can never silently sign production tokens.

Data (src/snapper/data/)

Persistence layer with SQLAlchemy:

- **ORM Models** (models.py):
  - Instrument — Financial instruments (natural key: symbol_public_id + exchange). Instrument.public_id is the stable identity used by fact tables (orders, executions, positions, signals, candles). The versioned business attributes symbol_public_id and exchange are resolved via ensure_instrument()

- Candle — OHLCV candles
- Trade — Transactions
- Order — Canonical order lifecycle facts (includes mode: "live" or "paper")
- Execution — Canonical confirmed fills
- Position — Derived position projection (includes mode: "live" or "paper")
- TradeCommand — Durable trade intent written by the engine before execution
- VenueEvent — Durable venue observations and acknowledgements persisted by executors
- TradeProjectionCheckpoint — Materialized position/cash snapshot for fast recovery
- PairedExecutionGroup, PairedExecutionLeg, PairedExecutionHalt — durable arming, compensation, and halt state for multi-leg paired execution
- Signal — Signal events
- User — System users
- Setting — Settings (encrypted)
- Symbol — Symbol identity with versioned attributes (native_symbol, base, quote, asset_type) via SCD Type 2. Archive symbols for filesystem paths are derived from the anchor row (first version per public_id)
- SymbolAlias — Exchange-specific symbol aliases (one row per (symbol_public_id, exchange, channel))
- SymbolExchangeCapability — Exchange-specific symbol capabilities
- ProcessRun — Background process execution records
- InstrumentSpec — Instrument trading specifications (tick_size, margins, expiry_at, instrument_kind)
- UnderlyingAsset — Canonical asset identity (e.g., S&P 500, Gold) linking instruments across exchanges
- InstrumentUnderlyingMapping — Temporal link from instrument to underlying (exact/derivative/proxy)
- MarketSnapshot — Real-time market data snapshots
- InstrumentFeedHealth — Current per-symbol feed-health snapshot keyed by coordinator, exchange, native symbol, stream kind, and timeframe
- UserLoginEvent — Authentication event log
- Control — Always-on audit for commands, auth events, subscribe/unsubscribe, REST mutations. Includes redacted payload, outcome, and client causation linkage (client_session_id, client_public_id)
- Telemetry — Toggleable high-volume table for pings, heartbeats, pongs, and GET read requests. Recording gated by TELEMETRY_RECORDING_ENABLED setting

Temporal ORM models carry provenance via TemporalMixin. Tables with independent lifecycle semantics do not inherit it, including token/OAuth inventory, AI delegate/review/research records, market views and sources, trade-integrity work items/cursors, and feed-health snapshots. The temporal provenance fields are:

- session_id (str, required) — producer session identity
- sequence_id (int, required) — per-table monotonic counter for gap detection

StrictDataSchema (Pydantic base) requires these fields at construction. Producers must obtain values from a SequenceTracker before creating the event, ensuring every payload is complete and identifiable from birth.

Models inheriting TemporalMixin use this dual-key identity pattern:

- id (INTEGER, internal PK) — row/version identifier for SCD2 close operations: SELECT → lock → close by id → INSERT new version. Never exposed outside the repository layer.
- public_id (UUID7 string) — logical entity identifier for domain joins and external references. Stable across SCD2 versions.
- timestamp (DateTime, system time / known_from)
- known_to (DateTime NOT NULL, default KNOWN_TO_MAX = 9999-12-31T23:59:59 UTC, active rows have known_to == KNOWN_TO_MAX; query pattern: WHERE timestamp <= :t AND known_to > :t)

No hard foreign keys between temporal entities. All domain joins use child.instrument_public_id == Instrument.public_id (or Execution.order_public_id == Order.public_id) with temporal filters. This enables future archival of closed SCD2 rows without breaking referential integrity.

- **SQLAlchemyRepository** (repository.py) — Async CRUD for SQLite/PostgreSQL
- **DatabaseRepository** (repository.py) — Sync access for scripts, archiver, and background updaters
- **CandleCacheArchiver** (archiver.py) — Exports active candle data to polygon-compatible CSV cache files per exchange/archive_symbol
- **EventArchiver** (archiver.py) — Exports append-only event rows (Tick, Trade, Signal, Execution, Telemetry, Control) to per-day CSV archive files with full temporal metadata, merge/dedup, and optional purge. Execution *purge* is refused (ExecutionPurgeUnsupportedError): deciding which execution rows are purgeable requires reading active anchor watermarks and per-scope max(scope_sequence) tips, but the archiver's reads and its delete are separate transactions in a separate process, serialized against the spot reconciliation anchor writer by nothing. An anchor committing between the protection read and the delete would strand rows above a live watermark and fail the counted range proof closed forever against an immutable anchor, so a filtered purge is only as sound as a lock that does not exist. Executions still export to CSV in full; only the DB delete is refused, and executions is rejected

from RETENTION_POLICIES at import. Lifting the refusal requires one cross-process lock shared from anchor capture through anchor commit and by the protection-plus-delete path. Execution exports include the full scope plane (wallet_public_id, exchange, mode, scope_sequence); execution *restore* is likewise refused (ExecutionRestoreUnsupportedError) because the generic audit restore bypasses the execution fence and counter protocol — an anchor-aware offline reconstruction protocol is a prerequisite. These process-level refusals are backed by a **physical append-only trigger** on executions (ledger_triggers.py): on both dialects UPDATE and DELETE (and, on PostgreSQL, TRUNCATE) are rejected at the database with an append-only error through any channel — ORM, Core, raw text() SQL, quoted_name(quote=False) identifier injection, or a console on the same connection — while INSERT stays permitted so the fenced max+1 ingest keeps working. The Python-layer _guard_immutable_ledger remains the fast, legible refusal for the six generic ORM mutation primitives; the trigger is the theorem that closes the raw-SQL / raw-identifier hole no target-parameter guard can reach. It installs from a single source both ways the table is ever built: an after_create DDL event on Execution.__table__ (every create_all-built database) and migration 0030 (the already-existing production table), so the DDL is byte-identical by construction

- **CandleAuditArchiver** (archiver.py) — Exports all candle SCD2 versions (closed + active) with full temporal metadata, grouped by open_at date
- **StateArchiver** (archiver.py) — Generic archiver for state-SCD2 tables (Order, Position, Instrument, Setting, Symbol, etc.) with closed_only filtering and Symbol anchor row protection
- **Archive symbol resolution** (archive_symbols.py) — Stable filesystem-safe symbol naming derived from Symbol anchor rows, with seniority-based collision handling

Strategies (src/snapper/strategies/)

Trading strategies framework:

- **BaseStrategy** (base.py) — Base class for all strategies
- **StrategySignal** — Trading signal structure
- **StrategyConfig** — Strategy configuration

Built-in strategies:

- RSIReversion (rsi.py) — Mean reversion on RSI
- MACDCrossover (macd.py) — MACD crossover
- CointegrationPairs (cointegration.py) — Pairs trading
- HeartbeatConsult (heartbeat_consult.py) — Scoped paper-only AI consultation exercise

Indicators (src/snapper/indicators/)

Technical indicators:

- rsi — Relative Strength Index (Python implementation)
- macd — Moving Average Convergence Divergence
- ta_lib_adapter — TA-Lib library adapter

Messaging (src/snapper/messaging/)

ZeroMQ pub/sub architecture:



Components:

- **Broker** (infrastructure/broker.py) — XPUB/XSUB proxy
- **Publishers** (publishers/) — Market data publication. When multiple timeframes are configured, a publisher subscribes to the venue's 1m stream only and a CandleAggregator (publishers/candle_aggregator.py) synthesizes the higher timeframes (5m/15m/30m/1h/4h/1d) client-side at UTC boundaries, publishing them on the same market.*.candles.{timeframe} topic family. Synthesized bars are also persisted to the candles table (tagged source='synthesized' with a complete trustworthy-boundary flag) when a higher timeframe is configured for persistence — off by default — under the same persist policy as 1m. Native 1m persistence runs through a NativeCandleFinalizer (publishers/native_candle_finalizer.py) — the persistence-side peer of CandleAggregator and TradeCandleBuilder: by default the DB stores only the final complete=true bar per native 1m window (eliminating intra-minute SCD2 churn), while the default-off persist_intermediate_candles setting restores per-frame persistence (ZMQ publishes every frame regardless). The candle source provenance vocabulary is native | calculated | synthesized (the ck_candle_source CHECK constraint on candles, mirrored as ck_shadow_candle_source on shadow_candles): native for venue OHLC bars, calculated for trade-built 1m bars (see the Kraken Spot sub-bullet below), and synthesized for aggregator-derived higher timeframes. The single-source read cutover is gated by the default-off candle_single_source setting (when ON, /api/candles serves 5m/15m/30m from the persisted plane instead of on-read derivation, after verify-candle-coverage confirms the plane is populated) (see docs/messaging.md "Higher-Timeframe Candle Synthesis"). An optional, default-off candle_forward_fill setting enables a time-driven flush that seals a wall-clock-ended window with no later 1m and forward-fills empty windows with a flat carried-close bar — for instruments whose warmup corpus is contiguous at the timeframe (24/7 crypto), never for session-based equities. The flag is gated per venue by the

publisher's _supports_forward_fill() (only kraken spot), so it is forced off (with a warning) elsewhere and in paper replay. The paper publisher synthesizes from replayed 1m too, anchoring the aggregator's live epoch at the replay start so historical higher-TF windows are emitted rather than suppressed.

- **Trade-built Spot 1m** — The Kraken Spot publisher (publishers/kraken.py) can build live 1m bars from the public trade feed instead of the native OHLC channel. A TradeCandleBuilder (infrastructure/exchanges/_trade_candle_builder.py) folds each TradeUpdate into its (symbol, minute) bucket and emits completed minutes once they age past trade_built_finalize_grace_seconds (default 12). This path is gated by the spot_candle_source setting (default native; set trade_built to enable), which tags persisted rows source='calculated' and disables the native candle-only liveness watchdog. The default-off spot_trade_built_shadow_enabled setting runs a parallel A/B writer that persists trade-built 1m bars to the separate shadow_candles table (source='calculated') while leaving the live native plane untouched — for comparing the two sources before any cutover.
- **Executors** (executors/) — Per-wallet order execution on exchanges. One executor process per (exchange, wallet) pair is spawned at boot from active wallet_credentials rows; each instance filters incoming commands by wallet_public_id and loads its credentials via CredentialResolver during startup. Persisted executor_<exchange> rows are templates only; runtime instances are named executor_<exchange>_w<wallet_short> and bare templates are not started as live executors.
- **Schemas** (schemas/) — Pydantic message models
- **Topics** (topics/) — ZMQ topic definitions

Message types (in messaging.schemas.data):

- TickData — Price tick
- CandleData — OHLCV candle
- SignalData — Trading signal
- TradeData — Trade execution
- OrderData — Order status
- ExecutionData — Order execution
- OrderRequestData — Order request
- AccountStateChangedEventData — Thin committed account-state invalidation
- HeartbeatData — Component heartbeat

Every Data class carries public_id: str (UUID7), type: Literal[...], and timestamp: datetime.

Account truth invalidation (executors)

Each per-wallet executor observes venue balances and positions independently of order reconciliation and persists the resulting account snapshot. After that durable write, and again after the matching portfolio-reconciliation evaluation commits, it best-effort publishes an AccountStateChangedEventData frame on portfolio.accounts.{wallet_public_id}. The frame is only an invalidation signal: it never carries balances or a precomputed reconciliation view.

The ZMQ-WebSocket bridge requires read:account_state at subscribe time and filters every frame against the destination principal's accessible-wallet set (global scope with effective impersonate:operator; malformed payloads and topic/payload wallet mismatches fail closed). The browser invalidates the active GET /api/portfolio/accounts query so the REST read path remains the only builder of the time-sensitive, fail-closed view. A 60-second safety-net poll covers lossy bridge delivery and raises the client transport-staleness ceiling after three missed polls.

Spot precision certification

Spot reconciliation consumes the persisted exchange column and evaluates each asset's balance and fee evidence planes independently. The pure spot_precision_certification.py predicate is the single certification authority: it binds exact provenance families to their venue and plane, recomputes content versions, enforces the current documentary floor and artifact dates, applies strict 12-hour freshness to observed evidence, and rejects a current plane whose decimals are NULL. Documentary evidence stays fresh only while its exact reviewed artifact remains current. Instrument-spec fallbacks use the same venue binding and digest checks; in particular, Kraken CCXT precision can certify only a Kraken account.

Walutomat's authenticated balance producer uses those same artifact checks before applying the documentary minimum, then persists the candidate's own exchange identity. A newer complete-provenance row with NULL decimals is an explicit plane revocation; neither a sibling plane nor an older ratcheted maximum can restore certification.

Spot anchor bootstrap

The immutable portfolio_spot_reconciliation_anchors row is the base case of spot execution replay: it certifies that one venue-reported balance snapshot at a named venue history point H0 corresponds exactly and completely to the local execution ledger's sealed prefix [1..W]. The account observer attempts the bootstrap each live cycle while an account is unanchored (gated on the venue's account_history_capability): it reads the venue history tip H0 BEFORE the watermark capture, rides the capture to the reconciliation task, then re-reads balances and the tip (H1) and gathers precision, margin,

chain-tip, and fill-witness evidence. Two pure authorities decide: spot_anchor_witness.py reconstructs each execution's venue history item ids by composition (the immutable exec-id cumulative partitions each order's account/history fills by monotone interval, cross-checked against the market_fx/orders cumulative totals in exact Decimal), and spot_anchor_bootstrap.py proves the full obligation set — quiescence (H0 == H1, equal double balance read), a reverse-coverage bijection between the sealed prefix and the attributed history page, per-asset precision certification, cash-only margin, and a ten-instant read-order chain the schema also CHECK-enforces. The writer (record_spot_reconciliation_anchor) re-reads the committed max(scope_sequence) inside its insert transaction and refuses unless it equals the anchor watermark, so the certified range is empty at birth. Everything fails closed: any missing read, unmappable exec id, witness inconsistency, or named refusal degrades to no-anchor-retry-next-cycle (the observer's balance snapshot is never at risk), and only a fully proven observation is sealed — once, immutably. The anchor's venue_cursor_value (H0) is the left endpoint for the subsequent replay-range proof.

Once an anchor exists, spot execution replay is live: the dispatch feeds the evaluator a transactionally consistent bundle — the anchor, the replay range (W_anchor, W_boundary] proven contiguous by count, instrument identities and specs pinned to the capture's as_of, the certified precision plane over the anchor∪venue∪replay assets, and the previously confirmed assets guarded by anchor epoch. The bundle also re-derives the execution hash chain from the anchor's sealed tip through the newest matched checkpoint to the boundary (regression, divergence, or a broken fold refuse by name), and every full verdict durably records its boundary tip as the next cycle's anti-rollback checkpoint. Walutomat permits price improvement, so the execution price is never the limit price: each fill carries the venue's EXACT opposite-currency amount (counter_amount_decimal), which the replay folds verbatim for the quote leg (dropping the half-tick price-improvement tolerance the size * price approximation required) — without it every improved fill produced a permanent false mismatched. That exact amount is sealed into the execution hash chain (bumped to v2); the extension was made while W = 0, before any chain tip is sealed, so no committed tip is invalidated.

Real balance divergence therefore fires mismatched immediately, and matched is gated behind the venue-cursor certificate: for an anchored account the observer additionally captures the history tip H_E before the watermark, then — after the snapshot commits — the fill legs of every in-range order, the confirming tip, and the ascending history range (H_anchor, H_E]; the pure walutomat_history_certificate validator then proves the tip double-read, the closed-range pagination, the full balanceAfter chain fold seeded from the anchor's balances, the operation taxonomy (external flows and correcting entries refuse), and the range-side witness bijection (reusing the bootstrap's composition authority), emitting a scheme-bound cursor with a digest of the folded range. Only a certified boundary reaches the evaluator with venue_cursor_certified=True; any refusal keeps the honest uncertified path, and its named reason replaces only the evaluator's generic uncertified_boundary incomplete — the mismatch-first gate order guarantees a

certificate failure never suppresses a computable mismatched. Every capture failure degrades to absent evidence with a named log; the balance observation itself is never at risk.

Two claim-domain facts are recorded rather than assumed. Leg completeness is enforced in code: every witnessed fill must carry exactly its base and quote legs, and each order's COMMISSION history items must sum exactly to the venue-reported cumulative commission, so an in-flight quote or fee leg refuses instead of sealing balances a later item would double-apply. And the un-ingested-fill / manual-activity checks are proven over the OBSERVED history window, not the account's whole past: activity older than the page is absorbed into the sealed balances (replay never re-applies pre-cursor items), and the anchor's provenance durably records the claim strength — :genesis when the page held the entire account history, :windowed otherwise.

Venue reconciliation (executors)

Executors can periodically poll exchange REST APIs to reconcile local pending-order state against the exchange's authoritative view. This detects two failure modes that WebSocket streams alone miss: **fill gaps** (exchange has more filled quantity than the executor observed) and **disappeared orders** (pending orders absent from the open-orders snapshot). The same cycle also works the durable command plane in both directions: **ghost orders** (open venue orders no executor is tracking) are adopted, and **dispatched commands** with no venue evidence are verified against venue truth (see below).

Always-on — runs alongside the executor's order handler, heartbeat, and private fill-stream tasks, all of which run as supervised loops (no feature flag; the defensive polling is cheap enough that gating it provided no value). Reconciliation cycles are serialized on an executor-level lock: the periodic 60s cycle and the fill-stream supervisor's post-reconnect heal cannot run concurrently and double-emit the same corrective. Each cycle — including lock acquisition — is bounded by a 300s timeout, so a hung venue call cannot hold the lock forever and wedge both loops; a cancelled cycle is safely recomputed next period because correctives carry stable synthetic ids.

_reconcile_fill_gap (in messaging/executors/base.py) emits a corrective ExecutionUpdate covering the observed cum-qty delta, stamping the synthetic execution with the deterministic exec_id=recon-<exchange_oid>-c<venue_cum> — re-emitting the same gap (after a failed publish, or again at the next startup recovery) dedupes at every consumer instead of double-applying, while a gap whose venue cumulative advanced gets a fresh id. Correctives carry venue-true fees instead of fabricating fee=0: the snapshot's order-level running commission, or the venue fills-summary total, rides the frame's cumulative cum_fee. When a venue-implemented fee source is transiently unusable (failed lookup, partial fills page, no usable rows yet) the corrective is deferred rather than freezing a fee-less emission under its stable exec id; the deferral is bounded

at five recon cycles, after which the corrective emits fee-less with a CRITICAL manual-reconcile signal. A venue reporting neither a snapshot-level running commission nor a per-order fills summary emits fee-less as before.

_reconcile_disappeared_order (in messaging/executors/base.py) calls get_order() on orders missing from the open-orders snapshot to determine their actual terminal status (CLOSED / CANCELED / EXPIRED), emits any residual fill gap first, then publishes the terminal ExecutionUpdate. Walutomat's polling client never guesses filled-vs-canceled for a disappeared order whose final-state query fails: the order stays tracked and the query retries every poll cycle, escalating to a single CRITICAL log after ten consecutive failures while still retrying — no terminal state is ever fabricated.

Ghost-order adoption and dispatched-command verification

A DISPATCHED trade-command row is durable pre-send intent, so the recon cycle also works the durable command plane.

Ghost adoption: an open venue order absent from the executor's pending set is attributed via a strict create/submit command lookup by client order id (cancels share the original order's cid; multiple active matches raise instead of guessing) and adopted with the original request rebuilt from the command row (application/trade/command_request.py, shared with the outbox publish path) so shard attribution survives adoption. Foreign/manual orders and other wallets' orders are never touched. Orders whose cid was already pending when the open-orders snapshot was taken never adopt from that snapshot (a stale snapshot would double-count a corrective). Adoption seeds the fill watermarks from the durable venue-event rows fail-closed — the engine books corrective deltas, so an untrusted zero watermark would double-book — and repairs the durable orders row so coordinator restart recovery re-arms engine intent.

Dispatched verification: active dispatched create/submit commands past a minimum age whose cid has no resolving venue evidence (a lone unknown-submit row does not resolve — parked entries lost to an executor restart land exactly here) are verified by client id under a per-cycle fairness cap with index rotation. A found order is adopted. Venue-verified absence twice in a row rejects only under publish-success-before-terminal semantics (the durable order_rejected row is written only after a confirmed REJECTED publish — a premature terminal row would exempt the command from the sweep while the engine guard stayed held), only while the command is young enough that the venue's bounded closed-order lookback keeps absence authoritative (older commands escalate WARN-only), and never when the dispatch TTL is disabled — a frame can then legitimately be in flight at any age, so absence may never auto-reject.

False-rejection heal: a command found OPEN on the venue after a durable REJECTED is adopted and its rejection durably restored to ACCEPTED through a retried restore queue; the coordinator's reconciliation loop runs a restart-proof resurrection pass for REJECTED rows whose live venue evidence postdates the rejection, and the next fold cycle re-derives the true state from the full event history.

Adopted-order intent re-arm: every adoption-shaped ACCEPTED publish (ghost adoption, ambiguous-submit verification, false-reject heal, startup recovery of a venue-verified-open row) carries reason="adopted" (ORDER_STATUS_REASON_ADOPTED, a shared wire constant), and the running coordinator re-arms a RELEASED engine's in-flight guard on exactly those frames — an engine that consumed a false REJECTED, or whose lazy timeout valve cleared, would otherwise emit a new order on the next signal while the adopted one still works the book. The re-arm never clobbers a newer live intent, skips frames whose registered shard mismatches the routed engine, and refuses cids already retired by an honest terminal (FILLED fill, cancelled/expired confirm); a refused or stale adopted frame also suppresses the TradeService shadow-write so it cannot regress a FILLED projection back to ACCEPTED. A failed adopted publish is retried each recon cycle because that frame is the running engine's only re-arm signal.

Market orders without price — venue-fills VWAP heal

If a market order's exchange snapshot reports price=None (Kraken Futures snapshots carry only limitPrice), the fill-gap path first asks the venue for the order's own fills via ExchangeClientBase.get_order_fill_summary (venues with a real per-order fills source set the supports_fill_summary capability flag; the same aggregate also supplies corrective fees) and uses the venue-true VWAP — but only when the fills page covers the **whole** filled quantity within a relative 1e-6 tolerance; a partial page is refused rather than silently skewing VWAP. The CCXT snapshot builder additionally backfills price from the executed average when the venue reports one. Only when the venue reports neither a price, nor an executed average, nor whole-coverage per-order fills does the path log an ERROR ("no price on market order, skipping corrective fill") and skip the corrective emission — recovery then depends on a later snapshot populating price, or manual reconciliation against the exchange's fill history. Operators seeing this ERROR should not patch Snapper to approximate from ticks; the skip is an intentional trade-off (predictable behaviour + observable gap) over silent approximation drift.

Private fill-stream supervision

The private execution WebSocket stream is supervised, not spawn-once. _supervise_execution_stream (in messaging/executors/base.py) treats ANY termination of the stream handler — venue SDK reconnect-budget exhaustion, a raised error, or a clean generator return — as death and respawns it with capped jittered exponential backoff (1 s → 60 s, reset after 300 s of healthy runtime), never abandoning. Previously the stream was started once: after an outage exceeding the SDK's internal reconnect budget (~2 min futures, ~5 min spot), fills went permanently dark while the process kept reporting RUNNING.

Death is detectable because the venue clients raise instead of exiting cleanly: the spot executions generator raises ConnectionError when its consume loop ends on a terminal SDK error (closing the poisoned client), ensure-connected on both venues rebuilds a slot

whose client carries a terminal SDK error instead of returning it as connected, and subscribe sends are bounded by a 5 s timeout so a wedged socket cannot hang an attempt.

Resubscribe is idempotent: subscription is snapshot-free at the source (spot subscribes with order/trade snapshots disabled; futures drops fills_snapshot frames) because startup recovery and the recon loop own snapshot-shaped state — a venue snapshot replayed on top of recovered watermarks would inflate cumulatives above venue truth. Before each post-death re-entry the supervisor runs a best-effort reconciliation pass so the dark-window fill gap is healed before the stream re-attaches.

Per-order fill booking is atomic: each tracked order carries a fill_lock serializing the dedupe gate, delta build, durable venue-event write, publish, and watermark advance across the stream task, recon correctives, orphan flush, and cancel handling. Two cumulative watermarks are kept per order: last_seen_cum_qty (committed — advances only on publish success, so ZMQ deltas anchor to what the engine actually received) and last_recorded_cum_qty (durable — advances on venue-event write success, so additive checkpoint replay of the durable rows sums to venue truth whichever prior step failed). The same dual-plane pattern covers fees: per-currency signed fee watermarks (last_recorded_fee durable, last_published_fee published) anchor fee attribution — cumulative cum_fee frames set their currency's entry, per-fill fee frames add — so successive gap correctives attribute exactly the not-yet-attributed remainder and never re-charge fees that live per-fill frames already booked.

Executor loop supervision and escalation

The same supervisor pattern covers every executor loop, not just the fill stream: a generic _supervise_loop (in messaging/executors/base.py) wraps the order handler, the heartbeat loop, and the reconciliation loop with the identical capped jittered exponential backoff (1 s → 60 s, reset after 300 s of healthy runtime). Before respawning the order handler the supervisor rebuilds its SUB socket — re-entering on a poisoned socket would just die again. Previously a died order-handler task was a permanent invisible order-execution outage while the process kept reporting RUNNING.

A _supervise_loop-wrapped loop that keeps dying for 1500 s without a 300 s healthy run escalates: the supervisor raises ExecutorTaskDeadError out of start(), sibling tasks are cancelled, ZMQ sockets are closed, and the process launcher rebuilds a fresh service instance — safe because startup recovery emits corrective fills. The fill stream's dedicated supervisor backs off indefinitely and never escalates. The ceiling deliberately exceeds the launcher's 1200 s healthy-uptime budget reset, so escalation-driven restarts retry slowly forever instead of exhausting the restart budget; only fast startup crash-loops exhaust it.

Executor heartbeats report honest status derived from these supervision seams instead of a hardcoded HEALTHY: active death streaks, a reconciliation progress clock (recon swallows its per-cycle failures by design, so it can fail forever without dying — only

progress age shows that), the age of the order command currently in flight (a command wedged inside its venue call never dies and never progresses), an unhealed accept-event backlog, and parked ambiguous submits all degrade the published status to WARNING or ERROR. The computation is crash-proofed: a raising status read degrades to WARNING with the failure in the reasons, never heartbeat absence or false HEALTHY. Restart-budget exhaustion at the launcher is paged through the same heartbeat pipeline (see "Restart watchdog and parking" under Processes).

Application (src/snapper/application/)

Business logic:

- **Engine** (engine/) — TraderCoordinator is the multi-symbol trade runtime coordinator; TradingEngineService is the per-instrument engine that turns signals into TradeCommand rows and order requests
- **Trade** (trade/) — Trade-domain services: trade_service.py (in-memory command and position read model, fill deduplication, funding accrual application, and reconciliation halt feedback), outbox.py (outbox-driven publishing with wake-up + polling fallback), reconciler.py (trade-command lifecycle fold, evidence-scoped stale-command scan, and reconciliation failure feedback), command_request.py (rebuilds the original order request from a durable command row — shared by the outbox publish path and the executor's adoption sweeps)
- **Process Manager** (process_manager/) — Process management
- **Services** (services/) — Application services, including market_cache.py (in-process 1m candle cache and pair-stat snapshots) and market_persist_policy.py (runtime policy deciding which market-data streams are persisted to DB versus cache-only)
- **Updaters** (updaters/) — Data updates (symbols, historical)
- **Risk** (risk/) — Risk defaults and sizing models
- **Portfolio** (portfolio/) — Signed long/short portfolio accounting
- **Retention** (retention/) — Periodic archive-and-purge scheduler (RetentionScheduler, scheduler.py) started by the FastAPI lifespan (a lifespan task, not a Process Manager process). Each RETENTION_INTERVAL_SECONDS tick (default 3600) drives the event archivers per the declarative RETENTION_POLICIES (v1: telemetry only, 1-day residency), exporting rows under RETENTION_OUTPUT_DIR before purge. RETENTION_DISABLED skips the eager run and the loop (the /api/metrics/retention route stays mounted, reporting disabled); RETENTION_DRY_RUN forces export-without-purge; executions is rejected at import (append-only — the archiver refuses its purge)

Infrastructure (src/snapper/infrastructure/)

External integrations:

- **Exchanges** (exchanges/) — Exchange clients (Kraken, Walutomat). kraken_sdk_patches.py is a version-coupled patch layer over python-kraken-sdk (3.3.x) that the Kraken venue implementations install at import time, so every process that builds a Kraken WS client — executors as well as publishers — is covered. Among its fixes is WS teardown hardening: the stock SDK reconnect backoff sleeps uninterruptibly (up to ~3 min), so a venue-bounded close() firing during it leaked one aiohttp ClientSession per rebuild cycle and the abandoned reconnect later spawned orphaned child tasks. The hardened reconnect polls the stop flag during backoff and reaps its children on every exit path, and force_close_ws_client() finishes teardown directly — cancel run tasks, bounded drain, close the session — whenever a venue's bounded close() times out or raises.
- **Market Data** (market_data/) — WebSocket feeds
- **Symbols** (symbols/) — Symbol mapping
- **Security** (security/) — Settings encryption
- **Logging** (logging/) — Logging configuration

Server (src/snapper/server/)

FastAPI application:

- **App Factory** (app.py) — create_app() with lifespan management
- **REST API** — HTTP endpoints
- **WebSocket** — Real-time streaming via ZMQ bridge with per-frame scope filters for ai_reviews.*, orders.events.*, portfolio.accounts.*, and alerts.*
- **Static Files** — Frontend dashboard
- **ClientProvenanceMiddleware** (provenance_middleware.py) — Extracts client provenance from mutation requests, runs per-session gap detection, records mutations to control table and GET reads to telemetry table

API (src/snapper/api/)

Shared API schemas and WebSocket auth helpers (not route definitions):

- **Schemas** (schemas/) — Pydantic request/response models (health, process, settings).

  Schema class hierarchy:

  ```
  text
  BaseModel
  ├── PartialBody           — extra="ignore", strict=True (GapEnvelope,
  WsTokenPayload)
  ```

```
├── StrictBody           – extra="forbid", strict=True (request bodies,
nested DTOs)
│   └── StrictDataSchema  – + provenance (all event payloads, REST/WS/ZMQ)
│       ├── PayloadRequest[T]
│       ├── PayloadResponse[T]
│       └── PayloadListResponse[T]
├── ExchangeResponse     – extra="allow", strict=True (exchange API
parsing)
├── ExchangeRequest      – extra="forbid", strict=True (outgoing exchange
requests)
└── BaseSettings         – BootstrapSettingsLoader (env var config)
```

- **Auth** (auth/) — WebSocket token service and schemas

Route modules live closer to their domains: server/app.py (assembly and data endpoints), the per-domain server/*_routes.py modules (orders, execution plans, trailing stops, position cycles, paired execution, backtests, wallets, operators, credentials, AI reviews/delegates, alerts, alert defaults, devices, metrics, market cache/coverage/feed-health, scope grants, egress health, processes, strategies), config/settings_routes.py, and auth/routes.py.

REST endpoints return the same Data schemas used by WebSocket messaging (messaging.schemas.data): OrderData, SignalData, ExecutionData, PositionData, CandleData. Old REST-specific schema modules were removed.

Auth (src/snapper/auth/)

Authentication system:

- JWT tokens with access/refresh via HTTP-only cookies
- CSRF protection
- Permission-based authorization. Roles are only stable names for the ai_researcher, ai_reviewer, ai_delegate, viewer, operator, and admin base permission sets, plus user-management values and display labels. Every REST, WebSocket, frontend, iOS, and MCP capability decision consumes effective permissions rather than role identity or rank.
- Role-bounded token downscoping. A token can remove permissions from its named set but cannot add permissions outside it. The structural impersonate:operator grant is non-downscopable, preserving historical admin-wide scope while allowing runtime scoping to consume effective permissions. Session profiles expose the resulting effective_permissions to clients.
- Read-only operator visibility for viewer, including market data and views, account state, scoped wallets and portfolio, orders, positions, signals, AI-review decisions, P&L, backtests, strategies, process state, health, AI integration, and notifications. It has no trading, process, strategy, backtest, diagnostic, AI-integration, or administrative control permissions; ownership-constrained notification-device self-service remains available. REST wallet reads combine operator scope grants with

personal wallet_user_read_grants; trade mutations use operator grants alone. WebSocket wallet filters and MCP tools currently still use the operator-grant plane, so a personal REST read grant does not by itself authorize those transports.

- Complete-catalog mapping for admin. Its named set is constructed from all 34 Permission values, including impersonate:operator for global wallet and operator scope.
- Dedicated mutation permissions keep read grants read-only: create:backtest_comparisons controls comparison creation, manage:runtime_diagnostics controls tracemalloc, and strategy configuration/start/stop use their matching strategy permissions rather than the coarse non-strategy manage:processes permission.
- WebSocket authentication

MCP (src/snapper/mcp/)

Model Context Protocol server mounted at /api/mcp as a Starlette sub-application. Exposes read/query tools plus permission-gated order actions (submit_manual_order, cancel_order) in tools.py, consumed by AI delegates over Streamable HTTP. The mount re-applies its own middleware stack: feature-flag gating, bearer auth (auth.py), and per-principal rate limiting (rate_limiting.py); each tool handler runs output sanitization (output_sanitizer.py) and returns a structured error envelope (error_envelope.py). See ai-integration.md for the delegate flow and the @mateusz-klatt/snapper-mcp client plugin.

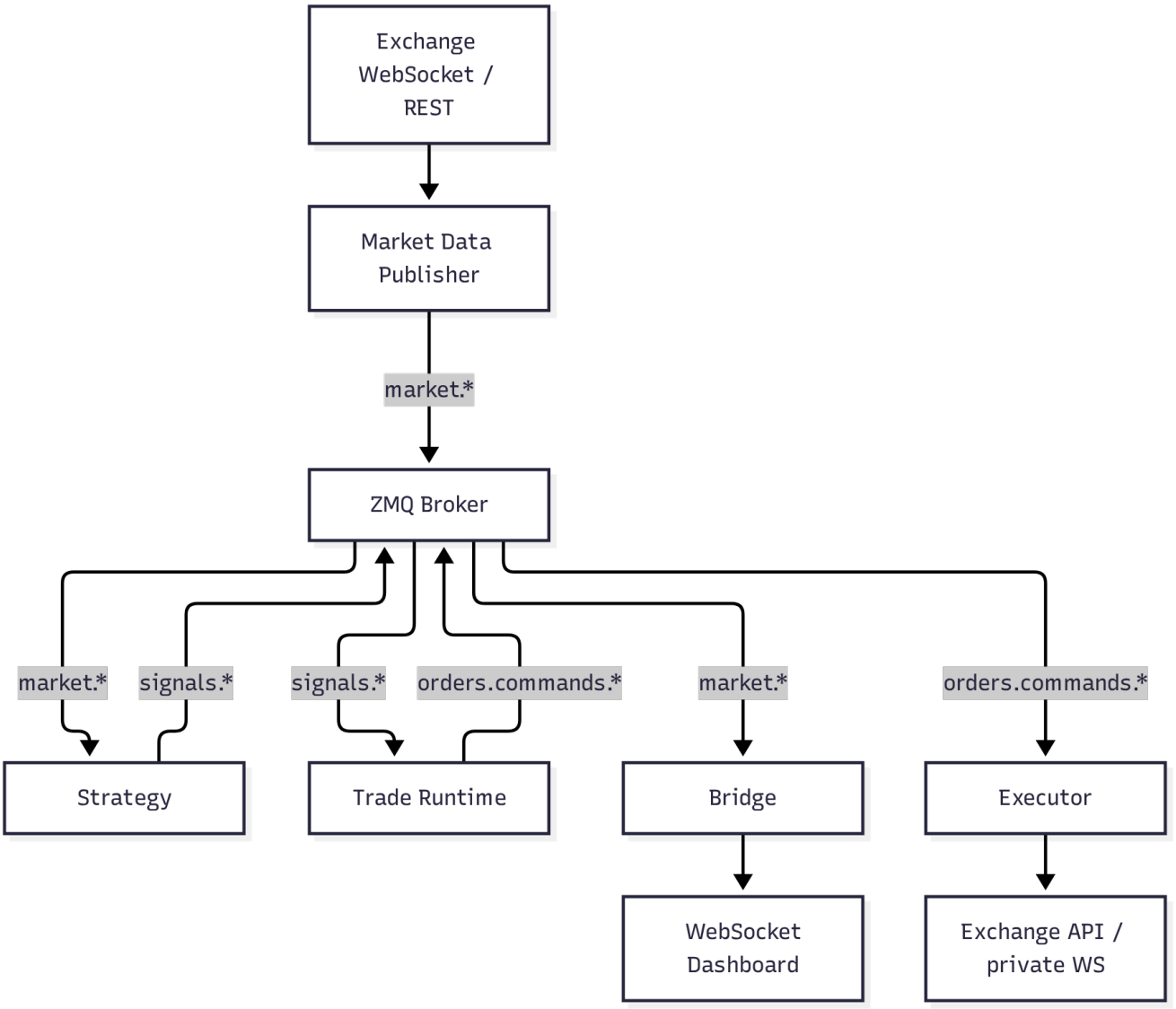
CLI (src/snapper/cli/)

Command-line interface (Typer):

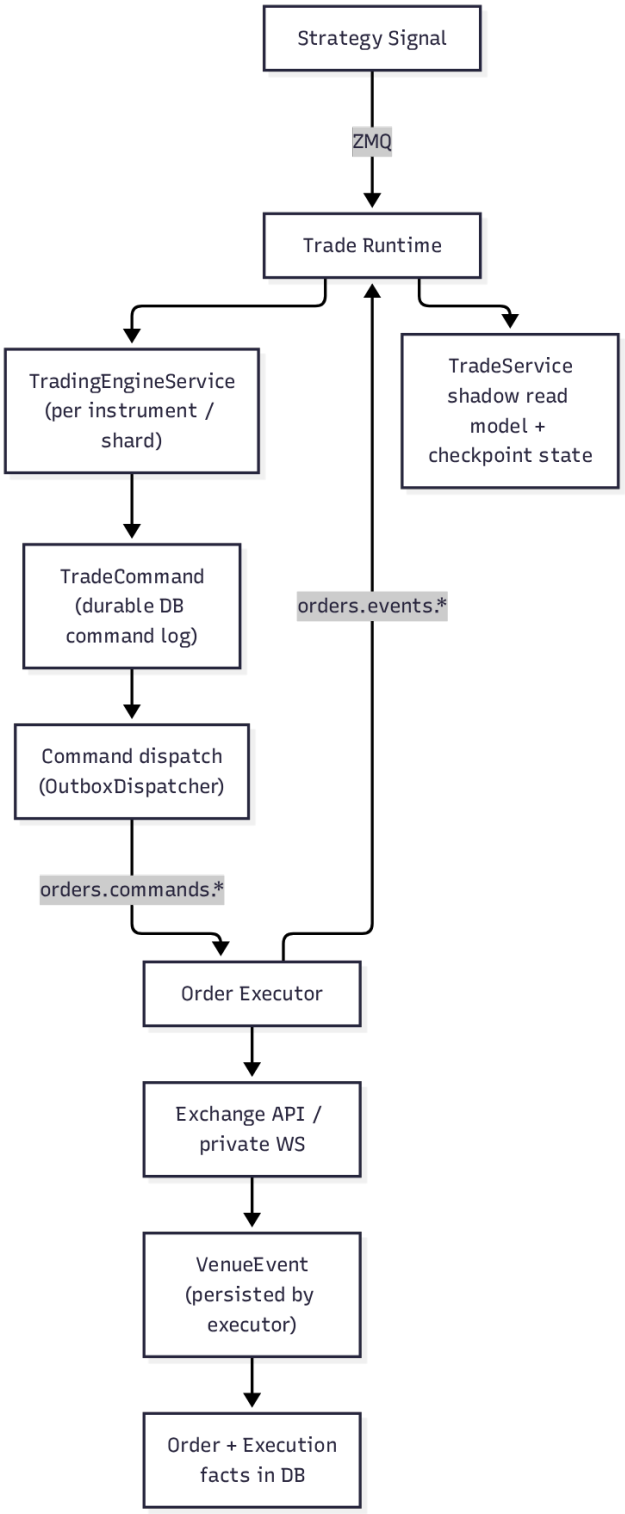
- Server and infrastructure management
- Database operations
- User management
- Market data updates

Data Flow

Market Data and Signal Flow



Order Flow



Database

SQLite for development, PostgreSQL for production.

Schema

```
-- Market data (joined to instruments via instrument_public_id)
instruments         -- Financial instruments (logical key: symbol_public_id +
exchange)
candles             -- OHLCV data
shadow_candles      -- A/B shadow plane for trade-built 1m candles
ticks               -- Real-time price snapshots
trades              -- Transaction history
market_snapshots    -- Real-time market data (SCD2 per instrument, one active
row each)
instrument_feed_health -- Current per-symbol feed health snapshots

-- Trading (joined to instruments via instrument_public_id)
orders              -- Order history (mode: live/paper)
executions          -- Order executions (joined to orders via order_public_id)
positions           -- Portfolio positions (mode: live/paper)
position_cycles     -- Open/closed cycles tracking per-cycle peak qty
signals             -- Strategy signals
execution_plans     -- Plan envelopes (brackets, trailing stops, manual)
execution_plan_checkpoints -- Materialized plan state snapshots
execution_plan_decisions   -- Per-tick plan decisions (audit trail)
execution_plan_decision_outbox -- Durable retry state for plans.decisions fanout

-- Trade runtime (durable command path)
trade_commands              -- Durable trade intent (engine writes before
execution)
venue_events                -- Durable venue observations and acknowledgements
from executors
trade_projection_checkpoints -- Materialized position/cash snapshots
paired_execution_groups     -- Multi-leg group FSM for arming and compensation
paired_execution_legs       -- Per-leg command binding and signed exposure
accounting
paired_execution_halts      -- Durable per-scope halt projection for broken/
exposed groups

-- Multi-tenant
wallets                     -- Trading accounts (label, is_paper)
wallet_credentials          -- Per-wallet exchange credentials (Fernet-
encrypted)
wallet_operator_scope_grants -- Per-wallet operator scope grants
operators                   -- Operator identities (multi-tenant trade actors)
user_operator_memberships   -- Operator memberships per user
user_trading_caps           -- Per-user trading-caps overrides

-- Symbol management
symbols              -- Symbol identity + versioned attributes (SCD2)
symbol_aliases       -- Exchange-specific symbol mappings
symbol_exchange_capabilities -- Exchange-specific capabilities
symbol_market_data_channel_capabilities -- Per-channel market-data capability
flags
underlying_assets    -- Underlying-asset definitions (YAML-sourced)
instrument_underlying_mappings -- Active instrument → underlying mappings
continuous_contract_configs -- Continuous-contract series config

-- Instrument specifications
instrument_specs              -- Trading specifications (tick_size, lot_size,
limits)
instrument_order_capabilities -- Per-instrument order-side capability flags
```

```
venue_fee_schedules           -- Maker/taker fee schedules per venue

-- Funding + accruals
funding_rates       -- Per-instrument funding rate history (perps)
accrual_ledger      -- Funding / rollover / borrow accrual entries

-- Backtest
backtest_runs           -- One row per backtest run
backtest_events         -- Lifecycle events (started/failed/completed/...)
backtest_signals        -- Strategy signals emitted during a run
backtest_trades         -- Trades minted by the backtest engine
backtest_equity_points  -- Equity curve samples
backtest_results        -- Final per-run metrics
backtest_comparisons    -- Pairing-grouping fingerprint anchors

-- AI Reviews + Delegates
ai_delegates        -- AI delegate principals (PAT-style auth)
ai_reviews          -- Human-in-the-loop review queue rows
ai_review_events    -- Per-review lifecycle audit trail

-- Notifications
alert_events        -- Shared alert history
alert_deliveries    -- Per-device delivery attempts
device_alert_prefs  -- Per-(device, alert_type, scope) preferences
notification_devices -- APNs tokens and separate Android FID bindings
android_push_configurations -- Active/disabled revision head and retired
configuration tombstones
user_alert_defaults -- Per-user fallback alert preferences

-- System
users               -- User accounts
user_active_tokens  -- Active JWT inventory (revocation)
settings            -- Settings (encrypted)
user_login_events   -- Authentication event log
process_runs        -- Background process execution records
control             -- Audit log (commands, auth events, mutations)
telemetry           -- High-volume operational events (toggleable)
```

Migrations

Alembic manages migrations:

```
snapper db-upgrade    # Apply migrations
snapper db-downgrade  # Rollback
```

Trade Runtime

The trade runtime always uses the durable (outbox-driven) dispatch path. TradingEngineService writes TradeCommand rows to the database; OutboxDispatcher polls the table and publishes undispatched commands to ZMQ. Executor VenueEvent writes are fail-closed for pre-acceptance events — a failed persist raises before the executor acknowledges the venue event. Once the venue has accepted an order

(returned an order id), a failed order_accepted persist no longer aborts the flow: it is logged CRITICAL, the entry is flagged accept_event_pending and queued, ACCEPTED is still published (the venue state is the truth), and the executor's recon loop retries the durable write each cycle until it sticks — probing for an existing row first, so a timeout after a committed write cannot insert a duplicate accept event.

Fresh ordinary, compensation, and atomic leg-flatten commands require an explicit nonblank string wallet_public_id before the repository opens a session. The coordinator drops signals with missing or blank wallet strings before creating an engine or persisting a paired group, then continues receiving signals. These checks preserve nonblank identifier spelling; they do not validate UUID syntax or repair historical wallet data.

trade_commands rows are a fold of the venue_events truth plane. The coordinator's per-exchange ReconciliationLoop folds each active command's venue events into a durable status advance through a lifecycle CAS (advance_trade_command_lifecycle): the first accept supplies acked_at and the exchange order id, the max cumulative fill decides partial-vs-filled, and the last terminal-class event maps to the terminal status. The fold is rank-monotonic — a command status never regresses, so duplicate and out-of-order events collapse idempotently. A later accept or fill supersedes an earlier REJECTED (rejections are legally retried by the outbox, and a resurrection pass restores durably-REJECTED rows whose live venue evidence postdates the rejection); other terminals are never resurrected. The fold covers create/submit commands past the outbox's territory (created rows stay the outbox's), and stale-command reporting is scoped by evidence: an aged active command with zero venue evidence WARNs, unknown-only evidence reports INFO, real evidence is silent — the fold advances it. Cancels keep the legacy stale WARN because they share the original order's cid. Coordinator-side reconciliation scan failures are recorded only against known or last-seen real shard keys owned by that coordinator. Executor heartbeat venue-health metadata can also halt known real shard keys for an exchange/wallet scope and drop new cold-path signals for that scope.

Five order-safety layers sit on the dispatch path:

- **Dispatch max-age TTL** (TRADE_COMMAND_DISPATCH_TTL_S, default 30 s): the outbox CAS-expires a stale CREATED create/submit command to terminal EXPIRED instead of publishing it (cancels and replaces are exempt), and the engine releases its in-flight intent via a synthetic expired event — an outage backlog cannot fire MARKET orders priced off old signals. The executor-side stale check is venue-truth-backed: a found order is adopted, an unverifiable venue drops the frame silently (recon resolves it), and only venue-verified absence rejects.
- **Duplicate-submit guard**: before placing, the executor drops outbox replays of an already-evidenced client_order_id (live pending entry, unhealed-accept queue, or a durable venue-event evidence probe) — fail-closed on probe failure, publishing nothing. A replay whose evidence includes a breaker-open or interlock-blocked event instead reruns that terminal disposition (below) so a crash between the evidence write and the REJECTED publish cannot strand engine intent.

- **UNKNOWN submit state**: when a venue submit outcome is ambiguous (timeout, connection drop mid-send), the executor parks the order as non-terminal OrderEventEnum.UNKNOWN instead of fabricating a REJECTED — a false reject would let the engine re-emit and double the position. It then verifies against venue truth by client id (find_order_by_client_id: could-not-check never counts as venue-says-no); a found order is adopted, two consecutive authoritative absences reject safely, anything else stays parked. The recon loop verifies parked entries each cycle under a per-cycle fairness cap with index-based rotation (a large parked set rotates through verification rounds instead of starving its tail or blowing the cycle's time budget), and the engine holds the in-flight guard until resolution. An order_unknown safety-critical alert (user-scoped, with read:system_status recipient fan-out for strategy orders) fires while an order is parked.
- **Breaker-open disposition**: a submit refused by the venue circuit breaker (typed CircuitBreakerOpenError) is authoritative not-submitted but NOT a venue rejection, and it must not blind-retry — the breaker may have closed, and a late redispatched frame would place an order the engine no longer tracks. The executor writes a probe-guarded durable order_breaker_open venue event (which counts as duplicate-submit evidence), CAS-fails the command row to terminal FAILED so the outbox can never re-fetch it, and only then publishes REJECTED with reason circuit_breaker_open so the engine releases intent. Any step failing parks the entry and the recon loop reruns the sequence until it completes.
- **Live-trading interlock** (operator kill-switch): every non-paper submit reads the durable live_trading_mode setting fresh from the DB (2 s timeout, fail-closed to blocked) via _is_live_trading_interlocked (messaging/executors/base.py). Only enabled proceeds; halted and reduce_only both block (reduce_only currently differs from halted only in the observability reason), and an unreadable, missing, or unrecognized mode blocks as live_trading_mode_unavailable. Paper venues always pass. A block runs a redispatch-safe disposition mirroring the breaker-open path — a durable order_interlock_blocked terminal event, CAS the command row to terminal FAILED, then publish REJECTED — so a frame replayed after the mode later flips to enabled can never execute.

The executor also gates order-type vocabulary before any venue send. The durable command plane (trade_commands, OrderRequestData) speaks the CORE vocabulary (market/limit/stop/stop_limit); one shared venue-request builder (_exchange_order_request_from_core in messaging/executors/base.py, used by both the submit path and ghost-adoption row repair) translates it to the venue wire vocabulary (stop-loss/stop-loss-limit for the stop types). A command whose order type has no wire mapping, or a stop-typed command without a stop_price trigger, raises inside the builder — before the network call, so the order is provably not placed — and the submit path's definitive-reject branch publishes REJECTED and writes the durable order_rejected venue event instead of letting a half-formed frame reach the venue.

Spot (Kraken) create_order is additionally excluded from blind network retry — an ambiguous network failure may have placed the order, and Kraken's cl_ord_id dedupe covers only open orders, so a blind retry of a MARKET order could double-place. The 429 rate-limit retry is kept (an exhausted 429 is a definitive venue-side rejection).

Executor startup recovery emits corrective fills instead of silently re-baselining: the old recovery seeded fill tracking from the venue's current cumulative, so every fill that landed while the executor was down became invisible to projections forever. Recovery now reads both truth planes per order — the durable venue_events fill rows and the executions log (rows exist only for successfully published fills) — seeds the committed/durable watermarks from what each plane proves, republishes recorded-but-unpublished fills under their original exec ids (idempotent — the engine, checkpoint replay, and the executions insert all dedupe by exec id), and emits the remaining venue-ahead gap as a recon corrective with a deterministic id, so repeated restarts and publish retries converge instead of double-applying. Orders that went terminal during the downtime get their fill gap healed before the terminal event is projected; orders the venue cannot verify at startup are parked with DB-derived seeds and retried each recon cycle. Fills without a venue exec id are never republished — they dedupe on an identity-shaped fallback key shared by the live engine and checkpoint replay, count toward the committed seed as already shown, and the durable rows heal a coordinator that missed them at its next restart. This class is shrinking: Walutomat's cumulative polls now stamp deterministic wal-{oid}-c{basis_units} exec ids on every emission (the disappeared-order terminal upgrade gets a -t suffix so a same-cumulative status upgrade is never identity-dropped), so only rows recorded before that carry no id. Operator notes are in docs/operations.md "Recovery-time corrective fills".

The TraderCoordinator class acts as the trade runtime coordinator and integrates TradeService, which owns command lifecycle, position and cash together as one read model. It spawns the outbox dispatcher (when a SQLAlchemyRepository is wired) plus per-exchange reconciliation loops that drive the trade-command lifecycle fold and feed the circuit breaker. Canonical Order and Execution rows are persisted on the exchange/executor path.

The coordinator completes startup recovery before accepting signals and owns the lifetime of its receive and background tasks. An unhandled receive, message-validation or handler failure ends the owning run with the original error. Unexpected normal completion of the required receiver also fails the run; normal completion of an optional worker is allowed. If multiple failures are observed together, a receiver error takes priority, followed by other worker errors in creation order. Existing message-specific refusal paths, such as dropping a blank wallet identifier, retain their normal behavior.

Stop and failed startup both await owned-task cleanup before releasing the coordinator's private ZMQ contexts with zero linger. Partially created workers and sockets are included. A secondary teardown failure does not replace the original operational error or cancellation, and repeated stop does not retry a failed context destruction. Shared

repository resources remain owned by their application lifecycle. This supervision adds no signal replay or listener-only restart; configured process supervision still decides whether to restart a failed coordinator.

Each process launch constructs a fresh coordinator instance. An instance accepts one start; concurrent or repeated starts are rejected. Calling stop before its first start remains supported and does not skip cleanup of resources acquired by that later start.

Multiple TraderCoordinator processes can run against the same DB + broker with deterministic SHA-256 shard partitioning via snapper.core.partitioning.ShardOwnership. Each coordinator owns ~1/N of the shard_key set; signals, venue events, recovery, outbox dispatch, and reconciliation all filter by ownership so no two instances process the same shard. Default --instance-count 1 is byte-identical to single-coordinator behavior. Scaling to N≥2 requires PostgreSQL: the coordinator fails fast (ValueError) at startup when instance_count > 1 on a SQLite backend, because SELECT ... FOR UPDATE row locking — which the money-path DALs depend on — is a no-op there. See docs/operations.md for the systemd template recipe and scale-up / scale-down / crash-recovery procedures.

Checkpoint recovery resolves wallet-aware shard keys from wallet_credentials at the checkpoint's checkpoint_at timestamp before falling back to the boot-time wallet_short cache. This keeps rolling restarts and credential rotations from recovering old .w{wallet_short} checkpoints into an empty or stale wallet scope.

Recovery is also **gap-aware**. The executor commits a fill_observed venue event BEFORE publishing the fill and inserts the executions row only after a successful publish, so a fill recorded but never consumed (publish/insert never completed, and not absorbed by a later cumulative fill) can be stepped over by the scalar last_venue_event_id watermark and dropped. Recovery detects this per shard by **cumulative coverage** — recorded gross fill quantity (deduped by venue identity) exceeding the consumed executions total for the shard's orders — which is exact in both the absorption case (equal totals, no gap) and the true-drop case. A gapped, non-funding shard is rebuilt chronologically from its durable venue_events: a checkpoint shard overlays only the fill-derived fields (position, entry, cash, realized PnL, turnover, dedup set, watermark), leaving command identity and peak_equity from the checkpoint untouched; a shard with no checkpoint is rebuilt from scratch. A dedicated venue-plane pass discovers shards with recorded fills that the execution-replay pass never visits (it early-returns on an empty executions table). Funding (futures) shards are left to status-quo recovery — a from-scratch replay cannot reconstruct pre-checkpoint accruals — so the rebuild is spot-scoped.

The coordinator also spawns the **paired-execution guard scanner** (PairedExecutionGuardScanner) as a background task: a DB-only liveness loop that assembles and arms multi-leg signal groups behind an arming barrier (no grouped command reaches a venue until every sibling leg is durably registered), breaks groups that miss their assembly/fill deadlines, cancels and reduce-only-flattens exposed legs, projects durable per-scope halts, and completes settled groups. The guard is dark by

default (PAIRED_EXECUTION_GUARD_ENABLED false): live multi-leg emission is refused fail-closed at the strategy layer, while the scanner's compensation backstop always runs. Operator surface: GET /api/paired-execution/incidents and POST /api/paired-execution/groups/{id}/terminalize — runbook in docs/paired-execution.md.

Bitemporal Model

Most entity tables inherit TemporalMixin which provides id, public_id, timestamp (bus-time / known_from), and known_to columns. Tables with their own lifecycle, such as token/OAuth inventory, AI delegate/review/research records, market views and sources, trade-integrity work items/cursors, and feed-health snapshots, opt out. Inspect the model base classes before applying temporal repository helpers to a table.

Key concepts:

- **SCD Type 2** — Business-state changes use a close+insert pattern: the current row is closed (known_to set to now) and a new row is inserted with the updated values and known_to = KNOWN_TO_MAX.
- **KNOWN_TO_MAX** — Sentinel value (9999-12-31T23:59:59 UTC) marking the active version of a row.
- **Point-in-time queries** — REST endpoints accept an as_of query parameter to retrieve the state of data at a specific moment.
- **Helpers**:
  - where_active(model, at) — SQLAlchemy filter for temporal queries. at: datetime is required — no silent default-to-now
  - where_active_now(model) — convenience for operations that genuinely mean "current state" (auth login, heartbeat). Not a shortcut for skipping as_of threading
  - close_and_insert() / close_and_insert_sync() — Repository helpers that atomically close the current row and insert a new version

Batch atomicity guarantees

True SCD2 batch writes (upsert_candles, upsert_market_snapshots) and single-row close+insert methods (revise_*, close_and_insert) commit as all-or-nothing transactions. The pattern is:

```
async with self.session() as s:          # opens a transaction
    for row in rows:
        # SELECT ... FOR UPDATE the active version
        # UPDATE close (set known_to = bus_time)
        # s.add(new row)
    await s.commit()                     # single commit
```

If any row in the loop raises (IntegrityError, serialization failure, connection drop, await cancellation), the transaction aborts and **no** close/insert pair from this batch lands in the DB. The batch must be retried in full; there is no partial-progress mode.

upsert_candles batch-loads existing versions for unique (instrument_public_id, timeframe, open_at) keys before closing and inserting rows. If the incoming batch contains duplicate candle natural keys, those rows keep the sequential close+insert path because an earlier row in that group can create the version a later row must close. Lookup batches are chunked to stay below SQLite parameter limits.

Append-only batch writes do NOT use the SCD2 close-insert pattern and split into two shapes:

- upsert_trades delegates to the dialect-aware _upsert_batch. On native dialects (SQLite / Postgres) it uses INSERT ... ON CONFLICT DO NOTHING in a single s.execute — atomic per-batch, and duplicates are silently skipped. On the fallback row-by-row path it wraps each row in a begin_nested() SAVEPOINT; successful rows stay in the outer transaction and are committed at the final await s.commit(), while rows that hit IntegrityError are skipped. This trades all-or-nothing for conflict tolerance.
- upsert_ticks issues a Core bulk insert via session.execute(insert(Tick), list(rows)). The function has two branches: when the caller passes a session, commit is external (caller-managed); otherwise it opens its own session and commits internally. Atomic per-batch on every dialect — it is not routed through _upsert_batch and has no duplicate-skipping behaviour.

Serialization on the same natural key: on **PostgreSQL**, row-level SELECT ... FOR UPDATE serializes concurrent transactions via MVCC row locks — a second session waiting on a locked active row observes the first session's close+insert only after the first commits; both batches complete without error and produce a clean SCD2 chain.

On **SQLite** (aiosqlite), SELECT ... FOR UPDATE is a no-op and the default transaction mode does not acquire a write lock at SELECT time — two concurrent tasks can both read the same active row before either writes. The first writer to COMMIT wins; the second writer's INSERT then collides on the public_id UNIQUE index and raises sqlalchemy.exc.IntegrityError. The final DB state is still a valid SCD2 chain (exactly one active row, carried-forward public_id preserved, no orphan inserts) — one writer is simply reported the conflict via exception rather than implicitly serialized. Callers on SQLite that race on the same natural key must either retry the failed batch or rely on the single-publisher-per-exchange invariant to avoid the contention entirely.

At-scale implication: under heavy multi-publisher contention on the same natural key, batches serialize strictly. If a long-running batch holds the lock for many milliseconds per row, throughput falls linearly in contention. The current single-publisher-per-exchange shape makes this a non-issue; revisit if multi-publisher contention on the same natural key becomes a real workload.

Non-atomic fallback (_upsert_batch row-by-row path): the dialect-agnostic fallback (_upsert_batch in data/repository.py) wraps each row in a begin_nested() SAVEPOINT and continues on IntegrityError. Successful rows are NOT committed immediately — they are held in the outer transaction and finalised only by the final await s.commit(). This trades all-or-nothing for conflict tolerance — the caller sees inserted = N_successful, not all-or-nothing. Used only for append-only paths where a duplicate key is a genuine no-op. Never used for SCD2 close-and-insert.

Processes

The system manages processes through Process Manager:

- **Core** — Broker, feed publishers, executors, trader coordinator, plan executor (required — startup is aborted if any enabled long-running CORE process fails to start; the ZMQ-WebSocket bridge is not a managed process — it starts with the FastAPI lifespan)
- **Strategy** — Trading strategies
- **Task** — One-time tasks
- **Backtest** — Backtesting processes

Processes can be:

- long_running — Run continuously (services)
- one_shot — Execute once (tasks)

Restart watchdog and parking

ProcessLauncherService reconciles died processes toward their desired state with backoff and two restart budgets (consecutive short-lived FAILED deaths, plus a lifetime backstop; both reset by healthy uptime). The watchdog drives native subprocesses and in-process async-task/thread processes through the same machinery — a died per-wallet executor task is respawned with the same backoff, budget, and stop-race guards as a native feed subprocess instead of being finalized FAILED and silently never restarted.

When a process exhausts its restart budget, a CORE market-data publisher escalates to a feed-container restart; any other process is parked. Parking is level-triggered and visible: /api/health reports ERROR while any process is parked (checked before every other branch, including the API-only early return and the TTL cache), and for per-wallet executor instances the launcher bursts synthetic ERROR heartbeats on the instance's own per-wallet heartbeat topic, re-bursting hourly while parked, so the existing critical-system-error alert pipeline pages the operator without any new alert type or topic family. A successful (re)start or a successful deliberate stop unparks the name; a failed stop or a failed manual restart keeps the marker.

Deployment Modes

By default, the FastAPI lifespan starts all enabled processes (all-in-one dev mode). Set SERVER_API_ONLY=true to skip process autostart -- the ZMQ-WebSocket bridge still starts, so the frontend receives live data from a separately-running engine. Useful when broker/strategies/executors run on different hosts.

Dedicated feed container (NYSE event-loop split): the market-data publishers (Kraken spot/equities/futures, Walutomat, paper) can be moved out of the FastAPI backend into a separate container so the API event loop stops sharing one CPU core with five WS publishers + their parse/persist work. Under NYSE open burst the shared loop saturated one core and starved the trade writer (drops); isolating each publisher onto its own process/core removes the contention.

PROCESS_AUTOSTART_PROFILE selects which registered processes a node autostarts (see snapper.core.types.ProcessAutostartProfileEnum):

- all (default) — every enabled process (single-container / dev).
- api — everything EXCEPT market-data publishers (the backend container: broker, executors, strategies, API).
- feed — ONLY market-data publishers (the feed container). Set by the feed-engine CLI command, which launches each publisher as its own OS subprocess (mode=PROCESS, uvloop) via ProcessLauncherService.start_feed_publishers. A market-data publisher is any registered process tagged both market-data and publisher.
- strategy — ONLY role-STRATEGY processes (the strategies container). Set by the strategies-engine CLI command, which starts them THREAD-mode off the backend loop; the backend excludes them via STRATEGIES_EMBEDDED=false.

A market-data publisher is filtered out of the backend's start_all_processes under the api profile, and get_core_health honours the same filter so a publisher that intentionally runs in the feed container is not reported as a missing CORE process on the backend.

The ZMQ broker runs in its own snapper-broker container in the compose split (snapper broker --xsub tcp://0.0.0.0:7500 --xpub tcp://0.0.0.0:7501), so restarting the backend no longer bounces the bus for the feed (and future strategies) containers. The backend opts out of its embedded broker via ZMQ_BROKER_EMBEDDED=false: the launcher excludes the zmq_broker CORE process from autostart and ownership resolution treats it as remotely managed (the dashboard hides Start/Stop, so no duplicate local broker can be

spawned). The default (true) keeps the embedded broker for single-container and dev flows. Cross-container wiring uses two endpoint pairs because ZMQ bind rejects hostnames while connect resolves them:

- The broker container receives its bind interfaces as explicit CLI flags (tcp://0.0.0.0:7500 / :7501); the ZMQ_BROKER_BIND_* env pair remains for embedded-broker deployments (empty default falls back to the connect endpoints, so single-container behaviour is unchanged).
- ZMQ_BROKER_XSUB / ZMQ_BROKER_XPUB — the connect endpoints every process (backend components, feed publishers, and the egress sidecar) dials. All compose services set these to the broker service name (tcp://snapper-broker:7500 / :7501).

The broker container is a bare CLI without a ProcessLauncherService: it emits no processes.events.summary.* snapshots, so its only supervision is the docker healthcheck (TCP probes on both ports) + restart policy — port liveness IS the health of a single-thread proxy.

Strategies run in their own snapper-strategies container (snapper strategies-engine, PROCESS_AUTOSTART_PROFILE=strategy): role-STRATEGY processes start THREAD-mode through the SAME start_all_processes path they used in the backend — boot wallet/grant scope enforcement, watchdog restart budgets, and parking included. The backend sets STRATEGIES_EMBEDDED=false, which excludes strategies from its autostart, reports them as remotely managed, and hard-refuses manual local starts so one strategy can never run twice. The engine keeps the container a first-class bus citizen with a decision-only bus.ai_review_decision listener (the consult fast path; delegate-offline and caps fanout stay on the FastAPI server) and a 5s summary loop so the API's RemoteSummaryCache (15s TTL) always sees fresh coord-2 snapshots even with zero strategies running. Out-of-tree strategies register via STRATEGY_EXTRA_PACKAGES + PYTHONPATH (compose mounts ./proprietary read-only).

Alert delivery runs in its own snapper-notify container (snapper notify): a bare CLI (no PROCESS_AUTOSTART_PROFILE, healthcheck disabled, SQLAlchemy pool clamped to DB_POOL_SIZE=2 / DB_MAX_OVERFLOW=3). It subscribes to the source event topics its notify rules watch (orders.events. / plans.decisions. / system.heartbeats. / bus.portfolio_drift_episode for the default rule set, plus admin.scope_revoked / admin.user_deactivated), evaluates each rule to mint an AlertEventData, persists it SCD2 (alert_events plus one queued alert_deliveries row per active device), publishes an AlertEventData frame on alerts.{user_public_id}.{alert_type} for the web live-refresh bridge, and fans each delivery out to the target user's eligible APNs or configured Android FID registrations via the outbox-backed NotifySidecar (application/notify/sidecar.py), retrying server/throttled failures on its own 30s scheduler. The same process owns a guarded periodic recovery scanner for sentinel-current open portfolio-drift episodes. An eager pass at startup and later PORTFOLIO_DRIFT_RECOVERY_INTERVAL_SECONDS ticks rebuild the exact Stage 1 owner page when its source bus frame was lost. Bus and scan inserts take the same transaction-serialized (user_public_id, drift.<episode_public_id>)

claim, so only the winner reaches web and provider fanout. The scanner never writes episode, reconciliation, position, order, rebase, or trading state. Its apns_* settings are read from the database. Optional Android sender configuration comes from an explicit runtime JSON; it is disabled by default and does not change APNs payloads or registration semantics. See Android push.

NotifySidecar.start supervises separate receive and retry workers. A retry failure, or an unexpected worker exit before shutdown, ends the owning run even while receive or dispatch is blocked. Shutdown cancels and awaits the owned workers and stops the recovery scanner. An existing failure or cancellation remains primary if ordinary cleanup also fails; repeated stop calls after teardown do not repeat scanner cleanup. Ordinary failures propagate through the CLI to the compose restart: unless-stopped policy. The outbox drain on the next start recovers queued deliveries left behind, using the existing delivery and retry rules.

Before each APNs call, the repository allocates and commits the next attempt number while replacing the exact active queued delivery version. A competing writer that replaces the selected version before its conditional close prevents that allocation from admitting a send. Retry scheduling preserves the committed counter and requires the caller's attempt number to match the active row. A delayed result from an older attempt therefore cannot overwrite a newer attempt's retry schedule. These guards do not provide exclusive ownership of an in-flight send or exactly-once delivery.

Android outbox rows additionally retain immutable registration/configuration/binding IDs and an expiry fixed at allocation plus 24 hours. Fresh user, registration, configuration, scope and preference checks precede sending, including after OAuth acquisition. All FCM terminal outcomes compare the charged attempt count, so an old worker cannot cancel a newer attempt. Retries respect the original expiry and a three-attempt budget, including attempts charged before admission fails. FCM sends only opaque data hints; private content waits for foreground authentication. The persisted active/disabled configuration revision prevents stale activation after a disable, while retired configuration UUIDs can never reactivate.

DB_POOL_SIZE / DB_MAX_OVERFLOW clamp each publisher subprocess's SQLAlchemy pool (PostgreSQL only). get_repository caches one engine per process, so splitting publishers across processes otherwise multiplies the default pool (5 + 10) per process toward Postgres max_connections. Keep count(publishers) * (DB_POOL_SIZE + DB_MAX_OVERFLOW) + backend pool under the server's max_connections.

Deploy notes (see docker-compose.yml snapper-feed service):

- Bring up the backend (api) and feed (feed) together. A partial docker compose up snapper with the api profile but no feed container stops all market-data ingest, so deploy both atomically.

- The broker's bound endpoint is seeded into its DB process-config parameters by registry sync on a fresh database. With the dedicated snapper-broker container that row is inert: the launcher never starts the excluded process (ZMQ_BROKER_EMBEDDED=false) and the broker container reads only its CLI flags — no row surgery is needed on existing deployments. (Embedded-broker deployments keep the old rule: a stale tcp://127.0.0.1:7500 row must be re-synced or cleared.) In multi-instance deployments only coordinator instance 0 runs this replicated registry sync; non-zero instances skip it during FastAPI lifespan startup to avoid racing the same temporal Setting rows.
- **Egress routing (infrastructure/network/egress_*)** — Outbound exchange traffic is split into two classes by an EgressIdentity (egress_context.py) carried on a ContextVar: its traffic_class is public for market-data style traffic and private for authenticated executor traffic. Executor client lifecycles enter an egress_identity(..., traffic_class="private") scope (executors/base.py), so private traffic is direct-first and never traverses the public allow-list. Private idempotent REST reads and the next private WebSocket reconnect may use private_fallback_route_id when direct is unavailable; private REST mutations remain direct-only. Public/market-data traffic instead egresses through the per-venue snapper-egress SOCKS5/WireGuard routes, selected from the EgressPool by the exchange allow-list (egress_pool.py). The API process initializes its own process-local pool from egress_pool; API-side Kraken public REST reads may use that pool when configured. Feed-publisher public routing is gated separately: by default feed publishers dial exchanges directly. Set feed_egress_enabled=true and restart snapper-feed to enable snapper-egress routing for feed traffic, then confirm publisher connections use the configured snapper-egress SOCKS routes. See docs/snapper-egress.md for the sidecar deployment model. Each pool-bearing process publishes a read-only system.egress.snapshot frame on the heartbeat cadence; the API process subscribes to those frames and merges the latest per-container snapshots into GET /api/health/egress for observability only. The snapper-egress sidecar also publishes system.egress.transfer samples from wg show <iface> dump; the API joins those onto SOCKS5 route rows by listener port without changing route selection.

Multi-instance deployment (AI-review fanout dedup): multi-worker uvicorn is a supported production topology. The shared ZMQ XPUB-XSUB broker delivers every internal bus event to every FastAPI worker's AiReviewService.start_bus_listener; each subscribed topic has its own dedup story so connected clients see exactly one external WS frame per source event regardless of worker count:

- bus.delegate_offline is idempotent at the DB layer — the handler runs a per-row atomic_dispatch_fanout_with_audit CAS UPDATE, so only one worker wins per affected review row even when every worker receives the bus event.

- bus.ai_review_decision is process-local — the handler resolves an asyncio.Future from the per-worker registry populated by the worker that ran create_review for the strategy await loop. Other workers stash the event in a bounded pending-resolution cache (1024-entry hard cap, 30s TTL); cache memory across the cluster is bounded by N × 1024 entries even at peak.
- bus.caps_violation_after_ai_approve is gated by deterministic partitioning. handle_caps_violation_bus_message short-circuits on ShardOwnership.owns(msg.review_public_id); exactly one worker (the SHA-256 owner of the review row) re-publishes the external ai_reviews.*.caps_violation frame. Other workers observe the bus event, log a debug skip, and return False. instance_count == 1 (default deployment) makes ShardOwnership.owns return True unconditionally so behaviour is byte-identical to single-worker.

The same partitioning primitive backs the Trade Runtime shard ownership; AI-review reuses it via the set_shard_ownership(ShardOwnership(coordinator_instance_id, coordinator_instance_count)) injection seam wired by the FastAPI lifespan after the publisher seam and before the bus listener starts.

Operational scaling: raise SNAPPER_COORDINATOR_INSTANCE_COUNT to N and start N FastAPI workers each with a distinct SNAPPER_COORDINATOR_INSTANCE_ID in [0, N) (env vars only — snapper server does not expose per-worker CLI flags; the --instance-id / --instance-count CLI flags exist on snapper trade-zmq for the Trade Runtime coordinator partitioning, not on the FastAPI server). Each worker subscribes to every bus topic; only the SHA-256 owner runs the caps-violation external fanout.

Execution Plans

The Execution Plans framework provides a unified control plane above the existing TradeCommand spine. Manual orders, brackets, and trailing stops use ExecutionPlan instances evaluated by pluggable PlanEvaluator classes running inside PlanExecutorService. Strategy signals also have a direct trade-runtime path that writes TradeCommand rows without creating a plan.

Tables: execution_plans (plan state + lifecycle), execution_plan_checkpoints (high-churn evaluator state), execution_plan_decisions (tiered decision log), execution_plan_decision_outbox (durable retry state for the plans.decisions ZMQ fanout — written in the same transaction as each decision row the plan executor logs, drained with capped exponential backoff after publish failures; REST-surface decision rows are logged without an outbox entry), instrument_order_capabilities (capability matrix), venue_fee_schedules (fee tiers), position_cycles (one open→close lifetime per shard, brackets attach by position_cycle_public_id).

Plan types: manual_once (shipped), bracket (shipped — attaches to a position_cycles row), trailing_stop (shipped — stateful ratcheting stop with checkpoint persistence, separate /api/trailing-stops route module), passive_mm, peg, and scheduler are reserved vocabulary without registered evaluators in application/plans/service.py; they are not executable plan types.

Manual order create flow: POST /api/orders creates a manual_once plan (pending), stamps child_client_order_id and native_instrument into plan.params, then inserts a create TradeCommand for the outbox dispatcher and transitions the plan to active. The command row stores the CORE order-type vocabulary (market/limit/stop/stop_limit) plus the stop_price trigger for stop types; translation to the venue wire vocabulary (stop-loss/stop-loss-limit) happens only at the executor's venue boundary. Cancel/flatten readers resolve the order type through core_order_type_from_plan_params, which still normalizes the legacy venue_order_type param carried by older plans. The OutboxDispatcher inside TraderCoordinator publishes the command as OrderRequestData on the orders.commands.{ex}.{instr}.submit topic.

Fill propagation: PlanExecutorService subscribes to orders.events. and market. on the broker XPUB and consumes every fill. Cumulative ExecutionData.size is compared against plan.filled_quantity with a small epsilon so duplicate or stale frames are dropped. Partial fills update filled_quantity and keep the plan active; full fills transition the plan to completed with completed_at stamped via SCD2.

Cancel flow: POST /api/orders/{plan_public_id}/cancel (plan id path) or POST /api/orders/by-client-order-id/{client_order_id}/cancel (UI convenience route) resolves the plan, verifies wallet scope, transitions the plan to cancel_requested, hydrates exchange_order_id from the active orders row, and inserts a cancel TradeCommand. On insert failure the plan is rolled back to failed and the route returns HTTP 500 — this is terminal (no retry). PlanExecutorService._recover_plans on the next service restart re-emits cancel commands only for plans still in cancel_requested state (deduplicated via has_pending_cancel_command); failed plans are not retried. The outbox dispatcher branches on command_type and re-hydrates exchange_order_id one more time at publish time so a late venue ACK is picked up, then publishes OrderCancelData on the orders.commands.{ex}.{instr}.cancel topic.

Listen loop resilience: per-message exceptions (parse failures, handler errors) are caught inside the loop so a single bad frame cannot silently stop the service. Transient recv_multipart errors retry with a 100 ms backoff. asyncio.CancelledError still unwinds the loop for graceful shutdown. After _setup_subscriber() the service sleeps 500 ms for ZMQ XPUB/XSUB slow-joiner stabilization before recovery to avoid losing terminal events triggered by stranded-cancel re-emits.

Position cycles: the position_cycles table represents one flat→non-flat→flat lifetime per shard via the standard TemporalMixin SCD2 envelope. A partial unique index uq_pc_shard_open_active enforces at most one open cycle per shard_key at any instant. Five repository methods cover the lifecycle: insert_position_cycle, close_position_cycle

(SCD2 close-and-insert), get_open_position_cycle (lookup by shard_key because paper shards embed strategy_tag), flip_position_cycle (TOCTOU-hardened atomic close+open with shard_key match assertion), and update_position_cycle_max_qty (monotonic — silent no-op when the new peak does not strictly exceed the existing one). The repository methods are called from the trader (not from TradeService, which stays sync and pure); ShardState carries trader-owned cache fields active_cycle_public_id and active_cycle_max_qty.

The trader's _sync_fill_to_trade_service samples old_qty before apply_venue_event and new_qty after, then delegates to _sync_position_cycle_on_fill. That helper classifies the transition via TradeService._detect_cycle_transition (returns Literal["open", "close", "flip", "scale_up"] | None with a 1e-12 epsilon to match the existing zero-snap) and issues the matching repository call. Four guards apply: degraded-identity fail-closed when engine.wallet_public_id is falsy, idempotency on open via a get_open_position_cycle pre-check, async instrument_public_id resolution via get_instrument_public_id_by_symbol, and a symmetric DB fallback on close/flip/scale_up paths so a degraded restart cannot reuse a stale cycle row from a prior life of the shard. When a flip's instrument resolution fails, the trader degrades to close-only — the old cycle is closed cleanly, the new leg is left uncovered with an audit-gap warning, and the cache is cleared so subsequent fail-soft paths handle themselves correctly.

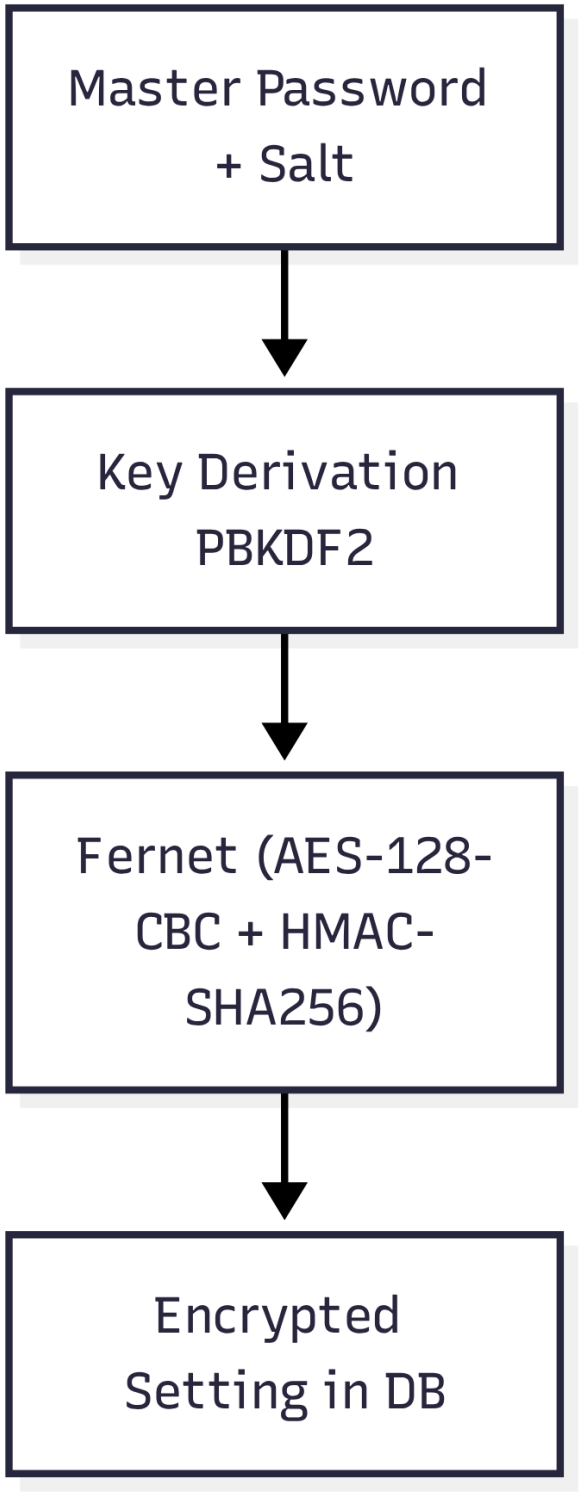
_recover_engine_state runs _reconcile_position_cycles as a fourth recovery pass, after engine state is rebuilt and before the trading loop starts. It iterates self.engines (engines, not TradeService, because full replay rebuilds engine.position_qty but not the projection) and handles four cases: recovered flat with a stale open row → close the row; recovered non-flat with a matching-direction open row → hydrate the cache and bump max_qty if downtime scaled-up beyond the stored peak; recovered non-flat with an opposite-direction row → atomic flip via flip_position_cycle (or degrade to close-only on unresolved instrument); recovered non-flat with no row → bootstrap a synthetic cycle. Brackets attach to position_cycle_public_id, not to an order, so a flat reopen does not inherit stale stop levels. Orphan cycles (open rows without a matching engine) can be detected via GET /api/position-cycles/open and closed individually with POST /api/position-cycles/close-orphan or in bulk with POST /api/position-cycles/sweep-orphans.

max_qty is per-cycle, not lifetime. Each position_cycles row tracks the peak absolute quantity reached within its own opened_at → closed_at window. A new cycle on the same shard starts a fresh accumulator at abs(opening_qty); it does not inherit or max against prior cycles' peaks. This matches the bracket-evaluator contract: a bracket attaching to cycle C1 is sized against C1's peak, not against all-time peak. A future UI surfacing max_qty must scope to the individual cycle row; summing or max-ing across cycles would conflate independent trading episodes.

Security

Settings Encryption

Sensitive data (API keys) are encrypted in the database:



Backtesting

The backtesting subsystem runs strategy simulations against historical candle data. It supports both direct repository-backed candle streaming and ZMQ replay through the same strategy processing contract.

Architecture:

- **BacktestConfig** (Pydantic): Strategy, instruments, date range, fill model
- **DirectDbEngine**: Candle-driven simulation loop with time-batched processing
- **ZmqReplayEngine**: Replay mode that publishes historical candles through ReplayPublisher on the same topic shape used by live feeds
- **ResultCollector**: In-memory artifact buffer (signals, trades, equity)
- **BacktestMetrics**: Pure-math metrics computation (Sharpe, Sortino, CAGR, etc.)
- **BacktestRunnerProcess**: ONE_SHOT RegisterableProcess orchestrating lifecycle
- **BacktestRepository**: Separate repo sharing session_factory, SCD2 status transitions

Lifecycle: pending -> running -> completed | failed | cancelled

Boot reconciliation: Orphaned runs (pending/running/cancel_requested) are swept to failed at server startup via reconcile_stale_runs(). This runs only on the normal startup path (skipped in SERVER_API_ONLY mode).

Storage: 7 bitemporal tables (BacktestRun, BacktestEvent, BacktestSignal, BacktestTrade, BacktestEquityPoint, BacktestResult, BacktestComparison) all with TemporalMixin.

Authentication

- JWT tokens via HTTP-only cookies (access: 15min, refresh: 7 days)
- CSRF tokens for mutating requests
- WebSocket tokens (one-time)
- Bcrypt password hashing

Kraken FCM market data (TradFi)

Kraken's FCM platform hosts index-futures, energy, metals, and yield contracts (collective name: "TradFi" in this project). Unlike crypto spot + perpetuals, FCM has **no public order API** — Snapper treats these instruments as observation-only.

Data sources

- REST instrument metadata: iapi.kraken.com/api/internal/markets/all/futures-contracts. Reverse-engineered from pro.kraken.com (requires Origin: https://pro.kraken.com + Referer: https://pro.kraken.com/ headers). Powers the kraken_equities_symbol_updater process.
- REST historical candles: iapi.kraken.com/api/internal/markets/{ws_symbol}/ticker/history. Same Origin/Referer requirement. Drives KrakenEquitiesExchangeClient.get_ohlcv() and the kraken_equities_aggregates_backfill process. Intervals {1, 5, 15, 30, 60, 1440} minutes accepted.
- WebSocket ticks + trades: wss://ws-equities.kraken.com by default (Kraken Spot WS v2 protocol with asset_class: futures_contract). The public feed is delayed ~10 minutes per FCM policy. When kraken_equities_realtime_ws_enabled=true, the publisher first mints a Kraken Spot WebSockets token from the configured kraken_equities_realtime_wallet_public_id override (a wallet public id or a label:<wallet-label> pin resolved to the single matching live wallet), or auto-selects the first active Kraken Spot api_key_secret wallet credential when the setting is empty, and connects to wss://ws-equities-auth.kraken.com/?f; subscribe params receive the token in memory only. Mint, credential, or autolookup failure falls back to the public delayed URL. Under feed_egress_enabled=true, the token mint is classified as public kraken_equities market-data traffic so it uses the same egress-pool route as the authenticated Equities WebSocket, preserving source-IP consistency for tokens that may be IP-bound. The outer envelope's delayed flag propagates onto every TickData.is_delayed value so ZMQ subscribers can distinguish delayed and realtime frames.

Capability model

Each TradFi symbol is seeded with a SymbolExchangeCapability(exchange="kraken_equities", can_trade=False, can_market_data=True) row. Every order-entry REST route (/api/orders, /api/execution-plans, /api/trailing-stops) calls snapper.server._capability_guard.require_tradable before emitting a TradeCommand — market-data-only submits fail with HTTP 422 error_code=instrument_market_data_only. The AST meta-test at tests/meta/test_order_entry_capability_guard.py enforces that every enumerated submit handler imports and calls the guard.

Frontend UX

GET /api/exchanges/{exchange}/instruments/detail exposes capability-aware rows (InstrumentDetailData — symbol + can_trade + can_market_data + instrument_kind + expiry_at). MarketData.tsx renders a "Market-data only" badge next to the page title when the currently-selected instrument is observation-only and marks every can_trade=false row in the instrument dropdown with the same pill. NewOrderModal.tsx consumes the same endpoint, shows an amber inline notice, suffixes option labels with — market-data only, and disables the submit button. Submit failures branch on APIError.details.error_code via errorMessages.ts.

Cross-asset strategy pattern (informational)

Strategies may subscribe TradFi candles via the normal market.kraken_equities.{symbol}.candles.{timeframe} topic and emit signals whose instrument targets a can_trade=True execution instrument (crypto spot, xStocks). See src/snapper/strategies/examples/tradfi_observe_crypto_execute.py for an illustrative EMA-crossover implementation + activation instructions in the module docstring. The backtest engines process multi-feed batches and route simulated fills to the signalled execution instrument. BacktestConfig.target_execution_exchange selects the fill venue when it differs from the source feed, and both direct-DB and ZMQ replay modes build topics for every configured exchange/instrument feed.

Operational runbook

See docs/operations.md "Kraken Equities (TradFi) market data" for enable-the-feed steps (SQL DB Setting toggles), backfill invocation, quarterly rotation cadence, and troubleshooting (iapi reachability checks, WS disconnect loops).

Backtesting

Snapper ships two backtest execution engines, both producing the same artifacts (signals, trades, equity points) so a strategy author can pick either path with confidence.

Authorization

Authorization uses the token's effective permissions from Snapper's 34-permission catalogue. The frontend opens the Backtests resource with read:backtests and independently hides and rechecks each mutation.

Surface	Required permission	Current named permission sets
List strategy classes, runs, artifacts, and comparisons	read:backtests	ai_reviewer, ai_delegate, viewer, operator, admin
Create, cancel, or rerun a backtest	manage:backtests	operator, admin
Create a comparison	create:backtest_comparisons	ai_reviewer, ai_delegate, operator, admin
Subscribe to backtest progress	read:backtests	ai_reviewer, ai_delegate, viewer, operator, admin

The current viewer set therefore has the full operator read surface but no backtest mutation. Wallet-bound REST endpoints revalidate the token's active wallet on every request. Reads accept operator scope grants plus personal wallet_user_read_grants; mutations require operator scope grants. A personal read grant therefore permits inspection without authorizing a run or comparison write. Neither named set receives global wallet visibility.

Execution modes

The execution_mode field on BacktestConfig selects which engine the runner instantiates:

- direct_db — DirectDbEngine reads candles directly from the repository and feeds them into the strategy in-process. Lowest overhead; recommended for fast iteration.
- zmq_replay — ZmqReplayEngine spins up a private XPUB/XSUB broker on OS-assigned local ports, attaches the strategy as a SUB, and publishes candles via a ReplayPublisher. Exercises the full live candle publication/subscription path on the private broker. It does not exercise production order dispatch, venue execution, or deployment wiring.

Both engines call the same process_time_batch helper for fill simulation, signal recording, and equity sampling, so artifact parity is structural rather than aspirational.

Choosing between modes

Concern	direct_db	zmq_replay
Speed	fastest (no IPC)	slower (per-candle ZMQ hop)
Fidelity	bypasses message bus	mirrors live wire path
Concurrency	at most one active run (global uq_bt_single_running invariant — see below)	at most one active run (same DB-enforced invariant)
Cancellation checks	between batches, throttled by cancel_poll_ms	per received candle, throttled by cancel_poll_ms

At-most-one-running invariant

Migration 0001_init includes a partial unique index uq_bt_single_running on backtest_runs(status) filtered to status='running' AND known_to=KNOWN_TO_MAX. This is a declarative DB guarantee — a second runner trying to transition pending → running receives IntegrityError at COMMIT, the runner catches it via is_single_running_conflict and transitions to failed with a clear error message. No engine, no broker, no ports consumed for the loser.

The constraint is dialect-aware (PostgreSQL via asyncpg + SQLite via aiosqlite); see snapper.data.backtest_conflict for detection details.

Cancellation

CancelProbe polls backtest_runs.status between time batches (Direct-DB) or per processed candle (ZMQ replay). Throttled by config.cancel_poll_ms (default 500 ms), each DB read uses asyncio.wait_for with probe_timeout_s (default 1.0 s). A timed-out probe logs a warning and the next safe point retries. Cancellation is cooperative: strategy work, database-driver cancellation cleanup, and terminal-status persistence can extend latency beyond those settings.

End-to-end cancel SLA target: ≤ 5 s from the API call POST /api/backtests/{run_id}/cancel (which sets status to cancel_requested) to terminal status cancelled.

start_date warmup gating

Candles before config.start_date feed strategy indicators (so SMA/MACD warm up properly) but do **not** generate fills, signals, or equity points. The process_time_batch helper enforces this with a single if batch[0].open_at < config.start_date: return guard before fill simulation.

Both engines honour the gate identically; the parity test tests/integration/test_backtest_hardening.py::TestHardeningIntegration::test_warmup_gating_parity_between_engines asserts equality of the per-signal (signal_time, signal_type, instrument) tuples and per-point (point_time, equity) tuples between the two engines.

Signal and fill artifacts

process_time_batch is the single execution/fill path for both engines. For every timestamp batch it updates latest_closes for all candles first, feeds each candle through the strategy, drops pre-start_date signals, then records kept signals/trades and one equity point for the timestamp.

Each kept signal receives a fresh UUID7 public_id before any fill is simulated. When a trade is produced, backtest_trades.signal_public_id carries that same value so Direct-DB and ZMQ replay artifacts are FK-linkable in parity tests. The signal row records:

- signal_time from the candle batch timestamp (open_at);
- signal_type from StrategySignal.side;
- instrument from StrategySignal.instrument;
- price from the resolved target close when fill attribution succeeds, or from StrategySignal.price on the missing-target-close fallback;
- timestamp from the run's snapshot_as_of bus-time anchor.

Live-execution fidelity boundary

Both backtest engines share the same simulator, but that simulator does not implement the live coordinator's absolute-position-target contract. A BUY spends a fraction of remaining cash; a SELL closes an existing position and cannot open a short from flat. A zero-strength BUY produces no fill. Repeated BUY targets can therefore buy repeatedly in a backtest even though the live engine would treat an unchanged target as a no-op. Multi-leg signals are simulated independently, without the paired-execution guard. Direct-DB/ZMQ artifact parity is not evidence of live sizing, shorting, or compensation parity.

The only current fill model is market. simulate_market_fill executes at the relevant close price adjusted by slippage_bps; commission is charged in basis points. Buy size is signal_strength * cash / cost_per_unit (default strength 1.0), and a sell flattens the current position quantity. Invalid close prices, zero/negative buy strength, no cash, or a sell with no position return None; the signal is still recorded, but no trade row is written.

When a fill exists, the trade row records:

- executed_at = fill.fill_at, which is the candle open_at timestamp passed into the fill model;
- instrument, side, quantity, price, fee, and pnl from the simulated fill (pnl is populated on sells from realized PnL delta);
- position_after from the portfolio after the fill;
- signal_public_id linking back to the triggering signal;
- timestamp from the same snapshot_as_of bus-time anchor.

ZMQ replay implementation notes

The replay path adds a few primitives the live system does not need:

- **Echo-ack handshake**: the publisher sends a sentinel WARMUP_PUBLIC_ID candle on every expected topic each round; the strategy mixin's _listen_loop ACKs each topic into state.acked_topics and only sets state.subscriber_ready when the set equals state.expected_topics. Up to WARMUP_MAX_RETRIES=5 rounds of WARMUP_READY_TIMEOUT_S=1.0 s each; failure raises BacktestReadinessTimeoutError.
- **Drain coordinator**: counts published vs processed candles. After the publisher streams the last candle it calls mark_done_publishing() and waits on drained.wait() bounded by DRAIN_TIMEOUT_S=10.0 s. Failure raises BacktestDrainTimeoutError with both counters in the message so an operator can tell publisher- stall from strategy-death.
- **Bounded cleanup**: the engine cancels publisher and listen tasks then asyncio.wait(timeout=2.0) — tasks that refuse to terminate are logged as leaks rather than silently kept alive.
- **Shielded DB writes**: the runner wraps every terminal-status DB write (cancelled, failed, fail-event insert) in asyncio.shield so a second cancel arriving mid-write does not interrupt the write coroutine.

Advanced metrics, live progress, comparison

The backtest surface ships three interlocking capabilities:

Advanced metrics

Eight additional metric columns on backtest_results — five promoted from the extra_metrics JSON blob (sortino_ratio, cagr, calmar_ratio, expectancy, avg_trade_pnl) plus three net-new metrics computed in snapper.application.backtest.metrics:

- **max_drawdown_duration_seconds** — longest peak-to-recovery window; unrecovered runs use the final suffix from last peak to end-of-curve. Degenerate input (fewer than two points, flat curve, monotonically non-decreasing (no drawdown ever observed)) returns None + a metric_warning event.
- **exposure_ratio** — fraction of total run duration during which the portfolio held a non-zero position; leading-edge interval attribution. Zero-trade runs with non-zero duration return 0.0 (not degenerate).
- **turnover_ratio** — total notional traded divided by mean equity. Zero-trade returns 0.0; non-positive mean equity returns None + warning.

Read-side fallback at GET /api/backtests/{id} collapses older extra_metrics JSON into the typed slots using explicit is not None coalescing (never Python truthiness — preserves legitimate 0.0 values). The response strips the five promoted names from the emitted extra_metrics so mixed-vintage rows never surface the same metric twice.

Live progress (4-segment WS topic family)

Runner emits BacktestProgressData events on backtest.{wallet_public_id}.{run_public_id}.{event} where event ∈ {started, progress, milestone, completed, failed, cancelled}. The existing ZMQ→WS bridge forwards the envelope to every subscribed client.

- BacktestProgressEvent is the canonical Literal source of truth in snapper.messaging.schemas.data; topic validator, emitter, and frontend handlers all derive from it.
- BacktestProgressEmitter throttles progress at 250 ms, emits milestone events exactly once per 25/50/75 pct bucket (bypassing the throttle so milestones + progress at the same step do not collide), and fires started / terminal events once. Degrades gracefully when total_candles=None by disabling milestones and pinning progress_pct=0.0.
- Per-subscription wallet scope: read:backtests admits the topic category, then effective impersonate:operator permits subscription to any wallet prefix. Every other caller may subscribe only to prefixes matching its active_wallet_public_id. Foreign-wallet or bare backtest. subscriptions are denied at the handler before the bridge is touched.
- BacktestProgressData.milestone carries a @model_validator enforcing the cross-field invariant milestone is not None iff event == 'milestone' so a malformed payload cannot reach the frontend chip logic.

Config hash for auto-pair

Every new run persists a config_hash via compute_fingerprint(config, for_pairing=True) — SHA-256 over the 10 pairing-stable fields (strategy_class, instruments, start_date, end_date, initial_balance, strategy_params, timeframe, fill_model, slippage_bps, commission_bps), plus target_execution_exchange when it is set on the config (see "Cross-asset execution" → "Fingerprint + pairing" below — default-None runs keep their exact prior hash). execution_mode, snapshot_as_of, warmup_bars, and buffer_size are explicitly excluded so Direct-DB and ZMQ replay runs on the same config share a hash.

GET /api/backtests?config_hash={hash} returns sibling runs for the auto-pair UI; the repository query is wallet-scoped and indexed on (wallet_public_id, config_hash, timestamp).

Comparison

POST /api/backtests/compare creates (or returns existing) a comparison row and requires create:backtest_comparisons. The current ai_reviewer, ai_delegate, operator, and admin sets contain that permission; viewer and ai_researcher do not. This preserves historical comparison creation for review and delegate principals without widening the read-only viewer. GET /api/backtests/compare/{id} requires read:backtests and returns the comparison metadata plus the diff **recomputed from current artifact rows** so metric-schema changes never stale a persisted diff. The diff surfaces four shapes:

- metrics_diff — per-name {run_a, run_b, delta, pct} with explicit is not None precedence.
- equity_overlay — full-outer-joined equity samples on point_time with nullable per-leg fields.
- trades_diff — multiset matching on (instrument, executed_at, side, quantized_qty, quantized_price) at 1e-8 precision so IEEE drift collapses but real ticks stay distinct. Common entries carry pnl_a/pnl_b/pnl_delta.
- signals_diff — multiset on (instrument, signal_time, signal_type).

Route ordering: compare endpoints are registered **before** /{run_id} so /compare is not captured as id="compare". Pair is normalised to lexical (min, max) before insert. Duplicate submit is idempotent via SELECT-then-INSERT with an IntegrityError rollback+re-SELECT race recovery path.

No-active-wallet fail-closed

Every wallet-bound backtest read endpoint (list, detail, trades, signals, events, equity, and comparison reads) returns **400 no active wallet selected** when principal.active_wallet_public_id is None. WS subscribes respond with a subscription_success frame whose status is denied (or partial when some requested

topics succeeded), with the offending topics surfaced in denied_topics. The picker triggers a selectWalletAndRefresh on change that mints a new JWT carrying the chosen wallet claim before swapping the client scope, so REST and WS both authorise against the same wallet after the picker moves.

GET /api/backtests/strategy-classes is the exception: it requires read:backtests but is a global registry read and does not require an active wallet.

Cross-asset execution

Cross-asset strategies observe market data on one venue / instrument and emit signals that execute on a *different* venue / instrument (for example TradFiObserveCryptoExecute observes MNQM6-CME candles on kraken_equities and emits BUY/SELL signals on BTC-USD / kraken). The backtest engine supports this through a single config field.

TradFiObserveCryptoExecute is an illustrative, non-runnable example: it is **not** auto-registered, so BacktestConfig.validate_strategy_class rejects the config below as written. To run it, first copy the example module to the strategies package root and decorate it with @register_strategy, or substitute a registered strategy class (RSIReversion, MACDCrossover, or CointegrationPairs). The current registry is queryable at GET /api/backtests/strategy-classes, which returns the sorted StrategyFactory keys accepted as strategy_class values (see api.md):

```
BacktestConfig(
    strategy_class="TradFiObserveCryptoExecute",
    instruments={
        "kraken_equities": ["MNQM6-CME"],
        "kraken": ["BTC-USD"],
    },
    target_execution_exchange="kraken",
    start_date=..., end_date=...,
    wallet_public_id=...,
    initial_balance=10_000.0,
    strategy_params={"fast_period": 12, "slow_period": 26, "min_candles": 30},
)
```

Attribution semantics

When target_execution_exchange is not None, the batch processor substitutes at simulated-fill time:

- target_exchange = str(config.target_execution_exchange) — carries the *venue label* only. It is NOT validated against the feed populating latest_closes[target_instrument]; the caller is responsible for configuring instruments so the target symbol's feed matches the target venue.
- target_instrument = signal.instrument — comes directly from the strategy's StrategySignal, unchanged since pre-cross-asset.

- target_close = latest_closes[target_instrument] — the *price* at which the simulated fill executes. The source feed's close price is no longer used on cross-asset runs.

When target_execution_exchange is None (default), the simulator passes event.exchange into the in-memory fill and uses signal.instrument plus latest_closes[signal.instrument] for execution. Persisted backtest trade and signal artifacts expose instrument and price only; source exchange and optional target_execution_exchange live on the run row, not per trade/signal. The single-leg emitters (rsi.py, macd.py) always set signal.instrument == event.instrument, so for those strategies the in-memory fill's exchange / instrument / price are byte-identical with the pre-v1.2 path. CointegrationPairs does not follow that convention: its on_candle returns a [primary, hedge] pair whose hedge leg targets the *partner* instrument, so that leg is attributed to event.exchange plus the partner symbol and fills at latest_closes[partner] (see "Multi-leg signal groups" below).

Multi-leg signal groups

A strategy callback may return a list[StrategySignal] (a leg group) instead of a single signal. The backtest path handles groups as follows:

- **Fail-closed group preflight** — _handle_candle_data runs the whole group through BaseStrategy._normalize_signal_group before returning anything: every element must be a StrategySignal, no two legs may share an instrument, and every leg must map to a configured output topic. A malformed group raises instead of emitting, so a backtest can never record half a spread.
- **Per-leg fan-out** — process_time_batch pairs each leg with the candle event that produced the group and processes the legs independently: each leg gets its own signal_public_id, fill simulation, and signal/trade rows.
- **Hedge-leg pricing** — the hedge leg fills at latest_closes[partner], the partner instrument's latest close from its own feed, not the observing candle's close. Because process_time_batch updates latest_closes for every candle in the timestamp batch before feeding the strategy, both legs price off same-timestamp data when both feeds emit. A partner symbol absent from latest_closes falls into the missing-target-close policy below.

CointegrationPairs additionally refuses to enter a one-sided position at the source: _build_paired_entry returns None (no signals at all) when the partner leg has no buffered price yet.

Missing-target-close policy

If signal.instrument has no entry in latest_closes (typically during the warmup overlap window where the strategy fires before the target feed has primed), the engine:

1. Skips simulate_market_fill + the trade row (no fill attempted).

2. Increments collector.cross_asset_blocked_fills by one, logged at DEBUG with reason='missing_target_close'.
3. Still records the signal row with price = float(signal.price) (source-close fallback) so downstream analytics see the strategy's intent.

At run end the counter surfaces through BacktestResultInsertRow.extra_metrics["cross_asset_blocked_fills"] *only when positive*. Single-feed runs keep extra_metrics == {} byte-identical with pre-cross-asset behaviour.

Scope limits

Two explicit non-goals in the current implementation:

- **Same-symbol multi-venue** — PortfolioTracker.positions is keyed by instrument only, so running BTC-USD on Kraken Spot + Kraken Futures simultaneously in one backtest is NOT supported. Cross-asset strategies in scope use distinct symbols across feeds.
- **Multi-target cross-asset** — a single target_execution_exchange per run means one strategy cannot emit signals for different target venues inside the same backtest.

Supported: cross-asset via REST

POST /api/backtests accepts an optional target_execution_exchange field on BacktestCreateBody. When set, simulated fills are attributed to that order-capable venue while candles still feed from exchange. When unset, the run stays single-exchange and byte-identical to pre-cross-asset behaviour. The same field round-trips through DB persistence + the rerun endpoint, and surfaces on BacktestRunData for frontend display. Allowed target values: paper / kraken / kraken_futures / walutomat.

Fingerprint + pairing

compute_fingerprint includes target_execution_exchange in the payload **only when non-None**, so default-None runs keep their exact pre-cross-asset hash in both the default and for_pairing=True paths. This preserves dedup cache validity and the auto-pair UI grouping logic in the _resolve_auto_pair resolver in backtest_routes.py. Explicit cross-asset runs (field set to a concrete venue like "kraken") generate distinct fingerprints so they never collide with single-venue baselines.

CLI

Snapper provides a command-line interface built on the Typer framework. All commands are available through the snapper command in an activated virtual environment, or as .venv/bin/python -m snapper from this checkout.

Usage

```
snapper [COMMAND] [OPTIONS]
```

Help for a specific command:

```
snapper [COMMAND] --help
```

Server and Infrastructure

server

Starts the FastAPI server with the web dashboard.

```
snapper server [OPTIONS]
```

Options:

Option	Type	Default	Description
--host	string	from env	Listen address
--port	int	from env	Server port
--reload / --no-reload	bool	from env (SERVER_RELOAD)	Auto-reload for development; explicit flag overrides the env-var setting

Environment:

Set SERVER_API_ONLY=true to skip process autostart while keeping the ZMQ-WebSocket bridge alive (useful when the engine runs as a separate process). Multi-worker uvicorn (multiple FastAPI processes sharing a broker) is supported — see the AI-review fanout dedup contract in docs/architecture.md (Deployment Modes); set SNAPPER_COORDINATOR_INSTANCE_ID + SNAPPER_COORDINATOR_INSTANCE_COUNT per worker (env vars only — snapper server does not expose per-worker CLI flags; the --

instance-id / --instance-count flags in the table below are on snapper trade-zmq) to enable the shared partitioning that prevents duplicate caps-violation WS frames under N>1.

Examples:

```
# Start with default settings (all-in-one dev mode)
snapper server

# Start on all interfaces
snapper server --host 0.0.0.0 --port 8000

# Development mode with auto-reload
snapper server --reload

# API-only mode (engine runs separately)
SERVER_API_ONLY=true snapper server
```

broker

Starts the ZeroMQ XPUB/XSUB broker for message routing.

```
snapper broker [OPTIONS]
```

Options:

Option	Type	Default	Description
--xsub	string	from env	XSUB endpoint (publishers)
--xpub	string	from env	XPUB endpoint (subscribers)

Example:

```
snapper broker --xsub tcp://127.0.0.1:7500 --xpub tcp://127.0.0.1:7501
```

trade-zmq

Starts the central trade runtime coordinator.

The process still runs as the trade-zmq command, but after the trade runtime redesign it also hosts TradeService, the outbox dispatcher, and the reconciliation loops. It writes TradeCommand rows, consumes orders.events.* to keep projections in sync, and persists checkpoints for recovery.

```
snapper trade-zmq [OPTIONS]
```

Options:

Option	Type	Default	Description
--signal-topics	string	signals.	Signal topics to subscribe
--instance-id	int	0 (or $SNAPPER_COORDINATOR_INSTANCE_ID)	Multi-instance partitioning: zero-based id for this coordinator in a multi-instance deployment. CLI flag > env var > default.
--instance-count	int	1 (or $SNAPPER_COORDINATOR_INSTANCE_COUNT)	Multi-instance partitioning: total coordinator instance count across the deployment. Must match across all coordinators.

Example:

```
snapper trade-zmq --signal-topics "signals.paper.BTC-USD.rsi_btc_1h,signals.kraken.BTC-USD.live"
```

Multi-instance deployment:

```
# Coordinator 0 of 2
snapper trade-zmq --instance-id 0 --instance-count 2

# Coordinator 1 of 2 (separate process, same DB + broker)
snapper trade-zmq --instance-id 1 --instance-count 2
```

Each instance owns ~1/N of the shard_keys via deterministic SHA-256 hashing. Default --instance-count 1 is byte-identical to single-coordinator behavior (every shard owned by the single coordinator).

Multi-instance deployment requires PostgreSQL. With --instance-count greater than 1 on a SQLite backend the coordinator fails fast at startup with a ValueError: SQLite compiles SELECT ... FOR UPDATE to a plain SELECT, so cross-instance row claims cannot serialize. A single-instance SQLite coordinator is allowed but logs a startup warning about the same locking caveat.

See docs/operations.md for systemd template unit + scale-up/down/crash-recovery runbooks.

executor

Starts the standalone order execution service for a selected exchange. Process-managed deployments use per-wallet executor instances instead: bare executor_<exchange> process configs are templates, and the launcher expands active wallet_credentials rows into runnable executor_<exchange>_w<wallet_short> instances. Start/stop the generated per-wallet instance names in the process UI/API; starting a bare executor template is rejected.

```
snapper executor [OPTIONS]
```

Options:

Option	Type	Default	Description
-e, --exchange	string	kraken	Exchange (kraken, walutomat)

wallet_short is the last 12 lowercase hex characters of the wallet UUID7. Each per-wallet instance subscribes to the same exchange command prefix as the template, then filters by wallet_public_id before loading credentials or placing orders.

Examples:

```
# Kraken executor
snapper executor -e kraken

# Walutomat executor
snapper executor --exchange walutomat
```

feed

Starts the direct Kraken market data publisher helper via WebSocket. For the dedicated feed-container topology, use snapper feed-engine instead; it starts every enabled market-data publisher from the process registry as a supervised subprocess.

```
snapper feed [OPTIONS]
```

Options:

Option	Type	Default	Description
--symbols	string	BTC-USD	Comma-separated native symbols passed to the Kraken publisher. Symbols must be in native dash format (e.g. BTC-USD); slash-format WebSocket symbols are skipped as unknown.

Example:

```
snapper feed --symbols "BTC-USD,ETH-USD"
```

feed-engine

Runs the dedicated feed-container entrypoint. It syncs the process registry, starts enabled market-data publishers as OS subprocesses, and exits non-zero if any supervised publisher crashes so the orchestrator restarts the feed container. It does not start a broker; publisher subprocesses connect to the backend's configured ZMQ_BROKER_* endpoints. Per-process RSS/CPU summary events are emitted best-effort when the backend broker is reachable, but metrics-publisher setup failure does not abort feed startup. Set PROCESS_AUTOSTART_PROFILE=feed to enable desired-state reconciliation and the remote process-command listener; the command still starts publishers without that profile, but disables those control loops.

```
snapper feed-engine
```

strategies-engine

Runs the dedicated strategies-container entrypoint. It syncs the process registry (including STRATEGY_EXTRA_PACKAGES mounts), then starts every role-STRATEGY process THREAD-mode inside this container exactly as they ran embedded in the backend (boot wallet/grant scope enforcement, watchdog, parking). Set STRATEGIES_EMBEDDED=false on the backend so it stops owning strategies, and run this container with PROCESS_AUTOSTART_PROFILE=strategy (required for the reconcile loop that converges running processes to the DB desired-state). It does not start a broker; it connects to the backend's configured ZMQ_BROKER_* endpoints. Exits non-zero only when a CORE process fails to start — strategy crashes are watchdog-managed and, after the restart budget, parked (visible via the coordinator summary frames), never container-fatal.

```
snapper strategies-engine
```

delegate-engine

Runs registered generic delegate workloads as managed subprocesses. Requires PROCESS_AUTOSTART_PROFILE=delegate and a loadable extra package that registers the runner. This command provides process lifecycle management; it does not implement model requests, consult handling, or egress itself. The optional Compose delegate profile instead starts the separate runner-only PID1 and does not invoke this coordinator.

```
snapper delegate-engine
```

zmq-logger

Monitors and logs ZMQ message traffic.

```
snapper zmq-logger [OPTIONS]
```

Options:

Option	Type	Default	Description
--payload/--no-payload	bool	false	Log full payload
--file/--no-file	bool	true	Write to audit file
--audit-file	string	data/zmq_audit.jsonl	Audit file path
--max-length	int	200	Max payload preview length

Example:

```
snapper zmq-logger --payload --audit-file logs/zmq.jsonl
```

egress

Runs the snapper-egress sidecar from the unified Snapper image. Additional arguments after snapper egress are forwarded verbatim to the egress entrypoint's argparse parser. This is the compose dispatch path for ENTRYPOINT ["snapper"] plus command: ["egress"]; the module entrypoint python -m snapper.egress remains supported for bare-shell invocations. The Typer wrapper propagates the egress entrypoint's integer return code as the process exit code.

```
snapper egress [--instance-id snapper-egress-prod]
```

See snapper-egress.md for tunnel settings and verification steps.

Database

retire-trade-projection-checkpoints

Preview and SCD2-close stale trade-projection checkpoints so the next trader boot rebuilds them from replay. The command never deletes a row or changes checkpoint values; it only closes the current knowledge interval by setting known_to.

Selection and confirmation are deliberately separate. Choose exactly one of --shard-key KEY or --all. Every invocation prints the credential-free database host and database name plus each candidate's shard key, position quantity, realized P&L, turnover, and checkpoint time. Without --confirm, the command is a read-only dry run.

```
snapper retire-trade-projection-checkpoints --all
snapper retire-trade-projection-checkpoints --all --confirm
```

Use --all when checkpoint age cannot distinguish clean state from state accumulated under older projection semantics. A confirmed invocation still prints the full preview before writing, revalidates that exact set in the repository transaction, and refuses if it changed.

annulment

Host-side execution-ledger correction commands use the configured DB_URL. Start with the read-only inspection:

```
snapper annulment inspect --wallet <wallet-public-id> --mode paper
```

Subcommand	Required options	Behavior
inspect	--wallet, --mode	Read unwitnessed executions, canonical row digests, existing annulments, and missing visibility observations. Optional --exchange and --limit bound discovery.
annul	--request-file, --confirm	Append one guarded annulment from a JSON request. There is no bulk mode.
complete-visibility	--wallet, --mode	Idempotently complete pending durability observations; optional --limit bounds one pass. This writes observations.
retire-derived	--wallet, --mode, --confirm	Retire affected active trade-projection checkpoints and positions. Repeat --checkpoint and --position to assert the expected active public IDs; execution-plan checkpoints are not retired.

Modes are live and paper. An annulment request must contain exactly target_execution_public_id, expected_execution_digest, wallet_public_id, exchange, mode, scope_sequence, annulled_by_user_public_id, correction_time, reason, and evidence. The writer validates the target, scope, digest, reason, actor, and absence of a durable fill witness. The acting user is an operator assertion checked against an active database user, not an authenticated session identity.

Exit 0 means the operation completed, 1 means refusal, and 3 means a correction is durable but its knowledge/visibility proof remains incomplete. Treat exit 3 as an incomplete operation requiring follow-up, not success.

db-init

Initializes the database schema.

```
snapper db-init
```

db-upgrade

Runs Alembic migrations to a specified revision.

```
snapper db-upgrade [OPTIONS]
```

Options:

Option	Type	Default	Description
--revision	string	head	Target revision

Examples:

```
# Migrate to latest version
snapper db-upgrade

# Migrate to specific revision
snapper db-upgrade --revision abc123
```

db-downgrade

Rolls back Alembic migrations.

```
snapper db-downgrade [OPTIONS]
```

Options:

Option	Type	Default	Description
--revision	string	-1	Steps to roll back

Examples:

```
# Roll back one migration
snapper db-downgrade

# Roll back to specific revision
snapper db-downgrade --revision base
```

daily-partitions

Inspects and operates PostgreSQL daily partitions for the static ticks, candles, and trades allowlist. Mutating subcommands are dry-run by default and require --apply. This lifecycle is separate from RETENTION_POLICIES; it never materializes rows for deletion.

```
snapper daily-partitions inspect trades
snapper daily-partitions adopt trades --anchor 2026-07-30T00:00:00+00:00
snapper daily-partitions ensure trades
snapper daily-partitions detach trades --day 2026-06-01
```

adopt is the explicit manual path for populated tables. It requires a UTC-midnight anchor and refuses an unsupported or drifted catalog, target-name collisions, or a host load above 1.5 per CPU. Add --apply only after reviewing its printed DDL plan. ensure maintains fourteen future leaves and refuses while the retained DEFAULT anomaly partition contains rows. detach enforces the seven-day ticks or thirty-day candles/trades horizon, uses plain bounded-lock DETACH, and leaves the detached table intact; dropping or archiving it is a separate operator action.

fx-proofs backfill

Reports the exact execution-minute FX conversion requirements sealed by the persisted watermark maps of every active P&L anchor and sample. Report mode is the default and performs no writes. Each election manifest is the consumer union's complete exact-minute set for one currency pair, never a collection of singleton manifests. This is the identity F4 must request: changing the minute electorate can change both the manifest digest and the coverage-ranked plane. The report groups post-run states by wallet, currency pair, and election scope and lists every unprovable instrument with its lost durable events.

```
snapper fx-proofs backfill
snapper fx-proofs backfill --apply
snapper fx-proofs backfill --apply --checkpoint data/fx-proof-backfill-
checkpoint.json
snapper fx-proofs backfill --apply --reset-checkpoint
```

--apply pins proofs and canonical refusal audits in fixed-size batches. The anchored checkpoint advances after each completed consumer union and is only a performance hint: every rerun asks the database whether each requirement is already pinned, so a

restored empty database is repopulated even when an old cursor survives. The checkpoint is removed only after post-apply verification proves every requirement has a proof or canonical refusal audit. Use --reset-checkpoint with --apply to discard the hint under the same exclusive host lock used by the writer.

The printed fully-resolved flag means every requirement has either a proof or a canonical refusal audit; F5's coverage gate must not interpret it as proof coverage. A partially proven requirement exits successfully when its remaining gap is carried by the embedded canonical refusal, which automation must treat as resolved but not fully proven.

All execution, symbol, candle, accrual, and proof reads use the active durable consumer union's maximum knowledge timestamp. The calculation-version label is also consumer-owned: anchors and samples can produce separate otherwise-equal election identities when their stored versions differ. The scope deliberately excludes spot-reconciliation anchors and never extends persisted cuts with a newly captured execution prefix.

Each backdated election timestamp asserts knowledge at the durable consumer horizon by design, so the historical as_of lookup can see exactly the proof or refusal that certifies that consumer.

pnl-minimum-gate

Builds the read-only evidence required before the F6 below-venue-minimum rule may change published equity. The command evaluates the existing temporal minimum proof against authoritative production balance observations, but it does not write samples, change the snapshotter, or alter an API response.

The venue denominator is not operator input. Every command loads the complete active credential catalogue, applies the same executor-topology policy and resolve_spot_venues semantics as the live snapshotter, and fails closed when the wallet has no executor-backed live spot venue. The artifact includes only non-secret scope provenance: credential public ids, exchange and credential type, executor disposition, and inclusion flags. It never serializes encrypted payloads, labels, or the raw pin declaration.

The one-time activation artifact is always exactly 24 hours, exactly 1440 finalized minute points, and bound to the operator's threshold:

```
snapper pnl-minimum-gate report \
  --wallet <wallet-public-id> \
  --max-bound-share <accepted-equity-fraction> \
  > pnl-minimum-gate.json
```

report ends its minute grid two finalized minutes behind the current clock. All commands are intentionally current-only: the credential catalogue, executor topology, fresh mint-wallet pin, market evidence, and P&L samples are read for one captured current horizon. There is no backdated --as-of surface that could mix historical topology with a current setting. Each would-be excluded leg is bounded independently of the snapshotter's

sparse minute price plane. For every certified direct USD plane, the report takes its latest causal close no more than 24 hours old, then uses the highest of those closes. The leg bound is min_order_size * selected_close.

When exclusions exist, their per-minute aggregate is divided by a conservative equity denominator. For each impacted minute, the denominator is the lowest positive complete current-epoch equity observed causally through that minute, provided the latest causal sample is no more than five minutes old. A complete sample with missing, non-finite, or non-positive equity, a missing causal sample, or a stale sample fails closed. The latest complete sample must also be within five minutes of the report end. The JSON names the exact minimum proof, candle version, close age, per-impact causal equity floor, Rule-W refusals, and every minute that lacked an authoritative basket. A finite positive bound overflow is recorded as a definite threshold breach rather than ambiguous evidence.

The current-time evaluator for a future standing alarm accepts a window of at least 24 hours; shorter checks are refused:

```
snapper pnl-minimum-gate check \
  --wallet <wallet-public-id> \
  --hours 24 \
  --max-bound-share <accepted-equity-fraction>
```

For diagnostics only, inspect permits a shorter bounded window. Its JSON always says "activation_eligible": false, even when every available point is complete:

```
snapper pnl-minimum-gate inspect \
  --wallet <wallet-public-id> \
  --hours 1
```

The check emits the same canonical JSON with a gate verdict. Exit codes are stable for cron, systemd, or another alert runner:

Exit	Meaning
0	Complete evidence and the observed bound does not exceed the stated share
1	Invalid input or the database read failed
2	Typer rejected the command-line syntax before the report ran
3	Evidence is incomplete, including no authoritative minutes, no exercised minimum proof, a missing or non-finite USD bound, or no positive complete equity denominator
4	At least one fully known bound exceeds the stated share

The threshold comparison is strict: equality passes. A known breach takes priority over incomplete evidence, so a partial authoritative window, a Rule-W refusal, or an unpriced leg cannot hide an already-proven alarm. Any of those conditions still remains visible and prevents a passing activation artifact. Redirect standard output to retain the review artifact; refusals go to standard error.

No standing alarm or scheduler is deployed by these commands. They provide the read-only evaluator only. As of this documentation update the F6 activation gate remains unsatisfied: no passing 24-hour threshold-bound production artifact has been accepted, no recurring alarm has been deployed, and the legacy sample-basis plus coordinated frontend/server prerequisites remain open. Nothing in this section claims S3/S4 activation is complete.

db-seed

Seeds the database with environment-specific data from TOML profiles. Automatically run by make migrate-dev and make migrate-prod.

```
snapper db-seed [OPTIONS]
```

Options:

Option	Type	Default	Description
--profile	string	dev	Seed profile name (e.g. dev, prod)

Users

init-admin

Creates the initial administrator user.

```
snapper init-admin [OPTIONS]
```

Options:

Option	Type	Default	Description
--username	string	admin	Username
--password	string	*(prompted, hidden, confirmed)*	Password — no default; the command prompts with hidden input and confirmation when the flag is omitted, so the value never lands in shell history.

Example:

```
# Prompts for password with hidden input + confirmation; no inline value
# means nothing lands in shell history
snapper init-admin --username admin
```

list-users

Displays a list of all system users.

```
snapper list-users
```

reset-password

Resets a user's password.

```
snapper reset-password USERNAME [OPTIONS]
```

Arguments:

- USERNAME — Username (required)

Options:

Option	Type	Default	Description
--new-password	string	(prompt)	New password

Example:

```
# Prompts for the new password with hidden input + confirmation; the secret
# never lands in shell history when --new-password is omitted
snapper reset-password john
```

token preflight

Read-only verification of an existing access credential against the deployment's signing configuration and token inventory:

```
snapper token preflight --token-file data/dev-pat.json
```

The required file can contain a raw JWT or the MCP bridge JSON envelope with SNAPPER_ACCESS_TOKEN. Run against the issuing deployment's DB_URL and MASTER_PASSWORD. The verifier checks the signature, lifecycle, active owner, access-token purpose, and signed jti agreement using the production token verification path. It

prints correlation IDs and lifecycle diagnostics, but neither the JWT nor its hash. Exit 0 means accepted; 1 means rejected or unreadable input. This does not prove permissions for a particular endpoint.

Symbol Synchronization

update-kraken-symbols

Synchronizes symbol mappings from the Kraken API.

Tokenized assets are classified as equity. Other pairs are classified as forex when both currency codes supplied by their mapping or discovery path are in USD, EUR, GBP, JPY, CAD, AUD, and CHF; remaining pairs are classified as crypto. This rule applies to REST, WebSocket-only, and BTNL records. A changed classification creates a new symbol version while preserving its public identity and history.

```
snapper update-kraken-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-kraken-futures-symbols

Syncs Kraken Futures crypto symbol mappings from the exchange API.

```
snapper update-kraken-futures-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-kraken-equities-symbols

Syncs Kraken Equities (FCM Futures) symbol mappings from the exchange API.

```
snapper update-kraken-equities-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-walutomat-symbols

Synchronizes symbol mappings from the Walutomat API.

```
snapper update-walutomat-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-polygon-symbols

Synchronizes symbol mappings from the Polygon.io API. By default it updates existing rows only; pass --insert-new to insert symbols that are not yet in the database.

```
snapper update-polygon-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update
--insert-new	bool	false	Insert new symbols

Note: This operation may take 10-15 minutes (44k+ symbols).

update-underlyings

Syncs underlying asset definitions from YAML to the database. It matches active instruments to underlyings via pattern rules, upserts mappings, and applies YAML-fallback instrument_type / expiry_override values to InstrumentSpec rows when API-sourced values are NULL.

```
snapper update-underlyings [OPTIONS]
```

Option	Type	Default	Description
-f, --force	bool	false	Bypass safety guard for stale cleanup

reconcile-symbol-aliases

Closes active symbol_aliases rows whose owning capability row is no longer tradeable and no longer market-data-enabled. The command is idempotent and can be scoped to one exchange.

```
snapper reconcile-symbol-aliases [OPTIONS]
```

Option	Type	Default	Description
-e, --exchange	string	all	Exchange to reconcile (kraken, kraken_futures, kraken_equities, walutomat, polygon, or all)

Market Snapshots

update-kraken-market-snapshot

Updates Kraken market snapshots with current prices.

```
snapper update-kraken-market-snapshot
```

update-kraken-futures-market-snapshot

Updates Kraken Futures market snapshots with current prices.

```
snapper update-kraken-futures-market-snapshot
```

update-kraken-equities-market-snapshot

Updates Kraken Equities market snapshots with current prices.

```
snapper update-kraken-equities-market-snapshot
```

update-kraken-futures-funding-rates

Backfills historical funding rates for Kraken Futures perpetuals into the funding_rates table. Duplicates are silently skipped via the partial unique index.

```
snapper update-kraken-futures-funding-rates [OPTIONS]
```

Options:

Option	Type	Default	Description
--symbol / -s	list[str]	None	One or more native perpetual symbols (-s BTC-USD-PERP -s ETH-USD-PERP)
--all	bool	false	Backfill every mapped Kraken Futures perpetual

update-walutomat-market-snapshot

Updates Walutomat market snapshots with current prices.

```
snapper update-walutomat-market-snapshot
```

Data Backfill

polygon-backfill-aggregates

Downloads historical aggregated data from the Polygon.io API to the on-disk CSV cache. This command is download-only: it writes CSV files and never touches the database. Populating the candles table is the separate second step, handled by polygon-load-csv.

The full workflow is two steps:

1. polygon-backfill-aggregates downloads candles into the CSV cache.
2. polygon-load-csv reads that cache and upserts the candles into the database (no Polygon API calls).

```
snapper polygon-backfill-aggregates [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	string[]	from settings	Symbols to download
--all	bool	false	All mapped symbols

Option	Type	Default	Description
-m, --multiplier	int	1	Timeframe multiplier
-t, --timespan	string	minute	Timespan (minute, hour, day)
-d, --days	int	None (effective default 30)	Days back
--resume/--no-resume	bool	true	Skip days already cached on disk
--csv/--no-csv	bool	true	Save to CSV files

Examples:

```
# Download specific symbols to the CSV cache
snapper polygon-backfill-aggregates -s AAPL -s MSFT --days 365

# Download all mapped symbols
snapper polygon-backfill-aggregates --all --timespan day

# Hourly download, skipping already-cached days
snapper polygon-backfill-aggregates -s SPY -t hour -m 1 --resume

# Then load the downloaded cache into the database
snapper polygon-load-csv --all --timespan day
```

polygon-load-csv

Loads candles from the on-disk Polygon CSV cache into the database. This is the cache-only counterpart to polygon-backfill-aggregates: it reads the CSV files produced by that command and upserts the candles into the candles table. It never contacts the Polygon API.

Run this after polygon-backfill-aggregates has populated the cache. Archive-symbol cache directories with no current Symbol identity are skipped with a warning, so a stale cache directory never aborts the load.

When neither --symbol nor --all is given, the symbols default to the settings-configured Polygon instruments (the instruments setting). The wildcard sentinel ["*"] in that setting is treated like --all and loads every cached archive symbol; an explicit settings list loads only those symbols.

```
snapper polygon-load-csv [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	string[]	from settings	Symbols to load (native or Polygon format)
--all	bool	false	Load every archive symbol present in the cache
-t, --timespan	string	day	Timespan subtree to read (minute, hour, day)
--since	string	None	Earliest day to load, inclusive (YYYY-MM-DD)
--until	string	None	Latest day to load, inclusive (YYYY-MM-DD)

Examples:

```
# Load every cached minute symbol into the database
snapper polygon-load-csv --all --timespan minute

# Load a single symbol within a date window
snapper polygon-load-csv -s X:BTCUSD --since 2024-01-01 --until 2024-01-31
```

polygon-repair-splits

Detects and repairs stale split-price bases in polygon equities. The incremental CSV cache freezes each day's file at the price-adjustment basis of its fetch time, so a stock split executed while the cache is being collected leaves an unadjusted discontinuity in both the cache and the database. This command automates the repair chain proven in the 2026-07-02 audit:

1. Pulls split events from the Polygon reference endpoint for the trailing --lookback-days window and maps them onto active polygon instruments (crypto/FX never split and are skipped by construction).
2. Confirms via the current 1d close history which symbols actually carry a consecutive-close break matching the split's ratio — the break sits at the fetch boundary between cache waves, not necessarily at the execution date. Symbols whose whole history was fetched after the split are already uniform and are skipped, so the command is idempotent and safe to run on every fetch cycle.
3. For confirmed symbols only: re-fetches the full --window-days range (adjusted as of today), prunes cache files older than the window (the plan's minute-data lookback limit means they can never be refreshed and would reintroduce the stale basis on a future full-range load), SCD2-supersedes every current candle row, reloads the refreshed cache, re-synthesizes the higher timeframes, and re-runs the detector.

Two safety behaviors to know about:

- Micro-splits whose ratio is closer to 1.0 than 1.4 (e.g. 20:21, 1000:1061) are reported as undetectable and skipped — an ordinary flat close would match them forever, so a price detector cannot repair them without endless churn; verify those manually.

- An EMPTY current 1d history for a candidate confirms the repair (it is the signature of a run that crashed between supersede and reload), so simply rerunning the command self-heals a partial repair.

Exits non-zero when a repaired symbol still shows the matching break afterwards. MUST run on the host, not in a container — the fetch and prune steps write the data/polygon CSV cache, which containers mount read-only (for the same reason this service is deliberately not registered with the process manager).

```
snapper polygon-repair-splits [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	string[]	all equities	Restrict the check to these native symbols
--lookback-days	int	45	Trailing window of split executions to inspect
--window-days	int	730	Re-fetch window in days (plan minute-data lookback limit)
--dry-run	bool	false	Detect and report stale split bases without repairing

Examples:

```
# After each fetch cycle: check recent splits, repair what is broken
snapper polygon-repair-splits

# Preview without mutating anything
snapper polygon-repair-splits --dry-run

# Targeted re-check of one symbol
snapper polygon-repair-splits -s NFLX
```

polygon-load-grouped-candles

Loads the on-disk Polygon **grouped-daily** cache into the database as 1d candles tagged source='native', complete=True. This is the cache-only counterpart to polygon-backfill-grouped (which only writes CSV): it reads the grouped-daily cache — the same corpus the strategy warmup reads — and upserts each leg's days into the candles table so the persisted plane holds the daily history the single-source candle read path and the DB-first warmup depend on. It never contacts the Polygon API.

--exchange is **required**: it is the venue the persisted 1d bars live under, which **must** be the same venue the leg's live read and warmup resolve (e.g. kraken for FET/RENDER) — *not* polygon. Instrument identity is per (symbol, exchange); live synthesized higher-TF bars persist under the live publisher's venue, so a Polygon-keyed backfill would be an orphaned plane that no live read or warmup ever sees. polygon here is only the CSV cache segment that supplies the OHLCV.

--cut-date is **required**: it is the first UTC day that synthesized live persistence may own. The backfill writes only days strictly *before* it, so under one venue native backfilled history and synthesized live bars never share a bar key (the candle unique key excludes source). Before writing, an ownership preflight — reading back under the same venue — fails closed if the persisted plane already holds a synthesized 1d row before the cut date or a native 1d row at/after it, so the operator resolves any overlap rather than silently version-thrashing.

When neither --symbol nor --all is given, the symbols default to the settings-configured Polygon instruments; the wildcard sentinel ["*"] is treated like --all and enumerates every Polygon-mapped native symbol.

```
snapper polygon-load-grouped-candles --exchange VENUE --cut-date YYYY-MM-DD [OPT
IONS]
```

Options:

Option	Type	Default	Description
-e, --exchange	string	*required*	Venue the persisted bars live under (e.g. kraken); the leg's live-read/warmup venue, never polygon
--cut-date	string	*required*	First UTC day synthesis may own (YYYY-MM-DD); writes only days before it
-s, --symbol	string[]	from settings	Native symbols to load
--all	bool	false	Load every Polygon-mapped native symbol
--lookback-days	int	800	Calendar-day cap on the backward cache walk per symbol

Examples:

```
# Load FET/RENDER daily history (under the kraken venue) before the cutover
boundary
snapper polygon-load-grouped-candles --exchange kraken --cut-date 2026-06-16 -s
FET-USD -s RENDER-USD

# Or via Make (EXCHANGE and CUT_DATE are required; unset fails fast)
make run-polygon-grouped-candles EXCHANGE=kraken CUT_DATE=2026-06-16
```

audit-candles

Audits one explicit persisted candle window without modifying the database. Both --window values are inclusive expected candle opens and must include a UTC offset. The command freezes one as_of instant, verifies the native symbol is active on the selected exchange in the same database statement that reads the bounded range, and checks empty or missing window edges, incomplete rows, OHLC/price/volume invariants, timestamp alignment, duplicates, ordering, interior gaps, and split suspects.

```
snapper audit-candles \
  --target polygon AAPL \
  --timeframe 1m \
  --window 2026-07-01T13:30:00+00:00 2026-07-01T19:59:00+00:00
```

Use --json for one stable machine-readable document. Exit status is 0 for a complete clean audit, 1 for a complete audit with anomalies, 2 for invalid command usage, and 3 when the audit cannot be completed. Windows are bounded to 100,000 expected slots and must contain only candles already closed at the captured as_of.

Intraday grids default to a zero-second UNIX-epoch anchor. Pass --anchor-offset-seconds N when a venue's opens use a different explicit anchor; the canonical range is 0 <= N < timeframe_seconds. Daily audits require offset zero and are limited to one slot until a calendar-aware expected open contract exists.

The gap policy is deliberately unsuppressed: every missing slot is an anomaly. Polygon carries equities, crypto, and FX, so the command never guesses a session calendar from the exchange name. For an intraday session market, request one expected trading session at a time. Calendar-aware multi-session audits, scheduling, automatic repair, and cache-directory continuity remain outside this command.

verify-candle-coverage

Read-only gate that verifies the persisted candles plane is ready to be the single source for >1m reads before the read-path cutover. For each (symbol, timeframe) under --exchange it RANGE-reads an explicit canonical window and checks the FULL expected slot grid (not just the newest rows) is fresh, gap-free, complete and correctly-tagged — 1d is native strictly before --cut-date and synthesized at/after, every other higher timeframe is synthesized. Exits 1 (and prints the failing (symbol, timeframe) reasons) when any pair is incomplete, so it can gate the cutover and the DB-first warmup; exits 0 when every pair passes.

It reads under the live venue — the same --exchange the read path and warmup resolve; passing -e polygon is rejected (that plane is orphaned). For 1d the window is extended back across the --cut-date seam, so the §4e invariant is enforced directly: a [cut_date,

publisher_start) gap (synthesis not yet persisting from the cut) surfaces as a missing-bar failure rather than silently staling the warmup. --min-bars is the window DEPTH verified — set it to the consumer's required lookback (e.g. the warmup depth) to verify that far back.

The derive_snaps OHLCV-parity rung (the "no derive regression" check) needs a genuinely live, SUB-socket-warm cache as an independent witness, so the standalone CLI runs **structural checks only**; parity is deferred to an in-process slice-4 cutover check.

```
snapper verify-candle-coverage --exchange VENUE --cut-date YYYY-MM-DD [OPTIONS]
```

Options:

Option	Type	Default	Description
-e, --exchange	string	*required*	Live venue the persisted bars + read path resolve under
--cut-date	string	*required*	Synthesis ownership boundary (YYYY-MM-DD); decides 1d provenance
-s, --symbol	string[]	from settings	Native symbols to verify
-t, --timeframe	string[]	5m 15m 30m 1h 4h 1d	Timeframes to verify
--min-bars	int	30	Window depth (canonical slots) verified per pair
--writer-lag-seconds	int	120	Grace seconds for the writer to flush the just-closed bar

Examples:

```
# Gate the FET/RENDER plane before the single-source cutover
snapper verify-candle-coverage --exchange kraken --cut-date 2026-06-16 -s FET-USD -s RENDER-USD

# Verify only the daily plane
snapper verify-candle-coverage -e kraken --cut-date 2026-06-16 -s FET-USD -t 1d
```

kraken-futures-backfill-candles

Backfills historical OHLCV candles for Kraken Futures perpetuals.

```
snapper kraken-futures-backfill-candles [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	str[]	None	Native symbols to backfill (e.g. BTC-USD-PERP)
--all	bool	false	Backfill all mapped Kraken Futures symbols
-t, --timeframe	str	1h	Candle interval (1m, 1h, 4h, 1d)
-d, --days	int	90	Days back to fetch
--resume / --no-resume	bool	--resume	Resume from latest stored candle

kraken-equities-backfill-candles

Backfills historical OHLCV candles for Kraken Equities (TradFi FCM) contracts via the internal iapi.kraken.com ticker-history endpoint. Response is ~10-minute delayed per FCM policy. This REST backfill path is independent of kraken_equities_realtime_ws_enabled: enabling the authenticated realtime WS makes the *live* tick/candle feed realtime but does not change what this endpoint returns. Per-chunk upstream failures raise RuntimeError so outages are distinguishable from legitimately-empty candle windows.

```
snapper kraken-equities-backfill-candles [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	str[]	None	Native symbols to backfill (e.g. MNQM6-CME)
--all	bool	false	Backfill all mapped Kraken Equities symbols
-t, --timeframe	str	1h	Candle interval (1m, 5m, 15m, 30m, 1h, 1d)
-d, --days	int	30	Days back to fetch
--resume / --no-resume	bool	--resume	Resume from latest stored candle

backfill-candles-from-trades

Backfills calculated 1-minute candles from persisted trades.

```
snapper backfill-candles-from-trades --exchange EXCHANGE --start START --end
END [OPTIONS]
```

Options:

Option	Type	Default	Description
--exchange	str	*required*	Exchange whose trades should be aggregated
--start	str	*required*	Inclusive UTC start date or datetime
--end	str	*required*	Inclusive UTC end date or datetime
-s, --symbol	str[]	None	Native symbols to backfill
--all	bool	false	Backfill all active symbols on the exchange

backfill-synthesized-candles

Backfills synthesized higher-timeframe candles from persisted 1m candles.

```
snapper backfill-synthesized-candles --exchange EXCHANGE --start START --end
END [OPTIONS]
```

Options:

Option	Type	Default	Description
--exchange	str	*required*	Exchange whose 1m candles should be aggregated
--start	str	*required*	Inclusive UTC 1m candle lower bound (date or datetime)
--end	str	*required*	UTC upper bound used to seal higher-timeframe windows
-s, --symbol	str[]	None	Native symbols to backfill
--all	bool	false	Backfill all active symbols on the exchange
--timeframes	str	5m,15m,30m,1h,4h,1d	Comma-separated higher timeframes to synthesize
--cut-date	str	None	UTC date (YYYY-MM-DD) where synthesized 1d ownership begins

build-continuous

Builds and displays a continuous-contract series for an underlying.

```
snapper build-continuous TICKER EXCHANGE CONTRACT_FAMILY [OPTIONS]
```

Arguments:

Argument	Description
TICKER	Underlying asset ticker (e.g. SPX, GOLD)
EXCHANGE	Exchange to source contracts from
CONTRACT_FAMILY	Product root (e.g. ES, GC)

Options:

Option	Type	Default	Description
-t, --timeframe	str	1d	Candle timeframe
-m, --method	str	panama	Adjustment method (unadjusted, ratio, panama)
--start	str	365 days ago	Series start time (ISO)
--end	str	now	Series end time (ISO)
--rollover-days	int	0	Days before expiry to roll (0..365)

polygon-backfill-grouped

Downloads grouped daily data from the Polygon.io API to the CSV cache. Like polygon-backfill-aggregates, this is download-only and writes CSV files without touching the database.

```
snapper polygon-backfill-grouped [OPTIONS]
```

Options:

Option	Type	Default	Description
--market / -m	str	crypto	Market type (crypto, stocks, fx)
--days / -d	int	3	Number of recent days to fetch
--locale / -l	str	global	Market locale (global, us)
--csv / --no-csv	flag	--csv	Save data to CSV files
--adjusted / --unadjusted	flag	--adjusted	Use adjusted prices

Data Archive

archive

Export data to CSV archive files. Supports candle cache projection, append-only event tables (ticks, trades, signals, executions, telemetry, control), and SCD2 state tables (orders, positions, instruments, instrument_specs, settings, symbols, symbol_aliases, symbol_exchange_capabilities, users, user_login_events, market_snapshots, process_runs, underlying_assets, instrument_underlying_mappings, continuous_contract_configs). Merges with existing files and deduplicates rows.

```
snapper archive [OPTIONS]
```

Options:

Option	Type	Default	Description
--table	TEXT	candles	Table to archive. Accepts candles, candles-audit, any event table (ticks, trades, signals, executions, telemetry, control), or any state table (see the section intro for the full list).
--exchange	TEXT	None	Exchange filter (e.g. polygon, kraken)
--symbol	TEXT	None	Native symbol filter (e.g. BTC-USD), resolved to stable archive_symbol
--timeframe	TEXT	1m	Candle timeframe (1m, 5m, 1h, 1d) — candles and candles-audit only
--day	str	None	Single day to archive (YYYY-MM-DD; parsed in the handler)
--from	str	None	Start of date range (YYYY-MM-DD; parsed in the handler)
--to	str	None	End of date range (YYYY-MM-DD; parsed in the handler)
--dry-run	FLAG	False	Report counts without writing files
--purge	FLAG	False	Delete exported rows from DB after writing (event tables **except executions**, plus candles-audit/state tables when combined with --closed-only). Refused for executions — see the execution exception note below.
--closed-only	FLAG	False	Only closed SCD2 versions (candles-audit and state tables)
--output-dir	str	data	Base output directory

Output structure:

```
Candles:  {output_dir}/{exchange}/cache/{timespan}/{archive_symbol}/{year}/
{date}.csv
Events:   {output_dir}/archive/{table}/{exchange}/{archive_symbol}/{year}/
{date}.csv
Flat:     {output_dir}/archive/{table}/{year}/{date}.csv  (telemetry, control)
```

Examples:

```
snapper archive --day 2024-03-15 --exchange polygon --symbol BTC-USD
snapper archive --from 2024-01-01 --to 2024-03-31 --dry-run
snapper archive --day 2024-03-15 --timeframe 1d
snapper archive --table ticks --day 2024-03-15 --exchange polygon
snapper archive --table trades --from 2024-01-01 --to 2024-03-31 --purge
snapper archive --table control --day 2024-03-15
snapper archive --table candles-audit --day 2024-03-15 --timeframe 1m
snapper archive --table candles-audit --day 2024-03-15 --closed-only --purge
snapper archive --table orders --day 2024-03-15
snapper archive --table symbols --day 2024-03-15 --closed-only --purge
snapper archive --table instruments --day 2024-03-15
```

Notes:

- --symbol resolves the current native_symbol to the stable archive_symbol via the Symbol anchor row. After a symbol rename, the CLI argument and the output directory may differ.
- --day or --from/--to is required.
- Daily timespan (1d) produces monthly CSV files to match Polygon layout.
- --purge is supported for event tables, plus candles-audit and state tables when combined with --closed-only (which protects active rows); it is **not** supported for the candle cache export.
- **Execution exception (--purge refused):** snapper archive --table executions --purge fails closed with ExecutionPurgeUnsupportedError before any file is written or any row is deleted. Executions are the replayable evidence behind spot reconciliation and precision certification; deciding which execution rows are safe to delete requires reading the active per-wallet anchors and unresolved spot/margin signals, which the archiver does not consult. Executions still export to CSV — only the DB delete is refused. As a direct consequence, execution storage is currently **UNBOUNDED**: no automatic retention prunes it (only telemetry has a configured retention policy), and no manual purge path exists. Bounding execution storage is deferred pending an anchor-aware retention design.
- Event archive CSV files include full temporal metadata (public_id, timestamp, known_to, session_id, sequence_id).
- Merge/dedup for events uses (public_id, timestamp, known_to) as key.
- Executions are partitioned by exchange/archive_symbol via order -> instrument.
- candles-audit exports all SCD2 versions (closed + active) grouped by open_at date. --purge requires --closed-only to protect active rows.

restore

Restore plain or zstd-compressed CSV data back into the database. The restorer streams .csv and .csv.zst files in bounded batches, validates each complete header against the table's positional archive contract, and deduplicates by the exact typed (public_id, timestamp, known_to) identity.

```
snapper restore [OPTIONS]
```

Options:

Option	Type	Default	Description
--table	TEXT	(required)	Table name to restore into
--file	str	None	Single CSV file to restore
--dir	str	None	Directory to scan recursively for .csv and .csv.zst files
--source	TEXT	audit	Restore mode (audit for full history)
--batch-size	INTEGER	10000	Maximum rows parsed and inserted per batch

Examples:

```
snapper restore --table ticks --file data/archive/ticks/polygon/BTC-USD/
2024/2024-01-01.csv
snapper restore --table trades --file /mnt/archive/trades-2024-01-01.csv.zst
snapper restore --table settings --dir data/archive/settings/
snapper restore --table candles --dir data/archive/candles/polygon/BTC-USD/
```

Notes:

- At least one of --file or --dir is required.
- Directory scans include both .csv and .csv.zst files.
- Headerless files, transposed columns, and other header mismatches are refused before any database access.
- The default 10,000-row batch size can be lowered to reduce peak memory or raised to favor throughput.
- Rows already present in DB (matching public_id + timestamp + known_to) are skipped.
- Audit restore inserts all temporal columns exactly as exported.
- **Execution exception (restore refused):** snapper restore --table executions fails closed with ExecutionRestoreUnsupportedError. The audit restore's direct bulk insert would bypass the per-wallet execution fence and the scope-counter protocol that

guarantee a contiguous, gap-free execution range proof, so restoring executions is refused. An anchor-aware offline replay is required before executions can be restored; that path does not yet exist.

Encryption Management

settings-rotate-encryption

Rotates encryption keys for settings in the database.

```
snapper settings-rotate-encryption [OPTIONS]
```

Options:

Option	Type	Default	Description
--new-password	string	*(prompted, hidden, confirmed)*	New master password — no default; the command prompts with hidden input and confirmation when the flag is omitted, so the value never lands in shell history. Inline --new-password X is still accepted for non-interactive automation but the same shell-history caveats apply.
--old-password	string	from env	Current master password (optional; reads from env/ bootstrap when omitted). Avoid inline use — passing a secret on the command line leaks it to shell history.
--dry-run	bool	false	Show changes without applying

Example:

```
# Simulate rotation — prompts hidden for the new master password
snapper settings-rotate-encryption --dry-run

# Actual rotation — same prompt flow, then re-encrypts every encrypted setting
snapper settings-rotate-encryption
```

The CLI prompts for --new-password with hidden input and confirmation when the flag is omitted (preferred), so the master password never lands in shell history, scrollback, or CI logs. After a successful rotation the command prints only a reminder to update MASTER_PASSWORD in .env — it does NOT echo the value you entered. Inline --new-password X is still accepted for non-interactive automation but the same shell-history caveats apply.

Example Workflows

New Installation Initialization

```
# 1. Initialize database
snapper db-init

# 2. Create administrator (prompts for password — never put it on the cmdline)
snapper init-admin --username admin

# 3. Synchronize symbols
snapper update-kraken-symbols
snapper update-polygon-symbols

# 4. Start server
snapper server
```

Full Trading System Startup

```
# Terminal 1: ZMQ Broker
snapper broker

# Terminal 2: Data feed
snapper feed --symbols "BTC-USD,ETH-USD"

# Terminal 3: Executor
snapper executor -e kraken

# Terminal 4: Trade runtime
snapper trade-zmq

# Terminal 5: Server with dashboard
SERVER_API_ONLY=true snapper server
```

Starting all services does NOT enable live trading. The live_trading_mode DB setting is seeded halted and gates every non-paper submit; live orders require the explicit operator action of setting it to enabled (see configuration.md).

Historical Data Backfill

Backfilling Polygon candles is a two-step workflow: download to the CSV cache, then load the cache into the database.

```
# Step 1 - download last 30 days for SPY to the CSV cache (no DB write)
snapper polygon-backfill-aggregates -s SPY --days 30 --timespan minute

# Step 1 - download daily data for all symbols to the CSV cache
snapper polygon-backfill-aggregates --all --timespan day --days 365
```

```
# Step 2 - load the downloaded CSV cache into the database (no API calls)
snapper polygon-load-csv --all --timespan day
```

Notifications

notify

Long-running notification sidecar (domain events → alert history, web fanout, APNs HTTP/2 and optional Android FCM HTTP v1 FID delivery). It evaluates the registered notify rules, fans each resulting alert out through the outbox-backed NotifySidecar, and retries server/throttled failures on its own 30-second scheduler. It also eagerly scans current open portfolio-drift episodes and repeats that recovery every PORTFOLIO_DRIFT_RECOVERY_INTERVAL_SECONDS (default 60), restoring any drift.<episode_public_id> page whose real-time bus event was lost. APNs configuration comes from the apns_* settings provisioned through the seed profile. Android is disabled unless an explicit SNAPPER_ANDROID_PUSH_CONFIG_FILE configures its sender; apns_enabled:false permits FCM-only startup without APNs credentials. See Android push.

```
snapper notify
```

Run under systemd / K8s with Restart=always — the sidecar exits only on unhandled errors or SIGINT; the outbox drain on the next start recovers any queued deliveries left behind. Drift recovery is notify-only: it reads episode/ownership/dedup evidence and writes only the normal alert/delivery outbox rows; it never trades, rebases, or mutates portfolio truth.

android-push-state

Read the configured database's current Android configuration head without loading sender credentials or changing state:

```
snapper android-push-state
```

The JSON contains revision and active_configuration_id. Copy the opaque revision into runtime expected_configuration_revision before a new activation or explicit disable. Both fields are null only before any decision; a disabled head retains its revision. This command neither enables delivery nor contacts Google. See configuration decisions.

AI Integration

mcp-oauth provision-client

Pre-registers one confidential MCP OAuth client in the configured database. Run migration 0052 before using the command. It generates a 256-bit client identifier and a 384-bit client secret, stores only the secret's bcrypt hash, and prints the raw secret exactly once after the database commit succeeds.

```
snapper mcp-oauth provision-client \
    --name "Private ChatGPT" \
    --redirect-uri "https://chatgpt.com/connector/oauth/callback"
```

Options:

Option	Type	Default	Description
--name NAME	string	required	Exact client name for the future consent screen
--redirect-uri URI	string, repeatable	required	Exact registered callback URI
--auth-method METHOD	string	client_secret_basic	client_secret_basic or client_secret_post
--scope SCOPE	string, repeatable	snapper.read, offline_access	Initial-release ceiling: only snapper.read or offline_access

Redirect URIs are stored without normalization because the authorization flow must compare them exactly. The command accepts HTTPS URIs and HTTP only for the exact loopback hosts localhost, 127.0.0.1, and [::1]; it rejects fragments, userinfo, whitespace, noncanonical spellings, and duplicates. Scope and redirect URI duplicates are refusals rather than silent deduplication. The dormant first connector is strictly read-only: provisioning rejects snapper.account.read and every write scope even though those names already exist in the future OAuth scope catalog.

Successful stdout is a one-line JSON object containing client_id, client_secret, and the registered public metadata. Save it immediately in the client's secret store: Snapper cannot recover or print the raw secret again. A validation failure or generated-ID collision exits 1 and prints no secret.

This command only creates the dormant client row. It does not enable MCP_OAUTH_ENABLED, publish OAuth metadata, or add authorize/token/revoke routes. Existing delegate PAT clients are unchanged.

dev-mint-pat

Mints a long-lived AI delegate JWT into data/dev-pat.json (mode 0600) for the local Snapper MCP bridge. See ai-integration.md for the wider Claude Code / Cursor / Windsurf wire-up.

```
snapper dev-mint-pat [OPTIONS]
```

Options:

Option	Env var	Default	Description
--base-url URL	SNAPPER_DEV_BASE_URL	http://localhost:8000	Base URL of the running backend
--admin-username USER	SNAPPER_DEV_ADMIN_USERNAME	None (resolved from seed profiles, mcp then dev)	Admin login
--admin-password PASS	SNAPPER_DEV_ADMIN_PASSWORD	None (resolved from seed profiles, mcp then dev)	Admin password
--output PATH	SNAPPER_DEV_PAT_OUTPUT	data/dev-pat.json	Output file path
--label LABEL	—	Local Dev MCP	Delegate label

The script drives the production REST endpoints (POST /api/auth/login + POST /api/ai-delegates) so the seeded delegate exercises the same code path the AI Integration UI uses. Tokens never appear in stderr — HTTP error bodies are passed through a JWT-shape redaction helper before printing.

Backtesting

backtest-run

Run a backtest synchronously. Creates a DB record, executes the engine in-process, and persists results (signals, trades, equity, metrics).

```
snapper backtest-run \
    --strategy MACDCrossover \
    --instrument BTC-USD \
    --exchange kraken \
    --start 2026-01-01 \
    --end 2026-06-01 \
    --timeframe 1h \
    --initial-cash 10000 \
    --params '{"fast": 12, "slow": 26, "signal_period": 9}'
```

Options:

Option	Type	Default	Description
--strategy	str	required	Registered strategy class name
--instrument	str	required	Native instrument symbol
--exchange	str	required	Exchange name
--start	str	required	Start date (ISO format)
--end	str	required	End date (ISO format)
--timeframe	str	1h	Candle timeframe
--initial-cash	float	10000	Starting cash balance
--wallet	str	cli	Wallet public ID
--params	str	{}	Strategy params as JSON
--execution-mode	str	direct_db	Engine type (direct_db or zmq_replay)
--fill-model	str	market	Fill simulation model
--slippage-bps	float	0.0	Per-fill slippage in basis points
--commission-bps	float	0.0	Per-fill commission in basis points

--execution-mode is persisted on the run, but the CLI computes the pairing config hash with execution mode excluded and currently instantiates DirectDbEngine directly. Use POST /api/backtests for a run that actually executes through zmq_replay.

backtest-list

List backtest runs with optional filters.

```
snapper backtest-list --status completed --limit 10
```

Options:

Option	Type	Default	Description
--strategy	str	None	Filter by strategy name
--status	str	None	Filter by status (pending, running, completed, failed, cancel_requested, cancelled)
--wallet	str	None	Filter by wallet public ID
--limit	int	20	Maximum number of rows to return

backtest-cancel

Cancel a pending or running backtest run.

```
snapper backtest-cancel <run-public-id>
```

backtest-rerun

Re-run a CLI-compatible backtest with the fields exposed by backtest-run, including strategy params. The CLI rerun path does not preserve target_execution_exchange; use the API/process runner for cross-asset reruns.

```
snapper backtest-rerun <run-public-id>
```

Configuration

Snapper's configuration system is based on two layers: environment variables (bootstrap) and database settings.

Environment Variables

Settings loaded from .env file or environment variables.

Database

Variable	Default	Description
DB_URL	sqlite+aiosqlite:///./data/snapper.db	SQLAlchemy connection URL

Database URL examples:

```
# SQLite (development default)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# PostgreSQL (production)
DB_URL=postgresql+asyncpg://user:password@localhost:5432/snapper
```

Encryption

Variable	Default	Description
SNAPPER_ENV	development	Deployment environment: development, dev, test, testing, and ci allow local placeholders; production, prod, and staging refuse placeholder secrets at startup/runtime. Any other value fails startup; matching is case-insensitive
MASTER_PASSWORD	snapper_default_master_password_v1	Password for encrypting settings in database (salt derived automatically)

Important: Set SNAPPER_ENV=production and change the default MASTER_PASSWORD in production. It is the ONLY secret you provision: every internal key (settings encryption, JWT auth signing, CSRF token signing, control-plane command signing) is derived from it with a distinct purpose tag (src/snapper/infrastructure/security/kdf.py), so production-like

modes fail fast only when MASTER_PASSWORD still uses the development placeholder. Changing MASTER_PASSWORD rotates every derived key at the next restart — see the rotation runbook in docs/operations.md.

HTTP Server

Variable	Default	Description
SERVER_HOST	127.0.0.1	Listen address
SERVER_PORT	8000	Server port
SERVER_RELOAD	false	Auto-reload for development
SERVER_PROXY_HEADERS	true	Enable proxy header parsing in uvicorn
SERVER_FORWARDED_ALLOW_IPS	127.0.0.1	Trusted proxy IPs/CIDRs for forwarded headers
SERVER_API_ONLY	false	Skip process autostart; serve API + WS bridge only (separate-engine boots). Multi-worker uvicorn (multiple FastAPI processes sharing a broker) is supported — see docs/architecture.md Deployment Modes for the AI-review fanout dedup contract under N>1. Set SNAPPER_COORDINATOR_INSTANCE_ID + SNAPPER_COORDINATOR_INSTANCE_COUNT per worker to enable the shared partitioning.
PROCESS_AUTOSTART_PROFILE	all	Select which enabled process configs this node starts: all for single-container/dev; api excludes market publishers and delegate workloads; feed selects market publishers; strategy selects strategies; delegate selects generic delegate workloads. STRATEGIES_EMBEDDED separately controls strategy ownership.
TELEMETRY_RECORDING_ENABLED	false	Record pings, heartbeats, and GET reads to telemetry table. High volume — enable for debugging only

The Default column reflects the Pydantic defaults on BootstrapSettingsLoader (src/snapper/config/bootstrap.py) for the server / encryption / ZMQ / coordinator / trade-safety rows. The observability rows further down (SYSTEM_METRICS_*, RETENTION_*, DB_METRICS_*, DB_POOL_*, TRADE_INTEGRITY_*, SNAPPER_*_PROBE) and the notify-sidecar PORTFOLIO_DRIFT_RECOVERY_INTERVAL_SECONDS row are NOT loaded through that class. application/system_metrics/snapshotter.py and application/db_stats/

snapshotter.py read their env vars directly via os.environ. The trade-integrity watchdog reads TRADE_INTEGRITY_MONITOR_INTERVAL_SECONDS directly in application/data_quality/trade_integrity.py. The retention env vars are read in two places: application/retention/scheduler.py (RETENTION_INTERVAL_SECONDS, RETENTION_DISABLED, RETENTION_OUTPUT_DIR) and application/retention/service.py (RETENTION_DRY_RUN), with fallback resolvers defined in application/retention/policies.py. The defaults shown for those rows reflect the per-module fallbacks.

The repo-root .env.example intentionally ships with Docker-friendly overrides for two server values: SERVER_HOST=0.0.0.0 (so the dev server binds inside containers) and SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1 (so requests forwarded by the default Docker bridge are accepted). Caveat: the make run-server / make dev-backend targets in the Makefile pass --host 0.0.0.0 on the command line, which wins over the .env value via Typer's host or s.server_host resolution — so editing .env only changes SERVER_HOST for direct snapper server invocations (no --host flag). SERVER_FORWARDED_ALLOW_IPS is honored on every server-start path because no CLI flag overrides it; make migrate-dev does not bind either value.

ZeroMQ

Variable	Default	Description
ZMQ_BROKER_XSUB	tcp://127.0.0.1:7500	XSUB endpoint (publishers connect here)
ZMQ_BROKER_XPUB	tcp://127.0.0.1:7501	XPUB endpoint (subscribers connect here)
ZMQ_BROKER_BIND_XSUB	empty	Optional broker bind endpoint for XSUB. Empty means bind ZMQ_BROKER_XSUB; cross-container deployments set this to tcp://0.0.0.0:7500.
ZMQ_BROKER_BIND_XPUB	empty	Optional broker bind endpoint for XPUB. Empty means bind ZMQ_BROKER_XPUB; cross-container deployments set this to tcp://0.0.0.0:7501.
ZMQ_BROKER_EMBEDDED	true	When false, this node's launcher excludes the zmq_broker CORE process (a dedicated snapper-broker container owns the bus); the dashboard shows the broker as remotely managed.
STRATEGIES_EMBEDDED	true	When false, this node's launcher excludes role-STRATEGY processes and refuses manual local starts (a dedicated snapper-strategies container owns them).
	empty	

Variable	Default	Description
STRATEGY_EXTRA_PACKAGES		Comma-separated extra top-level packages process discovery imports (fail-soft) so out-of-tree strategies (e.g. a mounted proprietary tree on PYTHONPATH) register their processes.

Bind endpoints exist because ZMQ bind() requires a local interface while cross-container clients connect through the Docker service name. In the compose split, the dedicated snapper-broker container binds 0.0.0.0 (via explicit CLI flags) and the backend, snapper-feed, and snapper-egress all connect to tcp://snapper-broker:7500/7501; the backend additionally sets ZMQ_BROKER_EMBEDDED=false so its launcher does not start a duplicate embedded broker.

Database Engine Pool

These optional variables are read directly by the repository engine factory and only affect PostgreSQL engines.

Variable	Default	Description
DB_POOL_SIZE	SQLAlchemy default	Per-process pool size.
DB_MAX_OVERFLOW	SQLAlchemy default	Per-process overflow connections.

Set them on the dedicated feed container when publisher subprocesses would otherwise multiply the default pool toward PostgreSQL max_connections.

Coordinator Sharding

The trade runtime supports horizontal sharding via these variables. With SNAPPER_COORDINATOR_INSTANCE_COUNT > 1 each instance owns a partition of the active wallets / outbox rows, enabling multi-worker uvicorn deployments and parallel trade-zmq instances. Multi-instance coordinator deployments require PostgreSQL; startup fails fast on SQLite when SNAPPER_COORDINATOR_INSTANCE_COUNT > 1 because SQLite does not enforce SELECT ... FOR UPDATE row claims. Process-registry DB sync at server startup runs only on instance 0 (or when the count is 1); other instances skip it with a log line so concurrent workers do not race on the registry setting rows.

Variable	Default	Description
SNAPPER_COORDINATOR_INSTANCE_ID	0	Zero-based shard index for this instance
SNAPPER_COORDINATOR_INSTANCE_COUNT	1	Total number of instances in the cluster

Variable	Default	Description
SNAPPER_COORDINATOR_OUTBOX_MAX_SCAN_ROWS	1000	Per-tick upper bound on outbox rows scanned by this shard; set to unbounded, none, or an empty value to disable the cap. Positive integers are accepted.

Trade Runtime Safety

These bootstrap variables gate stale trade dispatch and live multi-leg execution safety.

Variable	Default	Description
TRADE_COMMAND_DISPATCH_TTL_S	30.0	Max age, in seconds, for trade commands. Stale CREATED submits expire in the outbox before publishing; executor stale-frame handling verifies venue state before any order placement; the executor's reconciliation sweep auto-rejects an unresolved DISPATCHED command on verified venue absence only after max(120 s, 2 × TTL). Values <= 0 disable the cap and the absence auto-reject. Keep this below the engine's 60 s in-flight valve.
PAIRED_EXECUTION_GUARD_ENABLED	false	Fail-closed live multi-leg strategy gate. With the default, live exchanges refuse multi-leg groups while paper multi-leg remains allowed. Enable only after the paired-execution guard is deployed.
PAIRED_EXECUTION_ASSEMBLY_TIMEOUT_S	5.0	Seconds a paired-execution group waits for all sibling legs to register durable commands before the guard can break the group.
PAIRED_EXECUTION_FILL_TIMEOUT_S	30.0	Seconds an armed paired-execution group waits for fills before the guard treats a lagging leg as breakage and compensates.

Observability Pipeline

These variables tune the always-on background pipelines that emit system-metrics snapshots, run retention archive+purge sweeps, and sample DB stats. See observability.md for the underlying snapshots and the retention window math.

Variable	Default	Description
SYSTEM_METRICS_INTERVAL_SECONDS	5	Cadence for the in-process system-metrics snapshotter
SYSTEM_METRICS_HISTORY_CAP	17280	Maximum number of snapshots retained in memory (≈ 24 h at 5 s cadence)

Variable	Default	Description
SYSTEM_METRICS_DISK_FREE_WARN_BYTES	21474836480	Free-byte warning threshold for the sampled data partition
SYSTEM_METRICS_DISK_FREE_CRIT_BYTES	10737418240	Free-byte critical threshold for the sampled data partition
SYSTEM_METRICS_DISK_MOUNT_PATH	/	Mount path sampled for disk-pressure REST metrics and host/disk heartbeats
RETENTION_INTERVAL_SECONDS	3600	Cadence for the retention archive+purge tick
RETENTION_DISABLED	false	Disable the retention loop entirely (still allow on-demand archive via CLI)
RETENTION_DRY_RUN	false	Compute the window and still export CSV archives, but skip the purge (deletion) step
RETENTION_OUTPUT_DIR	data	Base directory for archive CSV writes (matches the CLI --output-dir default)
DB_METRICS_INTERVAL_SECONDS	60	Cadence for the per-table SCD2 row-count sampler
DB_METRICS_DISABLED	false	Disable the DB stats sampler
AI_RESEARCH_TRIGGER_INTERVAL_SECONDS	1800	Periodic latest-wins AI-research round cadence (floor 60)
AI_REVIEW_MAINTENANCE_INTERVAL_SECONDS	60	Cadence for expired-review reaping and offline-delegate fanout (floor 60)
MARKET_DATA_WATCHDOG_DISABLED	false	Park the silent-exchange market-data watchdog entirely
MARKET_DATA_WATCHDOG_INTERVAL_SECONDS	60	Poll cadence for the per-exchange candle-freshness check (floor 5)
MARKET_DATA_WATCHDOG_THRESHOLD_SECONDS	600	Whole-exchange silence threshold before the critical_system_error alert path fires (floor 120)
MARKET_DATA_WATCHDOG_EXCHANGE_THRESHOLDS	unset	Per-exchange overrides as exchange=seconds CSV; 0 disables one exchange (e.g. walutomat=1200,kraken_equities=900)

Variable	Default	Description
TRADE_INTEGRITY_MONITOR_INTERVAL_SECONDS	60	Coordinator-0 M1/M2 cadence, clamped to 30–60 seconds; row and timeout bounds are fixed in code
PORTFOLIO_DRIFT_RECOVERY_INTERVAL_SECONDS	60	Notify-sidecar cadence for scanning current open drift episodes whose owning users lack the lifetime drift.<episode_public_id> page; malformed or non-positive values use the default
SNAPPER_TICK_PROBE	unset	Enable per-stage tick hot-path histograms in publisher logs when truthy (1, true, yes)
SNAPPER_TRADE_PROBE	unset	Enable per-stage trade hot-path histograms in publisher logs when truthy (1, true, yes)

The two probe variables flush one log line roughly every 10 seconds with per-stage count, rate, p50, p95, p99, and max. Leave them unset outside targeted throughput investigations.

Indicators

Variable	Default	Description
USE_TALIB	true	Selects the indicator backend in src/snapper/indicators/ta_lib_adapter.py (read directly via os.getenv at module import). Values true/1/yes (case-insensitive) use the TA-Lib C library; any other value — or a failed TA-Lib import — falls back to the pure-Python RSI/MACD implementations. Not part of the .env allowlist (KNOWN_ENV_KEYS in config/env_contract.py), so set it as a process environment variable only — adding it to .env fails startup validation with UnknownEnvKeyError.

Database Settings

Sensitive data is stored encrypted in the settings table. Wallet-scoped exchange credentials (api keys, secrets, PEM material) live in a separate wallet_credentials table — see Wallet Credentials below. Shared market-data provider keys stay in settings.

Market Data API Keys

Key	Description
polygon_api_key	Polygon.io API key (shared market data — not wallet-scoped)

Push Notifications

The notification sidecar reads its APNs credentials and the push-beta rollout gate from the settings table:

Key	Description
apns_team_id	Apple Developer team ID
apns_key_id	APNs auth key ID
apns_bundle_id	iOS app bundle identifier (reverse-DNS form)
apns_topic	APNs topic — normally identical to apns_bundle_id for alert pushes
apns_environment	sandbox, production, or sandbox_and_production; the latter runs both APNs clients and routes by each device's registered environment
apns_private_key_p8_base64	APNs auth key (.p8 PKCS#8 PEM) encoded as base64 so it round-trips through the TOML seed
push_beta_config	JSON {"enabled": bool, "user_public_ids": [...]} rollout gate (category notifications). Disabled by default — every user receives pushes; when enabled, only allowlisted users do. An absent or malformed value falls back to the disabled default so a misedit can never silence pushes. Managed via Manage Push-Beta Gate

All six apns_* keys are required when APNs is enabled at sidecar startup (the default): load_apns_config fails loud at startup with the full list of missing keys rather than producing cryptic APNs errors at first-send time. They are seeded under category apns via the proprietary seed profile (proprietary/data/seed/{dev,prod}.toml).

Optional Android delivery uses SNAPPER_ANDROID_PUSH_CONFIG_FILE, an explicit runtime JSON file separate from the APNs settings. Empty means Android is disabled for that process; it does not retire a shared configuration. Set apns_enabled:false in the runtime file for FCM-only startup without APNs credentials. Credential-file paths resolve relative to the runtime JSON, and metadata exposes only public Firebase options. The shared push-beta gate still applies.

Use snapper android-push-state to read the opaque configuration revision before activation or explicit global disable. Runtime expected_configuration_revision compares that durable active/disabled head; initial null cannot become valid again after a decision. See Android push configuration for the exact JSON, immutable configuration IDs, disabled-state semantics and validation boundary.

Wallet Credentials

Per-exchange trading credentials live in the wallet_credentials table and are loaded by CredentialResolver during per-wallet executor startup. Each row carries:

Column	Description
wallet_public_id	UUID of the owning wallet ((label, is_paper) unique in wallets table)
exchange	Exchange identifier (kraken, kraken_futures, walutomat, paper)
credential_type	Envelope shape: api_key_secret, rsa_pem, oauth, or paper
encrypted_payload	Fernet-encrypted JSON envelope (shape depends on credential_type)
label	Human-readable description of the credential row

Envelope shapes by credential_type:

- api_key_secret — {"api_key": "...", "api_secret": "..."} (Kraken, Kraken Futures)
- rsa_pem — {"api_key": "...", "private_key_pem": "..."} (Walutomat)
- oauth — {"client_id": "...", "client_secret": "...", "refresh_token": "..."}
- paper — {"initial_balance": "10000.0"} (paper wallets)

Seed profile format version 2

A seed profile is a self-contained declarative input. Every file stands alone: there is no cross-file inheritance and no base/overlay merging. Format version 2 makes operators and user memberships authoritative for fresh bootstrap instead of deriving them from role permissions. The loader rejects any other version at parse time so a stale profile fails loud instead of silently receiving code-side defaults.

Required sections, all validated when the profile is parsed:

- [profile] with name, format_version and tier (1 = data/seed, 2 = proprietary/data/seed, 3 = package-bundled). No defaults. name must equal the profile name the file is loaded as, so a renamed or mis-copied file fails loud instead of claiming to describe a state it does not seed. tier records where the profile is *authored* to live and is deliberately not cross-checked against the tier it resolved from: the container image copies the proprietary tier-2 profiles into the package-bundled tier-3 slot, so one authored file legitimately resolves from different tiers depending on packaging.

- [runtime_owned] with user_roles — the roles whose users, memberships and caps are minted at runtime rather than declared by any profile. An empty list is a deliberate statement; an absent section is an omission and is rejected. Every name must be a real role, and no [[users]] entry may hold a role the same file calls runtime-owned.
- [[operators]] entries with label and description. Labels are unique.
- Per-user operators, primary_operator and readable_wallets. They may be empty but never absent. A non-empty, duplicate-free operators list requires exactly one primary_operator from that list; an empty list requires primary_operator = "". Usernames are unique.
- [[settings]] entries with key, value, category and description.
- [[wallets]] entries with label and is_paper (plus optional description and nested [[wallets.credentials]], which require exchange, credential_type and reconciliation_method). is_paper has no default because it completes the wallet's natural key, and a defaulted one would quietly become a live-money wallet. The (label, is_paper) pair is unique.
- Optional [[scope_grants]] entries with operator, wallet, wallet_is_paper, granted_by, scope_kind, note, plus exactly one of underlying (for scope_kind = "underlying") or instrument (for scope_kind = "instrument").

Unknown top-level sections and unknown keys inside any table are rejected rather than ignored, so a misspelled [[scope_grant]] fails loud instead of vanishing.

Wallets are addressed by the natural (wallet, wallet_is_paper) key because the wallets table is unique on (label, is_paper). Every operator label, granted_by username and wallet key referenced by a user or a scope grant must be declared in the same file. readable_wallets models read-only principals: a user whose role permissions include create:orders or impersonate:operator may not declare one.

```
[profile]
name = "dev"
format_version = 2
tier = 1

[runtime_owned]
user_roles = ["ai_delegate", "ai_researcher"]

[[operators]]
label = "default"
description = "Default seed operator for single-user deployment"

[[users]]
username = "admin"
email = "admin@snapper.local"
password = "change-me-after-first-login"
role = "admin"
operators = ["default"]
primary_operator = "default"
readable_wallets = []

[[users]]
```

```
username = "operator"
email = "operator@snapper.local"
password = "change-me-after-first-login"
role = "operator"
operators = ["default"]
primary_operator = "default"
readable_wallets = []

[[users]]
username = "viewer"
email = "viewer@snapper.local"
password = "change-me-after-first-login"
role = "viewer"
operators = ["default"]
primary_operator = "default"
readable_wallets = []

[[wallets]]
label = "paper"
is_paper = true
description = "Paper-mode wallet (10k USD bootstrap)"

  [[wallets.credentials]]
  exchange = "paper"
  credential_type = "paper"
  reconciliation_method = "unclassified"
  initial_balance = "10000.0"
  label = "paper bootstrap"
```

Seed profiles are resolved in three tiers: data/seed/{profile}.toml, then proprietary/data/seed/{profile}.toml, then the package-bundled src/snapper/data/seed/{profile}.toml. The bundled dev.toml seeds an open-source paper wallet with a paper credential; maintainers may have a proprietary override with live wallet credentials. On a fresh database, db-seed creates every declared [[operators]] entry with its exact label and description, every [[wallets]] entry, nested [[wallets.credentials]] rows, each requested live reconciliation-method config, and exactly the user/operator memberships declared by each user's operators and primary_operator fields. A role does not imply a membership: a viewer with operators = [] receives no desk membership. The shipped dev profiles declare the admin, operator, and viewer bootstrap users as members of the default desk; this applies only when the tenant graph is wholly fresh. Production profiles must not bootstrap extra active human accounts with shared default passwords: attach an existing production viewer through the desk membership API instead.

readable_wallets and [[scope_grants]] are currently parse-and-validation only; db-seed writes neither wallet_user_read_grants nor operator scope rows. Personal read grants are never synthesized from a role or a desk membership. Scope grants cannot safely be materialized during this phase: db-seed runs before run-static has populated the symbol and underlying reference rows, and the version-2 instrument field does not include an exchange, so an instrument symbol can be ambiguous across venues. Provision wallet scope through the supported runtime administration path after static reference data exists.

For local UAT, scripts/seed_demo.py runs after static reference data is available. It verifies that the seeded viewer is already a member of the default desk, then idempotently gives that desk an instrument scope on the required paper BTC demo market. The script fails on a missing membership or a scope owned by another desk; it never creates a personal read grant.

The first successful demo insert also creates the local ai_demo delegate with a cryptographically random password and prints that credential once, only after the database transaction commits. Treat the command output as sensitive. An idempotent rerun cannot recover the original password; rotate it with snapper reset-password ai_demo instead.

If the profile has no wallets, the loader falls back to a single default paper wallet with a 10000.0 initial balance. Re-running seed on an established database does not repair or merge multi-tenant state: if any seed-owned tenant row existed before the run (users, operators, memberships, wallets, credentials, or reconciliation-method configs), both user creation and the operator/wallet bootstrap are skipped as one unit. Settings retain their independent insert-if-absent behavior. In particular, production seeding does not add a viewer account, attach an existing viewer to a desk, or reset any password unless that account is explicitly present in the selected production profile on a fresh database. Attach an existing viewer with POST /api/auth/desks/{operator_public_id}/members/{username} and have that user log in again; membership exposes the wallet scopes already provisioned for that desk, but does not create a scope grant.

The seed loader supports the api_key_secret, rsa_pem, and paper envelope shapes. The oauth shape is accepted at the credential-resolver layer but **not** by the current seed loader, so OAuth credentials must be inserted through the REST wallet_credentials create/rotate routes (POST /api/wallets/{wallet_public_id}/credentials, POST /api/wallets/{wallet_public_id}/credentials/{credential_public_id}/rotate) rather than via seed.

The credential management REST surface AND the matching frontend UI (frontend/src/features/admin/CredentialManagement/) are both shipped. Day-to-day operators use the dashboard's Credential Management view; headless callers can hit the REST routes directly (curl / Postman — the snapper-mcp bridge does NOT expose credential CRUD as MCP tools). Other recovery options are re-running the seed against a clean DB or direct SQL surgery on the encrypted payload as a last resort.

Wallet sections of a seed file (the [profile], [runtime_owned] and [[operators]] sections above are required in the same file):

```
[[wallets]]
label = "default"
is_paper = false

[[wallets.credentials]]
exchange = "kraken"
credential_type = "api_key_secret"
reconciliation_method = "unclassified"
```

```
api_key = "your-kraken-api-key"
api_secret = "your-kraken-api-secret"

[[wallets.credentials]]
exchange = "walutomat"
credential_type = "rsa_pem"
reconciliation_method = "spot_execution_replay"
api_key = "your-walutomat-api-key"
private_key_pem_base64 = "your-pem-base64-encoded"
```

Two wallets named default — one with is_paper=true and one with is_paper=false — coexist via the (label, is_paper) unique index. The seed loader encrypts every payload with the master-password Fernet key before insert. Every seed credential requires reconciliation_method. The accepted values are futures_position, spot_execution_replay, margin_ledger_replay, and unclassified. A real method is inserted atomically with its credential; unclassified and paper credentials create no method-config row. Concrete adapter policy allows only futures_position for Kraken Futures, the two replay methods for Kraken Spot, and spot_execution_replay for Walutomat. Paper, market-data-only, unreviewed, and unknown adapters reject every real method. For an existing deployment, stop executors, migrate through revision 0026, classify each live credential through the administrative PUT endpoint, and then restart. An omitted classification remains fail-closed as unclassified; the migration never infers or backfills a real method.

Trading Parameters

Key	Default	Description
instruments	{"kraken": ["*"], "kraken_futures": ["*"], "kraken_equities": ["*"], "walutomat": ["*"], "polygon": ["*"]}	Instruments per exchange (* = all instruments)
timeframes	["1m"]	Candle timeframes (list)
risk_max_leverage	1.0	Maximum leverage
risk_r_per_trade	0.005	Risk per trade (0.5%)

Additional runtime settings are also DB-backed and can be managed through the Settings API/UI:

Key	Default	Description
ai_integration_enabled	true	Feature gate for /api/mcp, /api/ai-delegates/*, and /api/ai-researchers; false returns 503 {"error_code":"feature_disabled"} from /api/mcp and hides AI integration in the frontend

Key	Default	Description
feed_egress_enabled	false	Route feed publishers through their own process-local egress pools at startup
kraken_equities_realtime_ws_enabled	false	Use Kraken Equities authenticated realtime market-data WS when token mint succeeds; fallback remains the public delayed feed
kraken_equities_realtime_wallet_public_id	""	Optional wallet pin whose exchange='kraken' Spot API key/secret mints Kraken Equities realtime WS tokens: a wallet public id used verbatim, or label:<wallet-label> resolved at runtime to the single matching live wallet (fails closed to the delayed feed on zero/multiple matches); empty auto-selects an active Kraken Spot api_key_secret wallet
egress_pool	unset	JSON tunnel-pool definition, validated as EgressPoolConfig at startup. An absent, malformed, or validation-failing value disables egress routing with a log line. See snapper-egress.md
paper_instruments	{"kraken": ["BTC-USD", "EUR-USD"], "kraken_futures": [], "walutomat": ["EUR-PLN", "USD-PLN"]}	Source exchanges and symbols replayed by paper feeds
backfill_days	30	Default historical backfill window
risk_max_drawdown	0.15	Maximum portfolio drawdown threshold
log_level	INFO	Application log level
log_json	false	Emit structured JSON logs
ws_reconnect_max_delay	10	Maximum WebSocket reconnect delay in seconds
rest_retry_max_attempts	5	REST retry attempt cap
rest_retry_backoff_base	0.2	REST exponential-backoff base delay in seconds
rest_retry_max_delay	2.0	REST retry maximum delay in seconds
rest_circuit_failure_threshold	5	Failures before the REST circuit opens

Key	Default	Description
rest_circuit_reset_timeout	30	Seconds before the REST circuit attempts reset
zmq_heartbeat_interval_ms	1000	ZMQ heartbeat interval
recon_balance_threshold	1.0	Absolute balance mismatch threshold for reconciliation warnings

The trade runtime always uses the durable (outbox-driven) dispatch path. The engine writes TradeCommand rows to the database; the OutboxDispatcher picks them up and publishes to ZMQ. Stale CREATED create/submit commands older than TRADE_COMMAND_DISPATCH_TTL_S (see Trade Runtime Safety) are CAS-expired to terminal EXPIRED instead of published — cancels and replaces are exempt — and the engine releases its in-flight intent via a synthetic expired event. Executor-side VenueEvent writes are fail-closed for pre-acceptance events — a failed persist raises before the executor acknowledges the venue event; once the venue has accepted an order, a failed order_accepted persist no longer aborts the flow and the executor's recon loop retries the durable write each cycle.

Live-Trading Interlock (Kill Switch)

A DB-backed kill switch gates every live (non-paper) order submit. It is a settings row, not an env var, seeded halted under category risk.

Key	Default	Description
live_trading_mode	halted	Live-trading interlock: halted, reduce_only, or enabled. Only enabled admits non-paper submits.

The executor interlock (src/snapper/messaging/executors/base.py) reads this setting fresh from the database on every non-paper submit — never the ZMQ-refreshed cache, so a lost system.settings broadcast can never leave a stale kill switch — under a hard 2 s read timeout so a wedged database cannot starve the serialized per-wallet handler. Paper venues are exempt and always pass (the discriminator is the executor's venue, never the caller-supplied order.mode). The read is fail-closed: any failure mode — no settings service, timeout, query/decrypt error, a duplicate active row, a missing row, or a value that is not exactly one of the three accepted strings — blocks the submit with reason live_trading_mode_unavailable, a blocking sentinel distinct from a deliberate halted. Currently, reduce_only blocks like halted (differing only in the observability reason). A

blocked submit gets a durable order_interlock_blocked disposition and is then REJECTED, so a replayed frame can never execute after the mode later flips to enabled. Flipping to enabled is an explicit operator money-risk action.

Market Persist Policy

Market-data publishers always publish ZMQ frames, but DB persistence is controlled by these settings so high-volume feeds can be cache-first:

Key	Description
market_persist_ticks	Persistence mode for tick rows
market_persist_trades	Persistence mode for trade rows
market_persist_candles	Persistence mode for candle rows
persist_intermediate_candles	When false (default) only the FINAL bar per native window is persisted (the native finalizer eliminates intra-minute SCD2 churn); true also persists in-progress complete=false bars, superseded by the final. ZMQ publishes every frame regardless.
spot_trade_built_shadow_enabled	When false (default) Kraken Spot trade-built 1m candles are not persisted; true writes them to shadow_candles only for native-vs-trade-built A/B checks.
spot_candle_source	Kraken Spot live 1m source. Default native keeps venue OHLC unchanged; explicit trade_built switches live 1m bars to the trade stream. Read at feed startup: the live subscription source is fixed when the publisher subscribes, so a change takes effect only after a snapper-feed restart (a live settings broadcast refreshes the cached value but does not re-subscribe).
trade_built_finalize_grace_seconds	Seconds to wait after a trade-built minute closes before finalizing it. Default 12 absorbs late Kraken trades.
candle_forward_fill	When false (default) the higher-TF synthesis flush emits only data-backed bars; true forward-fills empty higher-TF windows with a flat carried-close bar. Enable ONLY for continuous-corpus venues (24/7 crypto like Kraken spot); leave off for session-based equities where it would manufacture non-trading-day bars.
candle_minute_completion	When false (default) the base 1m plane stays trade-driven and gappy. When true, a venue that supports it (Kraken spot in trade_built mode, Kraken futures) also emits a flat carried-close bar (open = high = low = close = vwap = prior close,

Key	Description
	volume = 0, trades = 0, complete = true, source = synthesized) for every minute it OBSERVED the venue live and in which the instrument did not trade, so every subscribed instrument has a candle at every minute boundary. Gated on live evidence: a minute the publisher did not witness edge to edge, a symbol whose trade subscription is not confirmed, and a symbol with no in-session close all produce NO row. Enabling it is roughly a five- to sixfold increase in daily candles rows on Kraken spot, on a table with no retention policy, and it shifts higher-timeframe open/close for windows whose first or last minute was tradeless — approve both first. Switching OFF takes effect at the next minute boundary; switching ON wants a snapper-feed restart so the higher-TF flush grace widens with it.
candle_single_source	When false (default) the /api/candles smart route derives 5m/15m/30m on-read from the 1m cache; true serves those frames from the persisted candles plane, closing the cache-vs-persisted dual-source hazard. Keep off until the persisted higher-TF plane is populated and verify-candle-coverage passes. 1m stays cache-served and 1h/4h/1d are already DB-served.
candle_read_gap_fill_minutes	Longest run of absent 1m bars the /api/candles façade bridges with carried-close volume=0.0 bars. Default 60; 0 disables it and serves the sparse series verbatim. A venue publishes a 1m bar only for a minute in which something traded, so a thin instrument's plane is full of holes — and because derive_snaps needs minutes_per_bar CONSECUTIVE bars, such an instrument otherwise yields almost no 5m/15m/30m cells at all. Filling is INTERIOR only (never past either end of the window), so a dormant instrument grows no flat tail. Anything longer than the bound stays a hole: read-side nothing distinguishes a quiet market from a blind collector. The bound also keeps session boundaries unbridgeable (an overnight equity gap is ~1050 minutes, a weekend ~4300). Affects only /api/candles; /api/candles/db and /api/candles/cache stay literal and as_of time-travel reads are never filled.
market_persist_extra	Explicit extra instrument allowlist
market_persist_exclude	Explicit instrument denylist

MarketPersistPolicy refreshes from settings and admin bus events, then injects immutable allow/deny snapshots into publishers. Invalid edits are rejected for that refresh cycle and the previous policy remains active. Use /api/market/cache/health and /api/market/coverage to verify the effective persisted universe.

Publisher Micro-Batch

Key	Default	Description
write_buffer_flush_ms	50	Flush age threshold in ms (cached at start)
write_buffer_candle_max_rows	100	Candle batch size trigger
write_buffer_tick_max_rows	500	Tick batch size trigger
write_buffer_trade_max_rows	500	Trade batch size trigger

These settings are cached when the publisher starts and require a process restart to take effect.

Authentication

The JWT signing key and the CSRF signing key are NOT settings: both are derived from MASTER_PASSWORD (purpose tags snapper-auth-token-signing-v1 / snapper-csrf-token-signing-v1). Legacy auth_secret_key / csrf_secret_key rows in the settings table are ignored and may be deleted.

Key	Default	Description
auth_algorithm	HS256	JWT signing algorithm
auth_access_token_expire_minutes	15	Access token lifetime
auth_refresh_token_expire_days	7	Refresh token lifetime
auth_refresh_token_expire_days_extended	30	Reserved extended refresh token lifetime. remember_me is accepted by the login schema but is not currently wired through by /api/auth/login, so refresh JWTs use auth_refresh_token_expire_days and browser refresh cookies use a fixed 7-day Max-Age
ws_token_ttl_seconds	900	WebSocket token lifetime
csrf_token_expire_minutes	60	CSRF token lifetime
session_secure	false	Require HTTPS for session cookies
session_same_site	lax	Session cookie SameSite mode

Key	Default	Description
session_domain	""	Session cookie domain override
ui_origin	""	Additional allowed UI origins for CORS and WS origin validation

These settings are ordinary tunables with working defaults; none of them are placeholders and none are refused at any SNAPPER_ENV value. When SNAPPER_ENV is production, prod, or staging, the only placeholder refused at startup is the development default for MASTER_PASSWORD (checked in BootstrapSettingsLoader._validate_production_secret_defaults), from which the JWT and CSRF signing keys are derived.

.env.example File

The block below mirrors the variables documented in .env.example at the repo root. The inline comments are abbreviated here for readability; the canonical version of the file (with full per-variable rationale and rotation guidance) lives in .env.example itself.

```
# Database connection
DB_URL=sqlite+aiosqlite:///./data/snapper.db
SNAPPER_ENV=development

# Encryption settings for sensitive data (API keys, secrets)
# IMPORTANT: Change this default in production!
MASTER_PASSWORD=snapper_default_master_password_v1

# Server settings
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
SERVER_API_ONLY=false
PROCESS_AUTOSTART_PROFILE=all
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1

# Telemetry recording (pings, heartbeats, GET reads — high volume)
TELEMETRY_RECORDING_ENABLED=false

# ZMQ broker settings
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
# ZMQ_BROKER_BIND_XSUB=tcp://0.0.0.0:7500
# ZMQ_BROKER_BIND_XPUB=tcp://0.0.0.0:7501

# PostgreSQL pool clamps for multi-process feed deployments
# DB_POOL_SIZE=2
# DB_MAX_OVERFLOW=3

# Coordinator sharding (multi-instance trade-zmq / multi-worker uvicorn)
SNAPPER_COORDINATOR_INSTANCE_ID=0
SNAPPER_COORDINATOR_INSTANCE_COUNT=1
SNAPPER_COORDINATOR_OUTBOX_MAX_SCAN_ROWS=1000
```

```
TRADE_COMMAND_DISPATCH_TTL_S=30.0
# PAIRED_EXECUTION_GUARD_ENABLED=false
# PAIRED_EXECUTION_ASSEMBLY_TIMEOUT_S=5.0
# PAIRED_EXECUTION_FILL_TIMEOUT_S=30.0

# Observability pipeline (system-metrics snapshotter)
SYSTEM_METRICS_INTERVAL_SECONDS=5
SYSTEM_METRICS_HISTORY_CAP=17280
SYSTEM_METRICS_DISK_FREE_WARN_BYTES=21474836480
SYSTEM_METRICS_DISK_FREE_CRIT_BYTES=10737418240
SYSTEM_METRICS_DISK_MOUNT_PATH=/

# AI-research trigger
AI_RESEARCH_TRIGGER_INTERVAL_SECONDS=1800

# AI-review maintenance
AI_REVIEW_MAINTENANCE_INTERVAL_SECONDS=60

# Market-data watchdog (silent-exchange alerting)
MARKET_DATA_WATCHDOG_DISABLED=false
MARKET_DATA_WATCHDOG_INTERVAL_SECONDS=60
MARKET_DATA_WATCHDOG_THRESHOLD_SECONDS=600
# MARKET_DATA_WATCHDOG_EXCHANGE_THRESHOLDS=

# Incremental trade-integrity monitors (coordinator instance 0 only)
TRADE_INTEGRITY_MONITOR_INTERVAL_SECONDS=60

# Notify-sidecar durable drift-page recovery
PORTFOLIO_DRIFT_RECOVERY_INTERVAL_SECONDS=60

# Retention policy framework
RETENTION_INTERVAL_SECONDS=3600
RETENTION_DISABLED=false
RETENTION_DRY_RUN=false
RETENTION_OUTPUT_DIR=data

# DB stats sampler (per-table row-count snapshots)
DB_METRICS_INTERVAL_SECONDS=60
DB_METRICS_DISABLED=false

# Publisher hot-path probes, disabled by default
# SNAPPER_TICK_PROBE=1
# SNAPPER_TRADE_PROBE=1
```

Accessing Configuration in Code

Bootstrap settings (without database)

```
from snapper.config.settings import get_settings

settings = get_settings()
db_url = settings.db_url
host = settings.server_host
```

With database access

```
from snapper.config.settings import get_settings_with_service

settings = get_settings_with_service(settings_service)
polygon_key = settings.polygon_api_key  # Decrypted from database
```

Wallet-scoped exchange credentials do NOT appear on AppSettings. They are loaded by CredentialResolver during per-wallet executor startup:

```
from snapper.config.credentials import CredentialResolver

resolver = CredentialResolver(repository)
envelope = await resolver.get_credentials(
    exchange="kraken",
    wallet_public_id="01975a8b-3c7d-7000-8000-0000000000a1",
)
# envelope == {"api_key": "...", "api_secret": "..."}
```

Managing Settings via API

List Settings

```
GET /api/settings
GET /api/settings?category=api
GET /api/settings?category=api&as_of=2026-01-18T12:00:00Z
```

Public Feature Flags

```
GET /api/settings/features
```

This route does not require authentication. It returns the public feature-flag projection used by the frontend, currently ai_integration_enabled.

List Categories

```
GET /api/settings/categories
GET /api/settings/categories?as_of=2026-01-18T12:00:00Z
```

Manage Push-Beta Gate

```
GET /api/settings/push-beta/users
POST /api/settings/push-beta/users
X-CSRF-Token: <csrf_token>
Content-Type: application/json

{
    "type": "update_push_beta_users_command",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "enabled": true,
        "user_public_ids": ["<user-public-id>"]
    }
}
```

The POST body replaces the full allowlist; it is not a merge.

Set Setting

```
POST /api/settings/{key}/set
X-CSRF-Token: <csrf_token>
Content-Type: application/json

{
    "type": "setting_update",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "value": "new_value",
        "category": "system",
        "description": "optional description"
    }
}
```

Remove Setting

```
POST /api/settings/{key}/remove
X-CSRF-Token: <csrf_token>
Content-Type: application/json

{
    "type": "remove_setting_request",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 2,
```

```
    "timestamp": "2026-01-18T12:00:01Z",
    "payload": {}
}
```

Environment Configuration

The application always reads .env (SettingsConfigDict(env_file=".env") on BootstrapSettingsLoader); switch environments by editing the values in that file — notably SNAPPER_ENV — not by renaming it.

Development

```
# .env — development values
DB_URL=sqlite+aiosqlite:///./data/snapper.db
SNAPPER_ENV=development
SERVER_HOST=127.0.0.1
SERVER_PORT=8000
SERVER_RELOAD=true
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1
```

Production

```
# .env — production values
DB_URL=postgresql+asyncpg://user:pass@db.example.com/snapper
SNAPPER_ENV=production
MASTER_PASSWORD=<strong_random_password>
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1
```

Docker

Variables can be passed via docker-compose.yml:

```
services:
  snapper:
    env_file: .env
    environment:
      - DB_URL=postgresql+asyncpg://snapper:password@postgres/snapper
```

Settings Encryption

Sensitive settings are encrypted with Fernet (AES-128-CBC + HMAC-SHA256):

1. Salt is derived from MASTER_PASSWORD via SHA-256
2. Encryption key is derived via PBKDF2-HMAC-SHA256 (100,000 iterations)
3. Values are encrypted with Fernet and stored in database

If MASTER_PASSWORD changes, encrypted settings become inaccessible. Use snapper settings-rotate-encryption to re-encrypt encrypted settings rows before changing it. Wallet credential payloads use the same master-password Fernet key but are not rewritten by that command; rotate or recreate wallet credentials separately before changing MASTER_PASSWORD.

Configuration Validation

The system validates configuration at startup via the shared .env allowlist check. The encrypt/decrypt round-trip is not a startup step; it runs as part of the snapper settings-rotate-encryption command.

The allowlist is the union of BootstrapSettingsLoader aliases and each subsystem's exported ENV_VARS set. Unknown keys fail startup with suggestions, so typos such as MASTER_PASWORD do not silently fall back to defaults. When adding a new env var, register it on the owning module's ENV_VARS or add it as a bootstrap settings field. Production-like SNAPPER_ENV values additionally reject placeholder secret material so a typo cannot silently sign tokens or encrypt settings with development defaults.

Development

Guidelines for developers working on the Snapper project.

Requirements

- Python 3.14+
- Poetry
- Node.js 26+ and pnpm 11+ (frontend engines, Docker UI build, and CI workflows all standardize on Node 26)
- Build tools for Python packages. TA-Lib is optional at runtime: the indicator adapter uses TA-Lib when importable and falls back to pure Python implementations otherwise.
- Pre-commit hooks

Docker builds use Python 3.14 and require binary distributions for all locked dependencies. The vendored CCXT package currently pins cffi==2.0.0, which has no compatible Python 3.15 wheel. Python image minor and major upgrades therefore require a separate dependency compatibility review; patch updates remain enabled.

Environment Setup

Initialize the public submodules before installing frontend tools or running the full gate; this HTTPS override also works without GitHub SSH credentials:

```
git -c url."https://github.com/".insteadOf=git@github.com: \
  submodule update --init frontend integrations/snapper-mcp ios android
```

The private proprietary and data/polygon submodules require separate access and are optional for the public checkout.

The native Android client has its own build and device tests. Open android/ in Android Studio, or run make android-check with JDK 17 and Android SDK Platform 37.0 / Build Tools 36.0.0. See the Android README for backend configuration, ADB setup and APK installation. The backend's make check-all does not require an Android SDK.

The separate manual Android release workflow builds an unsigned bundle without signing credentials, or verifies a signed bundle against the configured upload-certificate pin. Google Play publication requires separate account setup and credentials; both Play modes require successful native CI and secret scanning on master for the exact commit.

```
# Install system dependencies (Linux/macOS)
make system-deps

# Install Python dependencies
make setup

# Install frontend dependencies
make ui-setup

# Synchronize pre-commit hooks
make pre-refresh

# Refresh dependencies, local tools, and Dockerfile tool pins
make update
```

make refresh upgrades Python, frontend, and MCP dependencies, GitHub Actions, and pre-commit hooks. The Python refresh also reinstalls the current project so its installed dependency metadata matches the updated manifest and lock file. make update additionally refreshes Docker build-tool pins.

CCXT currently uses a reproducible dependency metadata patch so its exact urllib3 requirement includes the current security fixes. Keep the vendored wheel in source checkouts and Docker build contexts; the linked note documents verification, rebuilding, distribution limits, and retirement of the patch.

Isolated Worktrees for Parallel Sessions

When more than one agent or developer session works on the repository at the same time, a shared checkout makes every quality-gate verdict unattributable: a red result cannot be assigned to either change, and a green one proves nothing about either. Concurrent frontend coverage runs in one checkout also corrupt each other through the shared frontend/coverage/.tmp/ directory. Run each session in its own git worktree instead:

```
make worktree NAME=my-task
```

This creates ../snapper-wt-<NAME> on a fresh wt/<NAME> branch cut from master and prepares the full verification harness:

- initializes the frontend, integrations/snapper-mcp, ios, android, and proprietary submodules, because missing submodules manufacture failures that look inherited but are environmental
- writes a minimal .env pointing DB_URL at the isolated SQLite test fixture (./data/dev.db); the real .env is never copied because it carries live venue credentials
- builds a dedicated .venv and asserts that import snapper resolves inside the worktree rather than the main checkout

- installs frontend dependencies, reuses the main checkout's frontend/dist through a symlink when one exists, and installs snapper-mcp node modules

The target requires access to the private proprietary repository as well as a POSIX shell, and is not available in native Windows shells. For a public-only checkout, create a worktree with Git and initialize only the public submodules using the setup command above, then install its dependencies and build its UI. Tear a worktree down with:

```
git worktree remove --force ../snapper-wt-<NAME>
git branch -D wt/<NAME>
```

For native Android work in a prepared worktree, install its Android toolchain and run the client's own checks. The parent verification gate needs only the initialized checkout for scanner-policy checks.

Quality Gates

Main Command

```
make check-all
```

Executes the complete quality gate:

1. Formatting (ruff, black, isort)
2. Linting (ruff)
3. Type checking (mypy)
4. Docstring compliance
5. No # comments in Python
6. Canonical __main__ guards
7. Empty __init__.py files
8. No forbidden temporal mutations
9. Vendor-neutrality check (check-vendor-neutral)
10. Pydantic-only FastAPI I/O (check-pydantic-routes)
11. Egress compose safety and delegate/read-visibility boundary checks (check-egress-compose, check-delegate-boundary, check-read-visibility-boundary)
12. Frontend checks (ui-typecheck, ESLint, Prettier, dead code, i18n checks)
13. Generated frontend/iOS type and backend i18n drift check (ui-check-types)
14. MCP wire-contract drift, public prose, type, lint, test, and stdout checks (bridge-check)
15. No pragma/noqa/ignore exclusions
16. Backend tests with 100% line and branch coverage of the configured source roots
17. Frontend tests with 100% coverage

Individual Steps

Command	Description
make fmt	Check formatting
make fmt-fix	Fix formatting
make lint	Ruff linting
make lint-fix	Fix linting issues
make typecheck	Mypy type checking
make test	Tests (parallel)
make test-serial	Tests (sequential, debug)
make test-integration	Paper-mode E2E tests (tests/integration/)
make cov	Tests with coverage
make cov-xml	Export coverage XML for SonarCloud (after make cov)
make check-egress-compose	Reject unsafe snapper-egress compose wiring

make check-egress-compose scans Compose files and override files for the snapper-egress sidecar contract. The sidecar must share the unified Snapper image, run through command: ["egress"], run as root with NET_ADMIN, avoid host ports: / network_mode: host, and share the SQLite data mount when the monolith uses /app/data. The monolith stays unprivileged: no cap_add and no user: override.

Automatic Fixes

```
make fix        # Backend: fmt-fix + lint-fix + move-imports
make fix-all    # Backend + frontend
```

Code Standards

Types (MANDATORY)

All functions must have type annotations:

```
def calculate_rsi(prices: pd.Series, period: int = 14) -> pd.Series:
    """Calculate RSI indicator."""
    ...
```

```python
async def fetch_data(symbol: str) -> list[Candle]:
    """Fetch candle data."""
    ...
```

Test functions must have -> None:

```python
async def test_rsi_calculation() -> None:
    """Test RSI calculation returns expected values."""
    ...
```

Docstrings (Google style)

```python
def process_signal(signal: Signal, config: StrategyConfig) -> Order | None:
    """Process trading signal and generate order.

    Takes a trading signal and configuration, validates the signal,
    and generates an appropriate order if conditions are met.

    Args:
        signal: Trading signal to process.
        config: Strategy configuration dictionary.

    Returns:
        Generated order if valid, None otherwise.

    Raises:
        ValidationError: If signal data is invalid.
    """
    ...
```

No # Comments

comments are forbidden. Use docstrings:

```python
# WRONG
def foo():
    # This calculates something
    x = 1 + 1

# CORRECT
def foo():
    """Calculate the sum."""
    x = 1 + 1
```

Imports

Imports at the top of file, single lines:

```
from datetime import UTC
from datetime import datetime

import pandas as pd
from sqlalchemy import select

from snapper.config.settings import get_settings
from snapper.data.models import Candle
```

Forbidden:

- from __future__ import annotations
- if TYPE_CHECKING: blocks
- Re-exports in __init__.py

__init__.py

__init__.py files must be empty (or docstring only):

```
"""Snapper strategies module."""
```

Tests

Structure

```
tests/
├── conftest.py          # Shared fixtures
├── api/                 # API tests
├── application/         # Business logic tests
├── cli/                 # CLI tests
├── data/                # Data layer tests
├── indicators/          # Indicator tests
├── integration/         # Paper-mode E2E tests
├── messaging/           # ZMQ tests
├── strategies/          # Strategy tests
└── ...
```

Writing Tests

```
import pytest
from snapper.indicators.rsi import rsi


class TestRSI:
    """Test suite for RSI indicator."""

    def test_rsi_returns_series_with_correct_length(self) -> None:
        """
```

```
        Given a price series of length N
        When RSI is calculated with period P
        Then the result has length N.
        """
        prices = pd.Series([100, 102, 101, 103, 105])
        result = rsi(prices, period=14)
        assert len(result) == len(prices)

    def test_rsi_values_in_valid_range(self) -> None:
        """
        Given any price series
        When RSI is calculated
        Then all values are between 0 and 100.
        """
        prices = pd.Series([100, 102, 101, 103, 105, 104, 106])
        result = rsi(prices, period=3)
        assert (result >= 0).all()
        assert (result <= 100).all()
```

Async Tests

```
import pytest


@pytest.mark.asyncio
async def test_fetch_candles() -> None:
    """Test fetching candles from repository."""
    repo = get_repository(db_url)
    candles = await repo.get_candles("BTC-USD", limit=10)
    assert len(candles) <= 10
```

Tests must produce zero pytest warnings. A common pitfall: AsyncMock() makes **every** attribute on the mock async, including synchronous methods such as session.add() or session.expunge(). The fix is to override the synchronous attributes explicitly:

```
from unittest.mock import AsyncMock, MagicMock

session = AsyncMock(add=MagicMock(), expunge=MagicMock())
```

Run python -m pytest tests/ -W error::RuntimeWarning to fail the suite on any unawaited-coroutine warnings.

Fixtures

```
@pytest.fixture
def sample_candles() -> list[Candle]:
    """Provide sample candle data for tests."""
    return [
        Candle(open=100, high=105, low=95, close=102, volume=1000),
        Candle(open=102, high=110, low=100, close=108, volume=1200),
    ]
```

Integration Tests

```
make test-integration
```

Runs the paper-mode end-to-end tests in tests/integration/ with a 120-second per-test timeout. The fixtures spin up a real ZmqBrokerThread, a PaperOrderExecutor, and a TraderCoordinator against the SQLite test fixture, so the full order path is exercised without touching a live venue. These tests carry the integration pytest marker and are excluded from make test / make cov (the default addopts select -m 'not integration').

100% Coverage

The project requires 100% code coverage (TDD). Check:

```
make cov
```

Coverage is configured under [tool.coverage.run] / [tool.coverage.report] in pyproject.toml. The fail_under = 100 threshold enforces line and branch coverage for src/snapper, proprietary/src when present, integrations/snapper-delegate/src/snapper_delegate, and scripts. There is no file omit list.

Frontend

Setup

```
make ui-setup
```

Development

```
make ui-dev
```

Frontend at http://localhost:3000/ with backend proxy.

Quality Checks

Command	Description
make ui-lint	ESLint
make ui-format	Prettier check

Command	Description
make ui-typecheck	TypeScript type check
make ui-dead-code	Knip dead code
make ui-i18n-check	Hardcoded user-facing string scan (frontend TSX/TS)
make ui-i18n-check-alerts	iOS alerts.* xcstrings and frontend alerts.json in sync
make ui-i18n-check-market	Frontend market.* JSON and iOS xcstrings in sync
make ui-test	Vitest tests
make ui-cov	Tests with coverage

Fixes

```
make ui-fix   # lint-fix + format-fix + dead-code-fix
```

Type Generation

```
make ui-gen-types   # Frontend OpenAPI/WS types, Zod schemas, entity aliases,
permissions
make ios-gen-types  # Swift models from OpenAPI + WebSocket schemas
make ts-bridge      # snapper-mcp bridge wire contract only
make bridge-regen   # Regenerate bridge contract and run bridge checks
make bridge-check   # Verify committed bridge contract without regenerating
make ui-check-types # Non-destructive drift check for generated types and
backend i18n catalogs
```

The shared generator is scripts/generate_types.py. With no explicit target it runs --all, which covers the main-repo frontend/iOS export path but intentionally excludes the snapper-mcp bridge because that target writes across the integration boundary. Use --bridge (or the Makefile targets above) when the bridge wire contract must be updated. make bridge-check verifies the committed wire-contract file against the current backend schemas (scripts/bridge_check/check_drift.py and check_oss_prose.py) and runs the bridge npm stack (typecheck, lint, tests, stdout gate) without rewriting any files. scripts/check_type_drift.py backs up generated files, runs make ui-gen-types ios-gen-types gen-backend-i18n-catalog, compares the result, then restores the working tree.

Internationalization

The frontend i18n catalogs in frontend/src/locales/<lang>/*.json are the source of truth for translated UI strings across 45 locales. iOS mirrors the subset of strings used on the native client by porting them into ios/Snapper/Resources/Localization/

Localizable.xcstrings. Backend user.default_language validation accepts the union of the iOS and frontend catalog code forms: 45 iOS cases plus 45 frontend cases, with five naming divergences (pt-BR/pt, nb/no, zh-Hans/zh, sr-Latn/sr, my/my-MM) for 50 accepted codes. Backend lookup resolves those five frontend aliases to the corresponding iOS catalog; explicit Chinese scripts remain distinct. Initial client selection uses the preferred language and script, with a valid saved selection taking precedence. Display locale never determines an instrument's quote currency.

Backend alert catalog

scripts/gen_backend_i18n_catalog.py reads the iOS xcstrings file and writes committed backend JSON catalogs to src/snapper/i18n/catalogs/<lang>.json. The backend catalog is limited to alert title/body templates and application-owned argument labels across 45 languages, one JSON file per language. It is generated with make gen-backend-i18n-catalog and checked by scripts/check_type_drift.py.

Known side, health-status and fallback-reason values are localized at render time; persisted arguments remain stable so iOS can re-render after a language change. Instrument, venue and instance identifiers and arbitrary external diagnostics retain their original text. APNs receives server-rendered strings.

New fill alerts use alerts.body.order_fill_full_quoted with the existing five arguments. The price argument preserves execution precision and appends the quote currency from a matching historical instrument reference. Known currency-leg revisions are treated as ambiguous. If identity or unambiguous quote metadata is missing, it contains only the price rather than an assumed dollar amount. Older iOS clients fall back to the server-rendered body for this unknown key. Historical rows retain their recorded price precision; it cannot be recovered from localization arguments. New unresolved-order alerts similarly use alerts.body.order_unknown_unresolved so older clients retain the server's explicit instruction not to assume the position is closed.

Market catalog

The market.description.*, market.assetClass.*, market.sector.*, market.related.*, market.pairStats.*, and market.cacheBanner.* namespaces are ported from the frontend JSON into the iOS xcstrings by scripts/port_market_catalog.py. Run after editing any frontend/src/locales/<lang>/market.json file under one of those namespaces:

```
python scripts/port_market_catalog.py        # write
python scripts/port_market_catalog.py --check  # CI parity gate
```

The --check mode is wired into make ui-check via ui-i18n-check-market, so a forgotten regeneration fails the backend + frontend quality gate. Placeholder tokens ({{name}} / {{assetClass}}) are rewritten to xcstrings positional codes (%1$@, %2$@) based on appearance order in the EN template.

iOS-side coverage of the resulting catalog is enforced by CatalogParityTests (45 locales × every key declared in SnapperTests/I18n/ExpectedKeys.swift); adding a new key to the catalog requires adding it to ExpectedKeys.swift in the same iOS PR.

make ios-i18n-check (run from the main repo) delegates into the ios submodule Makefile and lints Swift call sites: bare String(localized:) usage under ios/Snapper is rejected in favor of the LocaleStrings helper, so catalog lookups follow the selected app locale rather than Locale.current. An allowlist file (ios/scripts/check_i18n_allowlist.txt) exempts specific paths.

Alerts catalog (historical direction: xcstrings → JSON)

Historically the iOS alerts catalog was the source of truth and frontend caught up via scripts/port_ios_alert_catalog.py. That direction is preserved for back-compatibility; new namespaces should flow frontend → iOS via the market-catalog pattern above. The alert port processes all alerts.* keys across 45 locales, writes one frontend/src/locales/<lang>/alerts.json file per locale, and updates common.nav.alerts from the alerts.navTitle xcstrings key. The five divergent locale directory names are remapped between iOS and frontend in the same way as SUPPORTED_LANGUAGES.

Scanner Behavior

The backend quality scanners are intentionally source-tree scanners, not import-time checks. check_docstrings.py scans src, tests, proprietary when present, and scripts; with the Makefile flags it enforces module/public docstrings, BDD-style test docstrings, and Google-style Args: / Returns: sections. check_no_comments.py tokenizes Python under src, tests, scripts, proprietary/src, and proprietary/tests; hashes inside strings and docstrings are allowed, but real comment tokens fail strict mode. check_pydantic_routes.py walks src/snapper/server, src/snapper/auth, src/snapper/config, and src/snapper/api, then filters for FastAPI route functions whose I/O must use concrete Pydantic schemas rather than dict, Any, or raw response types.

The remaining scanners are similarly path-scoped: coverage/lint exclusions scan Python in src, tests, scripts, optional proprietary code, and frontend TypeScript; temporal-mutation and vendor-neutrality scans walk src/snapper; check_init_files.py checks configured __init__.py roots for docstring-only files; and check_main_guard.py requires every discovered entry point to end with raise SystemExit(main()).

Database Migrations

Dev DB Location

The local server SQLite database lives at ./data/snapper.db. Test and coverage Make targets use an isolated SQLite fixture at ./data/dev.db by setting DB_URL=sqlite+aiosqlite:///./data/dev.db inside the Makefile. This keeps make test, make cov, and make check-all from mutating the database used by a running local server, even when .env points at ./data/snapper.db or PostgreSQL.

The fixture is rebuilt automatically before every SQLite test or coverage invocation. make migrate-dev-sqlite rebuilds it explicitly using the same locked, staged migration and dev-seed process. The builder validates the new database before atomically replacing ./data/dev.db; no manual deletion is needed.

The fixture URL is the TEST_DB_URL Make variable (default sqlite+aiosqlite:///./data/dev.db). Override it to run the suite against a PostgreSQL staging database:

```
make test TEST_DB_URL=postgresql+asyncpg://USER:PASS@HOST/DB
```

Never point TEST_DB_URL at a live server's database: tests mutate state, and the isolated_sqlite_db conftest fixture only protects SQLite paths.

Migration Strategy (Development)

The schema is built from a chain of sequential Alembic revisions (0001_init.py, 0002_instrument_feed_health.py, ...), each of which revises the previous one. To change the schema, add a new numbered revision rather than editing an existing one, then apply and reseed:

```
make migrate-dev
```

make migrate-dev applies Alembic migrations (via snapper db-init) then seeds development data (via snapper db-seed --profile dev). There is no standalone make migrate target -- always use make migrate-dev or make migrate-prod.

Applying (Production)

```
make migrate-prod
```

Rollback

```
snapper db-downgrade
```

Docker

Build

```
make docker-build-dev    # Development image
make docker-build-prod   # Production image
make docker-migrate-dev  # Initialize and seed the Docker SQLite database
```

Run

```
make docker-migrate-dev
make docker-run
```

Docker Compose

```
make docker-migrate-dev
docker compose up -d
```

CI/CD

The pipeline (.github/workflows/ci.yml) wraps the consolidated gate used locally in additional build and smoke stages:

1. Setup — make system-deps, make setup, make ui-setup
2. make ui-build — Production frontend bundle
3. make migrate-dev — Initialize and seed the database
4. make ui-check-types — Drift check for generated types and backend i18n catalogs
5. make check-all — Backend checks, frontend checks, generated-type drift, exclusion scan, and tests with coverage
6. make cov-xml — Export coverage XML, consumed by the SonarCloud scan step
7. Docker smoke stage — make docker-build-dev, make docker-migrate-dev, docker compose up for the snapper and snapper-web services, then a make server-check health check

SonarCloud analyzes checked-out backend code, data, migrations, and tooling without coverage, duplication, or issue suppressions. The parent project excludes frontend/, integrations/snapper-mcp/, ios/, and android/ because those public submodules have independent SonarCloud projects. It also carries one temporary exact-file analyzer exception for src/snapper/data/repository.py: the 35,000-line module exhausts the scanner JVM while the rest of the project remains analyzable. Local and CI lint, type, test, and coverage gates still include it. The exception must be removed after the repository is split into bounded domain modules. Parent-owned integrations/snapper-delegate/ remains in the backend analysis; each independent project likewise suppresses no owned sources or findings. tests/meta/test_sonar_configuration.py pins this boundary and also rejects equivalent workflow, command-line, and inline overrides. Every project workflow waits for the pin-versioned SonarCloud Quality Gate after its scan, so a failed gate fails CI instead of leaving the GitHub check green.

The private proprietary/ submodule is not initialized by the public CI workflow, so it is not present in the current SonarCloud analysis. Covering it requires either a CI credential with read access to that repository or a separate private SonarCloud workflow/project; the root configuration cannot compensate for an absent checkout.

The android/ submodule is initialized so the parent gate can validate its Sonar configuration alongside the other public projects. Its independent Android workflow builds the APK, runs JVM tests and lint, executes Compose interaction tests on an Android 17 emulator, and merges JVM/device coverage into one JaCoCo XML report. Its full-source SonarCloud project imports that report and blocks CI on its quality gate. The backend gate does not require an Android SDK.

For debugging a failing make check-all locally, the equivalent steps are:

1. make check — Backend quality checks
2. make ui-check — Frontend lint, format, dead-code, type, and i18n catalog checks (ui-lint ui-format ui-dead-code ui-typecheck ui-i18n-check ui-i18n-check-alerts ui-i18n-check-market)
3. make ui-check-types — Generated frontend/iOS type and backend i18n drift check
4. make bridge-check — MCP wire-contract drift, public prose, type, lint, test, and stdout checks
5. make check-exclusions — No pragma/noqa/ignore bypasses
6. make cov — Backend tests with coverage
7. make ui-cov — Frontend tests with coverage

Workflow

Before Commit

```
make check-all
```

Before PR

1. Ensure make check-all passes
2. Add/update tests
3. Update documentation if needed
4. Verify 100% coverage

Code Review Checklist

- [ ] Types for all functions
- [ ] Docstrings (Google style)
- [ ] No # comments
- [ ] Tests for new functionality
- [ ] 100% coverage
- [ ] make check-all passes

Project Structure

```
snapper/
├── src/snapper/           # Source code
│   ├── api/               # API schemas
│   ├── application/       # Business logic
│   ├── auth/              # Authentication
│   ├── cli/               # CLI interface
│   ├── config/            # Configuration
│   ├── core/              # Core types
│   ├── data/              # Data layer
│   ├── egress/            # snapper-egress sidecar entry point
│   ├── i18n/              # Backend i18n catalogs
│   ├── indicators/        # Technical indicators
│   ├── infrastructure/    # External integrations
│   ├── interface/        # WebSocket
│   ├── mcp/              # MCP server (delegate-facing tool surface)
│   ├── messaging/        # ZeroMQ
│   ├── server/            # FastAPI
│   ├── strategies/        # Trading strategies
│   └── utils/             # Utilities
├── tests/                 # Tests
├── frontend/              # React dashboard submodule
├── ios/                   # SwiftUI iOS client submodule
├── docs/                  # Documentation
├── scripts/               # Helper scripts
└── data/                  # Local data
```

Tools

Ruff

Linter and formatter. Configuration in pyproject.toml:

```
[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "C4", "SIM", "N", "PERF", "ISC",
"PLW", "PLC", "A", "TID", "D"]
```

Mypy

Type checker. Strict mode enabled:

```
[tool.mypy]
python_version = "3.14"
strict = true
```

Black

Formatter. Line length 100:

```
[tool.black]
line-length = 100
```

Pre-commit

Hooks run before commit:

```
pre-commit install
pre-commit run --all-files
```

Debugging

Logging

```
from loguru import logger

logger.info(f"Processing signal: {signal}")
logger.debug(f"Calculated RSI: {rsi_value}")
logger.error(f"Failed to execute order: {e}")
```

ZMQ Logger

```
snapper zmq-logger --payload
```

Serial Tests

```
make test-serial
```

pdb/ipdb

```
import ipdb; ipdb.set_trace()
```

Documentation

Docstrings

Every module, class, and public function must have a docstring.

docs/

Markdown documentation in docs/ directory.

PDF

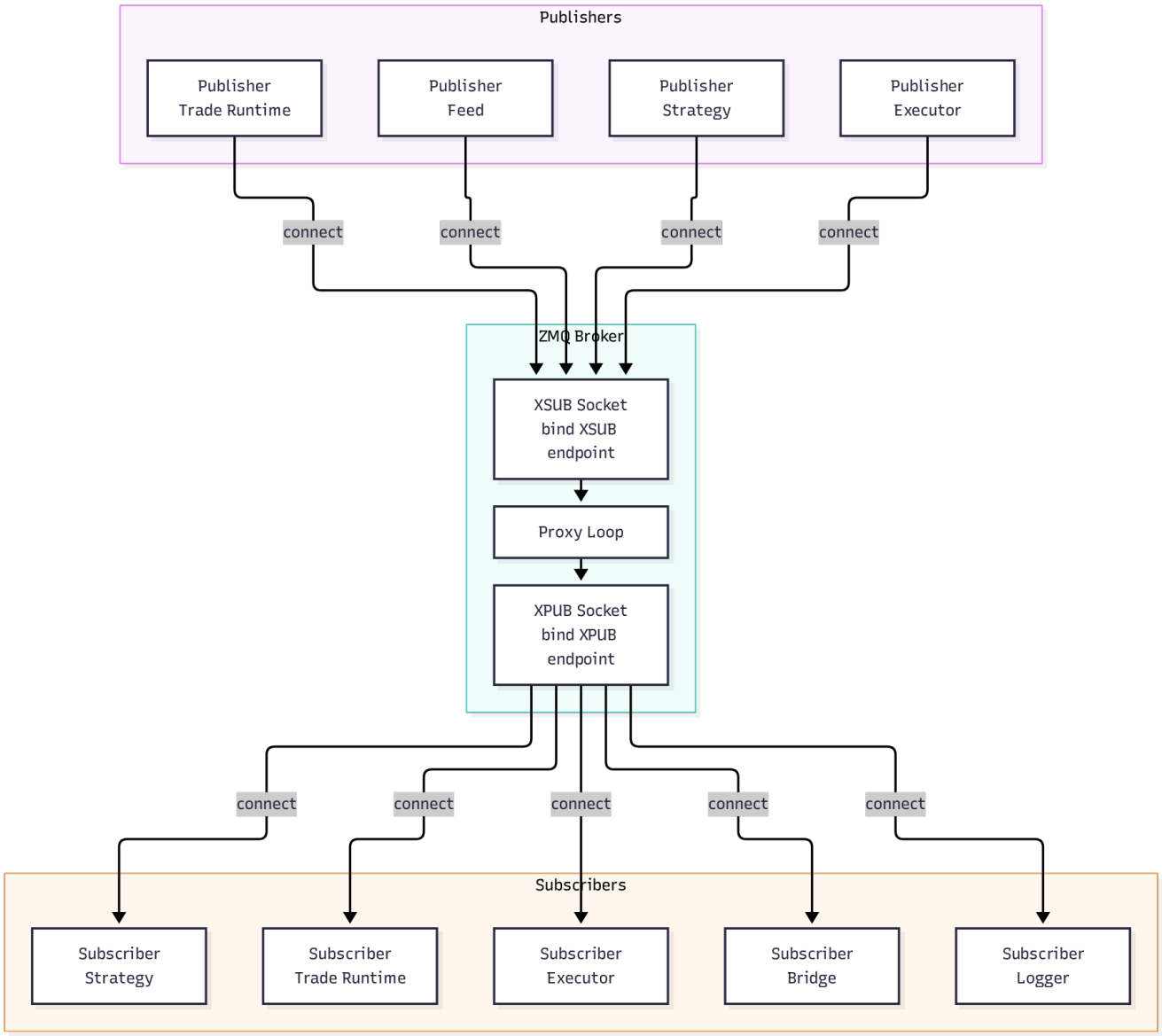
```
make docs-pdf
```

Generates frontend/public/snapper.pdf from README + docs/*.md so the docs ship with the frontend bundle and are linked from the login page. Because frontend/ is the snapper-frontend submodule, commit and push frontend/public/snapper.pdf from inside that submodule, then commit the updated frontend submodule pointer in the parent Snapper repository.

Messaging (ZeroMQ)

Snapper uses ZeroMQ for inter-process communication in a pub/sub architecture. The central component is an XPUB/XSUB broker that routes messages.

Architecture



XPUB/XSUB Broker

The broker is the central message hub:

- **XSUB** — Publishers connect here.
- **XPUB** — Subscribers connect here.

The broker binds to ZMQ_BROKER_BIND_XSUB / ZMQ_BROKER_BIND_XPUB when those bootstrap variables are set. When unset, the bind endpoints fall back to ZMQ_BROKER_XSUB / ZMQ_BROKER_XPUB, whose local defaults are tcp://127.0.0.1:7500 and tcp://127.0.0.1:7501. Docker deployments set the bind endpoints to routable container interfaces such as tcp://0.0.0.0:7500 and tcp://0.0.0.0:7501 while clients keep using the connect endpoints.

The broker forwards messages in both directions:

- Data: XSUB → XPUB
- Subscriptions: XPUB → XSUB

Starting the Broker

```
snapper broker
```

Or programmatically:

```
from snapper.messaging.infrastructure.broker import ZmqBrokerThread

broker = ZmqBrokerThread()
broker.start()
# ...
broker.stop()
```

Topics

Topic format varies by category (see per-category tables below).

Market Data

Topic	Description
market.kraken.BTC-USD.candles.1h	Hourly BTC/USD candles from Kraken
market.kraken.BTC-USD.ticks	BTC/USD ticks from Kraken
market.kraken_equities.MNQU6-CME.candles.1d	Daily Nasdaq-100 Micro futures candles from Kraken Equities (note: live market.* topics exclude polygon — Polygon is replay-only via market.paper.polygon.…)

Signals

Topic	Description
signals.paper.BTC-USD.rsi_btc_1h	Paper trading signals
signals.kraken.BTC-USD.live	Live signals from Kraken

Orders & Executions

Topic	Description
orders.commands.kraken.BTC-USD.submit	Order requests for Kraken
orders.commands.kraken.BTC-USD.cancel	Cancel requests for an existing Kraken order
orders.commands.kraken.BTC-USD.replace	Replace/modify requests; executors currently reject with a lightweight rejected replace event because atomic replace is not implemented
orders.events.kraken.BTC-USD.executed	Order executions from Kraken
orders.events.kraken.BTC-USD.submitted	Order submitted events
orders.events.kraken.BTC-USD.accepted	Order accepted events
orders.events.kraken.BTC-USD.rejected	Order rejected events
orders.events.kraken.BTC-USD.cancelled	Order cancelled events
orders.events.kraken.BTC-USD.expired	Order expired events
orders.events.kraken.BTC-USD.replaced	Order replaced events
orders.events.kraken.BTC-USD.unknown	Ambiguous submit outcome; non-terminal until venue verification resolves it

TradeCommand, VenueEvent, and TradeProjectionCheckpoint are durable database artifacts used by the trade runtime. They are not additional ZMQ topics.

Portfolio Account Invalidation

Topic	Description
portfolio.accounts.{wallet_public_id}	Thin AccountStateChangedEventData invalidation emitted after a venue-account snapshot or reconciliation evaluation commits

The payload carries wallet, exchange, mode, commit kind, and the standard provenance envelope; it deliberately carries no balances or reconciliation view. Browser clients subscribe to portfolio.accounts., invalidate active portfolio-account queries, and rebuild the fail-closed projection through GET /api/portfolio/accounts. A 60-second safety-net poll heals dropped or missed frames. Subscription requires read:account_state, and bridge fan-out uses the operator-grant accessible-wallet set for per-frame fan-out. REST account reads additionally accept personal wallet_user_read_grants; those grants do not currently widen this WebSocket filter. Malformed frames and topic/payload wallet mismatches fail closed.

System

Topic	Description
system.heartbeats.executor.{exchange}	Executor heartbeat for the single-wallet template path (4 segments)
system.heartbeats.executor.{exchange}.{wallet_short}	Per-wallet executor heartbeat (5 segments). wallet_short is the **last** 12 lowercase hex characters of the wallet UUID (see snapper.core.wallet_short)
system.heartbeats.strategy.{name}	Strategy heartbeat (e.g. strategy.rsi_btc_1h)
system.heartbeats.feed.{exchange}	Feed heartbeat (e.g. feed.kraken, feed.paper.kraken)
system.heartbeats.marketdata.{exchange}	Synthetic exchange-silence heartbeat from the API-side market-data watchdog; WARNING bursts on whole-exchange candle silence drive the critical_system_error alert pipeline (distinct from feed on purpose so its WARNING frames satisfy the 3-consecutive gate)
system.heartbeats.trade_integrity.{m1|m2}	Incremental trade-integrity monitor status; WARNING bursts report violations or excessive completed-coverage lag
system.heartbeats.ai_delegate.global	

Topic	Description
	Synthetic AI-delegate liveness and response heartbeat; WARNING bursts report no live delegate or a recent unanswered review through the existing critical_system_error pipeline
system.heartbeats.host.disk	API host disk-pressure heartbeat from SystemMetricsSnapshotter
system.egress.snapshot	Read-only process-local egress pool snapshot for API aggregation
system.egress.transfer	Read-only snapper-egress WireGuard transfer samples for API route load
system.settings	Configuration change notifications
system.symbol_aliases	Symbol cache invalidation

Heartbeat component values use dot notation matching the topic path after system.heartbeats.: executor.kraken, executor.kraken.019d6ca45f2e, strategy.rsi_btc_1h, feed.kraken, feed.paper.kraken, marketdata.kraken, ai_delegate.global, host.disk. The per-wallet executor heartbeat envelope additionally carries meta.wallet_public_id so subscribers that prefix-match the 4-segment parent topic can still disambiguate by reading the payload.

The host disk heartbeat reuses the existing critical_system_error pipeline. The snapshotter publishes HEALTHY, WARNING, and ERROR frames every tick when the API publisher is wired; the notify rule owns the three-consecutive non-HEALTHY gate, rolling cooldown, dedup, and fan-out to active users whose named permission set contains read:system_status. The current matching sets are ai_reviewer, ai_delegate, viewer, operator, and admin.

Pool-bearing processes publish system.egress.snapshot on the heartbeat cadence with an EgressPoolSnapshotEventData payload. The payload carries container (<role>@<hostname>) and snapshot, where snapshot is the process-local EgressPoolStatusSnapshot, including per-route connections rows for open WebSocket host counts and capped REST last-seen hostnames. Only hostnames are emitted; URL paths, query strings, headers, bodies, and authentication material are stripped before the snapshot leaves the process. The API process subscribes to the exact topic, caches the latest frame per container by local receive time, and merges those snapshots into GET /api/health/egress without changing routing decisions.

The snapper-egress sidecar publishes system.egress.transfer on the same cadence with an EgressTransferEventData payload. Each interface row contains interface, socks5_listen_port, cumulative rx_bytes and tx_bytes, optional byte-per-second rates,

latest_handshake_at, counter_reset, and sampled_at. The API caches the latest sample per interface by receive time and joins transfer data onto GET /api/health/egress routes by SOCKS5 listener port. Ambiguous port joins are left null.

Executor heartbeats derive status from supervised-loop health instead of reporting a fixed healthy value. ERROR fires on an active loop death streak of at least 600 s, no successful reconciliation pass for 900 s, or an order command in flight for 600 s; WARNING fires on 2+ deaths within an active streak, reconciliation progress age of at least 300 s, a command in flight for 120 s, an unhealed accept-event backlog, or parked ambiguous submits awaiting venue verification. A death streak counts as active only while its last death is younger than 300 s, so a loop that died once and then self-healed never ages into a false alarm. lag_ms carries the reconciliation progress age and meta.status_reasons lists the human-readable reasons; if the status computation itself raises, the frame still publishes as WARNING with the failure recorded in status_reasons — a broken computation can cause neither heartbeat absence nor a false healthy report. Executor heartbeats also carry venue REST reconciliation health in meta.venue_recon_failure_count, meta.venue_rest_reachable, meta.venue_health_halt_recommended, and meta.last_venue_recon_error. After three consecutive venue reconciliation failures, the coordinator uses the heartbeat to halt known real shard keys for the matching exchange/wallet scope and to drop new cold-path signals for that scope.

The launcher also publishes on the per-wallet executor topics: when a per-wallet executor exhausts its restart budget and is parked, the launcher emits bursts of three synthetic ERROR heartbeats on the instance's own topic, spaced 2 s apart so every frame clears the bridge's per-subscription forwarding throttle, re-bursting hourly while the name stays parked. These frames are recognizable by meta.synthetic: true, meta.origin: "launcher", and meta.reason: "restart_budget_exhausted"; they drive the existing critical-system-error alert pipeline without any new topic family. A successful restart or stop unparks the name and cancels the burst mid-flight.

Admin

Cross-cutting administrative notifications fanned out to every backend subscriber that cares about user/operator state changes.

Topic	Description
admin.user_deactivated	A user (or AI delegate) was deactivated; immediate revocation hook for in-process token caches, WebSocket close fanout, and notification fan-out via the notify sidecar. Auth listeners also poll the DB-backed users.is_active=False registry as broker-outage fallback
admin.membership_revoked	

Topic	Description
	A human viewer was detached from a desk; post-commit wake-up for token-cache invalidation, WebSocket retirement, and queued desk-notification cancellation
admin.scope_granted	create_grant published a new active scope row (instrument-exclusive grant just inserted)
admin.scope_handed_over	handover finalized a scope handover (old row closed, new row open, both committed)
admin.scope_revoked	revoke_grant closed an active scope row; revoke hook for any subscriber holding cached principal state

Shape: admin.{resource} — exactly 2 segments. The validator only enforces the 2-segment shape; the resource token itself is not character-class-checked, so consumers must use the payload's type discriminator as the authoritative routing hint.

admin.membership_revoked is a backend-internal event and is intentionally kept outside generated REST/WebSocket client schemas. UserService is its sole publisher and sends it only after the SCD2 membership close and any primary promotion have committed. Its payload identifies the closed membership, target user and username, removed operator, detach time, initiating user, optional promoted primary operator, and reason. The event is a wake-up signal; the committed database remains authoritative.

The consumers are:

- TokenManager, which invalidates every positive verification-cache entry for the target. Request-path verification also checks non-admin operator claims against current memberships, and the five-second database fallback evicts users with stale cached token JTIs or membership claims if the event is lost;
- WebSocketAuthManager, which snapshots the target's sockets, then retires and closes only connections fenced by the revoked membership generation or the post-detach active-token inventory. A delayed event therefore preserves a later active login. Its five-second database scan applies the same token and per-connection membership reconciliation during broker outages;
- the notify sidecar, which cancels queued deliveries for the exact (user_public_id, operator_public_id) membership while preserving deliveries for other desks. Cancellation is bounded by the event's detach time, inclusive, so delayed or duplicate events cannot cancel deliveries created after a later reattachment. A database membership check after the durable attempt bump and immediately before provider admission prevents a lost event from starting a new send under detached authority. A network call already admitted before the detach commit may still complete.

Attachment intentionally has no matching admin.membership_granted topic: new membership takes effect when the target next logs in and receives a fresh principal.

Accruals

Funding / rollover / borrow ledger entries that affect mark-to-market.

Topic	Description
accruals.kraken_futures.BTC-USD-PERP.funding	Perpetual funding
accruals.kraken.EUR-USD.rollover	FX rollover
accruals.kraken.BTC-USD.borrow	Borrow interest

Shape: accruals.{exchange}.{instrument}.{accrual_type} (4 segments). accrual_type must be one of funding, rollover, borrow.

Alerts

Per-user alert fanout. The notification sidecar consumes domain topics (orders.events., plans.decisions., system.heartbeats., and bus.portfolio_drift_episode), persists each alert, publishes it as an AlertEventData frame on this topic family before provider fanout, and also sends APNs and optional Android FID hints; the WS bridge is the consumer, forwarding frames to authenticated WebSocket clients with per-user scope enforcement. A periodic database scan in the same sidecar recovers open drift episodes whose real-time bus.portfolio_drift_episode frame was lost. It uses the same owner resolution, alert-row builder, and drift.<episode_public_id> identity as the bus rule; the insert claim is transaction-serialized so the two paths cannot double-page.

Android provider messages are separate data-only hints containing opaque configuration, binding, registration and alert IDs plus a fixed expiry. They do not forward the private AlertEventData payload. The client fetches authorized details in the foreground; see Android push. Existing WebSocket payloads and APNs content remain unchanged.

Topic	Description
alerts.<user_public_id>.order_fill_full	Order filled in full
alerts.<user_public_id>.order_rejected	Venue rejected the order
alerts.<user_public_id>.order_unknown	Order submit outcome ambiguous — parked as non-terminal UNKNOWN pending venue verification (safety-critical; user-scoped, with read:system_status recipient fan-out for strategy orders)
alerts.<user_public_id>.position_stop_loss_fired	Stop-loss fired on an open position

Topic	Description
alerts.<user_public_id>.margin_warning	Margin warning
alerts.<user_public_id>.critical_system_error	Critical system error
alerts.<user_public_id>.drift	Portfolio drift episode opened or resolved; open pages are safety-critical and deduplicated by episode public ID

Shape: alerts.{user_public_id}.{alert_type} (3 segments). user_public_id must be a UUID7; alert_type is the discriminated union in the AlertType Literal — the validator pulls the set via typing.get_args(AlertType) so adding a new alert type to the schema flows through automatically.

Plan Decisions

Bracket / trailing-stop / execution-plan decisions, emitted by the PlanExecutorService through a durable outbox written in the same transaction as each ExecutionPlanDecision SCD2 insert. The service attempts the ZMQ publish immediately, marks the outbox row sent after success, and retries pending rows with capped exponential backoff from a background drain loop after broker outages; the same drain also runs once at startup, so decisions that never published before a restart replay from their durable rows. A row whose third publish attempt fails is marked failed terminally. The notify sidecar subscribes to turn stop-loss / trailing-stop fires into configured-provider notifications (take-profit fires are explicitly excluded as non-loss outcomes).

Topic	Description
plans.decisions.<plan_public_id>	One topic per execution plan; payload carries the decision frame

Shape: plans.decisions.{plan_public_id} (3 segments) with plan_public_id in UUID7 form.

AI Reviews

Outbound delegate-consultation frames for the human-in-the-loop review queue. The bridge applies a per-frame scope filter so only delegates with matching live wallet and instrument grants receive frames. Every granted delegate receives the same single request publication; selected_delegate_public_id determines who may answer before fanout_after, not transport delivery. The fanout scanner opens a database CAS window and publishes no second request frame. Non-delegate WebSocket principals fail closed for this topic family even if they somehow request a subscription.

Topic	Description
ai_reviews.<user_public_id>.<strategy_public_id>.request	Strategy is asking for review; published once to granted delegates
ai_reviews.<user_public_id>.<strategy_public_id>.decision_ack	Operator's decision acknowledged
ai_reviews.<user_public_id>.<strategy_public_id>.caps_violation	Caps violation after approval

Shape: ai_reviews.{user_public_id}.{strategy_public_id}.{suffix} (4 segments). Both ID segments are UUID7; suffix is one of the three frame discriminators above. The payload must include string wallet_public_id and instrument_public_id values so the bridge can check the delegate's grant before forwarding.

AI Research

ai_research. is the registered WebSocket subscription root for research-round wakes. It belongs to its own ai_research authorization category and requires the effective submit:market_view permission. The current ai_researcher named set contains it. The ai_reviews. category instead requires both effective read:signals and create:orders; the current ai_delegate, operator, and admin named sets contain both, while ai_researcher does not. Concrete research frame shape is ai_research.{round_public_id}.request (3 segments), where the round identifier is UUID7 and request is the only valid suffix. The ai_research.request payload repeats round_public_id and carries the server-owned trigger. The round is already committed before this best-effort wake is published.

Backtest

Per-run lifecycle and progress events.

Topic	Description
backtest.<wallet_public_id>.<run_public_id>.started	Run kicked off
backtest.<wallet_public_id>.<run_public_id>.progress	Periodic progress tick
backtest.<wallet_public_id>.<run_public_id>.milestone	Engine progress milestone — payload's milestone field is one of 25pct/50pct/75pct (BacktestProgressMilestone Literal)
backtest.<wallet_public_id>.<run_public_id>.completed	Run finished
	Run failed

Topic	Description
backtest.<wallet_public_id>.<run_public_id>.failed	
backtest.<wallet_public_id>.<run_public_id>.cancelled	Run cancelled

Shape: backtest.{wallet_public_id}.{run_public_id}.{event} (4 segments). wallet_public_id and run_public_id are UUID7; event is drawn from BacktestProgressEvent in data.py — the validator's BACKTEST_EVENTS set is derived via typing.get_args(BacktestProgressEvent) so the wire schema and the validator share one source of truth. Subscription-only prefixes (backtest., backtest.{wallet}., backtest.{wallet}.{run}.) are accepted on the subscribe.py / bridge.py paths.

Bus

Internal cross-service event bus over the shared broker.

Topic	Description
bus.delegate_offline	Delegate session torn down
bus.ai_review_decision	AI review decision recorded — fast-path fanout to the in-process AI review bus listener that resolves the strategy's await future (WS fanout of the operator's ack is the separate external ai_reviews.<user>.<strategy>.decision_ack topic). The matching bus.ai_review_request topic is intentionally NOT published today
bus.caps_violation_after_ai_approve	Caps violation post-approval
bus.portfolio_drift_episode	Notify-only committed drift episode opened / resolved transition; emitted after reconciliation commit and never used to trade, rebase, or mutate portfolio truth

Shape: bus.{name} (2 segments) where name matches [a-z][a-z0-9_]* (snake_case). The payload's StrictDataSchema.type discriminator is the authoritative routing hint; the validator only enforces structural shape so the schema layer can reject unknown payload types.

Process Manager Events

Snapshot + per-run events emitted by ProcessLauncherService whenever the configured set of processes changes or a run transitions. Frontend subscribes via wsDispatcher and uses these to invalidate React Query caches (no REST polling).

Topic	Description
processes.events.summary.<instance_id>	Full snapshot of every configured process's current status (start/stop/crash/completion transitions)
processes.events.configured.<instance_id>	Snapshot of currently-known process names. Emits in four places: create_process_config (persisted-config-row create), spawn_per_wallet_executors (per-wallet executor instances appear), update_process_config (desired-state mutation), and update_process_config_parameters (strategy scope editor). It does NOT fire on persisted-row delete or on per-wallet teardown today. The snapshot's process_names is the union of persisted config names and instance_configs.keys().
processes.events.runs.<process_name>	Per-run lifecycle transitions: running / succeeded / failed / cancelled
strategies.events.list.<instance_id>	Snapshot of canonical class paths for every STRATEGY-role process config (drives the Strategies view)

Shape: 4 segments. The instance-id / process-name tail must match [A-Za-z][A-Za-z0-9_-]* (regex _PROCESSES_TOPIC_NAME_PATTERN). Numeric SNAPPER_COORDINATOR_INSTANCE_ID values are converted to topic-safe slugs by ProcessLauncherService.coordinator_topic_slug(); the default instance therefore emits on coord-0, and instance 7 emits on coord-7. The launcher emits these best-effort: if no MessagePublisher is wired the helper no-ops, and send failures are logged without corrupting process state.

The desired-state control plane adds two further ZMQ topic families that are validated by validate_topic but deliberately kept off WS RBAC (the payload HMAC signature, not a topic ACL, is the trust boundary). After a PATCH /api/processes/{name}/desired-state, the API coordinator publishes a signed ProcessCommandData nudge on processes.commands.<coordinator> (3 segments) so the owning coordinator reconciles immediately, and that coordinator replies with ProcessCommandAckData on processes.events.command_ack.<coordinator> (4 segments) so the API's blocking PATCH resolves against the ack (falling back to the periodic reconcile when no publisher/registry is wired or the ack times out).

Message Data Classes

All messages use typed Data classes (Pydantic) from snapper.messaging.schemas.data. Every Data class inherits from StrictDataSchema and carries:

Field	Type	Description
public_id	string	UUID7 external identifier, stable across REST and WS
type	string	Literal type discriminator
timestamp	datetime	Message creation time
session_id	string	UUID7 of the producer process session (required at construction)
sequence_id	int	Monotonic counter per topic within the session (required at construction)
topic	string | None	Routing key stamped onto the payload by publish_to() at the publish call site; None on REST-only items and control frames

session_id and sequence_id are allocated by producers from a shared SequenceTracker before the payload is constructed. MessagePublisher.send() serializes and forwards the already-complete payload without modifying provenance. Consumers can use these fields to detect message loss and producer restarts.

messaging.schemas.messages remains the parser/registry module: it exposes parse_message(), MessageParseError, GapEnvelope, MarketDataMessage, and MESSAGE_TYPE_MAP.

TickData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import TickData

tick = TickData(
    public_id="019e1a2b-0000-7000-8000-000000000101",
    session_id="019e1a2b-0000-7000-8000-000000000001",
    sequence_id=1,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="kraken",
    bid=41990.0,
    ask=42010.0,
    last=42000.0,
    volume=0.5,
)
```

Fields:

Field	Type	Description
type	string	"tick"
public_id	string	UUID7 external identifier
instrument	string	Instrument symbol
exchange	string	Exchange name
bid	float | None	Best bid price
ask	float | None	Best ask price
last	float | None	Last traded price
volume	float	Volume
is_delayed	bool	Whether the feed delivers exchange-delayed ticks (e.g. ~10 min for TradFi index futures); strategies must gate on this before treating the price as current
is_extended_hours	bool | None	Whether the tick occurred during an extended-hours TradFi session; None means the feed does not distinguish extended-hours ticks
timestamp	datetime	Timestamp
session_id	string	Producer session (inherited)
sequence_id	int	Monotonic counter per topic or logical stream (inherited)

CandleData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import CandleData

candle = CandleData(
    public_id="019e1a2b-0000-7000-8000-000000000102",
    session_id="019e1a2b-0000-7000-8000-000000000001",
    sequence_id=2,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="kraken",
    timeframe="1h",
    open_at=datetime(2026, 1, 18, 12, 0, tzinfo=UTC),
    open=42000.0,
    high=42500.0,
    low=41800.0,
```

```
    close=42300.0,
    volume=1234.56,
)
```

Fields:

Field	Type	Description
type	string	"candle"
public_id	string	UUID7 external identifier
instrument	string	Symbol
exchange	string	Exchange
timeframe	string	Timeframe (1m, 5m, 1h, etc.)
open_at	datetime	Exchange-provided candle interval start time
open	float	Open price
high	float	High
low	float	Low
close	float	Close price
volume	float	Volume
vwap	float | null	Volume-weighted average price (optional)
trades	int | null	Number of trades in the candle (optional)
complete	bool	Whether the bar's window has closed. false marks a provisional intra-minute update of a still-forming bar (the "living" candle a UI redraws in place); true marks the final bar. Defaults true. Native 1m bars derive it best-effort from the timeframe window; synthesized bars carry the aggregator's trustworthy-boundary flag.
timestamp	datetime	Timestamp

SignalData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import SignalData

signal = SignalData(
```

```
    public_id="019e1a2b-0000-7000-8000-000000000201",
    wallet_public_id="019e1a2b-0000-7000-8000-000000000301",
    session_id="019e1a2b-0000-7000-8000-000000000010",
    sequence_id=1,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="paper",
    side="buy",
    strength=0.85,
    reason="RSI <= 30",
    strategy_name="rsi_btc_1h",
    price=42000.0,
    fired_at=datetime.now(UTC),
)
```

Fields:

Field	Type	Description
type	string	"signal"
public_id	string	UUID7 external identifier
instrument	string	Symbol
exchange	string	Target exchange
side	string	"buy" or "sell"
strength	float	Signal strength 0.0-1.0
reason	string	Reason
strategy_name	string	Strategy name (required for paper exchange signals)
price	float | None	Suggested entry/exit price (optional)
fired_at	datetime	Domain time when signal was generated
timestamp	datetime	System timestamp
paired_group_id	string | None	Paired-execution group identifier; unset for standalone signals
paired_group_size	int | None	Number of legs in the group (must be >= 2)
paired_group_index	int | None	This leg's position in the group (0 <= index < size)
paired_group_policy	string | None	Coordination policy: simultaneous or sequential_handoff
paired_group_key	string | None	Canonical sorted {exchange}:{instrument}:{mode} leg-set key

Field	Type	Description
wallet_public_id	string	Owning wallet public id. The schema defaults to empty, but the trade coordinator drops signals whose wallet is empty or whitespace-only before engine/group persistence.
operator_public_id	string | None	Operator scope
user_public_id	string | None	User scope
ai_review_public_id	string | None	Citation of an approved AI delegate review (CONSULT outcome), threaded into the attribution-aware caps gate
ai_review_dispatch_version	int | None	Companion dispatch version for the bridge dedup contract (carried transport-only)

Paper signals (exchange == "paper") require strategy_name to be set. The schema enforces this invariant at construction time so invalid paper signals cannot be created.

Missing or blank string wallets are a normal coordinator refusal: the next valid signal can still be processed. Explicit JSON null is rejected by the signal schema before routing; this refusal does not provide recovery from all malformed frames. Nonblank wallet spelling is preserved.

The paired-group descriptor is fail-closed and all-or-nothing: a model validator requires either every paired_group_* field unset (a standalone signal) or all five set together, with paired_group_size >= 2, 0 <= paired_group_index < paired_group_size, and a non-empty paired_group_key. The descriptor (without paired_group_key) is also carried transport-only on the downstream order schemas — OrderRequestData, OrderData, ExecutionData, and OrderEventData — so subscribers can attribute order flow to its paired group.

OrderRequestData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import OrderRequestData

order = OrderRequestData(
    public_id="019e1a2b-0000-7000-8000-000000000301",
    session_id="019e1a2b-0000-7000-8000-000000000020",
    sequence_id=1,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    side="buy",
    order_type="limit",
    price=42000.0,
    quantity=0.1,
```

```
    strategy_id="rsi_btc_1h",
    mode="live",
)
```

order_type carries the core order-type vocabulary — market, limit, stop, or stop_limit (the OrderType Literal). Venue wire spellings such as stop-loss / stop-loss-limit never appear in orders.commands.* or orders.events.* payloads: the executor translates to each venue's wire contract at the venue boundary, immediately before the exchange call. stop_price is the trigger price, required for stop / stop_limit orders — before any venue send, the executor refuses a stop-typed request without a trigger (and any order type outside the core vocabulary), publishing a definitive rejected and recording a durable order_rejected venue event.

ExecutionData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import ExecutionData

execution = ExecutionData(
    public_id="019e1a2b-0000-7000-8000-000000000401",
    session_id="019e1a2b-0000-7000-8000-000000000030",
    sequence_id=1,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    exchange_order_id="KRAKEN-456",
    side="buy",
    price=42000.0,
    size=0.1,
    last_size=0.1,
    last_price=42000.0,
    fee=0.001,
    fee_asset="USD",
    status="filled",
    executed_at=datetime.now(UTC),
)
```

OrderData

OrderData is the canonical event for order state. It carries the request-time margin metadata (leverage, reduce_only) all the way to subscribers and the REST /orders endpoint, so the frontend can render shorts and reduce-only intent.

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import OrderData

order_status = OrderData(
    public_id="019e1a2b-0000-7000-8000-000000000501",
```

```
    session_id="019e1a2b-0000-7000-8000-000000000020",
    sequence_id=2,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    exchange_order_id="KRAKEN-456",
    side="sell",
    order_type="limit",
    size=0.1,
    filled_size=0.0,
    status="accepted",
    price=42000.0,
    created_at=datetime.now(UTC),
    leverage=3,
    reduce_only=False,
)
```

HeartbeatData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import HeartbeatData

heartbeat = HeartbeatData(
    public_id="019e1a2b-0000-7000-8000-000000000601",
    session_id="019e1a2b-0000-7000-8000-000000000001",
    sequence_id=1,
    timestamp=datetime.now(UTC),
    component="zmq_broker",
    status="healthy",
    sequence=1,
    lag_ms=5,
)
```

ProcessSummaryEventData

Full snapshot published on processes.events.summary.<instance_id> whenever the launcher detects a status transition. Frontend replaces its React Query cache entry wholesale — no diff reconciliation.

Fields:

Field	Type	Description
type	string	"process_summary_event"
coordinator	string	Topic-safe slug of the emitting node (default coord-0); mirrors the topic suffix so consumers can attribute rows to the coordinator that sampled them
	string | None	

Field	Type	Description
coordinator_label		Human-readable container label (API / Feed / Strategies) derived from the emitting node's autostart profile, None for the single-container ALL profile; lets consumers render a friendly name instead of the raw coord-<id> slug
processes	list[ProcessSummaryItem]	Unordered snapshot of per-process rows (configured first in repository row order, then runtime per-wallet instances — consumers must not rely on this order)
snapshot_at	datetime	Bus time the snapshot was assembled

ProcessSummaryItem fields: name, running, enabled, role, lifecycle, active_public_id, rss_bytes (int | None, subprocess resident set size in bytes; None for thread-mode/unsampled processes), cpu_percent (float | None, whole-process CPU% which can exceed 100 on multi-threaded children; None under the same conditions and 0.0 on the first sample after a restart), and owned (bool, default False; True when the emitting node's autostart profile runs this process or it is locally running — lets consumers attribute a not-running process to the container that owns it by profile rather than to whichever node most recently listed it). PID / command / exit_code are intentionally absent — they only exist for native subprocesses, so detailed status is fetched via REST when needed.

ProcessConfiguredEventData

Published on processes.events.configured.<instance_id> from four launcher sites: create_process_config (persisted-config-row create), spawn_per_wallet_executors (per-wallet executor instances appear), update_process_config (desired-state enabled/restart_nonce writes), and update_process_config_parameters (strategy scope editor). The launcher does NOT emit on persisted-row delete or on per-wallet teardown today. process_names is the sorted union of persisted config names AND instance_configs.keys() at snapshot time.

Fields:

Field	Type	Description
type	string	"process_configured_event"
process_names	list[string]	All configured process names at snapshot time
snapshot_at	datetime	Bus time the snapshot was assembled

ProcessRunEventData

A single process-run lifecycle transition. Per-run (not snapshot) so consumers can append to their run-history view without re-fetching.

Fields:

Field	Type	Description
type	string	"process_run_event"
process_name	string	Owning process
run_id	string	Stable identifier for this run
status	string	running / succeeded / failed / cancelled (mirrors ProcessRunStatusEnum.value)
started_at	datetime	Bus time the run kicked off
completed_at	datetime | None	Bus time the run finished; None while in flight
exit_code	int | None	Native subprocess exit code; None for async-task processes (the launcher folds non-zero exits into the run record's error field) and None while in flight

StrategyListEventData

Published on strategies.events.list.<instance_id> whenever a STRATEGY-role process config is created or its start/stop state transitions. The launcher derives the payload from **persisted** process configs filtered by role == STRATEGY — NOT from the in-memory StrategyFactory.STRATEGY_CLASSES registry (which is static after import time). Frontend uses this to refresh the Strategies view without REST polling.

Fields:

Field	Type	Description
type	string	"strategy_list_event"
strategy_classes	list[string]	Sorted canonical class paths of every active STRATEGY-role process config
snapshot_at	datetime	Bus time the snapshot was assembled

Publisher

MessagePublisher (recommended)

MessagePublisher sends fully constructed payloads to an explicit stream_key. Producers allocate session_id and sequence_id from a shared SequenceTracker before constructing the payload, then pass both the topic and payload to MessagePublisher.send().

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import
ValidatedPublisher, apply_hwm, HWM_MARKET_DATA
from snapper.messaging.infrastructure.publisher import MessagePublisher, Sequenc
eTracker
from snapper.messaging.topics.builders import topic_for_message

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.PUB)
apply_hwm(raw_socket, sndhwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7500")

tracker = SequenceTracker()
publisher = MessagePublisher(ValidatedPublisher(raw_socket), tracker)

topic = topic_for_message(candle_data)

await publisher.send(topic, candle_data)
publisher.close()
```

For brevity, the example assumes candle_data was already constructed with session_id and sequence_id allocated from tracker.

One SequenceTracker per component process; all MessagePublisher instances within that component share it. Counters survive socket reconnects — only a full component restart creates a new session.

Per-Topic Counters

SequenceTracker maintains monotonic counters keyed by **ZMQ topic** (e.g. market.kraken.BTC-USD.candles.1m, orders.events.kraken.BTC-USD.executed). Each topic has its own independent counter. Producers allocate provenance using the final ZMQ topic string and then pass that same topic to MessagePublisher.send().

For DB-only writes that do not flow over ZMQ (instruments, users, settings seed data), the counter key is the destination table name as a logical identifier.

For WS control and REST middleware paths, logical channel names are used: server.control, server.telemetry, rest.control, rest.telemetry.

For paper market data or other cases where the topic cannot be derived from the payload alone, pass an explicit topic override:

```
topic = "market.paper.kraken.BTC-USD.ticks"
await publisher.send(topic, tick_data)
```

ValidatedPublisher (low-level)

Direct access to the socket wrapper without StrictDataSchema serialization helpers:

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import
ValidatedPublisher, apply_hwm, HWM_MARKET_DATA

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.PUB)
apply_hwm(raw_socket, sndhwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7500")
publisher = ValidatedPublisher(raw_socket)

topic = "market.kraken.BTC-USD.candles.1h"
payload = candle_data.publish_to(topic)
await publisher.send_multipart(topic, payload)
publisher.close()
```

Subscriber

Subscribing to messages via validated socket wrapper around a raw ZMQ SUB socket:

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import ValidatedSubscribe
r, apply_hwm, HWM_MARKET_DATA
from snapper.messaging.schemas.messages import parse_message

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.SUB)
apply_hwm(raw_socket, rcvhwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7501")  # broker XPUB
subscriber = ValidatedSubscriber(raw_socket)
subscriber.subscribe("market.kraken.BTC-USD.")

while True:
    topic, payload = await subscriber.recv_multipart()
    envelope = parse_message(payload.decode())
    print(f"Received {envelope.type} on {topic}")
```

Market Data Publisher

Built-in publisher for exchange data:

```
snapper feed --symbols "BTC-USD,ETH-USD"
```

Programmatically:

```
from snapper.messaging.publishers.kraken import KrakenMarketDataPublisher

async def run_feed() -> None:
    publisher = KrakenMarketDataPublisher(symbols=["BTC-USD", "ETH-USD"])
    await publisher.start()
```

Candle Identity Cache

On startup the publisher loads the latest candle public_id per (instrument, timeframe) pair from the database into an in-memory cache. The startup load is bounded to a recent open_at window (_CANDLE_ID_CACHE_LOOKBACK, 2 days) so the warm-up rides the (instrument, open_at) index instead of scanning the full (multi-hundred- million-row) candles table — an unbounded scan otherwise blocked the publisher's startup for minutes before its WebSocket connected. Bounding it is safe: only the *current* open interval needs identity continuity, and intervals older than the window are closed (no further updates). When a new tick arrives for an existing candle interval (open_at matches), the cached public_id is reused so the same logical candle keeps a stable identity across ZMQ, WebSocket, and the database. When open_at advances to a new interval (or falls outside the warm-up window), a fresh UUID7 is minted and the cache entry is replaced. This avoids any DB reads on the hot path.

Higher-Timeframe Candle Synthesis

When more than one timeframe is configured (timeframes setting, e.g. ["1m", "1h", "1d"]), the publisher subscribes to the venue's 1m stream **only** and synthesizes every higher timeframe client-side from the finalized 1m candles. Each synthesized bar is published on the same topic family as a native candle (market.{exchange}.{instrument}.candles.{timeframe}), so subscribers cannot tell a synthesized 1h/1d bar from a venue-native one.

Key properties:

- **UTC boundaries** — every timeframe aligns to UTC (1d closes at 00:00 UTC, fixed intervals to floor(unix / interval) * interval). This matches Kraken's native OHLC boundaries and the Polygon historical corpus, so live bars line up with backtest/warm-up data.
- **Finalize once** — the venue emits many in-progress frames for the open minute; a minute is folded into the higher timeframes exactly once, with its final value, when a later minute is observed (per symbol).

- **Emit on close** — a higher-TF bar is published only when its window has closed (a finalized 1m at or after the window end). open_at is the window START; consumers must treat closed_at = open_at + timeframe as the true close — do not act on a bar before then.
- **Persisted (provenance-tagged)** — synthesized bars are published to ZMQ and, when a higher timeframe is configured for persistence (off by default — timeframes defaults to ["1m"]), also written to the candles table tagged source='synthesized' and carrying the aggregator's complete trustworthy-boundary flag, under the SAME persist policy (_should_persist_row) as the 1m stream. Provenance is source ∈ {native, calculated, synthesized} + complete (a trustworthy-boundary bool, NOT a full-minute-coverage assertion). native = venue-precomputed upstream OHLC (Kraken spot ohlc:1m, Polygon history, Kraken futures/equities 1m REST-aggregate backfills); calculated = Snapper-built live 1m from the trade/quote stream (Kraken futures/equities, Walutomat — tagged from migration 0012 forward); synthesized = higher-TF rollups. source names the MECHANISM that produced the bar, NOT the price it was marked on; the candles.price_basis column (migration 0040, nullable, no CHECK, vocabulary snapper.core.types.PriceBasis) names the price. Walutomat 1m bars are marked on the venue's top-of-book mid (bid + ask) / 2, carry no trade print (trades = 0) and no volume, and are tagged price_basis = 'quote_mid'. That convention changed FORWARD-ONLY at the cutover: before it those bars carried the venue's externally-sourced forex_now reference rate, which sat outside the tradable spread and could stay frozen for hours. No history is rewritten, so a Walutomat row with price_basis IS NULL is a pre-cutover reference-rate bar and the series is discontinuous at the cutover minute — that exact UTC minute is stamped in the deploy note for this change. The step was measured before deploy against production quotes: EUR-PLN −8.0 bps and USD-PLN −4.0 bps, ≈ −0.014 USD on 1520.29 equity, i.e. a phantom drawdown of ≈ 1e-5 against the 0.01 threshold that would have forced an explicit peak decision, so no risk peak was re-seeded. That measurement covered ONLY the majors the wallet basket touches: the exotic pairs' own forex_now-to-mid step was NOT measured, and on a book as wide as TRY-PLN (1197 bps) it may be percent-scale rather than bps-scale. The phantom_dd ≈ 1e-5 conclusion stands for the basket; it says nothing about the exotic series. The candle READ path single-source cutover is gated by the candle_single_source setting (default OFF): while OFF, the /api/candles smart route still derives 5m/15m/30m on-read from the 1m cache; when ON, those frames serve single-source from the persisted plane (get_candles), retiring the on-read derive_snaps rollup and closing the dual-source hazard. Enable it only after the persisted plane is populated and verify-candle-coverage passes (else the read would be empty/short). 1m stays cache-served and 1h/4h/1d are already DB-served regardless.
- **Native 1m persistence is final-only by default** — every native 1m frame is published to ZMQ (the living candle), but the NativeCandleFinalizer holds the in-progress minute and persists one final complete=True bar per window (later complete

corrections may create a new SCD2 version) — on the next minute's first frame (boundary), or via a 30s wall-clock flush for an illiquid/stalled symbol, or on shutdown drain. This eliminates the intra-minute SCD2 "temporary candle" churn. Operator caveat: a 1m bar's DB row therefore lags the live minute by up to one flush interval (~35s); the cache//api/candles live reads are ZMQ-fed and unaffected. A hard crash loses the held in-memory minute unless a later venue replay or backfill restores it; the finalizer itself is not a durable queue. Set persist_intermediate_candles=true to also persist the in-progress complete=false frames (superseded by the final).

- **Restart rebuild** — on startup the open window of each higher timeframe is rebuilt from the persisted 1m history (excluding the current open minute, which the live stream finalizes), so a mid-window restart does not truncate that window's bar. The seed reads each row's durable complete flag (not a write-time heuristic) to decide finality, so a finalized illiquid bar sealed with a pre-close write timestamp is correctly seeded.
- **Forward-fill (opt-in, default off)** — with candle_forward_fill enabled, a time-driven flush (every 30s) seals a wall-clock-ended window that no later 1m arrived to close, and forward-fills empty windows with a flat bar (open = high = low = close = vwap = prior close, volume = 0, trades = 0) so a thin instrument still yields a contiguous series. It operates only on the live region (windows that opened at or after the publisher's live epoch), so seeded/restart windows stay owned by the data path, and a late 1m for an already-sealed window is dropped (counted in late_rolls_after_close), never re-emitted. Enable it ONLY where the warm-up corpus is contiguous at the timeframe — 24/7 crypto qualifies; session-based equities (and 24/5 FX without verified weekend rows) must leave it off, or the live series would carry bars the strategy was never validated on. This is ENFORCED per venue: the global candle_forward_fill setting is gated by each publisher's _supports_forward_fill() (only the kraken-spot publisher returns true), so on any other venue — or in paper replay — forward-fill is forced OFF with a warning even if the flag is set. The flush wall-clock also applies a small grace so a just-ended minute is not sealed before the venue can deliver its final frame.
- **Feed breaks re-arm the live epoch** — the flush loop is process-scoped, so without this it keeps running across a disconnect and carries the pre-gap close forward into windows nobody observed, marking them complete. On a break the aggregator advances its live epoch, purges the buckets that opened before it, and drops the carried close and last-real-window marker. A window straddling the gap is therefore MISSING rather than truncated or flat-filled: a hole is honest, a fabricated bar is not. Forward-fill resumes on its own once a fresh real bar re-seeds the carry, so a break costs only the windows it actually spans. Four call sites feed it: stall detection, confirmed recovery (which moves the epoch to when data really resumed, closing the one-window hole detection alone would leave), the forced WebSocket restart (where meaningful time can pass between detection and the actual teardown), and the kraken SDK reconnect hook — SDK-internal reconnects never reach the liveness

watchdog at all, because the socket returns on its own and messages resume with no stall ever detected. Every one of those four sites ALSO opens a minute-completion feed break, and the pairing is not decorative. The two hooks bar different things — the epoch governs higher-TF forward-fill, the break window governs 1m completion — and the minute bar is a sliding now + resubscribe_settle, so an outage longer than that window would see the bar set at detection expire before data resumed. Re-marking at confirmed recovery is what covers the subscription-replay window.

- **Minute completion on the base 1m plane (opt-in, default off)** — with candle_minute_completion enabled, a venue whose 1m source is trade-driven and whose market never closes (Kraken spot in trade_built mode, Kraken futures) also emits a flat bar for every minute it observed the venue live and in which the instrument did not trade. The bar has the same shape as the forward-fill bar above (open = high = low = close = vwap = prior close, volume = 0, trades = 0, complete = true) and is published and persisted through the same synthesized path, so its row carries source = synthesized. The queryable signature is therefore timeframe = '1m' AND source = 'synthesized' AND trades = 0 AND volume = 0, which needs no schema change — but **all four conjuncts are required**. trades = 0 AND volume = 0 alone is NOT a discriminator: walutomat emits every real 1m bar with volume = 0 and trades = 0 (a genuine tick-mean price, no reported size), so the shorter test reads that venue's whole corpus as flat no-trade bars. The source clause is what makes the rule sound corpus-wide, and timeframe is what separates these from the higher-TF rollups, which are also synthesized. Emission is gated on live evidence and every gate is fail-safe — no evidence means NO row, never a guessed one:
  - the venue must have been witnessed by inbound public frames continuously from just before the minute opened to just after it closed (ticks, trades and real candle frames all count; a monotonic "time since last message" watermark cannot answer this, so the publisher keeps a small ring of per-minute witness spans);
  - no detected reconnect, WS restart or liveness recovery may overlap the minute;
  - the symbol's trade subscription must be confirmed and not quarantined right now, read from the exchange client's subscription-health tracker (never from the symbol mapper, which would name pairs the connection never subscribed, and never from the ticker channel, which a reconnect re-seeds as confirmed wholesale); and
  - the symbol must have printed at least one real bar in THIS process. The carried close is never seeded from the database: reconstructing a flat bar after the fact cannot distinguish "nobody traded" from "we were not listening", so a restarted publisher produces no flat bars for an instrument until it trades again. A minute that fails any gate is simply absent, and it is never retro-filled — the sweep considers only the single minute that has just settled. Refusals are counted per reason and surfaced on the feed heartbeat under meta.venue.minute_completion, so a hole in the plane is explainable rather than merely asserted. Two consequences worth planning for: the daily candles volume

on Kraken spot rises roughly five- to sixfold on a table with no retention policy, and a higher-TF window whose first or last minute was tradeless now opens or closes on the carried price rather than the first or last TRADED price (the price is a real prior print, but bars either side of the cutover are computed differently). Enabling also widens the higher-TF flush grace past the sweep settle, so higher-TF bars publish about 20 s later and now include their final minute, which the default 5 s seal could drop as late.

- **Native-feed replacement (kraken spot)** — kraken spot is the one venue with a native higher-TF OHLC feed; synthesizing those timeframes from 1m REPLACES it (a deliberate one-mechanism choice — the rolled-up VWAP/trades approximate the native bar). The publisher logs a one-time warning naming the replaced timeframes so the fidelity tradeoff is not silent.
- **Paper replay** — the paper publisher synthesizes higher timeframes from the replayed 1m history exactly as a live publisher does (so backtest bars match forward-test bars and no persisted higher-TF rows are required), but anchors the aggregator's live epoch at the REPLAY START rather than wall-clock now — so historical windows are trustworthy and emitted instead of being suppressed as pre-epoch. Paper does not run the restart seed (the full replay rebuilds every window) and never forward-fills (it is not a continuous-corpus live feed).

Micro-Batch DB Persistence

DB writes are decoupled from the ZMQ publish path via dedicated per-stream writer tasks. The producer loops (candles, ticks, trades) are ingest-only: each drains its exchange subscription iterator, publishes to ZMQ, and enqueues the resulting row onto a bounded per-stream write queue. A dedicated writer task per stream consumes its queue with a long-lived database session, accumulating rows and flushing on a size or age threshold — DB commit latency never blocks the producer loop:

- **Size trigger**: candles flush at 100 rows, ticks/trades at 500 rows
- **Age trigger**: all streams flush after 50 ms since the first batch item
- **Publish-first**: ZMQ delivery happens before DB persistence; subscribers receive data without waiting for the database

Thresholds are configurable via write_buffer_flush_ms, write_buffer_candle_max_rows, write_buffer_tick_max_rows, and write_buffer_trade_max_rows settings (cached at publisher start, require process restart to change).

The write queues are bounded (20 000 rows for ticks and candles, 5 000 for trades) with drop-oldest overflow: when a queue is full, the enqueue helper evicts the oldest queued row to make room for the new one and emits a rate-limited WARNING with the drop count. This is a persistence backlog, separate from wire delivery: those rows were already published over ZMQ, although publication does not prove subscriber receipt. Queue eviction skips their DB persistence.

If a writer task's database session is lost mid-flush, the task holds its unflushed rows, reopens the session with capped exponential backoff (1 s doubling up to 30 s), and resumes draining — queued and in-flight rows survive session loss while the process remains alive, subject to queue overflow; a process crash loses in-memory queue and batch contents.

On shutdown (launcher cancellation or direct stop), remaining rows are flushed by the writer drain loops, which keep draining the queue and the in-flight batch after the running flag goes false until both are empty; stop() joins each queue before awaiting its writer task. The producer loop's asyncio.wait poll still uses a persistent future that is never cancelled on timeout, preserving the underlying exchange subscription iterator — the age-based flush itself is driven by the writer task's batch timer.

Candle flushes that hit an IntegrityError (e.g. out-of-order timestamps) fall back to row-by-row retry, isolating the bad row without losing the rest of the batch. The repository upsert_candles() contract is unchanged.

Per-stream flush error counters (_flush_errors) drive the heartbeat status field: "warning" if any stream has errors, "healthy" otherwise. Counters reset to zero on the next successful flush of that same stream.

A feed heartbeat also carries whatever venue-specific health its publisher contributes through _venue_feed_health() (base: nothing). The venue's keys are nested under meta.venue — never merged flat — so no venue contribution can shadow the standard symbols, symbol_count or running keys; the key is omitted entirely when a publisher contributes nothing. Its degraded flag escalates status to "warning" alongside the flush-error condition.

Walutomat uses it for the fail-closed mark refusals:

- meta.venue.refused_marks — symbols whose top-of-book quote is currently unusable, or which have vanished from the venue payload altogether (an absent pair produces the same frozen mark and the same candle hole as an unusable book, so it is latched the same way).
- meta.venue.refused_mark_counts — cumulative refused-poll count per symbol, never reset on recovery.
- meta.venue.refused_mark_seconds — how long each currently-refused symbol has been *continuously* refused.
- meta.venue.refused_mark_fractions — the refused share of each symbol's polls over a one-hour rolling window, published only for symbols with at least 60 polls in the window and at least one refusal among them.

The feed degrades to "warning" when either escalation rule fires: a symbol refused continuously for WALUTOMAT_REFUSAL_WARNING_SECONDS (600 s = 60 consecutive refused polls at the default 10 s cadence), or a symbol whose windowed refusal fraction exceeds WALUTOMAT_REFUSAL_FRACTION_CEILING (0.5). The second rule exists because recovery pops the streak clock, so a book that is unusable on almost every poll but

flickers usable once in a while would never satisfy the first. Neither bound is measured — no distribution of refusal durations exists yet — so both are conservative operational choices to be revisited against real refused_mark_seconds history. Note that the bound is about *refusal* duration, not about tick cadence: the refusal latch clears on the first usable poll and per-symbol lag is refreshed before tick deduplication, so a healthy pair with an unchanged book is refusal-free and lag-fresh at every poll no matter how long it goes without emitting a tick.

Transition logs (mark refused / mark recovered) are rate-limited to one line per kind per symbol per 5 minutes, with the suppressed count carried on the next line that gets through. Without that bound a flapping pair emits thousands of lines a day on its own.

All of this exists because fail-closed is fail-STALE on the ticker plane: it applies no age gate, so a chronically refused pair keeps serving its frozen last-good mark.

Restart semantics. The refusal counts, the continuous-refusal clock, the rolling poll window and the notifier's own "3 consecutive WARNING heartbeats in 10 minutes" window are all in-memory, and every feed publisher runs under restart_policy=ALWAYS. Two consequences follow. A publisher restart cadence shorter than the 600 s escalation bound means a chronic refusal is never visible long enough to escalate — the clock restarts from zero on every respawn. And during incident review, post-restart counts describe only the current process lifetime, so they understate a days-long condition. The durable signal is the candle plane: a refused pair writes no bar, so the holes persist in the database across every restart and are the record to reconcile against when the in-memory counters disagree with the operator's memory.

lag_ms is the worst lag across the union of the concrete subscribed symbols and every symbol that has been seeded or has ever delivered. Every subscribed symbol is seeded into _last_data_timestamps at subscribe time, so a symbol that has never delivered reports the time since subscription; a symbol that delivered and then went dark reports its real growing lag. Without both halves such a symbol fell back to the FEED-level dark interval, which is near zero whenever the venue's other symbols are healthy — a permanently dark pair read as fresh forever after every restart.

The subscribe-all sentinel "*" is never a key in per-symbol state: it is a request, never a delivery key. A publisher that forwards it verbatim instead resolves it to concrete symbols at seed time through _wildcard_symbol_universe() (Walutomat: the connected client's pair list), so never-delivered pairs are seeded too. The base implementation resolves nothing, which keeps Kraken spot's ~1900-symbol wildcard on the feed-level fallback — expanding it would let one dark low-volume pair pin the whole feed to a degraded lag reading.

Where a venue caps symbols per connection, the cap is applied to the publisher's own symbol list, so the heartbeat's symbols / symbol_count and the lag loop describe the same set. The truncated tail is never subscribed and never delivers; it is reported at start as a WARNING advising multiple instances rather than carried in the heartbeat as a permanently stale subscription that does not exist.

Order Executor

Service executing orders:

```
snapper executor -e kraken
```

Executor:

1. Subscribes to orders.commands.{exchange}.{instrument}.submit, .cancel, and .replace topics, plus system.symbol_aliases and system.settings
2. Receives OrderRequestData and OrderCancelData from the durable trade outbox, and accepts direct OrderReplaceData command frames from compatible publishers
3. Executes order via exchange API
4. Persists VenueEvent rows for accepted (order_accepted), fill (fill_observed), terminal (order_terminal), rejected (order_rejected), ambiguous-submit (order_submit_unknown), breaker-open (order_breaker_open), and interlock-blocked (order_interlock_blocked) observations
5. Publishes ExecutionData, OrderData, or lightweight OrderEventData

Before the venue order-placement call, each submit passes the pre-placement safety checks. The duplicate-submit guard drops outbox replays of an already-evidenced client_order_id, checking the live pending entry, the unhealed-accept queue, and the durable has_order_submit_evidence probe (which excludes order_rejected rows so a legitimate retry after a definitive reject still flows). A failed durable probe is fail-closed — the command is dropped as if duplicate — and dropped duplicates publish nothing, with two exceptions: a replay whose durable evidence includes order_breaker_open or order_interlock_blocked reruns that terminal disposition described below instead of dropping silently, so an executor crash mid-disposition cannot leave the engine's intent held forever. The staleness gate then handles commands older than the dispatch TTL (TRADE_COMMAND_DISPATCH_TTL_S, default 30 s, measured from the published frame's signaled_at, which the outbox derives from the durable trade-command row's created_at) with venue-truth-backed outcomes: a single client-id lookup runs first; an order found on the venue is adopted as accepted, an unverifiable venue (lookup unsupported or unreachable) makes the frame drop silently, and only a verified absence publishes rejected and records an order_rejected venue event — the durable row is written only after a confirmed publish, so a failed publish leaves the command for the reconciliation loop to release durably.

When a submit fails ambiguously (the order may exist on the venue), the executor never fabricates a rejection. It parks the pending entry and verifies venue truth by client id via find_order_by_client_id: a found order is adopted as accepted, while a definitive rejection requires two consecutive authoritative not-found answers — a lookup that could not be checked never counts as the venue saying no. If verification cannot resolve, the executor publishes OrderData on orders.events.{exchange}.{instrument}.unknown (non-terminal; the engine holds its in-flight guard) and the reconciliation loop re-verifies parked entries each cycle until the order resolves to accepted or rejected.

A submit refused because the venue circuit breaker is open gets a distinct disposition instead of a fabricated venue rejection: the executor records a durable order_breaker_open venue event (counted as duplicate-submit evidence, so a late redispatch of the same command never resubmits), marks the durable command FAILED, and only then publishes rejected with reason="circuit_breaker_open" in the OrderData payload — subscribers can distinguish the local infrastructure refusal from a venue rejection. If any step fails, the pending entry is parked and the reconciliation loop reruns the sequence; the engine releases its in-flight intent only on the confirmed publish.

A live-trading interlock gates every non-paper submit before the venue order-placement call (snapper.messaging.executors.base). The discriminator is the executor's own venue — never the caller-supplied order.mode, which a client could forge — so paper venues always pass. For any other venue the live_trading_mode DB setting is read **fresh per submit** (deliberately bypassing the ZMQ-refreshed settings cache, so a lost system.settings broadcast cannot leave a stale kill-switch): only enabled proceeds, while halted, reduce_only (which blocks like halted today), and a missing or unreadable value (surfaced as live_trading_mode_unavailable) all fail closed. A block runs a distinct disposition mirroring the breaker-open path — a durable order_interlock_blocked venue event (counted as duplicate-submit evidence; a redispatch reruns this terminal disposition instead of reaching order placement), the durable command CASed to FAILED, and only then a published rejected — so a replayed frame can never execute after the mode later flips to enabled. If any step fails the entry is parked for the reconciliation loop to rerun.

Fills arrive on the venue's private execution WebSocket stream. A supervisor task owns the stream's lifecycle: any termination — SDK reconnect-budget exhaustion during an outage, a raised error, even a clean generator return — counts as death and triggers a respawn with capped jittered backoff (1 s doubling to 60 s; a stream that survived 5 minutes resets the backoff), so a dead stream can no longer leave fills permanently dark while the process keeps reporting healthy. Before each respawn the supervisor runs a best-effort reconciliation pass that heals the dark-window fill gap first, and the resubscribe is idempotent: both Kraken clients suppress subscribe-time snapshots (spot requests snap_orders/snap_trades as explicit false, futures drops fills_snapshot frames),

so neither a respawn nor an in-budget SDK reconnect that replays the cached subscription can re-deliver already-booked fills. Venues without WebSocket executions skip the stream entirely; the periodic reconciliation loop is their only fill source.

Fill booking is atomic per order: a lock on the pending entry serializes the dedupe gate, delta computation, durable fill_observed write, execution publish, watermark advances, and lifecycle removal across the live stream task, reconciliation correctives, and cancel handling. Duplicates are dropped via an exec-id LRU plus a cumulative-monotonic guard; the committed cumulative advances only after a successful publish, so a fill whose publish failed is absorbed by the venue's redelivery or by the next fill's cumulative-anchored delta and published deltas always sum to venue truth.

Failed VenueEvent persistence is fail-closed on the fill path: the fill_observed write precedes the execution publish, and a write failure aborts the publish. The accepted path differs because the order is already live on the venue: the accepted status is published anyway, the failed order_accepted write is parked in _unhealed_accept_events, and the reconciliation loop retries the durable write each cycle until it heals.

Replace commands are parsed and wallet-scoped like submit/cancel commands, but no exchange client implements atomic replace today; the executor logs the request and publishes a lightweight OrderEventData(event="rejected") on orders.events.{exchange}.{instrument}.rejected so callers can use an explicit cancel + new-order workflow.

The trade runtime subscribes to orders.events.* to keep TradeService in sync during normal operation. VenueEvent rows are used as the durable recovery and reconciliation backbone. At startup the executor recovers open orders from those rows and the executions log (rows exist only for successfully published fills): recorded-but-unpublished fills are republished under their original exec ids (every consumer dedupes by exec id), and the remaining venue-ahead gap is emitted as corrective fills with deterministic synthetic ids — fills that landed while the executor was down are projected instead of being silently re-baselined away (see docs/operations.md → Recovery-time corrective fills). Corrective fills carry real venue fees: the venue's cumulative commission is tracked per fee currency against what was already published, and each corrective's fee/fee_asset carries exactly the not-yet-attributed remainder, so per-fill live fees are never re-charged. A corrective emits fee-less only when the venue exposes no usable fee source, or after the bounded deferral for a transiently unavailable source expires with a CRITICAL log.

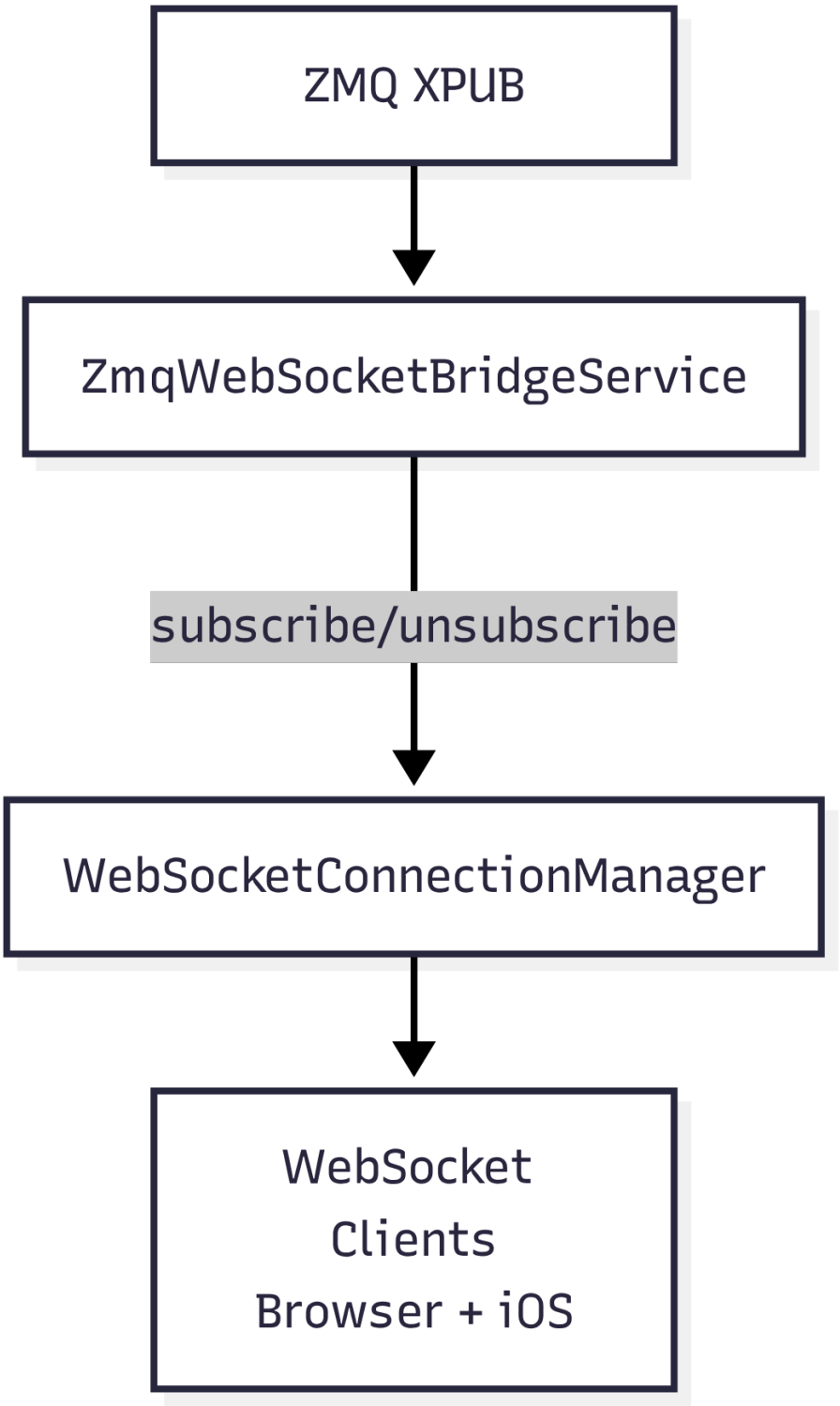
Because parked entries are in-memory, the executor's reconciliation loop also closes restart gaps against venue truth: open venue orders carrying our client order ids but missing from pending state are adopted under the original request rebuilt from the durable command row, and dispatched commands with no resolving venue evidence are verified by client id — a found order is adopted, while repeated authoritative absence (only with the dispatch TTL enabled, and bounded by an age cutoff) publishes rejected. Adoption-shaped accepted events carry reason="adopted" in the OrderData payload — the engine uses it to re-arm its in-flight guard for the adopted order — and a failed

adoption publish is parked and retried by the reconciliation loop until it lands. Subscribers may therefore see accepted or rejected events for orders the current executor process never submitted.

PlanExecutorService (see docs/architecture.md → Execution Plans) also subscribes to orders.events. and market. on the broker XPUB, routes ExecutionData → fill propagation, OrderData → plan status transitions, and TickData → evaluator dispatch. Plan cancels flow through the same TradeCommand outbox as submits, with command_type="cancel" branching _outbox_publish to emit OrderCancelData on orders.commands.{exchange}.{instrument}.cancel.

ZMQ-WebSocket Bridge

Bridge between ZMQ and WebSocket clients (browser and iOS):



Bridge automatically:

- Subscribes to ZMQ topics when WebSocket client subscribes
- Validates each inbound ZMQ message as JSON and runs GapDetector.check() before forwarding

- Drops messages with malformed JSON (logged as WARNING; counted in invalid_messages per topic)
- Applies per-frame scope filters for ai_reviews.*, orders.events.*, portfolio.accounts.*, and alerts.*; account invalidations require the exact typed schema, a UUID7 wallet topic, and topic-payload wallet equality, while every malformed scoped frame fails closed before fan-out
- Forwards messages via _forward_to_clients with per-subscription backpressure
- Throttles forwarding per subscriber with the matching TOPIC_REGISTRY interval; account invalidations use a 500 ms per-wallet window with one trailing delivery of the newest suppressed frame
- Drops market data when a client exceeds MAX_PENDING_MESSAGES_MARKET (100)
- Disconnects slow clients on trade topics when exceeding MAX_PENDING_MESSAGES_TRADE (1000)
- Unsubscribes when last client disconnects
- Records control events (subscribe errors, client disconnects) to the control table

Gap Detection

GapDetector tracks session_id and sequence_id partitioned by the tuple (received_topic, wallet_public_id) — wallet scoping avoids false gaps when interleaved per-wallet streams share a topic — and emits log messages when it finds sequence gaps or producer session resets.

```
from snapper.messaging.infrastructure.gap_detector import GapDetector

detector = GapDetector()
detector.check(received_topic, session_id, sequence_id, wallet_public_id=wallet_
public_id)
```

Rules:

- Messages without provenance (session_id == "" or sequence_id == 0) are **rejected** with a WARNING log and a rejected_unstamped counter increment. This makes unstamped traffic observable rather than silently passing through.
- First message on a (topic, wallet_public_id) stream: sets baseline. If sequence_id > 1 the subscriber joined mid-stream (logged INFO).
- Same (topic, wallet_public_id) stream, new session_id: producer restarted (logged INFO, counter resets).
- In-order message (sequence_id == expected): accepted silently.
- Gap (sequence_id > expected): logs WARNING with the missing range, then advances.
- Duplicate / reorder (sequence_id < expected): logs DEBUG, state unchanged.

GapDetector.check() returns True when the message carries valid provenance and was processed, False when rejected as unstamped.

GapDetector is wired into the ZMQ-WebSocket bridge and into subscriber loops inside the executor, trade runtime coordinator, and strategy _listen_loop().

Message Logger

Debug tool for monitoring ZMQ traffic:

```
snapper zmq-logger --payload --audit-file logs/zmq.jsonl
```

Logs all messages passing through the broker.

Configuration

Endpoints in .env:

```
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
```

Throttling

TopicSchema defines throttling per topic:

```
TopicSchema(
    pattern="market.",
    category="market",
    throttle_ms=100,  # Max 10 msg/s
)
```

Message Validation

ValidatedPublisher and ValidatedSubscriber validate **topic strings**, not message payloads. ValidatedPublisher.send_multipart() calls validate_topic() before sending; ValidatedSubscriber.subscribe() calls validate_subscription_pattern() before subscribing. Both raise TopicValidationError on invalid topics.

High Water Mark (HWM) Policy

ZMQ sockets use explicit high water marks via apply_hwm() from validated_socket.py to configure queue depth. Finite PUB/SUB queues may still drop messages under pressure, and a successful send is not a subscriber acknowledgement. Order-flow sockets use an unlimited local HWM, but the broker remains finite:

Tier	Constant	Value	Used by
Order flow	HWM_ORDER_FLOW	0 (unlimited)	Executors, trade runtime coordinator
Broker	HWM_BROKER	50 000	XSUB (rcvhwm), XPUB (sndhwm)
Market data	HWM_MARKET_DATA	20 000	Publishers, strategies, bridge, settings, symbol updaters
Audit	HWM_AUDIT	10 000	Message logger

apply_hwm() must be called **before** connect() or bind().

Retry and Resilience

Sockets automatically:

- Reconnect on connection loss
- Queue messages subject to socket connection/subscription state and HWM; disconnected or slow subscribers can miss publications
- LINGER=0 on close (discard pending)
- HWM applied before connect/bind (see above)

Topic Derivation

topic_for_message() in snapper.messaging.topics.builders derives the canonical ZMQ topic from a typed Data class. Producers typically call it before MessagePublisher.send().

```
from snapper.messaging.topics.builders import topic_for_message

topic = topic_for_message(candle_data)
await publisher.send(topic, candle_data)
```

Paper market data topics include a source_exchange segment that is not carried in the payload, so they must always be published with an explicit topic override.

WebSocket Message Provenance

All WebSocket protocol messages (authentication, subscription management, ping/pong, errors) inherit directly from StrictDataSchema, so they carry the same provenance fields as ZMQ data payloads: public_id, session_id, sequence_id, and timestamp. Every payload item in the system — whether it flows over ZMQ, REST, or WebSocket — has a uniform provenance envelope.

Audit Tables: Control and Telemetry

Two destination tables provide always-available observability for non-domain traffic:

- **control** — Always-on audit for commands, authentication events, subscribe/unsubscribe messages, and REST mutation requests. Recording is wired in three places:
  - **WS handlers** — _record_ws_control() records auth, subscribe, unsubscribe, and error events with transport="ws". Client causation linkage is extracted from inbound messages via _extract_client_provenance() and stored as client_session_id and client_public_id on the control row.
  - **REST middleware** — ClientProvenanceMiddleware._record_control() records every mutation (POST / PUT / DELETE / PATCH per _MUTATION_METHODS) with transport="rest", redacted payload, outcome (ok/error/exception), and server-side provenance.
  - **ZMQ bridge** — _record_bridge_control() records subscribe errors and client disconnects with transport="zmq".

  The REST middleware performs its control write in a finally block so mutations are recorded even when the handler raises; the WS dispatch loop records success/error outcomes inline and records an exception outcome from its top-level exception handler; the bridge records at its explicit error/disconnect sites. The write is non-blocking: any DB failure is logged and swallowed so the response already sent to the client is never invalidated.
- **telemetry** — Toggleable high-volume table for pings, heartbeats, pongs, and GET read requests. Recording is gated by the TELEMETRY_RECORDING_ENABLED environment variable (default false). When disabled, SequenceTracker counters still increment — only the DB write is skipped. Recording is wired in:
  - **WS handlers** — _record_ws_telemetry() records ping/pong/heartbeat events
  - **REST middleware** — _record_telemetry() records GET reads (health, status, entity endpoints)

Audit persistence is best-effort: write exceptions are logged and swallowed. REST middleware awaits its write after the application response has been sent; WS and bridge call sites may also await writes, so this is not a guarantee of zero latency or background-only persistence.

Best Practices

1. **One broker per system** — All components connect to the same broker
2. **Topic hierarchy** — Use hierarchy for prefix filtering (market.kraken.); wildcard * subscriptions are rejected
3. **Data types** — Always use typed Data classes from messaging.schemas.data
4. **Provenance** — Use MessagePublisher (not ValidatedPublisher directly) so every message carries session_id and sequence_id for gap detection
5. **One SequenceTracker per component** — Create it once at start() and share across all publisher instances; restart creates a new session
6. **Heartbeats** — Keep component-specific heartbeat cadences small and regular
7. **Graceful shutdown** — Close sockets with LINGER=0

Observability

Snapper exposes permission-gated health metrics so a dashboard can see the gradient toward exhaustion before a crash. The observability surface includes process-level counters sampled into an in-memory ring buffer, per-table SCD2 database statistics at /api/metrics/db/tables, and retention policy state at /api/metrics/retention. The retention definition of "archivable" drives the database statistics archivable counter.

Endpoints

Metrics reads require effective Permission.READ_SYSTEM_STATUS; the current ai_reviewer, ai_delegate, viewer, operator, and admin named sets contain that permission. The two tracemalloc mutations instead require effective Permission.MANAGE_RUNTIME_DIAGNOSTICS, which is present in the current ai_reviewer, ai_delegate, operator, and admin sets but not viewer or ai_researcher. Those POST routes also require CSRF on the cookie-authenticated path; Authorization: Bearer requests bypass CSRF per the project-wide auth contract. On this surface, viewer is a read-only operator: it can call every GET route below but cannot start or stop tracemalloc.

Method	Path	Purpose
GET	/api/metrics/system	Latest snapshot
GET	/api/metrics/system/history?since&until&limit	Windowed slice of the ring buffer
POST	/api/metrics/system/tracemalloc/start?duration_s	Arm Python tracemalloc with auto-stop deadline
POST	/api/metrics/system/tracemalloc/stop	Disarm tracemalloc + cancel pending deadline
GET	/api/metrics/notifications	Notify-sidecar outbox delivery counters
GET	/api/metrics/rest-rate	Rolling REST call rates + rate-limit utilization per exchange
GET	/api/metrics/retention	Retention scheduler status and policy counters
GET	/api/metrics/db/tables	Per-table row-count and SCD2 lifecycle counters

Sibling health endpoint

GET /api/health/egress is a sibling health endpoint outside the /api/metrics/* surface. It shares the same Permission.READ_SYSTEM_STATUS gate as the metrics routes and returns a per-container egress pool status snapshot, aggregated cross-process over the system.egress.snapshot ZMQ topic, with sidecar system.egress.transfer samples joined to matching SOCKS5 route rows. See api.md (GET /api/health/egress) for the full wire schema.

Data-liveness watchdog

Beyond the REST surface, each exchange feed runs a per-subscription data-liveness watchdog (SubscriptionHealthTracker, infrastructure/exchanges/_subscription_health.py): a confirmed subscription that receives no data for data_stale_threshold_s (default 300.0s; ack_timeout_s default 15.0s guards the initial subscribe ACK) is marked stale and emits a one-shot log line. Beyond the one-shot log line, each publisher periodically flushes the tracker's current state into the instrument_feed_health table. A token retaining read:system_status can query it through GET /api/market/feed-health (optional exchange and fresh_within_seconds filters; same Permission.READ_SYSTEM_STATUS gate as the metrics routes).

Market-data watchdog (whole-exchange silence)

The per-subscription tracker cannot alert on a whole exchange going dark: feed heartbeats report DB flush errors and venue-specific health contributions, but a healthy socket alone does not prove fresh candles; in-process freshness clocks reset on every publisher respawn, and a fully hung publisher emits nothing at all. The API lifespan therefore runs MarketDataWatchdog (application/market_data_watchdog/watchdog.py), which every MARKET_DATA_WATCHDOG_INTERVAL_SECONDS (default 60) queries the database for the newest candle per live feed exchange (kraken, kraken_futures, kraken_equities, walutomat). When an exchange's silence — measured from its newest candle minute END — exceeds MARKET_DATA_WATCHDOG_THRESHOLD_SECONDS (default 600, floor 120), the watchdog publishes a synthetic 3-frame WARNING heartbeat burst on system.heartbeats.marketdata.{exchange} (frames spaced 2 s to clear the bridge's 1 s heartbeat throttle). The existing critical_system_error rule then supplies the 3-consecutive gate, rolling hourly cooldown, hour-bucket dedup, and fan-out to every user with read:system_status — about one page per hour per silent exchange, with meta carrying silent_seconds, threshold_seconds, and latest_candle_open_at for forensics.

Scheduled venue closures are suppressed via the shared CME calendar (core/market_hours.py), whose schedule is defined in Chicago wall time (CT) and converted per call: kraken_equities never alerts inside the daily 16:00-17:00 CT maintenance break, the Friday 16:00 CT - Sunday 17:00 CT weekend closure, or a curated US-holiday closure, and

after a closure ends its silence clock restarts at the reopen boundary instead of the last pre-closure candle. Per-exchange thresholds can be overridden (or one exchange disabled with 0) via MARKET_DATA_WATCHDOG_EXCHANGE_THRESHOLDS (exchange=seconds CSV), and MARKET_DATA_WATCHDOG_DISABLED=true parks the watchdog entirely. Detection runs level-triggered — an exchange that stays silent keeps re-bursting each tick, so a notify sidecar restart cannot permanently miss an ongoing outage.

Trade-integrity monitors

Coordinator instance 0 runs two incremental database monitors every TRADE_INTEGRITY_MONITOR_INTERVAL_SECONDS (default 60, clamped to 30–60 seconds). M1 detects one non-null (instrument_public_id, trade_id) at differing executed_at values; M2 detects more than one active trades row for one public_id. The current U2 constraint and active-identity partial unique index make those conditions impossible, establishing the required zero baseline before U2 is widened. First startup does not pre-credit the window: completed coverage starts at the overlap boundary and becomes current only after the full initial pass completes.

Each monitor returns at most 25,000 recent sweep rows and 2,000 durable restore/import worklog rows per run, then performs only targeted index-backed identity probes for at most 27,000 distinct candidates; each probe reads at most two matching index entries. The repository rejects larger row limits. Each repository call has a 30-second timeout; one timeout does not prevent the other monitor from running. Across both monitors the scheduler therefore requests at most 50,000 sweep rows and 4,000 worklog rows per tick. Under normal production load each monitor is expected to finish well inside its 30-second ceiling.

Findings or excessive completed-coverage lag publish three WARNING heartbeats, two seconds apart, on system.heartbeats.trade_integrity.{m1|m2}. Clean passes publish one HEALTHY frame. The existing critical_system_error rule handles the three-frame gate, hourly deduplication, and operator fan-out. With successful calls and an available publisher, committed worklog identities with no backlog are normally detected in about 65 seconds for M1 and 99 seconds for M2 at the default cadence; worst tick ordering with successful calls consuming their full timeout is about 94 seconds for M1 and 128 seconds for M2. The raw six-hour overlap contains about 1.68 million rows at the current ingestion rate; arrivals during pagination expand a steady pass to about 2.06 million rows, or roughly 82 minutes at 25,000 rows per minute. Including the five-minute settlement grace, cadence, and warning spacing, a newly settled cursor-only violation normally takes at most about 89 minutes to alert. If every successful database call consumes nearly all of its 30-second budget, the same throughput calculation grows to roughly four hours. These are operational bounds, not mathematical guarantees: the live publisher retries retained batches without a time limit, so a commit stalled for more than the six-hour

overlap is outside enforced live-row coverage. Archive restore, historical paper replay, and the local-UAT trade import do not share that exposure because their trade write and exact monitor obligations commit in one transaction.

An M1 or M2 finding blocks the U2-to-U3 widening: inspect the identities in heartbeat metadata and database logs, stop the responsible ingestion/restore/import/replay writer, preserve the pending worklog evidence, and repair or quarantine the conflicting rows. A lag-only warning means the zero observation is not current; restore monitor coverage before treating the baseline as proven or proceeding with schema work.

AI-delegate watchdog

The API lifespan also runs AiDelegateWatchdog (application/ai_review/watchdog.py) for the single-operator AI review plane. It waits one 60-second poll interval at startup so a healthy delegate can reconnect before strict liveness is evaluated. Every 60 seconds thereafter it performs two database reads: delegate liveness across active AI-delegate users whose operator membership has an active scope grant, using the same strict 15-second heartbeat boundary as AiReviewService.create_review, and the count of reviews resolved as timeout_no_response during the preceding hour. No live delegate, or one or more recent unanswered reviews, publishes a synthetic three-frame WARNING burst on system.heartbeats.ai_delegate.global. A healthy tick publishes one HEALTHY frame to reset the alert rule's rolling state.

The existing critical_system_error rule owns the three-frame gate, rolling one-hour cooldown, hour-bucket deduplication, permission-based fan-out and configured provider delivery (including APNs localization). The one-timeout threshold is intentional: a single unanswered LIVE consult already vetoes a trading window, while the existing cooldown prevents repeated pages. The one-hour lookback keeps that failure visible across a notify-sidecar or API restart.

Failure contract

If the snapshotter singleton failed to start at lifespan time, the app.state.system_metrics_snapshotter attribute stays None and every /api/metrics/system* route returns HTTP 503 with body {"detail": "system metrics snapshotter not available"}. The rest of the application continues to serve other endpoints.

Cold-start contract

SystemMetricsSnapshotter.start() takes one eager synchronous sample BEFORE returning, so the first request after lifespan startup completes hits a populated buffer. When the API process has its ZMQ publisher wired, that same eager sample also publishes the first system.heartbeats.host.disk heartbeat.

Notification delivery metrics

GET /api/metrics/notifications does not touch the snapshotter or the ring buffer (the failure contract above does not apply to it). Each request aggregates active alert_deliveries rows (SCD2 predecessor versions excluded) into per-status delivery totals — sent, failed, unregistered, cancelled_scope — plus the queued backlog depth of the notify sidecar's retry loop. Counters aggregate APNs and Android rows without a provider breakdown. unregistered includes APNs 410 and explicit FCM UNREGISTERED outcomes; sent means provider acceptance, not device delivery. Android expiry and its bounded attempt budget can produce terminal failed rows. Statuses absent from the database report 0. See api.md for the NotificationMetricsResponse wire schema.

Use container state, restart history and sidecar logs to investigate retry-worker failure. The sidecar propagates unexpected worker failure through its CLI and awaits owned-task cleanup. An ordinary secondary cleanup failure is logged by stage and exception class while the original failure remains primary. Delivery totals describe persisted outbox state; they do not certify worker liveness or device receipt. Android metadata marked configured likewise proves local sender configuration, not Google reachability. See Android push for configuration and delivery diagnostics.

Snapshot fields

Each sample captures nine nested groups, a top-level bus_time timestamp, and two top-level flags. All field names are stable; downstream consumers (frontend, iOS) regenerate from the backend OpenAPI / JSON-Schema export and pin the wire shape.

process

Field	Type	Description
pid	int	Process ID.
uptime_seconds	float	Monotonic-clock seconds since the snapshotter was constructed.
status	str	psutil process status (running, sleeping, ...).
num_threads	int	Live thread count.
num_fds	int	Open file descriptor count.
num_connections	int	Open TCP/UDP socket count.

cpu

Field	Type	Description
process_percent	float	psutil.Process.cpu_percent reading at sample time.
user_time_seconds	float	Cumulative user-mode CPU.
system_time_seconds	float	Cumulative system-mode CPU.
cgroup_quota_microseconds	int | null	cgroup CPU quota microseconds per period; null when absent or unlimited.
cgroup_throttled_count	int | null	Cumulative throttle event count from cpu.stat nr_throttled; null when cgroup absent.

memory

Field	Type	Description
rss_bytes	int	Resident set size (RSS).
rss_peak_bytes	int	Peak RSS from /proc/self/status VmHWM; falls back to rss_bytes on non-Linux.
vms_bytes	int	Virtual memory size.
python_traced_bytes	int | null	Bytes tracked by tracemalloc; null when tracemalloc is OFF.
native_bytes	int | null	max(0, rss - python_traced) when tracemalloc is active; null otherwise — exposes "native dark matter" (numpy / pandas / zmq / aiosqlite native cache / pydantic-core Rust).
cgroup_limit_bytes	int | null	cgroup memory cap; null when absent or unlimited.
cgroup_current_bytes	int | null	Current cgroup memory consumption.
saturation_pct	float | null	cgroup_current / cgroup_limit when both available; null otherwise.

asyncio

Field	Type	Description
active_tasks	int	len(asyncio.all_tasks()).

Field	Type	Description
pending_tasks	int	Subset of all_tasks whose done() is False.

gc

The fields below describe the API/wire shape returned by /api/metrics/system. Internally, the snapshotter stores the generation counters as a tuple (collections_per_gen: tuple[int, int, int] on the GcMetrics group reached via SystemMetricsSnapshot.gc); the route mapper flattens that tuple into the three collections_gen{N} fields below before serializing.

Field	Type	Description
collections_gen0	int	Cumulative GC collection count for generation 0.
collections_gen1	int	Generation 1.
collections_gen2	int	Generation 2.
uncollectable	int	Cumulative count of objects GC could not free.
current_objects	int	Sum of gc.get_count() across all generations at sample time.

limits

Field	Type	Description
rlimit_nproc	int	Soft RLIMIT_NPROC.
rlimit_nofile	int	Soft RLIMIT_NOFILE.
rlimit_as_bytes	int	Soft RLIMIT_AS (address space).

saturation

Percentage toward exhaustion (0.0 - 1.0). The gradient operators actually need to see (vs raw counts) — the rate of increase here is the leading indicator of a thread leak or fd leak.

Field	Type	Description
threads_pct	float | null	num_threads / rlimit_nproc; null when rlimit_nproc is RLIM_INFINITY or zero.
fds_pct	float | null	num_fds / rlimit_nofile; null under the same conditions.

db_internal

Field	Type	Description
aiosqlite_live_connections	int	len(_live_aiosqlite_connections) — atomic read; each live aiosqlite connection runs its own worker OS thread. SQLite-only tracker; stays 0 on Postgres.
pool_size	int | null	The primary async engine's queue-pool base size (QueuePool.size()), for asyncpg/Postgres and file-backed SQLite. null for an in-memory StaticPool/NullPool or when no engine is injected.
pool_checked_out	int | null	Connections currently checked out of that pool (QueuePool.checkedout(), includes overflow). null under the same conditions as pool_size.

disk

The snapshotter samples the configured data-partition mount on every tick and exposes the counters through the REST snapshot. The API process also publishes the same status as a system.heartbeats.host.disk HeartbeatData frame when its shared ZMQ publisher is available. The existing critical_system_error rule consumes that heartbeat, gates on three consecutive non-HEALTHY frames in its rolling window, dedups by host/disk/hour, and fans out one alert row per current user SCD2 row whose named permission set contains read:system_status. The current matching sets are ai_reviewer, ai_delegate, viewer, operator, and admin; the fan-out helper excludes deactivated users (users.is_active = FALSE).

Field	Type	Description
mount_path	str	Mount path passed to shutil.disk_usage.
total_bytes	int | null	Total bytes on the mount, or null if the mount could not be sampled.
used_bytes	int | null	Used bytes on the mount, or null on sampling failure.
free_bytes	int | null	Free bytes on the mount, or null on sampling failure.
percent_used	float | null	Used percentage, or null on sampling failure.
disk_low	bool	true when free bytes are below the warning threshold or the critical threshold.

Field	Type	Description
disk_critical	bool	true when free bytes are below the critical threshold.
status	healthy | warning | error	warning below the warning threshold, error below the critical threshold, and warning when sampling the mount raises OSError.

Top-level flags

Field	Type	Description
bus_time	datetime (UTC)	Sample timestamp; window queries match against this field.
tracemalloc_active	bool	True iff tracemalloc.is_tracing() was True at sample time.
cgroup_version	"v1" | "v2" | null	cgroup detection result; null on hosts without cgroup.

Configuration

The snapshotter reads its environment variables directly (no AppSettings extension). Defaults match the in-code constants in snapper.application.system_metrics.snapshotter.

Variable	Default	Effect
SYSTEM_METRICS_INTERVAL_SECONDS	5	Seconds between sampler ticks. Empty / unparseable / non-positive falls back to the default.
SYSTEM_METRICS_HISTORY_CAP	17280	Ring buffer cap (snapshot count). 17280 ≈ 24h at the default 5s interval. Empty / unparseable / non-positive falls back to the default.
SYSTEM_METRICS_DISK_FREE_WARN_BYTES	21474836480	Free-byte threshold below which disk status becomes warning.
SYSTEM_METRICS_DISK_FREE_CRIT_BYTES	10737418240	Free-byte threshold below which disk status becomes error.
SYSTEM_METRICS_DISK_MOUNT_PATH	/	Mount path sampled by shutil.disk_usage. Empty falls back to /.

Operators can drop the ring buffer cap to reduce retained history (720 snapshots is approximately one hour at the default interval). Actual resident memory depends on Python object overhead and snapshot contents.

Publisher hot-path probes are separate, opt-in diagnostics:

Variable	Default	Effect
SNAPPER_TICK_PROBE	unset	Logs per-stage tick publisher timing histograms every ~10 seconds when truthy.
SNAPPER_TRADE_PROBE	unset	Logs per-stage trade publisher timing histograms every ~10 seconds when truthy.

Use these only during targeted throughput investigations; when unset the hot path pays only a branch and return.

Sampling cadence + ring buffer

- Sample interval: configurable, default **5 seconds**.
- Default cap: 17,280 snapshots; no fixed resident-memory byte budget is enforced.
- Eviction: oldest-first via collections.deque(maxlen=N) semantics.
- All buffer access goes through an asyncio.Lock so concurrent route reads see a consistent view (single-writer / multi-reader).

cgroup detection

Runs at every sample and degrades silently when paths are absent:

- v2 default in modern Docker (kernel 5.0+, Docker Engine 20.10+): /sys/fs/cgroup/memory.max, cpu.max, cpu.stat, memory.current.
- v1 fallback: memory/memory.limit_in_bytes, memory/memory.usage_in_bytes, cpu,cpuacct/cpu.cfs_quota_us (or the older cpu/cpu.cfs_quota_us mount), and the matching cpu.stat file for nr_throttled.
- cgroup_version reports the detected layout; null on dev hosts without cgroup (macOS, non-containerised Linux without unified hierarchy).

Tracemalloc

Off by default — enabling tracemalloc adds workload-dependent CPU overhead and holds extra metadata in process memory. A caller with manage:runtime_diagnostics arms it briefly to capture the python_traced_bytes byte counter for the native_bytes diagnostic, then auto-stop fires after a bounded window.

Knob	Default	Cap
duration_s (POST query)	600s	3600s (clamped)

Calling start while already armed REPLACES the deadline (cancels the previous timer, starts a fresh one with the new duration). The route response carries the clamped requested_duration_seconds, so the caller sees what was actually applied.

Example invocations

Latest snapshot:

```
curl -fsS \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
  https://snapper.example.com/api/metrics/system \
  | jq '.payload | {threads_pct: .saturation.threads_pct,
rss: .memory.rss_bytes, aiosqlite: .db_internal.aiosqlite_live_connections}'
```

Last hour, capped at 60 samples:

```
SINCE=$(date -u -d '-1 hour' +%FT%TZ)
curl -fsS \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
  "https://snapper.example.com/api/metrics/system/history?since=${SINCE}&limit=6
0" \
  | jq '.count, [.payload[].memory.rss_bytes]'
```

Arm tracemalloc for 5 minutes to inspect the native gap. The token used here must retain manage:runtime_diagnostics:

```
curl -fsS -X POST \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
  "https://snapper.example.com/api/metrics/system/tracemalloc/start?
duration_s=300" \
  | jq
sleep 30
curl -fsS \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
  https://snapper.example.com/api/metrics/system \
  | jq '.payload.memory | {rss: .rss_bytes, traced: .python_traced_bytes,
native: .native_bytes}'
```

Multi-instance note

The ring buffer is per-process. With N>1 instances each hosts its own buffer; aggregation across instances is outside the process-metrics scope. Operators who need cross-instance views should poll each instance directly; no cross-instance collector exists.

Retention

Snapper runs a **policy-driven retention loop** that periodically archives + purges old rows from event tables, complementing the existing manual snapper archive CLI. Policies are declarative (RetentionPolicy(table, retain_days, backlog_lookback_days)) and evaluated by RetentionService on every scheduler tick.

Endpoint

Method	Path	Purpose
GET	/api/metrics/retention	Most recent scheduler tick's per-policy summary

The route is gated by Permission.READ_SYSTEM_STATUS (same gate as the system metrics surface).

Failure / cold-start contract

The route returns HTTP 503 with one of three details:

- "retention scheduler not available" — singleton failed to start at lifespan time (the lifespan hook leaves app.state.retention_scheduler as None so the route falls through to 503).
- "retention scheduler disabled" — the operator set RETENTION_DISABLED=true; the scheduler is parked.
- "retention scheduler not yet run" — eager run did not populate the summary before the first request (defensive; should not happen in production because start() awaits the eager run_once).

Policy semantics

For each scheduler tick, with today_utc = datetime.now(UTC).date():

```
oldest_kept_day      = today_utc - retain_days       # rows on this day stay in
DB
last_eligible_day    = oldest_kept_day - 1d          # day_end (inclusive)
earliest_scanned_day = last_eligible_day - backlog_lookback_days  # day_start
(inclusive)
```

The archiver call covers the inclusive day-range [day_start, day_end], matching EventArchiver.export(...) semantics. Steady state after the backlog drains processes one new day per tick.

Concrete example with today_utc = 2026-05-01, retain_days = 1, backlog_lookback_days = 30:

- oldest_kept_day = 2026-04-30 (rows with timestamp on 2026-04-30 stay in DB).
- last_eligible_day = 2026-04-29 (last day archived + purged this tick).
- earliest_scanned_day = 2026-03-30 (oldest day touched this tick).
- The archiver query covers [2026-03-30 00:00 UTC, 2026-04-30 00:00 UTC) per the repository semantics.

The shipped default policy:

Table	retain_days	backlog_lookback_days
telemetry	1	30

Configuration

Variable	Default	Effect
RETENTION_INTERVAL_SECONDS	3600	Scheduler loop period. Parsed as a positive float; empty / unparseable / non-positive values silently fall back to the default.
RETENTION_DISABLED	false	Truthy ("1"/"true"/"yes", case-insensitive) parks the scheduler entirely.
RETENTION_DRY_RUN	false	Truthy forces purge=False on every archiver call regardless of policy. Operators flip to true for one cycle when adding a NEW high-volume policy to verify the window before any DB row is deleted.
RETENTION_OUTPUT_DIR	data	Filesystem root passed to EventArchiver; matches the snapper archive CLI default.

Roll-out checklist for a new high-volume policy

1. Add the new RetentionPolicy entry to RETENTION_POLICIES in src/snapper/application/retention/policies.py. The table must be a key in EVENT_TABLES except executions (append-only — the archiver refuses its purge); an invalid policy fails at module import.
2. Set RETENTION_DRY_RUN=true for one full interval and inspect GET /api/metrics/retention — check that the day_start / day_end window matches expectations and archived_rows is non-zero.

3. If the new policy targets a deeply backlogged table, run a one-time snapper archive --table=<...> --from=<old> --to=<recent> --purge to drain backlog before flipping RETENTION_DRY_RUN=false.
4. Flip RETENTION_DRY_RUN=false. Subsequent ticks will both archive AND purge.

Multi-instance note

The retention scheduler is single-instance only for v1. With N>1 instances, RETENTION_DISABLED=true on N-1 instances avoids double-archive / double-purge races. A coordinator-based extension is a future iteration.

Per-table SCD2 stats (DB metrics)

GET /api/metrics/db/tables returns the latest sampled per-table row counters for every registered event + state SCD2 table. The endpoint is operator telemetry: at a glance it answers "is telemetry growing as expected", "are there orders accumulating closed versions", "how many rows are eligible for the next retention cycle".

Database metrics endpoint

GET /api/metrics/db/tables — READ_SYSTEM_STATUS permission gate. Returns a DbStatsResponse envelope wrapping a DbStatsData payload with tables: list[TableStatsItem] in the sampler's deterministic order (STATE-table block first alphabetical, EVENT-table block second alphabetical).

Database metrics failure / cold-start contract

State	Status	Detail	Retry-After
Snapshotter helper failed before assigning	503	DB metrics snapshotter not initialized	(none)
DB_METRICS_DISABLED=true (operator opt-out)	503	DB metrics snapshotter disabled via DB_METRICS_DISABLED	(none)
Started but no sample yet (cold-start)	503	DB metrics snapshotter has not completed a sample yet	<interval_seconds>
Healthy	200	—	—

The cold-start 503 window opens at lifespan start and closes only after the first full sampling pass completes: the sampler sleeps one DB_METRICS_INTERVAL_SECONDS (default 60s) and THEN samples every table sequentially (30s per-table timeout), so a

degraded first pass can run well past a single interval. By design the sampler does NOT block lifespan startup on that first sample. Retry-After is a polling hint for frontend dashboards, not an upper-bound SLA — poll-with-backoff.

Snapshot fields

TableStatsItem

Field	Type	Notes
table	str	Table name. Usually a key in EVENT_TABLES or STATE_TABLES; candles is also sampled (as a state-kind table) despite living outside both archiver registries.
table_kind	"event" | "state"	Discriminates wire semantics.
total	int | null	Total row count. On PostgreSQL this is a pg_class.reltuples planner estimate (GREATEST(reltuples, 0) — fast, accurate within ANALYZE/ autovacuum drift, intended for growth-trend monitoring); on the SQLite dev fixture it is an exact COUNT(*). For state tables the published value is additionally clamped no lower than current. null only on per-table query failure with no prior sample to clone.
current	int | null	Active SCD2 versions (rows whose known_to equals the SCD2 sentinel) for state tables; null for event tables (no SCD2 lifecycle — reporting 0 would imply the dimension exists).
closed	int | null	Superseded SCD2 versions for state tables; null for event tables.
archivable	int | null	Row count in the policy retention window when a RETENTION_POLICIES entry applies; null when no policy applies (semantically distinct from 0).
is_stale	bool	True when the row was reused from a prior sample after a per-table query timeout or exception.
last_sampled_at	datetime	UTC timestamp of the row's source sample. On a stale clone this is the original timestamp, NOT the current cycle's clock.

DbStatsData

Field	Type	Notes
snapshot_started_at	datetime	When the sampler began the cycle.

Field	Type	Notes
snapshot_completed_at	datetime	When the sampler finished the cycle.
interval_seconds	int	Echo of the configured cadence.
tables	list[TableStatsItem]	One entry per registered table.

Per-kind semantics

- **EVENT tables** (append-only — ticks, trades, signals, executions, telemetry, control): total comes from the dialect-aware row-count primitive — a pg_class.reltuples planner estimate on PostgreSQL, an exact COUNT(*) on the SQLite fixture. The current / closed axes are null because event tables have no SCD2 lifecycle. archivable is non-null only for tables with a registered policy (currently only telemetry).
- **STATE tables** (SCD2-versioned — orders, positions, instruments, etc.): current = COUNT(known_to == sentinel) counts active SCD2 versions — exact, except on PostgreSQL for tables with a registered active-index estimate (currently candles, via the uq_candle_itf_open partial index), where current is a pg_class.reltuples planner estimate; on the SQLite fixture it is always exact; total comes from the dialect-aware row-count primitive (PG planner estimate / SQLite exact); total is clamped no lower than current (total = max(total, current)) to absorb stale PostgreSQL planner estimates where the exact current temporarily exceeds the estimated total; closed = max(0, total - current) is then derived from the clamped total (so it can never actually go negative), and panels always see Total >= Current >= 0 while PostgreSQL total and closed remain estimates. archivable is null until a policy is registered for the table.

Database metrics and retention alignment

The archivable counter computes the same window as the retention service's compute_retention_window(today_utc, policy): the half-open timestamp >= day_start midnight UTC AND timestamp < day_end + 1d midnight UTC predicate, with the policy's (retain_days, backlog_lookback_days) knobs. This means dashboard archivable and retention export use the same eligibility boundaries. Counts can differ between sampling and export if rows arrive or are purged, or the UTC day changes; equality requires the same data and evaluation day. The tests/application/db_stats/test_cluster_alignment.py integration test pins the equality contract.

Database metrics configuration

Variable	Default	Effect
DB_METRICS_INTERVAL_SECONDS	60	Sampler loop period. Empty / unset → default; a non-integer or out-of-range value raises ValueError at startup.
DB_METRICS_DISABLED	false	Truthy ("1"/"true"/"yes", case-insensitive) parks the snapshotter entirely. PG operators with deployments lacking psycopg2 in deps boot cleanly with this flag (the underlying repo factory is never called).

PER_TABLE_TIMEOUT_SECONDS is a module constant (30s, not env- configurable for v1). On per-table timeout, the snapshotter clones the prior TableStats with is_stale=True; if no prior sample exists the row carries all-null counters. Per-table failures NEVER abort the sampler tick.

Sampling order + cold-start

STATE tables sample first (alphabetical by name), EVENT tables second. Atomic swap of _latest_snapshot happens at the END of the cycle — readers wait one full interval_seconds before the first 200, regardless of order. STATE-first ordering is for resilience to EVENT timeouts: if a heavy EVENT table (e.g. ticks at 50M rows) times out, STATE counters were already computed and the snapshot still publishes with EVENT entries marked is_stale=True from a prior run.

telemetry.timestamp index

The database metrics archivable query on telemetry filters on Telemetry.timestamp (the bus-time column inherited from TemporalMixin). Without an index on that column, the query is a full table scan over a high-volume audit table. The ix_telemetry_timestamp index over Telemetry.timestamp is part of the consolidated 0001_init migration (src/snapper/data/migrations/versions/0001_init.py), so both the database metrics counter and the retention scan run in O(log n + k) (an index-assisted seek plus a scan of the k matching entries) from the first deployed schema.

Operations runbook — multi-instance trade coordinator

The trade runtime supports static-hash shard partitioning. At SNAPPER_COORDINATOR_INSTANCE_COUNT=1 (the default) a single TraderCoordinator owns every shard. Scaling to N >= 2 deploys multiple coordinators against the same DB and ZMQ broker, each owning ~1/N of the shards deterministically via SHA-256 of the shard key.

This document covers operating the N-instance trade runtime: scale-up, scale-down, and crash recovery. It is intentionally opinionated about systemd because systemd template units are the stable recipe. A contract for alternative orchestrators (Docker Compose, Kubernetes, Nomad) is also specified so operators can adopt them with empirical verification.

Prerequisites: systemd template unit

All subsequent recipes assume a template unit snapper-trade-zmq@.service that consumes %i as the instance id:

```
# /etc/systemd/system/snapper-trade-zmq@.service
[Unit]
Description=Snapper Trade Coordinator (instance %i)
After=network.target snapper-broker.service

[Service]
Type=simple
User=snapper
EnvironmentFile=/etc/snapper/coordinator.env
Environment=SNAPPER_COORDINATOR_INSTANCE_ID=%i
ExecStart=/opt/snapper/bin/snapper trade-zmq
Restart=always
RestartSec=2

[Install]
WantedBy=multi-user.target
```

With this template, snapper-trade-zmq@0.service and snapper-trade-zmq@1.service are distinct systemd units with separate cgroups and separate journal streams on the same host. Per-instance PID is queried via systemctl show -p MainPID snapper-trade-zmq@0.service.

/etc/snapper/coordinator.env holds the shared bootstrap env, including SNAPPER_COORDINATOR_INSTANCE_COUNT=2 during N=2 operation:

```
DB_URL=postgresql+asyncpg://snapper:...@db.internal/snapper
SNAPPER_ENV=production
ZMQ_BROKER_XSUB=tcp://broker.internal:7500
ZMQ_BROKER_XPUB=tcp://broker.internal:7501
MASTER_PASSWORD=...
SNAPPER_COORDINATOR_INSTANCE_COUNT=2
```

SNAPPER_ENV=production (also prod / staging) arms the placeholder-secret gate: every process loading this env file — coordinators included — refuses to start while MASTER_PASSWORD is unset or still the development placeholder (snapper_default_master_password_v1); BootstrapSettingsLoader raises ValueError naming the variable before anything binds. That is the whole gate: the JWT, CSRF, and control-plane signing keys are all DERIVED from the master password (one-root-secret model, src/snapper/infrastructure/security/kdf.py), so no separate secret rows exist to validate. Remediation: set a real MASTER_PASSWORD in the env file before flipping SNAPPER_ENV to a production-like value.

Rotating MASTER_PASSWORD (change the .env value, rebuild/restart):

- Derived keys (JWT, CSRF, command signing) change in each process when it restarts with the new value. Recreate all participating containers with the same environment; Compose does not switch the fleet atomically, and old-key and new-key processes cannot verify each other's signatures.
- Every session/JWT is invalidated: all users re-log-in, and MCP delegate long-lived tokens MUST be re-minted (they are signed with the derived auth key).
- CSRF cookies refresh transparently on the next page load.
- Fernet-ENCRYPTED rows do NOT re-encrypt themselves: run snapper settings-rotate-encryption for settings rows FIRST, and recreate wallet credentials separately (they share the Fernet key but are not rewritten by that command) — see .env.example for the step-by-step order.

SNAPPER_COORDINATOR_INSTANCE_COUNT >= 2 requires a PostgreSQL DB_URL. On a SQLite backend the coordinator refuses to start with a count above 1, raising ValueError before any signal is dispatched: SQLite compiles SELECT ... FOR UPDATE to a plain SELECT, so cross-instance row claims cannot serialize — use PostgreSQL for multi-instance deployments. A single-instance SQLite coordinator starts but logs a writer-concurrency warning.

Scale up N=1 → N=2 (systemd)

```
# 1. Edit /etc/snapper/coordinator.env — set
SNAPPER_COORDINATOR_INSTANCE_COUNT=2.

# 2. Stop the currently-running single-instance coordinator.
sudo systemctl stop snapper-trade-zmq@0.service
```

```
# 3. Verify graceful shutdown.
sudo systemctl status snapper-trade-zmq@0.service | grep -E "Active:|Main PID:"

# 4. Start both instances.
sudo systemctl start snapper-trade-zmq@0.service
sudo systemctl start snapper-trade-zmq@1.service

# 5. Verify both are live.
sudo systemctl is-active snapper-trade-zmq@0.service  # expect: active
sudo systemctl is-active snapper-trade-zmq@1.service  # expect: active
sudo journalctl -u snapper-trade-zmq@0.service -n 20 | grep "instance 0/2"
sudo journalctl -u snapper-trade-zmq@1.service -n 20 | grep "instance 1/2"
```

Scale down N=2 → N=1 (systemd)

```
# 1. Stop BOTH coordinator instances first — full cutover is required.
sudo systemctl stop snapper-trade-zmq@0.service
sudo systemctl stop snapper-trade-zmq@1.service

# 2. Verify nothing remains.
sudo systemctl list-units 'snapper-trade-zmq@*' --state=active --no-legend
# (expect empty output)

# 3. Edit /etc/snapper/coordinator.env — set
SNAPPER_COORDINATOR_INSTANCE_COUNT=1.

# 4. Start instance 0 only.
sudo systemctl start snapper-trade-zmq@0.service
```

Why full cutover is required

Scale up and scale down both require stopping ALL old-N coordinators BEFORE starting any new-N coordinator. Overlapping old-N and new-N instances is the failure mode to prevent.

Scenario: improper restart with overlap. If an operator skipped the cutover step and left instance 0 running as count=1 while starting instance 1 as count=2:

- Instance 0 believes it owns ALL shards (count=1 → every owns() returns True).
- Instance 1 believes it owns shards where hash(shard_key) % 2 == 1.
- Both coordinators dispatch the same TradeCommand for shards where hash() % 2 == 1 → duplicate venue orders OR a race on status='created' → 'dispatched'.

The ~10s dispatch pause for owned shards during the cutover window is the explicit no-HA trade-off. HA was excluded from scope by design.

Coordinator failure and shutdown

The required signal receiver is supervised together with the coordinator's background workers. An unhandled receive, validation or handler error, or an unexpected receiver exit, fails the owning run. Check its failure and the configured process restart policy when intake stops. Receiver failure initiates the owner's shutdown instead of silently leaving its sibling workers running.

Shutdown cancels and awaits owned workers before releasing the coordinator's private socket contexts. The same cleanup applies to partial startup and recovery failures. Concurrent stop callers join that shutdown, and a later stop does not repeat successful cleanup. If context destruction failed, later stop reports the retained cleanup failure without automatically retrying it. An earlier operational failure remains the primary run error. Zero socket linger does not provide a hard deadline for native resource teardown.

No message replay or independent receiver restart is performed by this cleanup. The systemd and application process-manager restart policies retain their existing roles; a replacement coordinator performs its ordinary startup recovery before receiving signals.

The API container probes /api/health directly with curl. Its connection budget is one second and its total request budget is two seconds, within Docker's three-second healthcheck timeout. The probe runs without a shell wrapper so curl is the process Docker supervises. A timeout reports an unhealthy probe; it does not diagnose the cause of an unresponsive API or automatically restart the container.

Crash recovery (systemd)

If instance K crashes:

```
# 1. Diagnose.
sudo journalctl -u snapper-trade-zmq@1.service -n 100 --no-pager

# 2. Check whether Restart=always brought it back.
sudo systemctl is-active snapper-trade-zmq@1.service

# 3a. If active: confirm the coordinator resumed with its original ownership.
sudo journalctl -u snapper-trade-zmq@1.service -n 200 | grep "instance 1/2"
sudo journalctl -u snapper-trade-zmq@1.service -n 200 | grep "recovery complete"

# 3b. If inactive (Restart=always gave up): manually restart.
sudo systemctl restart snapper-trade-zmq@1.service
```

During the outage window, shards owned by the crashed instance pause until the coordinator comes back. Typical outage is under 10s if Restart=always is configured.

Recovery rebuilds engines from:

1. Checkpoints (carry shard_key directly, filtered by ownership).

2. Live executions (filtered by recovered shard_key; paper rows are EXCLUDED under N>1 because ExecutionRow has no strategy_tag).
3. Active orders (same treatment as executions).

Under N>1 paper mode, state that never produced a checkpoint is not recovered by the restarted coordinator. Paper strategies that need to survive restart MUST persist checkpoints.

REST orders with wallet_public_id under N>1

REST manual orders are supported under N>1. POST /api/orders uses the same compute_shard_key(...) helper as the signal engine, executor, and MCP manual-order path. When wallet_public_id is non-empty, the shard key includes the .w{wallet_short} segment; REST manual orders pass strategy_tag=None, so they do not add the paper strategy segment.

The route writes that shard key to both execution_plans and trade_commands, inserts the command with ownership=None so any API worker may accept the request, and lets the trade coordinator outbox ownership filter decide which instance dispatches it. During outbox publish, the coordinator registers client_order_id -> shard_key, so the N>1 CID guard accepts the venue ACK/fill/cancel events only on the owning coordinator.

For manual orders, provide a wallet or omit the field to use the existing single-accessible-wallet resolution for the requested mode. Explicit blank values are rejected. Fresh command writes require a nonblank resolved wallet; the shard helper's legacy empty-wallet support does not permit unscoped new commands. Signal-driven paper orders may still add a strategy_tag; REST manual orders intentionally do not.

Non-systemd deployments (contract)

The operational recipes are anchored on the systemd template above. For non-systemd platforms, the deployment author writes a recipe that satisfies the same contract:

1. Two distinct supervised processes on the same host (or across hosts) with SNAPPER_COORDINATOR_INSTANCE_ID=0 and =1 respectively, and SNAPPER_COORDINATOR_INSTANCE_COUNT=2 in both.
2. Per-instance lifecycle (stop / start / restart / logs) via the orchestrator's native mechanism — never by process pattern match (pkill -f is too broad for multi-instance).
3. Scale-up and scale-down follow the same stop-all → verify-empty → start-new cutover flow as the systemd recipes above.
4. At least one concrete recipe per chosen orchestrator, verified empirically, with a provenance line:

```
Verified on YYYY-MM-DD against <platform-version>
```

The regex enforced by tests/meta/test_operations_runbook.py is ^Verified on \d{4}-\d{2}-\d{2} against \S+$ — exact casing, anchored to the start and end of the line. The meta-test searches the entire file body for at least one matching line; there is no required section placement.

Acceptable orchestrators for the non-systemd recipe are docker compose, kubectl (Kubernetes), or nomad — the meta-test rejects bare docker because docker run alone does not provide the per-instance lifecycle the contract requires.

Compose restart matrix (broker split)

With the dedicated snapper-broker service, restart domains are independent and each restart has explicit loss semantics on the at-most-once ZMQ bus:

- docker compose restart snapper (backend only) — the bus stays up; feed publishers and any strategies container keep publishing, BUT signals consumed by the trade runtime during the backend window are lost (the coordinator was down; signals have no replay). Feed frames keep flowing to other subscribers.
- docker compose restart snapper-feed — market-data gap for the restart window (publishers reconnect and resume; candle corpus heals per feed-resilience recovery below).
- docker compose restart snapper-broker — a brief bus blip for EVERY participant: PUB frames sent during the window drop, all sockets auto-reconnect. Treat as a rare, deliberate operation.
- docker compose restart snapper-strategies — strategies-only restart: in-memory strategy state is lost (candle buffers rebuild via DB warmup; open-position beliefs and one-decision floors do NOT — the signal-safety re-assertion gate governs arming anything beyond paper), the backend and feed keep running, and consults in flight time out and retry on the next window.
- docker compose restart snapper-egress — tunnel-routed public feeds fail closed to quarantine semantics (see egress runbook) until the sidecar is healthy again.
- docker compose restart snapper-notify — alert evaluation and configured APNs/Android delivery pause for the restart window; the outbox drain on the next start recovers queued deliveries, and rule dedup windows persist in the database. ZMQ events published while the sidecar is down are not replayed in general, so an event that never reached rule evaluation can be missed. Portfolio-drift pages additionally have a durable database recovery scan.

Android delivery is disabled by default. For a configured backend, inspect snapper android-push-state before changing the runtime JSON. Activation or global disable requires the current expected_configuration_revision; removing the file from one process

only disables that process. A global disable retires bindings and leaves a durable disabled revision, fencing old senders. Restarting with identical active configuration/options is idempotent; changed public options need a new UUID. Do not infer device delivery from metadata availability or a provider receipt. See Android push operations for credential-file resolution, FCM-only startup, fixed expiry and validation limits.

Flat-restart discipline (MANDATORY until venue-truth reconciliation lands)

A strategies-container restart orphans every standing target: the target re-assertion layer is in-session repair only (_target starts empty), so after ANY restart that touches snapper-strategies — including a full stack down/up — the operator MUST verify positions are flat and flatten anything standing (GET /api/positions, then close) BEFORE trusting strategy output again. Start every soak or arming window from flat. This discipline is the compensating control for the documented pre-LIVE gap (venue-truth position reconciliation + strategy target-state recovery are NOT built yet).

Paper-soak checklist and live go/no-go

The 2026-07-03 drill run verified on the split topology: all six restart domains heal (API ~25s, strategies ~30s incl. DB warmup, feed ~20s, broker ~30s with bus auto-reconnect, egress ~13s, full cold start), the signal-safety conventions live-fire (entry re-assert repriced each bar, long_only flat exits at strength=0.0, no same-bar duplicates), one poisoned bus frame is skipped without killing the listen loop, and the full MCP wake loop works container-to-host (consult admission → ai_reviews frame → host watch → delegate decision inside the deadline → resume → attributed target-flat consumed by the trade runtime).

Soak gating (all must hold over the soak window, ≥24h before LIVE go/no-go):

- No unexplained container restarts; every strategy heartbeat stays fresh (no zombie: a dead listen loop must surface as a process restart, not a healthy heartbeat).
- Zero signal-safety violations: no side=sell strength>0 rows from long_only strategies in signals scoped to the soak window.
- Consult admission succeeds each heartbeat window while the delegate watch is connected; missed windows are gating.
- The live-trading kill switch (live_trading_mode DB setting) stays halted for the entire soak; LIVE arming is the explicit, separate act of setting it enabled — only that value admits non-paper submits (reduce_only and any unreadable/unknown value also block).
- Transient SQLite database is locked noise is acceptable ONLY if all drills, heartbeats and signal writes heal in-window (a SQLite soak does not validate Postgres contention).

- Still OPEN by design before LIVE: venue-truth reconciliation, strategy target-state recovery after restart, proprietary parlay/spy exit conversion, Postgres for prod, migration head applied in prod (0034 as of 2026-07-20; 0015 was current at the 2026-07-03 drill).

Verified environments

Verified on 2026-04-18 against docker-compose-v2.29.7

Kraken Equities (TradFi) market data

TradFi index futures (Kraken FCM: MNQ/ES/YM/RTY/NKD/M2K) are shipped in market-data-only mode. The symbol updater remains disabled by default at code level, while the publisher is registered as an enabled market-data process. Persisted DB Setting rows still control runtime enable/autostart state, so rollout + rollback happen without code deploys.

Contracts used by this flow:

- REST: iapi.kraken.com/api/internal/markets/all/futures-contracts for instrument metadata; .../{ws_symbol}/ticker/history for historical candles. Requires Origin: https://pro.kraken.com + Referer: https://pro.kraken.com/ headers.
- WS: wss://ws-equities.kraken.com for live delayed (~10 min) ticks + trades. The outer envelope's delayed flag propagates into every TickData message as is_delayed.
- Optional realtime WS: wss://ws-equities-auth.kraken.com/?f, gated by kraken_equities_realtime_ws_enabled=false by default. The client mints a short-lived token from the configured exchange='kraken' Spot wallet credential and injects it only into outbound subscribe params. If the wallet id is empty, the credential row is missing, or Kraken rejects the token mint, the publisher logs the fallback and stays on the public delayed feed. When feed_egress_enabled=true, the token mint is routed through the same kraken_equities public egress pool selection as the Equities WebSocket so the mint and WS handshake originate from the same configured tunnel.
- No order API — every submit route (POST /api/orders, POST /api/execution-plans, POST /api/trailing-stops) calls snapper.server._capability_guard.require_tradable and rejects TradFi instruments with HTTP 422 error_code=instrument_market_data_only.

Realtime authenticated feed

The realtime feed is market-data only. It does not enable orders, execution subscriptions, or account channels in Snapper. To enable it:

1. Seed or select a wallet credential row with exchange='kraken', credential_type='api_key_secret', and Kraken's WebSocket interface permission.
2. Optionally set kraken_equities_realtime_wallet_public_id to pin a specific wallet — either <wallet_public_id> used verbatim, or label:<wallet-label> resolved at runtime to the single matching live wallet (fails closed to the delayed feed on zero or multiple matches; label form is seed-file friendly because public ids are minted per database). When this setting is empty, Snapper auto-selects the first active Kraken Spot api_key_secret wallet credential in deterministic repository order.
3. Set kraken_equities_realtime_ws_enabled=true.
4. Restart snapper-feed; these settings are read when the kraken_equities_feed_publisher constructs its exchange client.
5. Verify incoming ticker frames have is_delayed=false. If token minting fails, or no matching Kraken Spot credential exists, the same publisher keeps running against wss://ws-equities.kraken.com and frames remain is_delayed=true.

Tokens are never persisted in SubscriptionRequest.parameters_json or any settings row. Reconnect recovery rebuilds the SDK client and replays Snapper's stable subscription cache with a freshly minted token. Although Kraken exposes GetWebSocketsToken under /0/private/, Snapper treats this read-only market-data session credential as public Equities feed traffic for egress routing so it shares the auth WebSocket source IP. If feed egress is off or no pool is configured, the mint remains direct.

Market diagnostics

Use these REST endpoints during feed rollout and incident response:

- GET /api/market/feed-health — current per-symbol publisher health. Filter with exchange=kraken_equities and optionally fresh_within_seconds=<n> to hide stale rows from stopped publishers.
- GET /api/market/coverage — per-exchange counts for active instruments, fresh ticks, fresh candles, gated-off instruments, and dark instruments. Tune tick_window_seconds and candle_window_seconds to match the venue's expected cadence.
- GET /api/market/cache/health — cache and persist-policy snapshot: number of cached instruments, cached pair-stat entries, and persisted instrument universe size.
- GET /api/market/cache/stats/configured and /api/market/cache/stats/{exchange_a}/{symbol_a}/{exchange_b}/{symbol_b} — cached Pearson/cointegration diagnostics for configured pairs.

The feed-health table is current-state rather than SCD2 history. Its natural key is (coordinator, exchange, channel, symbol), where channel encodes the stream kind and timeframe together (e.g. ohlc:1m) and symbol is the wire-format product id, so split-feed deployments can show which coordinator is publishing a stale or missing stream.

Feed resilience and outage recovery

Market-data publishers self-recover from multi-minute network outages on both the public (egress) and private (direct) paths without operator action:

- **Persistent recovery.** When a feed goes silent past its liveness threshold, recovery runs as a single-flight loop with capped exponential backoff (1→60 s + jitter) that retries until messages resume or the publisher stops — it never gives up on a timeout.
- **Per-venue liveness thresholds.** Kraken Spot and Futures recover after 60 s of message silence (their wildcard/continuous universes always tick, so 60 s reliably means a dark feed); Kraken Equities uses 120 s and is fully suppressed during scheduled CME closure windows.
- **Kraken Equities live-trade age guard.** Each live WebSocket trade is parsed against one reception timestamp shared by its frame. Trades more than 24 hours old are rejected before they can refresh feed health, enter the raw-trade queue, or update the calculated-candle builder; a trade exactly 24 hours old is accepted. This leaves ample room for the public feed's normal ~10-minute delay while blocking historical replay after a reconnect. Rejections are emitted as rate-limited warning summaries. Historical recovery remains the responsibility of the REST backfill path.
- **Native-candle liveness.** The message watchdog above shares one watermark across ticks, trades, and candles, so a venue whose 1m bars arrive on a dedicated native channel (Kraken Spot's ohlc:1m) can have that channel stall silently while ticks/trades keep the watermark fresh (the 2026-06-18 incident: trades alive ~21 h, candles dark). Spot therefore also runs a venue-wide candle watchdog: if no native 1m candle arrives for 300 s (well above the ~60 s venue-wide cadence) while a candle subscription is active, it spawns the same WS-restart recovery (which re-subscribes the dead channel). This recovery is candle-aware — it is only declared successful once a fresh native candle resumes, not on trades alone — and it never escalates to the dark-feed process exit, so a candle-only stall cannot kill the still-live trade feed.
- **Transport keepalive.** Every Kraken WebSocket handshake is opened with an explicit ping_timeout/close_timeout so a silently dead socket (a yanked link with no close frame) surfaces in tens of seconds rather than waiting out the app-level silence threshold.

- **Bounded connect.** The WebSocket connect itself (SpotWSClient/ FuturesWSClient start()) is wrapped in a 20 s timeout (_WS_CONNECT_TIMEOUT_S). The vendored SDK's start() polls for the socket with a connect timeout that never fires, so on a prolonged blackout — where the connector exhausts its reconnect ceiling and never establishes a socket — an unbounded start() would hang forever and wedge in-process recovery (only a process restart would clear it). Bounding it makes a stalled handshake fail fast so the recovery loop tears the partial client down and retries with a fresh one, reconnecting as soon as the network returns.
- **Bounded teardown.** Tearing a client down is bounded too: every venue WS close runs under a 10 s timeout, and a close that times out or raises escalates to a last-resort force close that cancels the SDK's connector run tasks, drains them (5 s bound), and closes the SDK's internal aiohttp session directly. Previously a close landing during the SDK's reconnect backoff sleep (up to ~3 min; the SDK's stop flag did not interrupt it) abandoned the SDK's own session cleanup, so each rebuild cycle during a prolonged blackout leaked one HTTP session and the abandoned reconnect later spawned orphaned background tasks. The reconnect backoff is now hardened to re-check the stop flag every 0.5 s and to reap its child tasks on every exit path, so blackout-driven rebuild cycles no longer accumulate leaked sessions. This covers every process that tears down a Kraken WS client — the executors' private clients included, not just the publishers. A close timed out - forcing cleanup warning followed by kraken WS force-close: leaked aiohttp session closed is the cleanup working, not a fault to act on.
- **Bounded subscribe sends + serialized reconnects.** Every Kraken Futures SDK subscribe/unsubscribe send is wrapped in a 5 s timeout (_SDK_SEND_TIMEOUT_S): the Futures SDK's send_message spins forever on a client whose socket never came up, and one such send once wedged the shared subscribe throttle lock — starving the post-reconnect subscription replay and leaving the feed connected-but-dark until a process restart. Spot and Equities market-data subscribe sends are not wrapped — the spinning send_message is a Futures-SDK behavior, so those venues rely on the bounded connect, the ping/close timeouts, and the 180 s per-attempt recovery bound instead; the Spot *private* executions subscribe send IS bounded by the same 5 s timeout (see "Private fill-stream supervision"). Client (re)connects are serialized per venue (_ws_connect_lock), slot writes are compare-and-clear, and a connect raced by a disconnect closes its own client instead of leaking it. Each recovery attempt is additionally bounded to 180 s (_RECOVERY_ATTEMPT_TIMEOUT_S) so even an unforeseen hang becomes a logged, retried failure rather than a silent permanent wedge. Futures dark auto-recovery covers the trade channel too (its candles are synthesized from trades).
- **Dark-feed watchdog.** If a feed stays dark for ~25 min despite recovery (a wedged SDK or recovery bug), the publisher exits non-zero and ProcessLauncherService respawns it. The exit ceiling is kept above the launcher's lifetime-restart reset

window, so a prolonged outage produces an unbounded slow restart-and-retry cadence that self-heals the instant connectivity returns — it never permanently abandons the feed.

Watch recovery via GET /api/market/feed-health / GET /api/market/coverage and the pub:<exchange> log lines (liveness recovery triggered, feed recovered after N attempt(s), feed dark for …s … exiting for launcher restart).

Order-path retry semantics (Kraken Spot REST)

Order **creation** is deliberately excluded from the REST network-retry wrapper: a network-class failure such as a request timeout is ambiguous (the order may have reached Kraken and executed even though the response was lost), Kraken deduplicates cl_ord_id only among *open* orders, and strategy orders are market orders that fill instantly — so a blind re-send could double a real position. During a network blip an order submit therefore fails fast (one venue call, logged as Network error (no retry, non-idempotent call)) instead of silently retrying up to three times; the failure still feeds the REST circuit breaker so a sustained outage trips fail-fast mode. Idempotent calls (cancel, fetch, balance) keep the 3-attempt network retry. Expect more transient submit failures in logs during connectivity incidents — that is the guard working as intended, not a regression.

Ambiguous submits — the UNKNOWN order state

On every venue client, an otherwise-successful submit response whose snapshot or venue order id is missing or empty is placement-unproven and enters this ambiguity path; it never directly fabricates REJECTED. The same applies to a submit that fails *after the request may have left the process* (request timeout, connection reset, gateway 5xx, or an unparseable success body) where the adapter can establish that timing. On the Kraken Spot ccxt path the same treatment covers a submit whose placement is simply *unproven*: a venue error ccxt could not classify — the bare ccxt.ExchangeError type — parks and verifies rather than rejecting, because "not proven safe to reject" is not the same as "rejected". The executor first verifies against venue truth — up to three lookups by client id (cl_ord_id on Kraken Spot via open+closed orders, cliOrdId on Futures via order status; 15s bound each). The spacing before each lookup depends on which kind of ambiguity produced it. A *lost response* backs off 2s/5s/10s, because the venue may still be materializing an order the request just created and its listings lag the write. An *unclassified venue error* takes its first lookup immediately and then 2s/5s, because the venue answered synchronously and is healthy: nothing is in flight, so a collision can only be against an order already resting from an earlier delivery. That matters operationally — verification runs inline on the executor's single order-command loop, so a delay there also delays a queued cancel for an unrelated live order. The two-consecutive-absence rule is identical on both schedules. A found order finalizes as accepted (an already-terminal one additionally projects its fills through the disappeared-order reconciler); two consecutive

authoritative not-found answers make the rejection venue-truth-based and safe. Only when verification cannot resolve — venue unreachable or lookup unsupported — does the executor park the order, record a non-terminal order_submit_unknown venue event, and publish a single orders.events.*.unknown message:

- the engine **holds its in-flight guard indefinitely** (the 60 s timeout valve is disabled while UNKNOWN) so no replacement order can double exposure while the original may be live;
- a safety-critical order_unknown operator alert fires ("do not assume flat", 5-minute dedup window);
- failures that provably happened *before* any send (circuit breaker open, credentials, symbol/order-type validation) and authoritative venue answers (4xx, success=false, exhausted 429) still reject immediately on every venue — only genuine ambiguity parks. On the Kraken Spot ccxt path "authoritative" means ccxt *classified* the error into a typed ExchangeError subclass (InvalidOrder, InsufficientFunds, BadSymbol, PermissionDenied, BadRequest, …); those still reject at once. An error ccxt left unclassified is matched on runtime TYPE, never on the venue's error text — Kraken documents no duplicate-cl_ord_id error and ccxt maps none, so the wire text for the motivating case is unknown and a string match would be a guess. A breaker-open refusal additionally lands a durable terminal disposition before its REJECTED — a venue event recording the refusal plus the command row moved to FAILED — and publishes the rejection with reason circuit_breaker_open, so consumers can tell an infra refusal from a venue rejection and a redispatched frame can never resubmit the command after the breaker closes; if any step fails, the entry parks and the 60s reconciliation loop reruns the disposition until engine intent is released. Connection refused rejects immediately only on Walutomat, where httpx surfaces connection-setup errors distinctly; on Kraken Spot and Futures connection-setup failures park as UNKNOWN by design, because the ccxt and requests transports cannot reliably distinguish sent from not-sent (a connection error can fire mid-body), and a false park merely alerts while a false rejection could double a position.

Walutomat verification checks active orders first. When an order filled during the ambiguity window and is no longer active, the adapter also checks the newest account-history page for the submitted submitId. It adopts a non-terminal OPEN existence witness only when history positively supplies both opposite-signed MARKET_FX currency legs, each carrying the requested submit id, and one unambiguous venue order id. The witness carries the observed fill-so-far but never claims the requested amount or completion, which account history cannot prove. History absence, an incomplete leg pair, multiple matching order ids, or a zero-fill cancellation does not prove refusal and therefore continues to park UNKNOWN.

Kraken Futures uses python-kraken-sdk for the private send-order call. The SDK's check_send_status raises only for status strings present in its exception assignment and otherwise returns the payload unchanged; it also returns payloads with no sendStatus.

The Snapper Futures mutation call site therefore validates the returned order_id and raises a venue-answered ambiguity when sendStatus or its identity is malformed. It accepts only explicitly classified placement statuses, keeps the SDK's known definitive-refusal behavior, and parks any unmapped status even when an order id is present. This closes the unmapped-error path without patching the shared SDK boundary used by live market-data and read endpoints.

Resolution: a late accepted event clears the UNKNOWN flag (the order resumes its normal lifecycle with the guard still held until fills); a COMPLETE fill or a rejection releases the guard entirely. A partial fill alone neither clears the flag nor the guard — the order stays guarded until its terminal event. A PARKED UNKNOWN order is retried by the 60s reconciliation loop (fresh venue-verification round each cycle); if it stays unresolved across cycles, check the venue's open and closed orders for the client id from the alert before taking any manual action. The same 60s loop also heals fill gaps: when the venue reports more filled quantity than the executor has recorded, it emits a corrective fill, pricing market orders that lack a snapshot price from the VWAP in the venue's own per-order fills summary (get_order_fill_summary). Only when no venue price resolves at all does it log Recon: fill gap ... no price on market order, skipping corrective fill — a fill mismatch needs manual reconciliation against the venue's fill history only if that ERROR persists across cycles. Correctives also carry the venue's real cumulative fee — the order snapshot's running commission, or the fills-summary total — instead of fabricating a zero fee. When the snapshot carries no fee and the fills summary cannot answer (failed lookup, partial page, an implemented source with no rows), the corrective is deferred with a WARNING for at most 5 cycles, then emitted fee-less under a single CRITICAL log so the order can still reach its terminal state. After that CRITICAL the quantity is healed but the fee is not — reconcile the fee manually against the venue's fill history.

On Walutomat, where fills come from polling, an order that disappears from the active list is never guessed terminal: a failed final-state query keeps the order tracked and retries every poll cycle (a wrong FILLED would create phantom position; a wrong CANCELED would free engine intent while the order may have filled). Each failed attempt logs a WARNING; the 10th consecutive failure escalates to a single CRITICAL and the retries continue. On that CRITICAL, check the order on the venue — no manual terminal injection is needed, and the 60s reconciliation loop converges the pending entry independently once the venue answers.

Stop orders — venue support and pre-send rejects

Order commands speak the core order-type vocabulary end-to-end — market, limit, stop, stop_limit — across REST, MCP, the durable trade_commands rows (a stop's trigger is the trade_commands.stop_price column), and the outbox frames. Venue wire names (stop-loss, stop-loss-limit) appear only at the executor's venue boundary, where the request is

translated immediately before the exchange call — never on the bus or in trade_commands. Older execution plans may still carry the legacy venue_order_type wire value in their params; readers normalize it back to the core vocabulary. Venue support:

- **Kraken Spot** and **Kraken Futures** accept stop and stop_limit: the trigger comes from stop_price; once triggered, stop executes at market and stop_limit places a limit order at the command's price.
- **Walutomat** (its FX market API has no stop orders) and **paper trading** (no trigger simulation — a stop would fill immediately, misrepresenting the protective semantics) reject every stop-typed submit.

A stop-typed submit that cannot be sent whole is rejected PRE-SEND: the failure is provably-not-placed, so it takes the definitive-reject disposition (REJECTED published, durable order_rejected venue event, engine in-flight intent released) — never an UNKNOWN park, never a venue round-trip. The pre-send gates are:

- an order type with no venue wire mapping (vocabulary error);
- stop / stop_limit with no stop_price;
- stop_limit with no price (the limit leg) on the Kraken venues;
- any stop on Walutomat or paper.

POST /api/orders and the MCP submit_manual_order tool refuse the same malformed shapes up front — unknown order_type, limit/stop_limit without price, stop/stop_limit without stop_price — before any command row is persisted (HTTP 422 on REST; the same evaluator rule surfaces as a tool error on MCP). An executor-side pre-send stop rejection in the logs (Error processing order ... followed by the REJECTED publish) is therefore safe to act on: nothing reached the venue, engine intent is released, and the command can be corrected and resubmitted.

Duplicate-dispatch guard and dispatch TTL

Two executor/outbox gates close the remaining order-flow loss windows:

- **Duplicate-submit guard.** A coordinator crash between the ZMQ publish and the CREATED→DISPATCHED commit (or a failed bulk write) re-publishes the same client_order_id. The executor now drops such replays silently: an entry already pending, an accepted order awaiting its durable-event heal, or durable venue-event evidence (accepted/fill/terminal/unknown/breaker-open/interlock-blocked — rejections excluded so legitimate retries still flow) all block the re-submit before the venue order-placement call. Two evidence kinds are exceptions to the silent drop: a replay carrying breaker-open OR order_interlock_blocked evidence reruns its own terminal disposition (above) so a crash mid-disposition cannot leave engine intent held. A failed evidence check drops FAIL-CLOSED.

- **Dispatch max-age TTL** (TRADE_COMMAND_DISPATCH_TTL_S, default 30s, <= 0 disables). Stale CREATED create/submit commands expire at the outbox to the terminal EXPIRED status (never published; engine intent released through the regular expired pipeline), and stale frames buffered through an outage (the order-flow socket has no high-water mark) are resolved at the executor against venue truth: a frame whose order EXISTS under the client id is adopted as accepted (a crash-window replay that actually placed), an unverifiable frame drops silently (the row stays DISPATCHED for the verification sweep below and the engine valve), and only a venue-verified-absent frame rejects. A stale row that DOES carry submit evidence publishes anyway and is absorbed by the duplicate guard — expiring it would fabricate a terminal state for a possibly-live order. Cancels and replaces are exempt: expiring a stale cancel would strand a live order, and replace outcomes are reported on the lightweight cancel/replace event path. Keep the TTL below the engine's 60s in-flight valve.

Behind the two gates, the durable command plane reconciles itself. The coordinator's reconciliation loop folds the append-only venue_events rows into trade_commands status advances (ack, partial/full fill, terminal), which rescopes its stale-command logging: a stale command ... no venue evidence WARN fires only for an over-age create/submit command with ZERO venue evidence (a real anomaly); unknown-only evidence logs at INFO (executor verification is already working it); a command with real evidence is silent — an old open limit order is healthy, not stale. Cancel commands keep the legacy WARN. The executor's 60s cycle works the zero-evidence queue from the venue side: each unresolved DISPATCHED command gets a client-id lookup — a found order is adopted under its original request (this also recovers parked-UNKNOWN entries lost to an executor restart; the DISPATCHED row is the durable record of the send intent), and only two consecutive authoritative absences on a command younger than an hour auto-REJECT to release engine intent. Absence never auto-rejects when the dispatch TTL is disabled (nothing expires frames, so one may legitimately still be in flight at any age) or past the hour bound (venue closed-order lookback makes absence non-authoritative) — those cases WARN for operator attention instead. Consecutive executor venue reconciliation failures are exposed on executor heartbeats; after three failures, the coordinator halts known real shard keys for that exchange/wallet scope and drops new cold-path signals for the same scope until a healthy heartbeat clears the scope gate. Existing shard halts remain explicit operator/release decisions.

The same cycle adopts ghost orders — open at the venue under our client id with no in-memory entry — by rebuilding the request from the command row; an open order with NO command row is foreign/manual, warned about once and never touched. A durably REJECTED command later found open (or with live venue evidence postdating the rejection) is healed loudly: the order is adopted and the command resurrected to ACCEPTED, so a false rejection cannot silently strand a live order.

Adoption also re-arms the engine. Every adoption of a venue-LIVE order — ghost adoption, ambiguous-submit verification, the false-reject heal, startup recovery of a venue-verified-open row — publishes its ACCEPTED with reason="adopted"; terminal-snapshot adoptions stay reason-less (re-arming for a dead order would block honest emission). On that marker the coordinator re-arms a running engine's RELEASED in-flight guard (it consumed the false REJECTED, or the 60s timeout valve had already cleared it), so the next signal cannot emit a second order while the adopted one still works the book. A RE-ARMED in-flight intent for adopted order … WARN in the coordinator log is that guard working, not a fault. An engine already in flight for a DIFFERENT order is never clobbered (WARN — both orders stay live and fills of the adopted one still book position), and a client id retired by an honest terminal (FILLED fill or cancelled/expired confirm; rejections never retire) refuses late duplicate adopted frames. A failed adopted ACCEPTED publish parks on the executor and the 60s reconciliation loop retries it until it lands.

Private fill-stream supervision

The private execution (fills) WebSocket in each executor is supervised, not spawned once. Previously any termination of that stream — typically SDK reconnect-budget exhaustion after ~2 min (Futures) / ~5 min (Spot) of network outage — left fills permanently dark while the executor kept reporting RUNNING until a manual restart: Futures surfaced a single error line, Spot exited silently and kept treating the dead client as connected, blocking any rebuild. Now:

- **Death is detectable.** A Spot stream whose connection is terminally lost raises instead of ending cleanly and closes the poisoned client, and on both venues ensure-connected rebuilds a slot holding a terminally-failed client instead of returning it as connected. The Spot private subscribe send is bounded by a 5 s timeout (_SDK_SEND_TIMEOUT_S) so a wedged socket fails the attempt fast instead of hanging the resubscribe.
- **A supervisor respawns dead streams.** Any termination — a clean generator return included — triggers a respawn with capped jittered exponential backoff (1→60 s); it never abandons the stream. A stream that ran healthy for 5+ minutes resets the backoff so an old incident's ceiling is not inherited by the next blip.
- **The dark window is healed before re-entry.** Every post-death respawn first runs a best-effort reconciliation pass (serialized with the periodic 60 s cycle so two concurrent cycles cannot double-emit the same corrective fill), then resubscribes.
- **Resubscribe is idempotent.** Spot subscribes without order/trade snapshots and Futures drops fills_snapshot frames at the source, so neither a supervised respawn nor an SDK in-budget reconnect replays already-booked fills.

- **Per-order fill booking is atomic.** A per-order lock plus exec-id dedupe serializes booking across the live stream, recon corrective fills, and cancel handling, so concurrent paths cannot double-book a fill, and a publish failure no longer silently corrupts fill tracking (the gap is re-absorbed by later frames or the recon corrective).

During a private WS outage expect recurring executor-log warnings — Execution stream died (...) - respawn #N after Ns backoff or Execution stream returned cleanly - respawn #N ... — that is the supervisor retrying; no manual restart is needed, and fills plus the dark-window gap reconcile automatically once connectivity returns. The supervisor exits permanently only on shutdown and on venues without WebSocket execution streaming, where the 60 s reconciliation loop is the only fill source.

Recovery-time corrective fills

Executor startup recovery no longer re-baselines fill tracking to the venue's current cumulative (which silently swallowed every fill that landed while the stack was down). Instead, recovery reads both truth planes per order — the durable venue_events fill rows and the executions log (rows exist only for successfully published fills) — seeds the committed and durable watermarks from what each plane proves, republishes recorded-but-unpublished fills through the normal pipeline under their original exec ids (every consumer dedupes by exec id, so this is idempotent), and emits the remaining venue-ahead gap as a recon corrective with a deterministic id (recon-{order}-c{cum}), so repeated restarts and publish retries converge instead of double-applying. Orders that went terminal during the downtime get their fill gap healed BEFORE the terminal event is projected. Orders the venue cannot verify at startup are parked in tracking with DB-derived seeds — the 60s recon loop retries them every cycle instead of dropping them. If the durable plane is unreadable for an order, recovery degrades to the legacy venue-truth seeding for that order only and logs an ERROR (the downtime gap for it stays unhealed — fix the DB and restart).

Executor task supervision and service restarts

Every executor core loop (order handler, reconciliation, heartbeat — plus the fill stream covered above) runs under an in-process supervisor: a loop that dies (exception or unexpected clean return) respawns with capped jittered backoff (1s doubling to 60s; a 5-minute healthy run resets it), preserving all in-memory order state. The order handler's supervisor rebuilds its SUB socket before re-entry; poison frames (already consumed by ZMQ) are logged and skipped, never respawn-looped. Reconciliation cycles are bounded (300s) so a hung venue call cannot hold the recon lock forever, and parked-UNKNOWN verification is fairness-capped (3 per cycle, index-rotated) so a large parked set cannot starve its tail. On the core loops (order handler, reconciliation, heartbeat — the fill stream's supervisor only backs off, it never escalates), a death streak older than 25 minutes escalates: the whole service crashes out of start() (siblings cancelled, sockets

closed) and the process launcher — whose task-completion handler now drives the same restart watchdog as native subprocesses — rebuilds a FRESH instance, made safe by recovery-time corrective fills. The escalation ceiling deliberately exceeds the launcher's healthy-uptime reset, so escalations retry forever at a slow cadence instead of exhausting the restart budget; only fast startup crash-loops (bad credentials, DB down at boot) park the executor after the budget.

Executor heartbeats report derived health, not a hardcoded HEALTHY: each beat is computed from the supervised-loop seams, so degradation pages operators through the existing system-degradation alert (critical_system_error: 3 consecutive non-HEALTHY beats, deduped to about one page per hour per instance) instead of staying green while a loop is dead. An active death streak reports WARNING from its second death and ERROR once the streak is 10 minutes old (well before the 25-minute escalation); a reconciliation loop with no completed pass for 5 / 15 minutes reports WARNING / ERROR (recon swallows per-cycle faults by design, so only its progress clock can expose a recon that fails forever without dying); an order command stuck in flight for 2 / 10 minutes reports WARNING / ERROR (a wedged handler neither dies nor progresses); an unhealed accept-event backlog or parked-UNKNOWN submits report WARNING. The executor heartbeat's lag_ms is the age of the last successful reconciliation pass. Per-wallet executor instances publish on per-wallet heartbeat topics (system.heartbeats.executor.{exchange}.{wallet_short}), and the alert rule parses those — previously per-wallet executor heartbeats were silently discarded, so a degraded executor could never page.

A parked executor is paged, not just logged: on giving up, the launcher speaks for the dead instance on its own per-wallet heartbeat topic with a synthetic 3-frame ERROR burst, re-bursting hourly while the instance stays parked — the page is level-triggered (a notify restart that loses one burst is healed by the next) while the alert dedup still caps it at about one page per hour. A successful (re)start or a successful deliberate stop unparks the instance and cancels the burst. GET /health additionally reports ERROR while any process is parked — checked before every other branch, including API-only mode — as the backstop for a lost burst.

Fault-injection testing

scripts/resilience_fault_injection.py simulates a real exchange outage and asserts recovery. It drops outbound HTTPS (:443) **inside the feed container's network namespace** with a throwaway privileged helper (docker run --net=container:<feed> --cap-add=NET_ADMIN), holds the outage, restores it, and measures candle-freshness recovery against the SLA. Only :443 is dropped, so the ZMQ bus and DB writes keep working — the outage is market-data only (order execution, a separate container, is unaffected).

It targets the feed's own netns because the runtime image has no iptables (docker exec <feed> iptables fails). This matches the default direct-feed path (feed_egress_enabled=false). If feed egress is enabled, publishers connect to snapper-egress SOCKS ports instead, so drop the SOCKS/egress path rather than feed-container :443. Restore is safety-critical and proven, not best-effort: every step is exit-code-checked, the helper runs a prebuilt snapper-fault-helper image (no package install while the link is down), removal is proven with iptables -C, and if removal cannot be proven the feed container is recreated (fresh netns) and re-verified — the harness raises loudly rather than reporting an uncertain restore. It builds the helper image on first run.

```
# Default: drop the feed's :443 for 180 s, expect spot+futures back within 180
s.
DB_URL=... python scripts/resilience_fault_injection.py --hold-s 180

# Longer outage + a wider recovery SLA (e.g. validating the dark-feed watchdog).
DB_URL=... python scripts/resilience_fault_injection.py --hold-s 900 --sla-s 600

# Freshness bar vs SLA: --fresh-s is HOW FRESH the newest candle must be to
# count as recovered; --sla-s is the wall-clock budget for getting there.
DB_URL=... python scripts/resilience_fault_injection.py --hold-s 240 --sla-s
360 --fresh-s 120

# Override the target container, port, or verified exchanges.
DB_URL=... python scripts/resilience_fault_injection.py \
    --container snapper-feed --port 443 --exchanges kraken,kraken_futures
```

Run it in a low-impact window — it darkens live market data for the hold duration.

Egress routing for feeds (feed_egress_enabled)

By default the feed publishers connect **directly** to the exchanges and do not initialize an egress pool in their own processes. Set the feed_egress_enabled setting to enable per-publisher egress routing: each feed subprocess then initializes its own process-local egress pool at startup, so the Kraken connect shim and Walutomat's pooled HTTP transport route through the configured tunnels (per-exchange pins apply; direct stays the fallback).

This is gated off by default because routing live market data through shared VPN IPs changes latency and can make Kraken/Cloudflare throttle the feed differently than the host IP. Roll it out cautiously:

1. Set feed_egress_enabled=true and restart the feed (the setting is read at publisher startup, not live).
2. In snapper-feed, confirm ss -tnp shows connections to snapper-egress:1081-1085 (SOCKS) rather than direct exchange :443.
3. Compare candle freshness, tick/trade lag, and dark-instrument counts to the direct baseline; watch the reconnect logs for throttling/close codes.
4. Revert instantly by setting feed_egress_enabled=false and restarting the feed.

With routing enabled, a TCP/SOCKS connect-level failure through a proxy route — timeout, connection refused, or a tunnel that went dark — quarantines that route for 30 s (_WS_CONNECT_ERROR_QUARANTINE_S), so the next reconnect fails over to another tunnel or the direct fallback instead of re-dialing the dead route. Public direct routes are not quarantined: a connect error with no proxy means the exchange or the local uplink is down, not the route. Private executor WebSocket direct connect errors are the exception: direct is quarantined briefly (~15 s) so the next reconnect can use private_fallback_route_id.

Spot candle source (spot_candle_source)

By default the Kraken Spot feed reads live 1m candles from the venue-native ohlc:1m WebSocket channel (spot_candle_source=native). Set the spot_candle_source setting to trade_built to switch the live Spot 1m source to candles synthesized locally from the already-consumed spot trade stream: the publisher then consumes KrakenExchangeClient.subscribe_trade_built_candles and **drops** the ohlc:1m subscription entirely (the native-candle subscription set goes empty — a bandwidth saving), tagging the persisted rows source='calculated' instead of source='native'.

The setting value is read at feed startup, where the Spot 1m subscription (native ohlc:1m vs the trade-built stream) is established once — so **a flip only takes effect after a snapper-feed restart**. A live settings broadcast over ZMQ (_handle_settings_update) refreshes the publisher's cached value but does **not** re-establish the subscription, so it cannot switch the live source on its own; always restart the feed. Roll it out cautiously:

1. Set spot_candle_source=trade_built.
2. Restart snapper-feed (required — the subscription source is fixed at startup; the live ZMQ broadcast updates the cached value only).
3. Verify live Spot 1m rows are landing with source='calculated', that no new native ohlc:1m subscription is opened (the native-candle subscription set is empty), and that the feed stays healthy.
4. Note the illiquid-pair tradeoff: a minute with zero trades emits **no** base 1m bar, and candle_forward_fill fills only higher timeframes — not the base 1m — so thinly-traded pairs will show 1m gaps that the native ohlc:1m channel would have carried.
5. Revert instantly by setting spot_candle_source=native and restarting the feed. The default is native.

Per-wallet executors

Executor process configs are templates named executor_<exchange>. At runtime, ProcessLauncherService spawns one instance per active wallet_credentials row, named executor_<exchange>_w<wallet_short>. Those instances load credentials through CredentialResolver and ignore commands for other wallets. Operators should start/stop the generated wallet instances, not the bare templates.

Persistent process control across containers

The read-side process catalog routes (GET /api/processes/available, /configured, /schema/<name>, and /runs) require effective READ_PROCESSES; GET /api/processes/summary requires effective READ_SYSTEM_STATUS. The current viewer named set contains both read permissions, so a viewer can inspect the same process surface as an operator but has none of the lifecycle permissions described below.

POST /api/processes/<name>/start and /stop are LOCAL runtime operations on the node that receives them, so in the split topology (feed in snapper-feed, strategies in snapper-strategies, executors under their coordinator) they cannot control a process owned by a different container. For STRATEGY targets, local start requires effective START_STRATEGIES and local stop requires effective STOP_STRATEGIES; both operations require effective MANAGE_PROCESSES for non-strategy targets. The persistent, cross-container-safe control is PATCH /api/processes/<name>/desired-state. Non-strategy targets require effective MANAGE_PROCESSES; a STRATEGY enable requires effective START_STRATEGIES, disable requires effective STOP_STRATEGIES, and restart requires both. These mutation routes also require CSRF on the cookie-authenticated path; Authorization: Bearer requests bypass CSRF per the project-wide auth contract. The desired-state endpoint mutates only the DB state (enabled / restart_nonce) of the process_<name> config — the source of truth — and never starts or stops anything locally; the owning coordinator's reconcile loop converges the running state. A restart requires the process to be enabled (409 otherwise) and a client-minted restart_nonce (^[A-Za-z0-9_-]+$, 8-64 chars, 422 if missing; idempotent — resending the same nonce does not double-bounce). A per-wallet executor instance (404) and a bare executor template (422) have no desired-state row; enabling a STRATEGY re-runs the operator/wallet/grant scope check fail-closed before the write.

POST /api/processes uses the same target-aware model when it creates a configuration. A STRATEGY target requires effective CONFIGURE_STRATEGIES and, when created enabled, effective START_STRATEGIES; a non-strategy target requires effective MANAGE_PROCESSES.

Retargeting a strategy's scope (restart required)

PATCH /api/processes/<name>/config retargets an existing STRATEGY config's scope (operator / wallet / AI-reviewer reference-identity params) without hand-editing the config JSON. It requires effective CONFIGURE_STRATEGIES; cookie-authenticated requests also require CSRF. The route fully enforces the caller's authorization at edit time via _enforce_strategy_scope (operator membership, wallet grant, output coverage → 403/400). A non-strategy target → 400, an executor instance / unknown name → 404, a bare executor template → 422, and an undeclared reference-identity key → 400. operator_public_id / wallet_public_id may be a concrete public_id or a label:<name> reference: a label is resolved at edit time scope-qualified to the caller's own operator memberships (so it can never resolve outside the caller's authority), and an in-scope unambiguous label is accepted while a blank/unresolved/ambiguous label → 400 and a concrete foreign operator → 403. The value is persisted verbatim — a label: reference is stored as-is and resolved to a concrete UUID only at the next (re)start (the authorization check runs on a throwaway resolved copy). The edit does NOT restart the process: the response carries restart_required: true (surfaced in the UI as a "restart required" banner) and the new scope takes effect only on the next (re)start of the owning process (via the desired-state restart action above, or a snapper-strategies restart subject to the flat-restart discipline above).

Enable the feed

```
# 1. Populate Symbol + SymbolExchangeCapability rows from iapi.
#    (kraken_equities entry must be present for is_tradeable
#    cache hits; default deny on missing row.)
snapper update-kraken-equities-symbols --force

# 2. Verify the rows landed. SCD2 active rows have known_to set to the
#    canonical sentinel (9999-12-31 23:59:59.000000 on SQLite — see
#    _KNOWN_TO_ACTIVE_SQLITE in src/snapper/data/models.py).
sqlite3 data/snapper.db \
  "SELECT COUNT(*) FROM symbol_exchange_capabilities
   WHERE exchange='kraken_equities'
         AND known_to='9999-12-31 23:59:59.000000';"
# expect >= 38 (indices only; full catalog up to ~180 contracts)

# 3. Trigger an immediate runtime launch of the symbol-updater +
#    feed-publisher processes. `POST /api/processes/<name>/start`
#    invokes `ProcessLauncherService.start_process_by_name`, which
#    reads the existing persisted Setting row and applies the
#    request payload as RUNTIME-ONLY overrides — it does NOT
#    persist a new config row, and it does NOT go through
#    `ProcessRegistrySyncer`. (Persistent enable/autostart state
#    lives on the create / configure path: `POST /api/processes` or
#    the Settings page; the launcher reads that persisted state at
#    startup, while the registry syncer only seeds and updates the
#    DB rows from the code registry.) With
#    SNAPPER_COORDINATOR_INSTANCE_COUNT >= 2 only coordinator
#    instance 0 runs the startup registry sync; other instances log
```

```
#    "Skipping process registry sync on coordinator instance
#    <id>/<count>; instance 0 owns it" so replicas never race the
#    same temporal Setting rows.
#    The body is the full request envelope: `json_body` calls
#    `ProcessStartRequest.model_validate_json` on the raw body, and the
#    envelope (StrictDataSchema, extra="forbid") REQUIRES session_id,
#    sequence_id, public_id, and timestamp. A bare `{"payload":{}}` is
#    rejected with HTTP 422 before the handler runs. `payload` may carry
#    optional `mode` / `parameters` runtime overrides.
curl -X POST http://localhost:8000/api/processes/kraken_equities_symbol_updater/
start \
     -H "Authorization: Bearer <token>" \
     -H "X-CSRF-Token: <csrf>" \
     -H "Content-Type: application/json" \
     -d
'{"type":"process_start_request","session_id":"<sid>","sequence_id":1,"public_id
":"<uuid7>","timestamp":"2026-06-07T00:00:00Z","payload":{}}'

curl -X POST http://localhost:8000/api/processes/kraken_equities_feed_publisher/
start \
     -H "Authorization: Bearer <token>" \
     -H "X-CSRF-Token: <csrf>" \
     -H "Content-Type: application/json" \
     -d
'{"type":"process_start_request","session_id":"<sid>","sequence_id":1,"public_id
":"<uuid7>","timestamp":"2026-06-07T00:00:00Z","payload":{}}'

# To stop a running process later:
# curl -X POST http://localhost:8000/api/processes/<name>/stop ...

# 4. `/start` returns once the process is launched in the runtime;
#    there is no "wait one syncer interval" step on this path.
#    If you need the new state persisted across restarts, do that
#    via `POST /api/processes` (create / configure) rather than
#    via `/start`.
```

Backfill historical candles

```
# 30-day 1-hour backfill of the configured KRAKEN_EQUITIES instruments.
# The default setting is the wildcard ["*"], which expands at runtime to
# the full mapped venue universe (all ~180 contracts).
snapper kraken-equities-backfill-candles -t 1h -d 30

# Or the Makefile wrapper (equivalent)
make backfill-kraken-equities-candles

# Single symbol, daily candles, 90 days
snapper kraken-equities-backfill-candles -s MNQU6-CME -t 1d -d 90 --no-resume
```

The endpoint is undocumented/internal — upstream application-layer failures (HTTP 200 with result=null or non-empty errors) raise RuntimeError so they are distinguishable from legitimately-empty windows. See src/snapper/infrastructure/exchanges/implementations/kraken_equities.py:get_ohlcv for the error contract.

To seed daily history for the single-source read cutover and the DB-first warmup, load cached Polygon grouped-daily rows as 1d candles. This loader is cache-only — it reads the on-disk grouped-daily cache and never contacts the Polygon API:

```
# Load every cached day strictly before the cut date as
# source='native', complete=True 1d candles under the live-read venue.
snapper polygon-load-grouped-candles --exchange kraken --cut-date 2026-01-01

# Or the Makefile wrapper (equivalent)
make run-polygon-grouped-candles EXCHANGE=kraken CUT_DATE=2026-01-01

# Single symbol; or --all for every Polygon-mapped native symbol.
snapper polygon-load-grouped-candles -e kraken --cut-date 2026-01-01 -s BTC-USD
```

--exchange is the **live-read venue** the persisted bars live under (e.g. kraken) — never polygon; the loader writes the 1d history under the same venue the read cutover and warmup resolve. --cut-date must sit at or before the first UTC day live synthesis owns: the loader writes only days **strictly before** it, keeping native backfill and synthesized live bars on disjoint day ranges. There is **no** Docker variant of this loader (make docker-polygon-grouped wraps the separate grouped-daily CSV download, not the candle load). See docs/cli.md for the full option reference (--symbol/-s, --all, --lookback-days).

Quarterly rotation

The default KRAKEN_EQUITIES instruments setting is the wildcard ["*"], which expands at runtime to the full mapped venue universe, so the default scope tracks every available contract automatically. Rotation discipline applies when an operator narrows the setting to an explicit allowlist: the chosen TradFi symbols are quarterly expiry contracts (MNQU6-CME = Sep 26, etc.) and must be rotated before the InstrumentSpec.expiry_at timestamp on any listed symbol drops below 14 days. Rotation cadence:

1. Pull the current expiry list. native_symbol lives on the symbols table (joined to instruments via symbol_public_id); expiry data lives on instrument_specs keyed by instrument_public_id:

sqlite3 data/snapper.db \ "SELECT sym.native_symbol, datetime(spec.expiry_at) FROM instruments AS i JOIN symbols AS sym ON sym.public_id = i.symbol_public_id AND sym.known_to = '9999-12-31 23:59:59.000000' JOIN instrument_specs AS spec ON spec.instrument_public_id = i.public_id AND spec.known_to = '9999-12-31 23:59:59.000000' WHERE i.exchange = 'kraken_equities' AND i.known_to = '9999-12-31 23:59:59.000000';" 2. Identify the next quarterly (e.g. MNQZ6-CME Dec 26 when MNQU6-CME Sep 26 drops below 14 days). 3. Add the new nearest contract by either changing the default KRAKEN_EQUITIES entry inside the AppSettings.instruments property default literal in src/snapper/config/app.py (code path — requires a deploy; roll back via git

revert), OR overriding the instruments DB Setting row via the Settings page or POST /api/settings/instruments/set (immediate, no deploy; requires CONFIGURE_SYSTEM + CSRF). instruments is a read-only @property, so the DB Setting row is the only runtime override.

Troubleshooting

- **Zero instruments after update-kraken-equities-symbols --force**: iapi reachability / Origin header mismatch. Verify curl -H 'Origin: https://pro.kraken.com' -H 'Referer: https://pro.kraken.com/' \ 'https://iapi.kraken.com/api/internal/markets/all/futures-contracts?delayed=true' | head -c 200 returns JSON with result.data[...], not result:null,errors:[...].
- **WS disconnect loops on ws-equities.kraken.com**: inspect Kraken status page; no operator action required — the WS client backs off on reconnect. If the symbol_updater keeps returning zero rows across two consecutive runs, fire a monitoring alert.
- **Submit returns 422 instrument_market_data_only**: expected. TradFi is observation-only. Point the strategy at a can_trade=True instrument (crypto/xStocks) for execution.
- **Fragmented Kraken Equities 1m candles** (a settled minute holding more than one historical candle version, written from partial trade sets): the live-synthesis builder fix (065463de, live since 2026-06-10) stops new fragmentation, and the historical damage window (2026-05-24 → 2026-06-10) was repaired in full on 2026-06-11 (158 444 candles rebuilt from raw trades, rerun-idempotent). The one-off repair tooling was removed after execution; if fragmentation ever reappears, recover the repair-equity-candle-fragmentation CLI and its service/repository code from commit 50c98684 (ix_trades_executed_at from migration 0007 is still in place).

Operations runbook — paired-execution guard

The paired-execution guard coordinates multi-leg strategies such as CointegrationPairs. When a group fails to fill completely, it cancels the live remainder, attempts reduce-only compensation of filled exposure, and halts exposed pair scopes until the books are square or an operator resolves a manual-intervention incident. Compensation depends on venue availability and may require retries; it is not an atomic venue transaction. The scanner handles simultaneous groups; sequential_handoff is outside its compensation policy. PAIRED_EXECUTION_GUARD_ENABLED defaults to false: the strategy layer refuses new live multi-leg signals while it is off. The scanner still runs as a recovery backstop for durable groups, including after the flag is disabled. This runbook covers the operator's share of the lifecycle: reading incidents, resolving manual_intervention groups at the venue, and attesting the resolution so the halt clears.

Authorization is based on effective permissions from Snapper's 34-permission catalogue:

Surface	**Required permission**	**Current named permission sets**
GET /api/paired-execution/incidents	read:positions	ai_reviewer, ai_delegate, viewer, operator, admin
POST /api/paired-execution/groups/{id}/terminalize	manage:paired_execution	operator, admin

State model in sixty seconds

A multi-leg signal creates one **group** (assembling → armed → broken → compensating → completed, with manual_intervention as the operator hand-off state) and one **leg** per instrument. Each leg tracks signed accounting:

- filled_signed_qty — cumulative original-order fill (buy +, sell -),
- compensated_signed_qty — cumulative reduce-only flatten fills, negated so it moves toward filled_signed_qty,
- open_qty = filled_signed_qty - compensated_signed_qty — the residual exposure the guard still owes a flatten. open_qty > 0 means a net-long residual (flatten by SELLING |open_qty|); open_qty < 0 means net-short (flatten by BUYING).

A failing group's scope — (wallet, strategy, group_key) where group_key is the canonical sorted exchange:instrument:mode token set — carries one durable halt row in paired_execution_halts. The halt blocks NEW grouped signals on the same pair and is

mirrored into each trade coordinator's in-memory shard halts. The guard scanner interval is max(1.0, PAIRED_EXECUTION_ASSEMBLY_TIMEOUT_S / 2), which is 2.5 seconds with the default 5-second assembly timeout.

The automatic lifecycle (no operator action)

Breaks (assembly timeout, fill timeout, a leg rejected/cancelled/expired before filling) are handled end-to-end by the scanner: held commands are cancelled, live originals are cancelled at the venue, filled exposure is flattened with reduce-only market orders (re-flattening on late fills with a fresh compensation_seq each round), and once every leg is settled at zero open exposure the group COMPLETES, the scope's halt clears, and the pair may trade again — typically within one scanner cycle of the last fill settling. If you see a broken or compensating incident with automation still in flight, the correct action is usually to wait one cycle.

A submit the executor refuses locally because its venue circuit breaker is open counts as a leg rejection too: the order is rejected with reason circuit_breaker_open and its command is durably FAILED, so it can never fire late once the breaker closes. A flatten refused the same way counts as that flatten's terminal — the leg reopens to filled and the sweep re-flattens it next cycle, retrying until the breaker closes. Breakage does not depend on live messages alone: the coordinator's reconciliation loop folds durable venue events into command statuses, and the scanner projects a durably-terminal original command onto its stuck leg, so a lost reject/cancel message still breaks the group within a cycle.

When the guard hands off to you: manual_intervention

The guard escalates a leg (and its group) to manual_intervention instead of flattening when an automatic flatten would be unsafe:

- the venue/instrument has no reduce-only capability,
- the instrument spec is missing, or its lot size is missing/non-positive,
- the residual rounds below one lot or the venue minimum order size (dust),
- the instrument cannot be resolved (e.g. delisted mid-compensation).

A manual_intervention group stays HALTED every cycle by design until you resolve and attest it. Clearing the halt row by hand does NOT help — the scanner re-creates it on the next cycle while the group remains exposed.

Triage: read the incident

```
curl -s -H "Authorization: Bearer $SNAPPER_TOKEN" \
    "$SNAPPER_BASE_URL/api/paired-execution/incidents" | jq
```

One incident per halted/exposed scope, carrying the durable halt (reason, owning group), every exposed group with its status and failure_reason, and per-leg exposure: status, filled_signed_qty, compensated_signed_qty, open_qty, compensation_seq. Notes:

- halt_missing: true flags an exposed scope whose halt row is transiently absent (a crash/clear window). The scanner re-halts it within a cycle; it is surfaced so you never lose sight of exposure.
- A broken group with zero-fill legs (e.g. an assembly timeout) is NOT an incident — the guard deliberately leaves it free to re-assemble.
- Requires read:positions. Effective impersonate:operator exposes every wallet; every other caller, including viewer, operator, and ai_delegate, sees only its readable wallets (operator grants plus personal wallet_user_read_grants). Terminalization separately requires the operator-grant trade scope; a personal read grant cannot authorize it.
- Any token retaining read:positions can inspect incidents. A long-lived AI-delegate token can therefore read incidents, but it receives 403 on terminalize because its effective grant lacks manage:paired_execution. The viewer set is read-only for the same reason.

Resolve at the venue

For each leg with status = manual_intervention and open_qty != 0:

1. Confirm the actual venue position for that instrument/wallet.
2. Close the residual at the venue (UI or API): SELL |open_qty| when open_qty > 0, BUY |open_qty| when open_qty < 0. Use reduce-only if the venue supports it.
3. Verify the venue position is flat (or back at the strategy's intended level).

Orders you place outside Snapper's order flow do NOT update compensated_signed_qty — the leg's books will still show the residual. That is expected: the leg keeps the true record of what THE GUARD did, and your attestation (below) is the durable record that the remainder was resolved manually.

Attest: terminalize the group

```
curl -s -X POST -H "Authorization: Bearer $SNAPPER_TOKEN" \
    "$SNAPPER_BASE_URL/api/paired-execution/groups/$GROUP_ID/terminalize" | jq
```

Requires manage:paired_execution, which is present in the current operator and admin sets. On success the group becomes completed with terminalized_by=<you> stamped into its failure_reason, legs keep their true accounting, and the scanner clears the scope's durable halt and every coordinator's in-memory mirror within one cycle — the pair can trade again. Responses:

- 404 — no such group in your tradable wallets.
- 409 — not attestable right now. The detail's status is freshly read. Causes: the group is not manual_intervention (nor a compensating group re-opened onto a manual leg); a sibling leg still has automation in flight (a working original or an unfinished flatten — wait a cycle and retry); the group has no legs; or a held original command still exists.

If a LATE original fill lands after your attestation, the guard re-opens the group to compensating; automation re-flattens what it can, and if the manual leg is the residual holder the group will refuse auto-completion again — re-resolve at the venue if needed and POST terminalize again (the re-attestation path accepts a compensating group with a manual leg).

Known windows and boundaries

- **Reopen right after a clear**: a late fill landing immediately after a halt cleared leaves the scope exposed and un-halted for at most one scanner cycle; the next cycle re-halts. Self-healing, no action.
- **Missed not-tradeable reject**: a flatten rejected locally as not-tradeable (a delisted instrument) writes no durable venue event; if its live message is also lost, the leg stays compensating + halted, and the group is NOT attestable while it does (terminalize requires every non-manual leg settled, so POST terminalize returns 409). The executor's dispatched-command verification sweep normally heals this: the flatten's command row is still dispatched with no venue evidence, so on a venue that can look an order up by its client order id the sweep verifies it absent twice (no earlier than max(120 s, 2 × dispatch TTL) after creation), publishes the REJECTED status, and records the durable order_rejected event — the compensating sweep then settles the leg, and the residual re-flattens or escalates to manual_intervention under the normal sweep rules. The sweep does NOT auto-reject when the dispatch TTL is disabled (a frame may then legally be in flight at any age), when the command has aged past one hour (venue closed-order lookback makes absence non-authoritative — WARN-only escalation), or when the venue has no client-order-id lookup; in those cases the scope stays halted (fail-safe) until a durable venue event for the flatten appears or the rows are reconciled by hand.

- **Dispatch max-age TTL applies to paired commands too**: a paired CREATED command older than TRADE_COMMAND_DISPATCH_TTL_S (default 30 s) is CAS-expired by the outbox instead of dispatched — a group wedged pre-dispatch longer than the TTL breaks via the assembly/fill deadlines rather than firing stale legs.
- **Late fills older than 7 days**: startup recovery replays fills into groups completed within the last 7 days. Older late fills are reconciliation territory.
- **Reconciliation halts are separate**: a shard halted by the recon circuit breaker is NOT released by paired-execution machinery (and vice versa — the paired release is reason-scoped). A wedged recon halt currently requires a coordinator restart.
- **Do not hand-edit guard tables.** Clearing halts re-halts within a cycle while exposure remains; editing leg statuses falsifies the books the accounting relies on. The only supported operator mutation is the terminalize endpoint.

Enablement

The guard ships dark. Before enabling real-money multi-leg execution with PAIRED_EXECUTION_GUARD_ENABLED=true, complete a staging trial, verify the outbox-gate query plan on the production database, and define rollback criteria.

snapper-egress sidecar — operator runbook

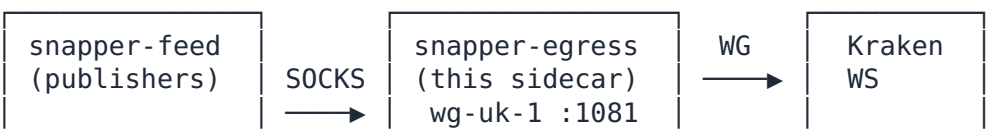
The snapper-egress sidecar runs N WireGuard tunnels plus N matching SOCKS5 listeners inside one container, so Kraken Spot/Futures/Equities WebSocket publishers and Walutomat HTTP polling can route public market-data traffic out through alternate egress IPs without changing source code.

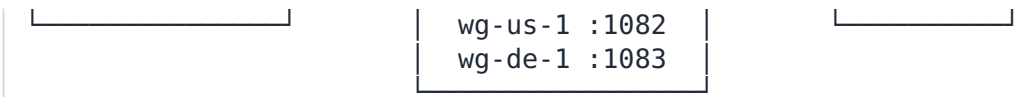
This sidecar is part of the egress-multiplexer subsystem.

Prerequisites

- **Docker Compose v2.** depends_on.condition: service_healthy is Compose v2 syntax. Compose v1 silently ignores it.
- **Kernel WireGuard on the host when tunnels are configured.** Verify with lsmod | grep wireguard or modprobe wireguard. The sidecar bind-mounts /lib/modules and uses the host kernel module via netlink (no userspace WireGuard). Empty/default deployments with no declared tunnels still expose /ready without probing WireGuard. Use /readyz when you need strict "every declared tunnel is up" semantics.
- **MASTER_PASSWORD** in the same .env the snapper (API) container uses. The sidecar reads encrypted egress_tunnel_*_private_key settings via the same Fernet path; a mismatched master password makes the settings load raise (Failed to decrypt encrypted setting) and the sidecar exits at startup, so the container crash-loops, never reports healthy, and blocks snapper, snapper-feed, and snapper-strategies via depends_on: service_healthy.
- **DB_URL and ZMQ_BROKER_XSUB** are both hard-required by the sidecar entrypoint (snapper.egress.__main__). The process exits with **code 2** at startup if either is missing: DB_URL lets the SettingsService read tunnel descriptors + encrypted keys, and ZMQ_BROKER_XSUB wires SettingsService change-broadcast support. DB_URL is supplied via env_file: .env. In the Compose split, ZMQ_BROKER_XSUB MUST be the routable broker host tcp://snapper-broker:7500, NOT the .env default tcp://127.0.0.1:7500 — the docker-compose.yml snapper-egress service overrides it for exactly this reason (and depends on the snapper-broker service being healthy). snapper-egress runs in a separate container, so localhost would not reach the broker living in the snapper-broker container.

Architecture (short)





Per tunnel: 1. WireGuard interface wg-<id> lives in the sidecar container. 2. Source-based routing rule sends packets from the tunnel's IP through the matching WG interface. 3. SOCKS5 listener binds outbound sockets to the tunnel's IP → kernel routes them via the tunnel.

Kraken WS handshakes go through the EgressPool (socks5h://snapper-egress:<port>) inside the feed-publisher subprocesses of the **snapper-feed** container (the API profile excludes market-data publishers; the FEED profile runs only them). Each publisher process builds its own process-local pool at start, gated on the feed_egress_enabled DB setting (default **off** — publishers dial direct until the gate is flipped). The API process also initializes its own process-local pool from egress_pool; API-side Kraken public REST reads may route through that pool even when feed_egress_enabled is false, because the gate applies only to feed-publisher subprocess pools. The pool reserves a route per Kraken WS handshake. A route is quarantined — and the next handshake fails over to an alternate (or the direct fallback) — on any of the QuarantineReason values: http-429 (a handshake 429 carrying a positive numeric Retry-After header; a 429 without one does not quarantine the route), close-1015 (Cloudflare close frame), http-connect-error (REST connect failure via the pooled transport), and ws-connect-error (a WS connect-level failure — TCP/SOCKS timeout, connection refused, or a silent blackhole — through a proxy route, quarantined ~30 s via _WS_CONNECT_ERROR_QUARANTINE_S). Connect-level failover matters because a dead tunnel often blackholes rather than returning 429: without it the reconnect loop would re-dial the same dead route indefinitely.

Adding a tunnel

Preferred path: run the provisioning helper from a checkout that has scripts/ available. The production Docker image does not copy scripts/ into /app, so run it on the host or bind-mount the checkout into a maintenance container with the same DB_URL.

```
poetry run python scripts/provision_egress_tunnel.py \
  --tunnel-id wg-uk-1 \
  --interface wg-uk-1 \
  --address 10.64.12.34 \
  --prefix-length 32 \
  --private-key-file ./secrets/wg-uk-1.key \
  --peer-pubkey "<peer-public-key>" \
  --peer-endpoint vpn.example.com:51820 \
  --socks5-listen-port 1081 \
  --priority 10 \
  --dry-run
```

Remove --dry-run after validation. The helper writes the descriptor, encrypted private/optional preshared key settings, and merges the SOCKS5 route into egress_pool. It can also restart the sidecar with --restart-sidecar. After egress_pool changes, restart **snapper-feed** (each feed-publisher process builds its own process-local pool at start) in addition to snapper (the API tier) — restarting only the API rebuilds the API process's pool but not the pools the Kraken/Walutomat feeds actually dial through.

Manual path:

Three DB settings per tunnel:

Key	Encrypted?	Type	Example
egress_tunnel_<id>	no (JSON descriptor)	text	{"interface":"wg-uk-1", "address":"10.64.12.34", "prefix_length":32, "peer_pubkey":"...", "peer_endpoint":"vpn.example.com:51820", "socks5_listen_port":1081, "priority":10}
egress_tunnel_<id>_private_key	yes (auto-encrypted because the key contains the private_key substring)	text	base64 WG private key
egress_tunnel_<id>_preshared_key	yes, optional	text	base64 WG PSK (or absent / empty)

Constraints:

- id MUST NOT contain underscores (it would collide with the _private_key / _preshared_key naming convention). Use hyphens.
- interface MUST start with wg- and be ≤ 15 chars total (Linux IFNAMSIZ).
- socks5_listen_port MUST be 1024-65535, unique across tunnels. Convention: 1081 for the first tunnel, then 1082, 1083, ….
- address must parse as IPv4 or IPv6; IPv4 prefix ≤ 32, IPv6 ≤ 128.

Once the three keys exist in the DB, restart the sidecar:

```
docker compose restart snapper-egress
```

The sidecar orchestrator (run_sidecar) calls load_declared_tunnels(service) → wg_control.bring_up(...) → Socks5Server.start() for each tunnel. Failures are best-effort: one bad tunnel does NOT block the others.

Wiring the route into the EgressPool

Edit the egress_pool setting to add the new route:

```
{
  "enabled": true,
  "on_all_quarantined": "wait",
  "private_fallback_route_id": "wg-uk-1",
  "routes": [
    {"id": "default", "kind": "direct", "priority": 100, "enabled": true},
    {"id": "wg-uk-1", "kind": "socks5",
     "proxy_url": "socks5h://snapper-egress:1081",
     "region": "uk-lon",
     "exit_ip": "203.0.113.20",
     "provider": "wireguard-uk",
     "priority": 10, "enabled": true}
  ]
}
```

region, exit_ip, and provider are optional operator metadata fields. They do not affect routing; they are surfaced through GET /api/health/egress so an operator can identify where a route exits.

private_fallback_route_id is an optional top-level key (default null). It names a declared route used as the **private-traffic fallback only when no healthy direct route exists** — so executor (order) traffic CAN ride a SOCKS5 tunnel when configured, even though private traffic normally goes direct (see *Public vs private traffic* below). It is validated at config-load: when set, it MUST reference an existing route id, otherwise EgressPoolConfig rejects the pool. Its allowed_exchanges is **ignored** on the private fallback path (the route may be publicly pinned to one exchange while still serving as the private fallback for another). Kraken authenticated REST wires this route for private executor REST reads.

Priority semantics: EgressPool.reserve() sorts by (priority, in_use_count) and picks the **lowest** number first. For SOCKS5 to actually win selection, its priority MUST be less than the default route's priority. Bootstrap configurations that ship default: priority=0 and a new tunnel at priority=10 would silently keep using direct egress because 0 < 10. Set the direct route to a high number (e.g. 100) when you want it to act as fallback only. Keep at least one enabled direct route whenever egress_pool.enabled=true; current schema does not support disabling direct fallback for an enabled pool.

Per-exchange routing — pinning a tunnel to one exchange

To restrict a SOCKS5 route to specific exchanges, add an allowed_exchanges field listing the exchange names (matching ExchangeEnum *values* — "walutomat", "kraken", "kraken_futures", "kraken_equities", "polygon", "paper"):

```
{
  "id": "wg-eu-1", "kind": "socks5",
  "proxy_url": "socks5h://snapper-egress:1084",
```

```
  "priority": 10,
  "allowed_exchanges": ["walutomat"],
  "enabled": true
}
```

A non-empty allowed_exchanges restricts the pool: when pool.reserve(exchange=...) is called with an exchange name not in the list, this route is filtered out before the (priority, in_use_count) sort. An empty allowed_exchanges (the default) means the route serves any exchange — the back-compatible path for existing pool entries.

The direct fallback path IGNORES allowed_exchanges by design. When every preferred route is quarantined, EgressPool returns the direct route as a last resort regardless of any exchange pin. If you want SOCKS routes preferred for a specific exchange, pin those routes with allowed_exchanges and give the direct route a worse priority; current schema does not support per-exchange denial of direct fallback. Unknown exchange names (typos) are rejected by EgressPoolConfig at config-load time, so a "krakeen" typo never silently makes a route unreachable.

Public vs private traffic

Routing depends on the reservation's **traffic class** (TrafficClass = Literal["public", "private"], defaulting to "public"):

- **Public** market-data traffic (Kraken WS publishers and other feeds) routes over the VPN/SOCKS5 tunnels by the public selection path — the (priority, in_use_count) sort honouring each route's allowed_exchanges allow-list, with the direct route as fallback.
- **Private** executor (order) traffic is routed **direct-first**. Executors wrap their work in egress_identity(traffic_class="private", owner="executor"), and the pool routes traffic_class="private" through a separate selection path (_pick_private_locked) that prefers a healthy direct route regardless of route priority and ignores allowed_exchanges, bypassing the public sort entirely. When direct is unavailable and private_fallback_route_id is configured, private idempotent REST reads and the next private WebSocket reconnect can ride that fallback route. Private mutations remain direct-only. Kraken authenticated REST ops are tagged the same way.

In short: **public = VPN/SOCKS5 preferred** (priority + allow-list, with direct fallback), **private = direct-first**. A healthy direct route always wins for private traffic; private_fallback_route_id is used only when direct is unavailable, and private mutations remain direct-only. Private executor WebSocket direct connect errors briefly quarantine the direct route for about 15 seconds so the next reconnect can use the configured private fallback. Public direct routes are not quarantined on connect errors; proxy routes quarantine for about 30 seconds.

Then enable the feed gate (DB setting feed_egress_enabled=true) if not already on, and restart both the API and the feed tier — the feed publishers hold their own process-local pools:

```
docker compose restart snapper snapper-feed
```

The process-local egress preflight validates each SOCKS5 route before installing the pool: it checks that python-socks is importable, resolves the proxy host, opens the proxy port, and sends a SOCKS5 NO_AUTH greeting. A route that fails preflight is auto-disabled in that process's effective config, so restart every process that should use the updated route set.

The orchestrator logs:

```
egress_pool: configured with 2 route(s), on_all_quarantined=wait
```

Verification

1. **Sidecar healthy?**

   ```
   docker compose exec snapper-egress \
     python -c "import urllib.request; print(urllib.request.urlopen('http://
   127.0.0.1:8081/ready', timeout=2).read())"
   ```

   Expect a 200 with the running + failed tunnel id arrays. This is the Compose health endpoint and intentionally stays 200 after bootstrap even if an individual tunnel failed.

2. **All tunnels up?**

   ```
   docker compose exec snapper-egress \
     python -c "import urllib.request; print(urllib.request.urlopen('http://
   127.0.0.1:8081/readyz', timeout=2).read())"
   ```

   Expect 200 only when every declared tunnel is up; otherwise it returns 503 with the failed tunnel ids.

3. **Tunnel status detail?**

   ```
   docker compose exec snapper-egress \
     python -c "import urllib.request; print(urllib.request.urlopen('http://
   127.0.0.1:8081/tunnels', timeout=2).read())"
   ```

   Each tunnel id maps to {"status": "up", "reason": null}.

4. **Backend egress snapshot visible?** From an authenticated session whose effective grant contains read:system_status (including a read-only viewer session), call GET /api/health/egress. Expect payload.enabled=true, one row per configured route, route metadata (region, exit_ip, provider) when configured, quarantine state, merged in_use_count, per-reservation container, connections rows for open WebSocket host counts and capped REST last-seen hostnames, and a transfer object on SOCKS5

routes when snapper-egress has reported matching WireGuard counters for that listener port. Transfer rows include cumulative rx/tx bytes, current byte rates when a reliable delta exists, the latest WireGuard handshake timestamp, sample age, and stale/reset flags. Direct routes, missing samples, and ambiguous shared ports show transfer=null. Connection rows expose hostnames only, never URL paths, query strings, headers, request bodies, or credentials. A missing snapper-feed row means the API process has not received a system.egress.snapshot frame from that container yet; a stale row remains visible with stale=true.

5. **WireGuard handshake established?** (operator debugging — uses wireguard-tools and iproute2 shipped in the image)

   ```
   docker compose exec snapper-egress wg show wg-uk-1 latest-handshakes
   docker compose exec snapper-egress wg show wg-uk-1 transfer
   docker compose exec snapper-egress ip -d link show wg-uk-1
   docker compose exec snapper-egress ip rule show table 5000
   ```

   A nonzero recent handshake and increasing transfer counters after a test request establish peer connectivity. An UP interface, /readyz success, or a tunnel status: up only proves local configuration succeeded. The routing rule should pin from <tunnel_addr> → table N.

6. **Kraken WS uses the tunnel?** The pool is a process-local singleton inside each feed-publisher process — get_egress_pool() in a fresh docker compose exec interpreter returns None, and quarantining inside the API process would not touch the publisher pools, so a REPL-based quarantine cannot exercise the failover. Instead, steer by configuration: give the direct route the worst priority in the egress_pool setting, make sure feed_egress_enabled=true, then restart snapper-feed and observe the next handshake. The publisher log line should show the tunnel route id; Cloudflare's view of the source IP should match the VPN exit. Restore the normal direct-route priority when done.

7. **Stable tick flow?**

   ```
   SELECT i.exchange, COUNT(*) AS n
   FROM ticks t JOIN instruments i ON i.public_id = t.instrument_public_id
   WHERE t.timestamp > NOW() - INTERVAL '60 seconds'
   GROUP BY i.exchange ORDER BY n DESC;
   ```

   Compare counts and freshness with this deployment's baseline for the same subscriptions and market activity. Tick persistence policy also affects these counts; a drop alone does not identify an egress failure. Correlate it with publisher logs, handshake age, and transfer counters.

Threat-model + security notes

- The SOCKS5 listener runs WITH NO AUTHENTICATION. Isolation comes entirely from the Docker snapper-internal network. The CI lint hook scripts/check_egress_compose.py rejects any ports: entry, network_mode: host, or env-interpolated bypass on the snapper-egress service so an operator cannot accidentally publish the listener to the host.
- The same lint hook also enforces the unified-image runtime contract: snapper-egress must use the same image as snapper, run command: ["egress"] as user: "0:0", include NET_ADMIN, and mount /app/data when the monolith does. The snapper service must remain unprivileged, with no cap_add and no user override.
- All traffic on snapper-internal is trusted equally. Do NOT attach untrusted containers (other tenants, debugging shells from unknown sources) to that network. If you must, add SOCKS5 username/password auth in socks5_server.py (≈10-line change).
- For extra defense-in-depth, run docker compose config | grep -A5 snapper-egress after any compose edit to verify the effective config still has no host port mapping. The lint hook catches the static cases but not every override pattern.
- WG private keys are stored Fernet-encrypted in the Postgres settings table. The encryption key is derived from MASTER_PASSWORD (env var). Loss of master password = loss of every encrypted setting (not just WG keys).

Decommissioning a tunnel

1. Remove the route from egress_pool (set enabled: false on that route OR delete the entry); restart the API and feed tiers (docker compose restart snapper snapper-feed).
2. Delete the three settings (egress_tunnel_<id>, egress_tunnel_<id>_private_key, egress_tunnel_<id>_preshared_key).
3. Restart snapper-egress. The stopping sidecar's shutdown pass brings the removed tunnel's interface / ip-rule / routing-table entries down, and the container restart recreates the network namespace; the idempotent cleanup-before-bring_up in wg_control.bring_up only re-cleans interfaces and routing tables of tunnels that are still declared.

Rotating private keys

1. Generate a new private key + matching peer config on the VPN side.
2. Update the egress_tunnel_<id>_private_key setting (SettingsService re-encrypts on write).
3. Restart snapper-egress to pick up the new key. The orchestrator's bring_up is idempotent and recreates the WG interface from scratch.

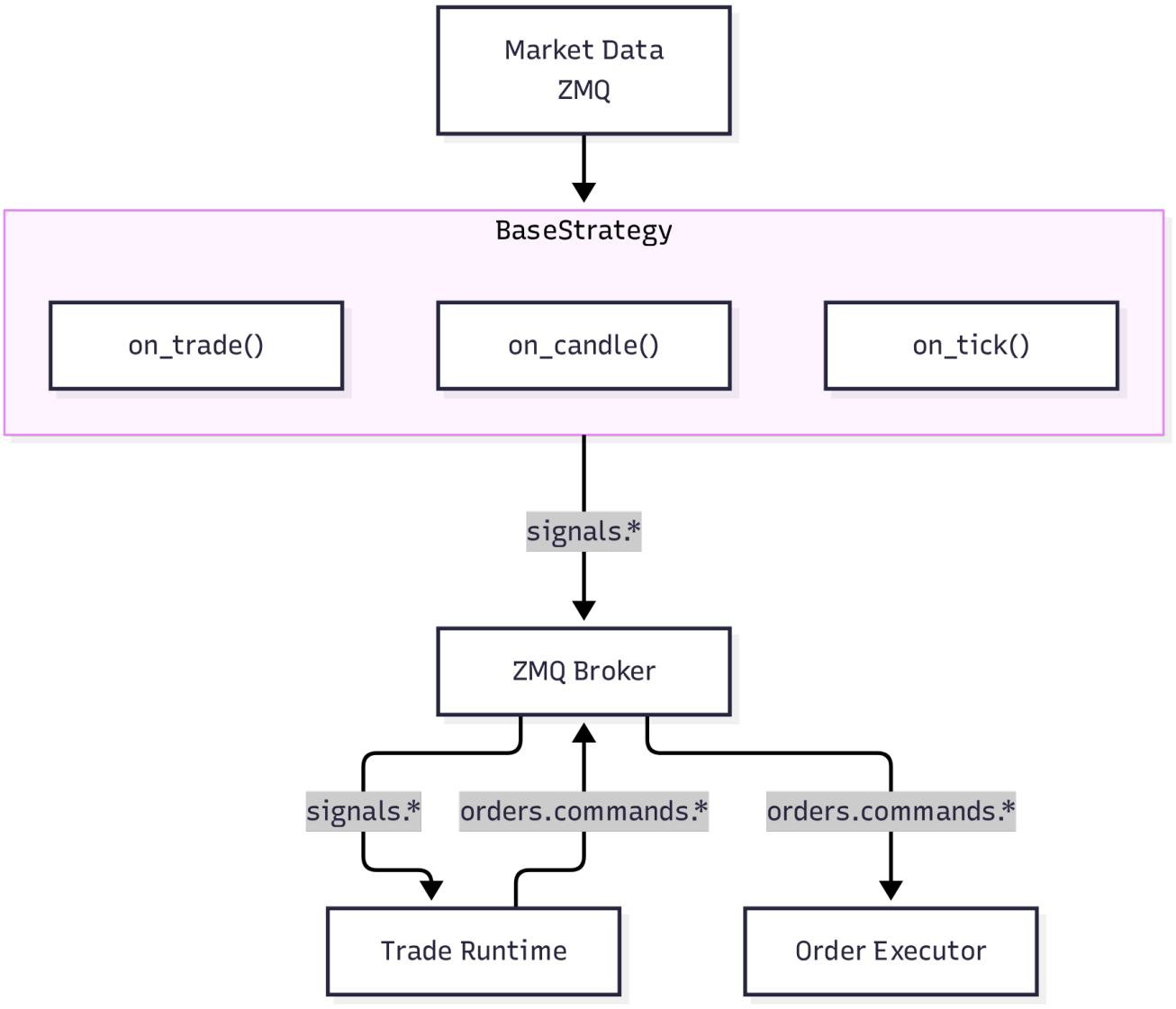
Operational expectations

- Sidecar startup: ~5-15 s (DNS resolution of peer endpoints is the dominant cost).
- HTTP 429 failover: ~1-2 s (route quarantined → next handshake picks alternate).
- WS connect-error failover (ws-connect-error): the dead proxy route is quarantined ~30 s on a connect timeout/refused/blackhole, so the next reconnect rides an alternate route (or the direct fallback) instead of re-dialing the dead tunnel.
- Tunnel handshake age: WireGuard re-keys every 2 minutes; if wg show <iface> latest-handshakes returns > 180 s ago, the peer is unreachable. The backend also surfaces the latest handshake from sidecar system.egress.transfer samples on the matching SOCKS5 route.
- SOCKS5 listener latency: ≤ 1 ms additional vs direct route (single process, single asyncio loop, no auth handshake).

Trading Strategies

Snapper provides a framework for creating trading strategies based on ZeroMQ messaging. Strategies subscribe to market data and publish signals. Strategies produce intent only; the trade runtime owns order lifecycle, positions, and cash projections.

Strategy Architecture



Authorization

Authorization uses effective token permissions from Snapper's 34-permission catalogue, not a role hierarchy. The frontend opens the Strategies resource with read:strategies, derives each control from its exact mutation permission, and rechecks that permission when the action runs.

Surface	Required permission	Current named permission sets
GET /api/strategies	read:strategies	ai_reviewer, ai_delegate, viewer, operator, admin
Read the process catalogue, configured rows, schemas, and runs	read:processes	viewer, operator, admin
Register a disabled strategy or edit its scope	configure:strategies	operator, admin
Register an enabled strategy	configure:strategies and start:strategies	operator, admin
Start or enable a strategy	start:strategies	operator, admin
Stop or disable a strategy	stop:strategies	operator, admin
Restart desired state	start:strategies and stop:strategies	operator, admin
Create a backtest from a strategy card	manage:backtests	operator, admin

The current viewer set contains both read permissions, so it sees the full operator read surface while every strategy mutation remains unavailable. Strategy targets use these target-specific permissions; manage:processes continues to authorize only non-strategy process targets. Scope configuration and launch also validate operator membership, wallet grants, and live-output coverage.

Creating Strategies

Basic Structure

```
from snapper.strategies.base import BaseStrategy, StrategySignal, StrategyConfig
from snapper.strategies.decorators import register_strategy, create_strategy_pro
cess
from snapper.messaging.schemas.data import CandleData


@register_strategy("MyStrategy")
@create_strategy_process(
    process_name="my_strategy_btc",
    default_config={
        "name": "my_strategy_btc",
        "inputs": ["market.kraken.BTC-USD.candles.1h"],
        "outputs": ["BTC-USD"],
        "exchange": "paper",
```

```
        "params": {
            "threshold": 0.5,
            "period": 14,
        },
    },
)
class MyStrategy(BaseStrategy):
    """Strategy description in docstring."""

    def __init__(self, config: StrategyConfig) -> None:
        """Initialize strategy."""
        super().__init__(config)
        self.threshold = self.params.get("threshold", 0.5)
        self.period = self.params.get("period", 14)

    async def on_candle(self, instrument: str, candle: CandleData) -> StrategySi
gnal | None:
        """Process candle and generate signal."""
        candles = self.candle_buffer.get(instrument, [])
        if len(candles) < self.period:
            return None

        closes = [c.close for c in candles]
        current_price = closes[-1]

        if self._should_buy(closes):
            return StrategySignal(
                instrument=instrument,
                side="buy",
                strength=1.0,
                price=current_price,
                reason="Buy condition met",
            )

        if self._should_sell(closes):
            return StrategySignal(
                instrument=instrument,
                side="sell",
                strength=1.0,
                price=current_price,
                reason="Sell condition met",
            )

        return None

    def _should_buy(self, closes: list[float]) -> bool:
        """Buy condition logic."""
        return False

    def _should_sell(self, closes: list[float]) -> bool:
        """Sell condition logic."""
        return False

    async def reset(self) -> None:
        """Reset strategy state (for replay)."""
        pass
```

Decorators

@register_strategy(name)

Registers strategy class in factory under given name.

```
@register_strategy("RSIReversion")
class RSIReversion(BaseStrategy):
    ...
```

@create_strategy_process(process_name, default_config)

Creates process wrapper for strategy and registers in process manager.

```
@create_strategy_process(
    process_name="strategy_rsi_eth_1h",
    default_config={
        "name": "rsi_eth_1h",
        "inputs": ["market.kraken.ETH-USD.candles.1h"],
        "outputs": ["ETH-USD"],
        "exchange": "paper",
        "params": {"period": 14, "upper": 70.0, "lower": 30.0, "cooldown": 2},
    },
)
class RSIReversion(BaseStrategy):
    ...
```

Strategy Configuration

StrategyConfig

Field	Type	Description
name	string	Unique strategy instance name
strategy_class	string	Strategy class name
inputs	list[str]	List of ZMQ topics to subscribe
outputs	list[str]	List of instruments for signals
exchange	string	Target exchange (paper, kraken, kraken_futures, walutomat)
params	dict	Strategy-specific parameters
	string	

Field	Type	Description
wallet_public_id		Wallet that will execute orders for this strategy. Still defaults to empty and is NOT validated by StrategyConfig itself (the dataclass permits unscoped configuration). Process routes validate active operator/ wallet grant coverage when both this field and operator_public_id are populated; a wallet without an operator is rejected. The trade coordinator drops signals with missing or blank wallet strings before engine/group persistence. Strategies using create_ai_review_and_await() also require a non-empty wallet at the caps guard — see "AI delegate consultation" below.
operator_public_id	string	Trading-identity operator that owns this strategy instance. Empty default; validated against the launching principal's operator_public_ids when populated, and used with wallet_public_id for active grant and live-output coverage checks.

Scoped-strategy reference identities

operator_public_id and wallet_public_id accept either a concrete public id or a label:<name> reference; the launch-time scope resolver (snapper.application.process_manager.strategy_scope) rewrites each label: value to a canonical UUID against the active operator / wallet catalogue (fail-closed on a blank, unknown, or ambiguous label).

A strategy can also declare identity references nested inside params:

- REFERENCE_IDENTITY_PARAMS: ClassVar[Mapping[str, str]] maps a nested params key to its reference kind ("operator", "wallet", or "user"). A label:<name> value under that key is resolved the same way at launch (a nested reference of any kind fails closed when the launch has no resolved operator to scope it).
- SEEDED_IDENTITY_PARAMS: ClassVar[tuple[str, ...]] names nested keys that carry a generate-once UUID7 correlation token; they are minted fill-if-empty by the persistence layer, never label-resolved, and never rotated once populated.

HeartbeatConsult is the canonical scoped strategy — it declares REFERENCE_IDENTITY_PARAMS = {"ai_review_user_public_id": "user"} and SEEDED_IDENTITY_PARAMS = ("ai_review_strategy_public_id",).

Candle history warm-up

A strategy that needs historical bars before it can compute indicators overrides required_candle_history() -> int (e.g. CointegrationPairs returns its lookback_window). When that value is > 0, BaseStrategy.start() prefills the candle buffer from history BEFORE subscribing to the live feed, so the strategy is indicator-ready from a cold start instead of waiting that many live periods (a restart re-warms the same way).

Warm-up is **DB-first** and **opt-in via required_candle_history() > 0**: it reads the persisted plane of each leg's own timeframe (get_candles under the leg's live venue) so the warmed series is continuous with the live synthesized bars and the read path is single-source. Only when the persisted plane is short for some leg (e.g. before the operator has applied the daily backfill) does it fall back to the local Polygon **crypto daily** cache as a non-canonical bootstrap (logged as such) — that fallback alone is crypto-scoped and 1d-only.

- Set params["warmup_market_type"] = "crypto" to enable ONLY the non-canonical Polygon crypto daily cache fallback for aligned 1d legs; the DB-first warm-up itself runs for any strategy declaring a positive history requirement, with or without this param. This keeps a non-crypto strategy from ever loading a crypto ticker.
- Set params["buffer_size"] (default 100) >= the lookback, or warm-up is skipped with a warning (no silent under-warm).
- Optional params["polygon_cache_root"] (default data/polygon/cache) — the bootstrap fallback root, used only when the DB is short.
- Candle input legs are warmed from the persisted DB plane under their own timeframe (1h, 1d, ...) for single-leg strategies and for multi-leg strategies whose legs SHARE one timeframe — mixed-timeframe leg sets skip warm-up entirely (live-only fill); only the Polygon crypto cache bootstrap fallback is restricted to 1d legs with warmup_market_type="crypto".
- **Aligned all-or-nothing for multi-leg strategies**: a pair (e.g. cointegration) is warmed only if every leg shares at least the required number of common UTC days; on any shortfall NO leg is warmed (symmetric live-only fill), so the spread can never start date-misaligned. A warm-up error never crashes startup.

Note: warm-up does NOT arm a strategy. For the FET/RENDER daily cointegration forward test, the spread's per-leg positional alignment is a separate hard pre-arming gate: it aligns legs by open_at, not list position.

AI delegate consultation

Strategies that consult an AI delegate via create_ai_review_and_await() and then forward the approved outcome to emit_signal(outcome=...) MUST set StrategyConfig.wallet_public_id to a non-empty value. The trader-coordinator's caps gate (TradingCapsEnforcer.guard_with_ai_review_attribution) fail-closes early when the engine submission's wallet_public_id is empty:

```
CapsViolationError: missing_wallet_for_ai_review_attribution
    submission.wallet_public_id required for the strategy AI gate;
    engine state must populate wallet on the submission before
    invoking this guard
```

The error is loud and operator-actionable: it identifies the exact missing config field. Without wallet_public_id, the engine cannot map the AI delegate's approved trade to a wallet for cap evaluation (per-user notional / open-orders / quantity caps key on wallet+user). Pure non-AI strategies that never invoke create_ai_review_and_await() continue to run without a wallet; they bypass caps via guard_service_principal() per the existing audit-bypass contract.

The ai_review_public_id carried on the published SignalData is transport-only end-to-end through the ZMQ wire. The companion ai_review_dispatch_version (dedup key) is also transport-only — the strategy citation validator does not compare it; the bus publisher reads dispatch_version from the cited row at publish time so downstream caps-violation fanout events are always tagged with the row-of-record version.

Input Topics

Common formats:

- market.{exchange}.{instrument}.candles.{timeframe} — Candle streams
- market.{exchange}.{instrument}.ticks — Tick streams
- market.{exchange}.{instrument}.trades — Trade streams
- market.paper.{source_exchange}.{instrument}.{type} — Paper replay streams from a historical source

Examples:

- market.kraken.BTC-USD.candles.1h — Hourly BTC/USD candles from Kraken
- market.kraken.ETH-USD.ticks — ETH/USD ticks from Kraken
- market.paper.polygon.AAPL.trades — Paper replay of AAPL trade tape from Polygon

Output Topics (Signals)

Generated automatically:

- Paper: signals.paper.{instrument}.{strategy_name}
- Live: signals.{exchange}.{instrument}.live

StrategySignal Structure

```
@dataclass
class StrategySignal:
    """Trading signal."""

    instrument: str
    side: TradeSide
    strength: float
```

```
    reason: str
    price: float
    timestamp: datetime | None = None
```

Signal-safety convention (MANDATORY)

A signal is an **absolute position target**, never an event. The trade coordinator maps (side, strength) to a signed target (desired_units = +strength for buy, -strength for sell) and executes delta = desired_units - current_position, so an in-sync target is a no-op (abs(delta) < 1e-12).

Two rules keep strategies safe across the independent restart domains (the strategies container can restart while the trade runtime keeps running, and vice versa):

1. **Express an EXIT as a flat target (strength=0.0), NEVER as an opposite-side strength>0 signal.** Use BaseStrategy.emit_flat(...) (a signal builder — return it from on_candle). An opposite-side strength>0 "exit" is an absolute *reversed* target: after an at-most-once-dropped entry (engine flat), it opens a naked reversed position instead of closing nothing. A flat target against a flat engine is a delta no-op — no reversal. A genuine long→short reversal is a deliberate signed target, distinct from an exit.
2. **Re-assert the standing target every decision bar** (opt in via reasserts_targets() returning True). Because the same target is a delta no-op, re-asserting is idempotent on the trade side and self-heals a dropped signal *within a session*. This does NOT recover a standing target after a process restart (the map starts empty and is repopulated from the first live decision) — restart discipline (start flat; flatten after any restart) covers that until venue-truth position reconciliation lands.

RSIReversion and MACDCrossover default to long_only=True, clamping a would-be short to emit_flat so they never emit a negative target. CointegrationPairs already exits with strength=0.0 and re-asserts a standing ENTRY as one paired group (atomicity preserved); a flat EXIT is re-asserted as independent per-leg flats (flattening needs no cross-leg coordination, and a flat paired group every bar would churn broken zero-leg groups). Re-assertions are repriced to each leg's current close and never overwrite the standing decision. Direct emit_signal callers (e.g. an AI-consult heartbeat) bypass the bar-driven re-assert by design.

Built-in Strategies

RSIReversion

Mean-reversion strategy based on RSI.

Parameters:

Parameter	Default	Description
period	14	RSI period
upper	70.0	Overbought threshold (sell)
lower	30.0	Oversold threshold (buy)
cooldown	0 (class fallback) / 2 (registered process preset)	Candles between signals

Logic:

- Buy when RSI <= lower **or** RSI was at or below lower on the previous candle (r_prev <= lower) and price has since reversed upward.
- Sell when RSI >= upper **or** RSI was at or above upper on the previous candle (r_prev >= upper) and price has since reversed downward.
- The previous-threshold + reversal branch lets the strategy capture exits that miss the strict edge-touch but confirm via price action.

```
from snapper.strategies.rsi import RSIReversion
```

MACDCrossover

Trend-following strategy based on MACD.

Parameters:

Parameter	Default	Description
fast	12	Fast EMA
slow	26	Slow EMA
signal_period	9	Signal line EMA period

Logic:

- Buy when MACD histogram crosses from negative to positive
- Sell when histogram crosses from positive to negative

```
from snapper.strategies.macd import MACDCrossover
```

Cointegration

Pairs trading strategy based on cointegration. Trades the spread between two cointegrated instruments, entering when the z-score crosses entry_threshold and exiting on mean reversion past exit_threshold.

The spread aligns the two legs **by open_at** (the days both legs have a bar), never by list position — a live feed delivering one leg's bar before the other cannot desync the regression. A signal fires only when the triggering candle is **both legs' current bar** (open_at == latest common day == latest seen), so the z-score and both legs' order prices are always the same executable day; the completing (second-arriving) leg of each period carries the signal. At most one new decision group is emitted per open_at interval (hourly or daily according to the input timeframe), and no new decision fires at/under the warm-up high-water mark (prefilled history is context, not tradeable). This matches the date-aligned daily backtest.

```
from snapper.strategies.cointegration import CointegrationPairs
```

CointegrationPairs is the canonical 2-leg consumer of MultiLegSpreadMixin — its 2-leg invariants are enforced by self._init_legs(expected_count=2) and instrument1 / instrument2 are backwards-compatible aliases for self.legs[0] / self.legs[1]. It declares PAIRED_EXECUTION_POLICY = SIMULTANEOUS, so its entry/exit pairs emit as one synchronized paired-execution group. See the next section for the generalized N-leg API.

Parameters:

Parameter	Default	Description
beta	0.05	Hedge ratio between legs (long leg − beta × short leg)
entry_threshold	2.0	Z-score threshold for entry (in standard deviations)
exit_threshold	0.5	Z-score threshold for mean-reversion exit
lookback_window	50	Window length for rolling spread statistics
min_data_points	30	Minimum buffered candles before any signal is emitted

Two processes register this class out of the box: strategy_cointegration_btc_eth trades hourly BTC-USD/ETH-USD paper candles with the defaults above, and strategy_cointegration_fet_render is a forward-test preset that trades daily FET-USD/RENDER-USD paper candles with a screened hedge ratio (beta=0.257463, lookback_window=60). It opts into the Polygon crypto daily warm-up fallback and sets buffer_size=100, so a fresh default process can prefill the 60-bar spread window from the persisted 1d history (falling back to the local Polygon crypto cache only when the DB plane is short for a leg).

HeartbeatConsult

A paper-only heartbeat that emits one AI-delegate CONSULT per completed candle to exercise the AI-review / consult wake path end-to-end (strategy -> ai_reviews -> MCP-delegate wake -> decision -> resume). On an approved decision it publishes a signal whose strength is the configurable heartbeat_signal_strength param: the default 0.0 stays target-flat (the emit + AI-review attribution path runs without ever opening a position), while a value in [0.0, 1.0] opens an actionable paper long so the full signal -> order -> fill -> position plane is exercised. The constructor rejects any non-paper exchange and requires a scoped config (both wallet_public_id and operator_public_id), and validates ai_review_user_public_id / ai_review_strategy_public_id as canonical UUID7 fail-fast.

The consult round runs DETACHED from the listen loop: on_candle spawns the round and returns immediately, so the strategy keeps draining feed heartbeats and its health lag never absorbs the decision deadline (raising ai_review_deadline_seconds has no alerting side effects). At most one round is in flight; stop/reset cancel it. Each consult's signal_envelope carries the proposed action plus a self-contained market snapshot (SMA 20/50, Wilder RSI 14, 1-bar and 24-bar percent changes, 24-bar high/low, 24-bar realized volatility) computed from the trailing 200 complete persisted bars bounded at the trigger window; null market fields mean that indicator is still warming up. A sibling macro snapshot reads complete persisted 1m kraken_equities candles for the configured CME equity-index contract at the same consult-time as_of anchor. It carries exactly symbol, as_of, age_minutes, session, change_1h_pct, change_since_session_open_pct, and realized_vol_24h_pct, with a presence-only stale: true field on an incident. The percentage fields compare the latest complete print with the print at least one hour earlier and with the current CME session's opening price; realized volatility is the population standard deviation of one-minute simple returns from the 24 wall-clock hours ending at the last print. age_minutes is computed at envelope-build time from the newest candle's persisted bus timestamp, never copied from stored metadata.

The shared CME calendar classifies the macro session as open, closed, or halted. When CME is open and age_minutes exceeds macro_stale_after_minutes, the snapshot reports session: "halted", adds stale: true, and sets all three percentage/volatility values to null. During the daily break, weekend, or a scheduled holiday closure, it instead reports session: "closed" without a stale key and keeps the old last print as the calculation anchor, its honest age, and all available derived values. The configured contract is never replaced by a crypto proxy during a closure. Market and macro construction are independently fail-soft, so a read or calculation error omits only the affected section and never prevents the consult.

Parameters:

Parameter	Default	Description
ai_review_user_public_id	""	UUID7 of the strategy owner stamped on review rows; accepts a label:<username> reference (declared reference-identity param).
ai_review_strategy_public_id	""	Stable UUID7 identifying this strategy instance; seeded-once (minted fill-if-empty), never label-resolved.
ai_review_deadline_seconds	25	Per-round decision deadline, bounded [5, 300].
heartbeat_signal_strength	0.0	Approved-round emit strength, bounded [0.0, 1.0]; 0.0 = target-flat.
macro_contract_symbol	"MNQU6-CME"	Native quarterly CME contract used for the macro snapshot. The default is the September 2026 Micro Nasdaq-100 future; update this parameter during the operator's quarterly contract roll.
macro_stale_after_minutes	10.0	Maximum healthy age for the newest complete 1m candle while CME is open. Exceeding it marks the session halted and suppresses derived values; scheduled closures are exempt.

```
from snapper.strategies.heartbeat_consult import HeartbeatConsult
```

The registered paper process is strategy_heartbeat_consult_btc_1h (1h BTC-USD candles).

Proprietary strategies

When the proprietary submodule is present, additional strategy classes live under proprietary/src/strategies. They are registered by their modules and use the same BaseStrategy and process-wrapper contracts as open-source strategies. Ignore proprietary/plans and proprietary/memory historical notes unless a current README explicitly points to them as live docs.

SPYBTCMomentum

SPYBTCMomentum is a cross-asset momentum strategy: completed/current SPY hourly movement drives same-direction BTC signals. It does **not** trade a correlation spread and does **not** emit SPY orders.

The default paper process subscribes to 1-minute SPY and BTC candles, aggregates both into hourly bars, and emits at most one BTC signal per UTC hour during the configured US session window. A positive SPY return above spy_threshold emits BUY on BTC; a negative SPY return below -spy_threshold emits SELL on BTC. Signal price comes from the current BTC hourly aggregate.

Parameters:

Parameter	Default	Description
spy_threshold	0.002	Absolute SPY hourly return threshold before a BTC signal can fire.
us_session_start_utc	14	Inclusive UTC hour at which signals may start.
us_session_end_utc	21	Exclusive UTC hour at which signals stop.
spy_instrument	SPY	Instrument identifier used to route incoming SPY candles.
btc_instrument	BTC-USD	Target BTC instrument emitted in StrategySignal.instrument.

ParlayCascade

ParlayCascade is a sequential N-leg compounding strategy. It uses MultiLegSpreadMixin for the ordered leg roster, but it does not open all legs at once. It starts on leg 0 only after the trailing-window drift gate clears, then advances through the configured legs one at a time.

Each active leg exits by one of three paths:

1. Favourable move reaches tp_pct: exit the current leg. If another leg remains, the callback returns [exit_current, enter_next] so the handoff uses the multi-leg list return contract. Note: ParlayCascade does not currently declare PAIRED_EXECUTION_POLICY, so this two-signal handoff only works on the backtest recording path; on the live/paper ZMQ path BaseStrategy fail-closes an undeclared multi-leg group with ValueError at emission and the strategy's listen loop stops (see "Signal pairing semantics"). Declaring PairedExecutionPolicyEnum.SEQUENTIAL_HANDOFF on the class is required before the registered paper process (parlay_cascade_btc_eth_sol) can emit the handoff.
2. Adverse move reaches sl_pct: exit the current leg, mark the cascade busted, and start the cooldown.
3. max_bars_per_leg candles elapse: exit the current leg as a timeout bust and start the cooldown.

The min_edge_bps gate is measured over lookback_window candles on leg 0. For long entries, positive drift is favourable; for short entries, the sign is flipped so positive still means "edge in our favor". After a bust, cooldown_bars candles must drain before the next attempt can start. A successful full cascade moves to complete state without cooldown and then returns to idle on the next eligible cycle.

Parameters:

Parameter	Default	Description
tp_pct	0.01	Per-leg take-profit threshold as a fraction. Must be positive.
sl_pct	0.01	Per-leg stop-loss threshold as a fraction. Must be positive.
max_bars_per_leg	24	Per-leg timeout in candles. Must be positive.
entry_side	buy	Direction used for every leg entry; sell inverts favourable/adverse tests.
lookback_window	20	Number of trailing candles used by the drift edge gate.
min_edge_bps	5.0	Minimum favourable drift, in basis points, required before leg 0 can enter.
cooldown_bars	4	Number of candles to wait after a bust before another attempt.

Multi-leg / N-leg basket strategies

Snapper ships a reusable mixin for strategies that emit synchronized signals across N legs (pair-trades, baskets, calendar spreads, cross-exchange arb). The 2-leg cointegration strategy is the canonical consumer; the same mixin generalizes to N ≥ 2.

```
from snapper.strategies.multi_leg import MultiLegSpreadMixin, resolve_legs
```

Two invocation shapes

resolve_legs(config, expected_count=None) accepts either:

1. **Live ZMQ process** — inputs is a length-N list of market-data candle topics, one per leg. Leg instruments are parsed from each topic via parse_market_topic.

```
config = StrategyConfig(
    name="basket_btc_eth_sol",
    inputs=[
        "market.paper.kraken.BTC-USD.candles.1h",
        "market.paper.kraken.ETH-USD.candles.1h",
```

```
        "market.paper.kraken.SOL-USD.candles.1h",
    ],
    outputs=["BTC-USD", "ETH-USD", "SOL-USD"],
    ...
)
legs = resolve_legs(config, expected_count=3)
```

2. **Direct-DB backtest** — DirectDbEngine synthesises a single candles.{exchange}.synthetic.{timeframe} input and passes the leg instruments through outputs.

```
config = StrategyConfig(
    name="basket_btc_eth_sol",
    inputs=["candles.kraken.synthetic.1h"],
    outputs=["BTC-USD", "ETH-USD", "SOL-USD"],
    ...
)
legs = resolve_legs(config, expected_count=3)
```

resolve_legs raises ValueError if neither shape matches or if expected_count is set and the resolved tuple has the wrong length.

Mixin API

Subclassing both BaseStrategy and MultiLegSpreadMixin gives you:

Attribute / method	Purpose
self.legs: tuple[str, ...]	Resolved leg instruments in declaration order. Populated by self._init_legs(expected_count=N) from __init__.
self._partner_legs(current)	Tuple of leg names other than current, in declaration order.
self._partner_prices(current)	{leg: last_close} for partners with at least one buffered candle. Partners with empty buffers are omitted (same warmup gate as 2-leg cointegration).
self._build_partner_signals(current, builder)	Pure builder: calls builder(leg, last_close) for each buffered partner and returns the resulting list[StrategySignal]. The host strategy concatenates these after its primary signal and returns [primary, *partners] from on_candle; BaseStrategy validates the group before sending its legs sequentially (see "Signal pairing semantics"). No side-channel queue.

Worked example — 3-leg equal-weight basket

The strategy builds its partner legs with _build_partner_signals and returns the whole group as [primary, *partners] from on_candle. BaseStrategy validates the complete group before emission or backtest recording. The live/paper path sends separate ZMQ frames

with a shared group descriptor; process_time_batch simulates each leg independently. The example illustrates the callback shape, not execution or sizing parity between paths. Its relative-price deviation is not a volatility-normalized statistical z-score.

```
from snapper.messaging.schemas.data import CandleData
from snapper.core.types import PairedExecutionPolicyEnum, TradeSide
from snapper.strategies.base import BaseStrategy, StrategyConfig, StrategySignal
from snapper.strategies.base import StrategySignalResult
from snapper.strategies.decorators import register_strategy, create_strategy_pro
cess
from snapper.strategies.multi_leg import MultiLegSpreadMixin


@register_strategy("EqualWeightBasket")
@create_strategy_process(
    process_name="basket_btc_eth_sol",
    default_config={
        "name": "basket_btc_eth_sol",
        "inputs": [
            "market.paper.kraken.BTC-USD.candles.1h",
            "market.paper.kraken.ETH-USD.candles.1h",
            "market.paper.kraken.SOL-USD.candles.1h",
        ],
        "outputs": ["BTC-USD", "ETH-USD", "SOL-USD"],
        "exchange": "paper",
        "params": {"lookback_window": 50, "z_entry": 2.0, "z_exit": 0.5},
    },
)
class EqualWeightBasket(BaseStrategy, MultiLegSpreadMixin):
    """Z-score each leg vs the basket mean and rebalance when it diverges."""

    PAIRED_EXECUTION_POLICY = PairedExecutionPolicyEnum.SIMULTANEOUS

    def __init__(self, config: StrategyConfig) -> None:
        super().__init__(config)
        self.lookback_window = int(self.params.get("lookback_window", 50))
        self.z_entry = float(self.params.get("z_entry", 2.0))
        self.z_exit = float(self.params.get("z_exit", 0.5))
        self._init_legs(expected_count=3)

    async def on_candle(
        self, instrument: str, candle: CandleData
    ) -> StrategySignalResult:
        partner_prices = self._partner_prices(instrument)
        if len(partner_prices) < len(self.legs) - 1:
            return None

        basket_mean = (candle.close + sum(partner_prices.values())) / len(self.l
egs)
        z = (candle.close - basket_mean) / basket_mean

        if abs(z) < self.z_entry:
            return None

        side: TradeSide = "sell" if z > 0 else "buy"
        primary = StrategySignal(
            instrument=instrument,
            side=side,
```

```
            strength=min(1.0, abs(z) / self.z_entry),
            price=candle.close,
            reason=f"basket_z={z:+.2f}",
        )
        opposite: TradeSide = "buy" if side == "sell" else "sell"
        partners = self._build_partner_signals(
            instrument,
            lambda leg, price: StrategySignal(
                instrument=leg,
                side=opposite,
                strength=primary.strength,
                price=price,
                reason=f"basket_partner_of_{instrument}",
            ),
        )
        return [primary, *partners]
```

Signal pairing semantics

A strategy callback (on_candle / on_tick / on_trade) returns its legs directly: None, a single StrategySignal, or a list[StrategySignal] in the order the strategy chooses. There is no side-channel queue. BaseStrategy normalizes the return to a list and runs a **fail-closed group preflight** at the callback boundary before anything is published or recorded:

- every entry must be a StrategySignal;
- no two legs may share an instrument;
- every leg's instrument must map to a configured output topic;
- a multi-leg group on a **live** (non-paper) exchange is rejected outright unless PAIRED_EXECUTION_GUARD_ENABLED is true — the fail-closed gate that keeps real-money spreads dark until the paired-execution guard is deliberately switched on.

If any check fails the whole group is rejected (the call raises) and **nothing is emitted** — a malformed multi-leg return can never leave one leg of a spread naked. On success, _listen_loop emits each leg in order and process_time_batch records each leg. Group validation is shared, but transport and fill semantics differ: see the live-execution fidelity boundary in backtesting.md.

Every multi-leg strategy **must declare a coordination policy** via the PAIRED_EXECUTION_POLICY class attribute (PairedExecutionPolicyEnum.SIMULTANEOUS for same-instant spreads, as CointegrationPairs declares, or SEQUENTIAL_HANDOFF for exit-then-enter chains); emitting a list[StrategySignal] without one raises ValueError. On emission BaseStrategy stamps the group with a shared paired_group_id (uuid7), paired_group_size, per-leg paired_group_index, the declared paired_group_policy, and a canonical paired_group_key — the descriptors the paired-execution guard correlates on.

> **Scope — validation and correlation.** Preflight rejects malformed groups before publishing any leg. Valid legs are separate awaited ZMQ sends; a send failure or dropped frame can leave only part of a group delivered. Neither

publication nor venue execution is atomic. If one leg rejects, partially fills, or fills late, exposure is still possible because the two legs are independent sends to independent executors/coordinators (and at N≥2 instances the legs can be owned by *different* coordinators, since the shard key includes the instrument). Venue-side atomicity is the **paired-execution guard**'s job: group-id correlation through an arming barrier, halt-both-shards on one-leg failure/timeout, and compensating reduce-only flattens. The guard is implemented but ships dark — with PAIRED_EXECUTION_GUARD_ENABLED false (the default) BaseStrategy refuses to emit live multi-leg groups entirely, while paper multi-leg is always allowed. Operator surface: GET /api/paired-execution/incidents and POST /api/paired-execution/groups/{id}/terminalize — see docs/paired-execution.md.

When NOT to use the mixin

- Single-leg strategies (RSI, MACD, VWAP) — there are no partners to pair with; subclassing the mixin adds no value.
- Strategies whose legs use heterogeneous timeframes — legs assumes one timeframe per strategy process. Run one process per timeframe and coordinate via signals.
- Calendar spreads where one leg trades on a derived feed (e.g. funding-rate vs spot) — the input topic shape doesn't fit resolve_legs; subclass BaseStrategy directly.

Data Access

Candle Buffer

```
async def on_candle(self, instrument: str, candle: CandleData) ->
StrategySignal | None:
    candles = self.candle_buffer.get(instrument, [])

    recent = candles[-20:]

    import pandas as pd
    closes = pd.Series([c.close for c in candles])
```

CandleData

```
from snapper.messaging.schemas.data import CandleData
```

Fields:

Field	Type	Description
type	string	"candle"
instrument	string	Trading pair symbol (e.g. BTC-USD)
exchange	string	Source exchange
timeframe	string	Candle duration (1m, 1h, 1d, ...)
open_at	datetime	Exchange-provided candle interval start time
open	float	Opening price
high	float	High price
low	float	Low price
close	float	Closing price
volume	float	Total traded volume
vwap	float | None	Volume-weighted average price (optional)
trades	int | None	Number of trades in the candle (optional)
complete	bool	True marks the final bar for its window; False marks a provisional intra-minute update of a still-forming "living" bar (default True)

> Note: strategies receiving non-final bars (complete == False) may want to gate signal logic on candle.complete so they act only on closed candles and ignore provisional intra-minute updates.

Technical Indicators

RSI

```
from snapper.indicators.rsi import rsi
import pandas as pd

closes = pd.Series([c.close for c in candles])
rsi_values = rsi(closes, period=14)
current_rsi = rsi_values.iloc[-1]
```

MACD

```
from snapper.indicators.ta_lib_adapter import macd

macd_line, signal_line, histogram = macd(closes, fast=12, slow=26, signal=9)
```

TA-Lib (via adapter)

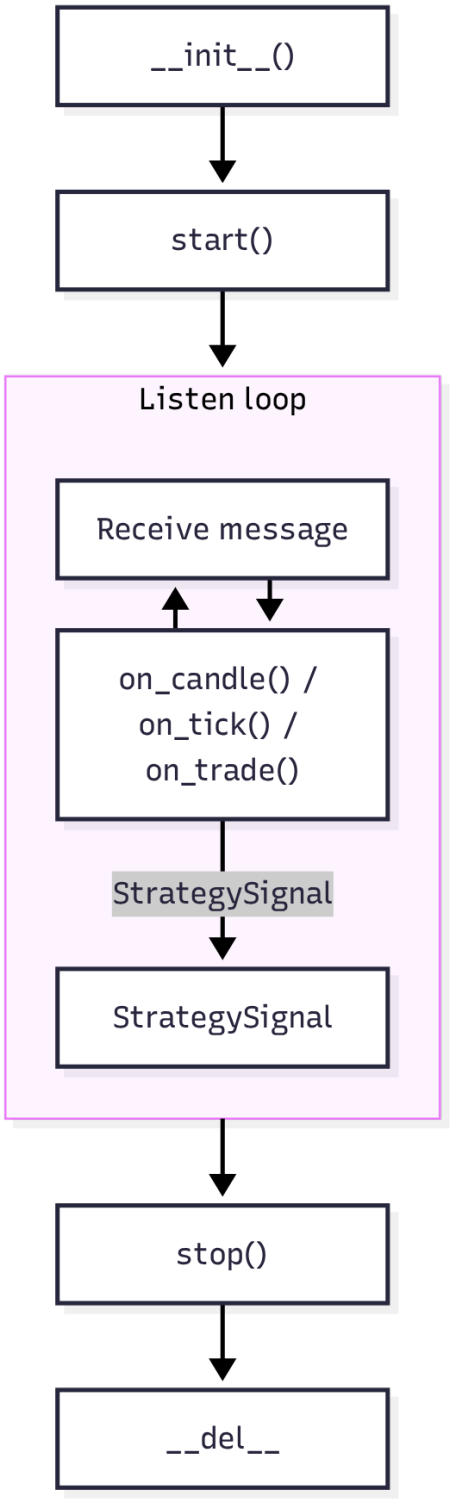
```
from snapper.indicators.ta_lib_adapter import rsi, macd
```

Configuration Validation

Framework automatically validates:

1. **Strategy name** — cannot be empty
2. **Inputs** — at least one required
3. **Outputs** — at least one instrument must be defined
4. **Exchange** — must be one of: paper, kraken, kraken_futures, walutomat
5. **Instruments** — must be available on selected exchange
6. **Paper/live mixing** — mixing paper inputs with live exchange not allowed

Strategy Lifecycle



BaseStrategy does not expose a separate cleanup() hook — teardown happens in stop() (cancel heartbeat + listen tasks, unsubscribe inputs, close the ZMQ publisher + context) and the implicit __del__ (cancel any surviving tasks, close subscriber/publisher sockets, terminate the ZMQ context).

Running Strategies

Via Process Manager (recommended)

Strategies are automatically managed by the server:

```
snapper server
```

Strategies can be enabled/disabled in the dashboard.

Standalone (for testing)

```python
import asyncio
from snapper.strategies.factory import StrategyFactory
from snapper.strategies.base import StrategyConfig

# Importing the strategy module fires the @register_strategy
# side effect that populates the factory's registry. Snapper's
# strategies/__init__.py is intentionally empty (no re-exports
# per project convention), so each concrete strategy must be
# imported by hand before the factory can resolve it by name.
import snapper.strategies.rsi  # noqa: F401  # register RSIReversion

config = StrategyConfig(
    name="test_rsi",
    strategy_class="RSIReversion",
    inputs=["market.kraken.BTC-USD.candles.1h"],
    outputs=["BTC-USD"],
    exchange="paper",
    params={"period": 14, "upper": 70, "lower": 30},
)

factory = StrategyFactory()
strategy = factory.create_strategy(config)

async def main() -> None:
    # `BaseStrategy.start()` is non-blocking — it wires sockets and
    # spawns the listen + heartbeat tasks, then returns. Keep the
    # event loop alive while the background tasks process data.
    await strategy.start()
    try:
        await asyncio.Event().wait()  # block until Ctrl-C / cancel
    finally:
        await strategy.stop()

asyncio.run(main())
```

Testing Strategies

Unit Tests

```
import pytest
from snapper.strategies.rsi import RSIReversion
from snapper.strategies.base import StrategyConfig
from snapper.messaging.schemas.data import CandleData


@pytest.fixture
def rsi_strategy() -> RSIReversion:
    config = StrategyConfig(
        name="test_rsi",
        strategy_class="RSIReversion",
        inputs=["market.kraken.BTC-USD.candles.1h"],
        outputs=["BTC-USD"],
        exchange="paper",
        params={"period": 14, "upper": 70, "lower": 30},
    )
    return RSIReversion(config)


async def test_buy_signal_on_low_rsi(rsi_strategy: RSIReversion) -> None:
    # Prepare data with low RSI
    ...
    signal = await rsi_strategy.on_candle("BTC-USD", candle)
    assert signal is not None
    assert signal.side == "buy"
```

Backtesting

For BacktestConfig runs, direct_db instantiates strategies with a synthetic input prefix (candles.{exchange}.synthetic.{timeframe}), while zmq_replay publishes market.{exchange}.{instrument}.candles.{timeframe} on a private broker. The separate paper_feed_publisher replay process is the path that uses the dedicated market.paper.{source_exchange}... topic shape:

```
default_config={
    "inputs": ["market.paper.kraken.BTC-USD.candles.1h"],
    "exchange": "paper",
    ...
}
```

replay.* is not a valid ZMQ topic — BaseStrategy._listen_loop filters incoming frames to market.* and routes them through BaseStrategy._dispatch_market_data. The canonical builder is market_topic(...) in src/snapper/messaging/topics/builders.py; paper topics are built by passing exchange="paper" together with the venue under source_exchange=... (there is no separate paper_market_topic symbol).

Trailing Stop Plans

Trailing stops are server-side evaluated execution plans that ratchet a stop price as the market moves favorably and fire a market close when the stop is breached. Attach a trailing stop to an open position_cycle via POST /api/trailing-stops (see docs/api.md).

Parameters

Field	Type	Bounds	Meaning
trailing_pct	float	0 < x < 100	Distance from peak price, as percentage. 5.0 means "trigger 5 % below the highest favorable price seen".
min_lock_pct	float	0 <= x < 100 (default 0)	Profit threshold the market must cross before the trailing stop arms. 0 means "arm immediately on attach"; 5.0 means "wait until price has moved 5 % in your favor before tracking a stop".

Behavior

- **Peak ratchet:** every tick where the last price moves favorably updates the peak; the stop level recomputes as peak * (1 - trailing_pct/100) for longs or peak * (1 + trailing_pct/100) for shorts.
- **Stop monotonicity:** for longs, the stop only moves UP (never down); for shorts the stop only moves DOWN. An adverse tick never loosens the stop.
- **min_lock dead zone:** while min_lock_pct > 0 and the market has not yet crossed that threshold, the trailing stop is attached and armed but NOT tracking — a close is NOT emitted no matter how far the price moves against you. A decision warning is logged on the trailing_stop_created row at create time.
- **Single trailing stop per cycle:** enforced by the partial unique index uq_ep_active_trailing_stop_per_cycle. A second create on the same open cycle returns HTTP 409.
- **Cycle close sweeps the plan:** if the cycle closes externally (reconciliation flat, manual close, bracket fill), _sweep_cycle_closures() cancels the trailing stop on the next clock tick.
- **Checkpoint persistence:** peak_price and current_stop are persisted every 10 s via the plan-executor checkpoint loop. On restart, state is restored and peak_price is floored at entry_price (long: max(peak, entry); short: min(peak, entry)).

- **Downtime semantics:** after restart, the first received tick is evaluated against recovered state. A breach fires only if that tick satisfies the stop condition; a crossing and recovery entirely during downtime is not replayed. Uncheckpointed peak changes can also be lost. There is no placed trailing-stop order on the exchange — the evaluator is server-side.

Interaction with brackets

A position cycle may carry both a bracket and a trailing_stop plan at the same time (the two partial unique indexes are disjoint per plan_type). Whichever fires first closes the cycle; the other is swept by the cycle-close handler.

Limitations

- entry_price and total_quantity are frozen at attach time. Dollar-cost averaging (scaling into a position) requires POST /cancel + re-attach after the new average is known.
- No exchange-side native trailing stops — every decision is made by PlanExecutorService against live ticks.
- Requires an instrument capability row with supports_reduce_only, like brackets. Admission uses the persisted capability matrix rather than a hard-coded futures-only venue list.
- No breakeven-move feature (ratchet stop to entry after N % profit).
- No time-based activation (arm after N minutes regardless of price).
- Requires a running plan executor and an accessible open position cycle. Orphan-cycle inspection and cleanup endpoints are implemented; see architecture.md.

Cross-asset market-data pattern

Snapper enables strategies to subscribe to a market-data-only feed (SymbolExchangeCapability.can_trade=False) and emit signals whose target is an execution-capable instrument on a different venue. Kraken FCM index futures (kraken_equities) are the primary driver: they have no order API, and the default public feed publishes ~10-minute-delayed candles + ticks. When the kraken_equities_realtime_ws_enabled DB setting is on and the authenticated realtime WS token mint succeeds, the feed is realtime instead; on any token/auth failure the publisher falls back to the public delayed feed. Ticks carry TickData.is_delayed accordingly.

Rules

- Strategies MAY subscribe to any combination of can_market_data=True topics regardless of the can_trade state of the source instrument.
- Every emitted StrategySignal.instrument MUST point at an can_trade=True instrument on the strategy's configured exchange. Snapper's order-entry capability guard (snapper.server._capability_guard.require_tradable) rejects submits against can_trade=False pairs with HTTP 422 error_code='instrument_market_data_only' — the signal would be dropped before reaching the venue.
- Tick-driven strategies that consume delayed feeds MUST gate on TickData.is_delayed=True before treating the price as current. Candle-driven cross-asset strategies inherit the source-feed latency implicitly; document the lag in the live runbook.
- Output-coverage enforcement at startup: whenever operator_public_id is populated on the process config, the launch path resolves the wallet (an explicit wallet_public_id passes through; an empty one is auto-resolved to the operator's single accessible wallet, failing the launch if zero or several match) and then enforce_strategy_outputs_covered checks that every instrument in StrategyConfig.outputs falls within the operator's active scope grants — a config with only operator_public_id populated does not skip the check. Coverage applies to live outputs only (paper-exchange configs return early), and the fully-unscoped paper path (neither field set) is still permitted for backwards compatibility on the REST start route; live launches without an operator are rejected.

Reference implementation

See src/snapper/strategies/examples/tradfi_observe_crypto_execute.py for a minimal EMA-crossover strategy that observes MNQM6-CME on kraken_equities and targets BTC-USD on kraken. The file is NOT registered with the process registry — it ships as a copy-paste starting point. Activation steps live in the module docstring; tests/meta/test_reference_strategy_not_registered.py enforces the non-registration invariant so future maintainers cannot accidentally promote the illustration into a live process.